

The Adjudication Layer for the Agentic Economy
Source: <https://docs.genlayer.com>

GenLayer uses decentralized AI-validator consensus to resolve contracts that require judgment, not just code.

Intelligent Contracts interpret language, process unstructured data, and pull live web inputs. No oracles, no intermediaries.

- **Bitcoin** – Trustless Money
- **Ethereum** – Trustless Computation
- **GenLayer** – Trustless Adjudication

The agentic-commerce stack is being built in public – Coinbase's [x402] (<https://www.x402.org/>) for payments, Ethereum's [ERC-8004] (<https://eips.ethereum.org/EIPS/eip-8004>) for trustless agent identity, the Linux Foundation's [A2A] (<https://a2a-protocol.org>) for agent interoperability, plus Stripe/OpenAI's ACP, Visa's Trusted Agent Protocol, Google's AP2, and Mastercard's Agent Pay. Every layer engineers the happy path. None ships dispute resolution. GenLayer fills that gap.

Explore [use cases](/understand-genlayer-protocol/typical-use-cases) or jump straight into building.

```
> Note:
> Start with Skills 🤖 – The fastest way to build a contract or run a validator is the
[GenLayer Skills](https://skills.genlayer.com/) plugin for Claude Code. One command
scaffolds, deploys, and operates – no manual setup.
>
> ``bash
> claude /plugin marketplace add genlayerlabs/skills
> # then: genlayer-dev (build contracts) or genlayernode (run a validator)
> ``
```

Get Started

- [Discover the Protocol](/understand-genlayer-protocol): What GenLayer is, how Optimistic Democracy works, and what makes it different.
- [For Developers](/developers): Build Intelligent Contracts in Python. Deploy to testnet or Studio.
- [For Validators](/validators/setup-guide): Run a validator node and participate in the GenLayer network.

Community and Careers

Join the community on [Discord](<https://discord.gg/8Jm4v89VAu>) or [Telegram](<https://t.me/genlayer>) to ask questions, share feedback, and showcase your builds.

We're hiring – [see open roles](<https://x.com/GenLayer/jobs>).

What is GenLayer?

GenLayer is the first AI-native blockchain built for AI-powered smart contracts—called Intelligent Contracts—capable of reasoning and adapting to real-world data. Its foundation is the Optimistic Democracy consensus mechanism, an enhanced Delegated Proof of Stake (dPoS) model where validators connect directly to Large Language Models (LLMs). This setup allows for non-deterministic operations—such as processing text prompts, fetching live web data, and executing AI-based decision-making—while preserving the reliability and security of a traditional blockchain.

Core Technology

At the heart of GenLayer lies Optimistic Democracy—an enhanced Delegated Proof of Stake (dPoS) consensus mechanism that integrates AI models directly into validator operations. This synergy delivers three capabilities traditional blockchains cannot match:

1. **On-Chain AI Processing**

Validators connect to leading AI models (GPT, LLaMA, Meta, etc.) to execute complex reasoning on-chain, from natural language comprehension to data-driven predictions.

2. **Consensus-Backed Security**

Multiple validators vote on outcomes, ensuring collective agreement and robust reliability for every transaction—even those involving non-deterministic AI outputs.

3. **Intelligent Contracts**

Smart contracts in GenLayer gain reasoning abilities, allowing them to understand natural language, process real-world data, and adapt to evolving conditions.

Technical Implementation

To integrate AI seamlessly with the blockchain, GenLayer employs a distributed neural consensus network, wherein validators run specialized software connected via API to advanced AI models. This approach unifies:

- Delegated Proof of Stake (dPoS) for efficient block production and governance.
- Neural Consensus for non-deterministic transactions requiring advanced AI reasoning.

This architecture supports autonomous DAOs, self-executing prediction markets, and dynamic DeFi protocols that react to real-world data in real time.

Optimistic Democracy: How Consensus Works

Source: <https://docs.genlayer.com/understand-genlayer-protocol>

Optimistic Democracy is GenLayer's consensus mechanism for merging probabilistic AI systems with deterministic blockchain rules so the network can reach secure and accurate consensus at scale. Inspired by **[Condorcet's Jury Theorem](https://jury-theorem.genlayer.com/)** (click the link to check out our interactive model), the process uses validator recomputation and majority agreement as a safety net for AI-driven computations.

1. **User Submits a Transaction**

A user sends a transaction request to the network (see the diagram's Step 1).

2. **Leader (Validator) Proposes Result**

The network selects a Leader, who processes the request and proposes an outcome (Step 2).

3. **Validators Recompute**

A group of Validators independently re-compute the transaction (Step 3). If the output aligns with the Leader's proposal, they approve; otherwise, they deny.

Validator Selection Mechanism

Token holders bolster network security by delegating tokens to validator candidates. A deterministic function $f(x)$ then randomly designates Leader-Validator and Validators for each transaction. This process promotes fairness, helps decentralize validation power, and strengthens GenLayer's security and trustlessness.

Validator Operational Framework

Each GenLayer validator node integrates:

- **Validator Software**

Handles core blockchain functions: networking, block production, and transaction management.

- ****AI Model Integration****

Connects to Large Language Models (LLMs) or other AI services for complex reasoning, natural language processing, and real-time data retrieval.

Validators seamlessly manage both:

1. ****Deterministic Transactions**** typical of traditional blockchains.
2. ****Non-Deterministic Transactions**** that leverage AI-driven logic (e.g., searching the internet, analyzing data, making probabilistic inferences).

By splitting tasks between standard deterministic transactions and advanced AI-powered transactions, GenLayer ensures high performance without compromising on security.

Putting It All Together

With Optimistic Democracy guiding consensus and validators empowered by AI, GenLayer enables a new class of blockchain applications. From DAOs that self-govern based on real-time data to DeFi protocols that dynamically adjust parameters in response to market changes, developers can now build truly intelligent decentralized solutions.

What is GenLayer

Source: <https://docs.genlayer.com/understand-genlayer-protocol/what-is-genlayer>

The Adjudication Layer for the Agentic Economy

GenLayer uses decentralized AI-validator consensus to resolve contracts that require judgment, not just code.

Intelligent Contracts interpret language, process unstructured data, and pull live web inputs. No oracles, no intermediaries.

- ****Bitcoin**** – Trustless Money
- ****Ethereum**** – Trustless Computation
- ****GenLayer**** – Trustless Adjudication

Where Bitcoin reached consensus on the **order** of transactions and Ethereum on the **execution** of code, GenLayer reaches consensus on the ****meaning**** of transactions. The technical primitive that makes this possible is the [Equivalence Principle] (/developers/intelligent-contracts/features/non-determinism): two answers can be different in form yet equivalent in meaning, and a network of independent AI validators can agree on that.

The Missing Layer

The agentic-commerce stack is being built in the open – and every layer of it stops at the happy path.

Layer	Standard	What it handles	Dispute resolution
Payments	[x402](https://www.x402.org/)	(Coinbase)	Internet-native, agent-friendly
Identity & reputation	[ERC-8004](https://eips.ethereum.org/EIPS/eip-8004)	(Ethereum)	Trustless agent identity
Agent interoperability	[A2A](https://a2a-protocol.org)	(Linux Foundation / Google)	Delegated to external validation protocols
How agents discover each other and exchange tasks			[A2A](https://a2a-protocol.org)

Other players are stacking in alongside them – Stripe + OpenAI's ACP (already powering ChatGPT checkout), Visa's Trusted Agent Protocol, Google's AP2, Mastercard's Agent Pay + BVNK, and Tempo (Stripe / Paradigm L1). The shape is the same in every case: the payment goes through, the job is accepted, the reputation is updated – and the moment a single material dispute appears, the stack reaches for a function it does not have.

That function is ****adjudication****. Not a sovereign court. A credible, machine-speed mechanism for evaluating contested commitments, weighing evidence, interpreting language,

reaching a verdict, and attaching consequences to it.

GenLayer is that layer.

What Intelligent Contracts Can Do

Subjective Decisions

Evaluate context and nuance. Turn judgment calls into enforceable on-chain outcomes – content moderation, claim assessment, quality evaluation.

Internet Access

Fetch live web data directly on-chain. Contracts can read websites, call APIs, and verify real-world information without oracles or intermediaries.

Natural Language Processing

Interpret human-readable inputs via LLMs. Contracts can analyze text, extract meaning, and make decisions based on qualitative criteria.

Image & Visual Processing

Pass images to LLMs for analysis – screenshot a webpage and verify its content, check visual evidence for claims, analyze receipts or documents. Contracts can capture screenshots via ``gl.nondet.web.render()`` and send them to LLMs via ``gl.nondet.exec_prompt(images=[...])``.

Unstructured Data

Process text, images, audio transcripts, and qualitative evidence. Handle real-world complexity that traditional smart contracts cannot.

How It Compares

Feature	Traditional Smart Contracts	Intelligent Contracts
Language	Solidity, Rust	Python
Data sources	On-chain only (or oracles)	On-chain + live web data
Decision logic	Deterministic only	Deterministic + subjective
AI integration	Not possible	Native LLM access (text + images)
Consensus	All nodes must agree on exact output	Validators assess equivalence of results

Architecture: Two Layers

GenLayer operates as two integrated layers:

GenLayer Chain – an EVM-compatible L2 (zkSync Elastic Chain). Holds account balances via ghost contracts, handles standard Ethereum operations (``eth_*` methods), and anchors to Ethereum's security model.

GenVM – the execution environment for Intelligent Contracts. A WebAssembly-based VM (built on [Wasmtime](https://wasmtime.dev)) that runs a Python interpreter with native access to LLMs, web data, and non-deterministic operations. Can also execute compiled native code.

Every Intelligent Contract has a corresponding **ghost contract** on the chain layer at the same address. Ghost contracts hold the contract's GEN balance, relay transactions to consensus, and execute external messages. See [Messages](/developers/intelligent-contracts/features/messages#ghost-contracts) for details.

Transactions enter via ``addTransaction`` on the chain layer. GenVM executes the contract logic. Results settle back on-chain.

Develop in Python

Intelligent Contracts are Python classes extending ``gl.Contract``:

```
```python
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8jzp09h6" }
```

```

from genlayer import *
import json

class WizardOfCoin(gl.Contract):
 has_coin: bool

 def __init__(self):
 self.has_coin = True

 @gl.public.write
 def ask_for_coin(self, request: str) -> None:
 if not self.has_coin:
 raise gl.vm.UserError("I don't have a coin!")

 prompt = f"""
 You are a wizard guarding a gold coin.
 An adventurer says: {request}
 Should you give them the coin?
 Respond as JSON: {{"give_coin": true/false}}
 """

 def leader_fn():
 return gl.nondet.exec_prompt(prompt, response_format="json")

 def validator_fn(leaders_res) -> bool:
 if not isinstance(leaders_res, gl.vm.Return):
 return False
 my_result = leader_fn()
 return my_result["give_coin"] == leaders_res.calldata["give_coin"]

 result = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)
 if result["give_coin"]:
 self.has_coin = False

 @gl.public.view
 def get_has_coin(self) -> bool:
 return self.has_coin
...

```

Full SDK available: [genlayer-js](/api-references/genlayer-js) (TypeScript), [genlayer-py](/api-references/genlayer-py) (Python), [CLI](/api-references/genlayer-cli).

[Get started →](/developers/intelligent-contracts/first-contract)

---

## # How GenLayer Works

Source: <https://docs.genlayer.com/understand-genlayer-protocol/optimistic-democracy-how-genlayer-works>

GenLayer works through **Optimistic Democracy**, a consensus mechanism where validators running diverse AI models independently evaluate transactions and vote on outcomes. This page explains the transaction lifecycle, how GenLayer reaches consensus on non-deterministic results from Intelligent Contracts, and how appeals lead to finality.

## ## Optimistic Democracy

GenLayer uses **Optimistic Democracy** for consensus – a mechanism where validators running diverse AI models independently evaluate transactions and vote on outcomes. It applies [Condorcet's Jury Theorem]([https://en.wikipedia.org/wiki/Condorcet%27s\\_jury\\_theorem](https://en.wikipedia.org/wiki/Condorcet%27s_jury_theorem)): a group of independent reasoners is more likely to reach the correct answer than any individual. This is what lets GenLayer act as an **adjudication layer** for the agentic economy – judgments emerge from a diverse validator set rather than from any one model, operator, or jurisdiction.

Transactions are accepted if a majority of validators agree. Anyone can appeal an accepted result, triggering re-evaluation by a new, larger validator set. This process can escalate through multiple rounds until a final decision is reached.

## ## Transaction Lifecycle

Every transaction moves through these stages:

1. **Pending** – queued, waiting to be picked up
2. **Proposing** – a leader validator executes the contract and proposes a result
3. **Committing** – other validators execute independently and submit encrypted votes
4. **Leader Revealing** – the leader reveals execution data and decryption keys
5. **Revealing** – validators reveal their votes
6. **Accepted** – majority consensus reached; transaction enters the appeal window
7. **Finalized** – appeal window closed, result is permanent and irreversible

If consensus is not reached, the transaction may be marked **Undetermined** or rotate to a new leader.

See [Transaction Execution](/understand-genlayer-protocol/core-concepts/transactions/transaction-execution) for the full state machine.

## ## Non-Determinism and Consensus

Because Intelligent Contracts use LLMs and web data, validators may produce different outputs for the same input. GenLayer provides several strategies for reaching consensus on non-deterministic results:

- **Strict equality** – all validators must produce the exact same output (for deterministic operations)
- **LLM-based comparison** – an LLM compares validator outputs against developer-defined criteria
- **Custom validation** – developers write explicit leader/validator function pairs with full control over consensus logic

See [Non-determinism](/developers/intelligent-contracts/features/non-determinism) for implementation details.

## ## Appeals and Finality

After a transaction is accepted, it enters a **finality window** during which anyone can appeal the result.

- An appeal triggers a new round with a fresh, larger validator set
- Appeals can escalate through multiple rounds
- The final round's decision is binding

Once the finality window closes without appeal (or after the final appeal round), the transaction is **finalized** – permanent and irreversible.

See [Appeal Process](/understand-genlayer-protocol/core-concepts/optimistic-democracy/appeal-process) and [Finality](/understand-genlayer-protocol/core-concepts/optimistic-democracy/finality) for details.

---

## # Use Cases

Source: <https://docs.genlayer.com/understand-genlayer-protocol/typical-use-cases>

GenLayer use cases are commitments where outcomes depend on judgment, such as evaluating evidence, interpreting language, or assessing quality, and where a deterministic smart contract alone cannot resolve them.

If you are deciding whether a feature belongs on GenLayer or in a normal backend, start with the [builder fit checklist](/developers/intelligent-contracts/when-to-use-genlayer).

The use cases below are grouped by where the need for adjudication is most acute today.

## ## 1. Performance & Milestone Adjudication

Payments, rewards, or recognition that depend on whether some obligation was actually fulfilled under criteria that are partly measurable and partly interpretive. The money is already on-chain. The commitment is already written down. The dispute already happens – and today it is resolved by human bottlenecks that do not scale.

- **Bounties** where payout depends on whether the work met the spec
- **Grant milestones** where tranche release turns on contested deliverables
- **Retroactive funding rounds** where allocation depends on impact assessment
- **Creator economies** where AI-scored rewards generate disputes with no resolution layer
- **Prediction markets** with subjective outcomes ([example contract] (/developers/intelligent-contracts/examples/prediction))
- **Contributor performance** tied to vesting or continued compensation
- **Freelance and gig work** – was the deliverable satisfactory? AI consensus replaces subjective back-and-forth
- **Chargebacks** – buyer/seller disputes resolved by analyzing shipping records, communication logs, and transaction history

GenLayer can support agreed dispute-resolution workflows, but it is not a court and does not automatically make a result legally binding. For legal or contractual use cases, parties still need the appropriate agreements, jurisdiction, and escalation process around the Intelligent Contract.

## ## 2. Adjudication Inside the Agentic-Commerce Stack

GenLayer plugs into the infrastructure the industry is building right now: payment rails ([x402](https://www.x402.org/)), agent identity and reputation ([ERC-8004](https://eips.ethereum.org/EIPS/eip-8004)), agent-to-agent task exchange ([A2A](https://a2a-protocol.org)), plus Stripe/OpenAI's ACP, Visa's Trusted Agent Protocol, Google's AP2, and Mastercard's Agent Pay. Each standard ships the happy path and carves the moment of disagreement out as someone else's problem. GenLayer is that someone else.

- **Agent-executed job disputes** – was the task delivered to spec?
- **Escrow release on ambiguous completion** – agent counterparties needing neutral resolution
- **SLA enforcement on agent work** – quality, latency, or scope claims
- **Reputation claims contested between counterparties** in ERC-8004-style identity systems
- **Coverage claims on agent-to-agent commitments** – parametric and evidence-based
- **Multi-agent workflows** where responsibility for failure has to be assigned across participants

## ## 3. Rule & Constitution Verification

Check whether something meets a set of criteria defined in natural language – a foundational primitive that many applications reduce to.

- Does a new prediction market meet listing guidelines?
- Does a DAO proposal comply with the organization's charter?
- Does a content submission follow community standards?
- Does a transaction comply with regulatory requirements?

## ## 4. Adjacent Surfaces

The same primitive shows up across digital commerce:

- **Insurance** – parametric and evidence-based claims evaluated by AI validators. Contracts fetch weather data, flight statuses, or photographic evidence to assess claims and trigger payouts automatically – no adjusters, no weeks of waiting.
- **Social content verification** – AI validators assess quality, detect plagiarism, and distribute rewards based on originality and engagement. Replaces centralized moderation with consensus-driven evaluation.

- **Code & work quality assurance** – staked submissions where reviewers are economically incentivized to find issues; AI validators assess deliverable completeness or compliance with specifications.
- **AI-governed organizations** – DAOs where proposals are written in natural language, evaluated against real-time data, and executed automatically when conditions align.
- **Compliance automation** – real-time screening against sanctions lists, KYC/AML requirements, and changing regulations. Contracts read authoritative sources directly – no manual updates needed.
- **Argumentation and debate markets** – structured debates where participants stake positions and AI consensus determines outcomes, a new primitive for information markets.

## ## What Makes These Possible

All of these share a common pattern: they require **judgment** that traditional smart contracts can't perform. GenLayer's Intelligent Contracts can:

- Fetch and interpret live web data
- Process natural language and unstructured inputs
- Make subjective decisions through multi-validator AI consensus
- Execute outcomes on-chain with full finality – justice in minutes, not months

See [projects building on GenLayer](https://portal.genlayer.foundation/#/) for live examples.

[Start building →](/developers/intelligent-contracts/first-contract)

---

## # Core Concepts

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts>

GenLayer core concepts are the fundamental building blocks that explain how Intelligent Contracts remain secure, efficient, and reliable in a non-deterministic environment. Use these topics to understand GenVM, transactions, Optimistic Democracy, the Equivalence Principle, the appeal process, finality, staking, slashing, and unstaking.

- [ GenVM](/core-concepts/genvm)
- [ Transactions](/core-concepts/transactions)
- [ Optimistic Democracy](/core-concepts/optimistic-democracy)
- [ Equivalence Principle](/core-concepts/optimistic-democracy/equivalence-principle)
- [ Appeal Process](/core-concepts/optimistic-democracy/appeal-process)
- [ Finality](/core-concepts/optimistic-democracy/finality)
- [ Staking](/core-concepts/optimistic-democracy/staking)
- [ Slashing](/core-concepts/optimistic-democracy/slashing)
- [ Unstaking](/core-concepts/optimistic-democracy/unstaking)

---

## # Validators and Validator Roles

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/validators-and-validator-roles>

Validators are GenLayer network participants that validate transactions, help maintain blockchain integrity and security, and participate in Optimistic Democracy consensus. They verify deterministic and non-deterministic transactions, use the Equivalence Principle for non-deterministic operations, take part in leader selection, vote on proposed outcomes, and stake tokens to earn validation rights and rewards.

## ## Overview

Validators are essential participants in the GenLayer network. They are responsible for validating transactions and maintaining the integrity and security of the blockchain. Validators play a crucial role in the Optimistic Democracy consensus mechanism, ensuring that both deterministic and non-deterministic transactions are processed correctly.



## ## Key Responsibilities

- **Transaction Validation**: Validators verify the correctness of transactions proposed by the leader, using mechanisms like the Equivalence Principle for non-deterministic operations.
- **Leader Selection**: Validators participate in the process of randomly selecting a leader for each transaction, ensuring fairness and decentralization.
- **Consensus Participation**: Validators cast votes on proposed transaction outcomes, contributing to the consensus process.
- **Staking and Incentives**: Validators stake tokens to earn the right to validate transactions and receive rewards based on their participation and correctness.

## ## Validator Selection and Roles

- **Leader Validator**: For each transaction, a leader is randomly selected among the validators. The leader is responsible for executing the transaction and proposing the result to other validators.
- **Consensus Validators**: Other validators assess the leader's proposed result and vote to accept or reject it based on predefined criteria.

## ## Becoming a Validator

- **Staking Requirement**: Participants must stake a certain amount of tokens to become validators.
- **Validator Configuration**: Validators must configure their nodes with the appropriate LLM providers and models, depending on the network's requirements.
- **Reputation and Slashing**: Validators must act honestly to avoid penalties such as slashing of their staked tokens.

---

## # GenVM (GenLayer Virtual Machine)

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/genvm>

GenVM is the execution environment for Intelligent Contracts in the GenLayer protocol, processing and managing contract operations across the GenLayer ecosystem. GenVM executes Intelligent Contracts that may use non-deterministic code while preserving blockchain security and consistency.

[Source code at GitHub](<https://github.com/genlayerlabs/genvm>)

## ## Purpose of GenVM

The only purpose of the GenVM is to execute Intelligent Contracts, which can have non-deterministic code while maintaining blockchain security and consistency.

In summary, the GenVM plays a crucial role in enabling GenLayer's unique features, bridging the gap between traditional smart contracts and AI-powered, web-connected Intelligent Contracts.

## ## Key Features That Make the GenVM Different

Unlike traditional blockchain virtual machines such as Ethereum Virtual Machine (EVM), the GenVM has some advanced features.

- **Integration with LLMs**: the GenVM facilitates seamless interaction between Intelligent Contracts and Large Language Models
- **Web access**: the GenVM provides access to the Internet
- **User friendliness**: Intelligent Contracts can be written in Python, which makes the learning curve much more shallow

## ## How the GenVM Works

1. **Contract Deployment**: When an Intelligent Contract is deployed, the GenVM compiles and executes the contract code.

2. **Transaction Processing**: As transactions are submitted to the network, the GenVM executes the relevant contract functions and produces the contract's next state.

## ## Developer Considerations

When developing Intelligent Contracts for the GenVM:

- Utilize Python's robust libraries and features
- Consider potential non-deterministic outcomes when integrating LLMs
- Implement proper error handling for web data access
- Optimize code for efficient execution within the rollup environment

---

## # Optimistic Democracy

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/optimistic-democracy>

Optimistic Democracy is GenLayer's consensus method for validating transactions and operations of Intelligent Contracts. It is designed to handle unpredictable outcomes from transactions involving web data or AI models, helping keep the network reliable and secure.

## ## Key Components

- **Validators**: Participants who stake tokens to earn the right to validate transactions. They play a crucial role in both the initial validation and any appeals process if needed.
- **Leader Selection**: A process that randomly picks one validator to propose the outcome for each transaction, ensuring fairness and reducing potential biases.

## ## How It Works

Optimistic Democracy relies on a mix of trust and verification to ensure transaction integrity:



1. **Initial Validation**: When a transaction is submitted, a small group of randomly selected validators checks its validity. One is chosen as the leader. The leader executes the transaction, and the other validators assess the leader's proposal using the Equivalence Principle.
2. **Majority Consensus**: If most validators accept the leader's proposal, the transaction is provisionally accepted. However, this decision is not final yet, allowing for possible appeals during a limited window of time, known as the **Finality Window**.  
  
> **Note**:  
> If any validator fails to vote within the specified timeframe, they are replaced, and a new validator is selected to cast a vote.
3. **Initiating an Appeal**: If a participant disagrees with the initial validation (if it's incorrect or fraudulent), they can appeal during the Finality Window. They must submit a request and provide a bond. After the appeal starts, a new group of validators joins the original ones. This group first votes on whether the transaction should be re-evaluated. If they agree, a new leader is chosen to reassess the transaction, and all validators then review this new evaluation.
4. **Appeal Evaluation**: The new leader re-evaluates the transaction, while the other validators assess the leader's proposal using the Equivalence Principle. This step involves more validators, increasing the chances of an accurate decision.
5. **Escalating Appeals**: If the appealing party is still not satisfied, the process can escalate, with each round involving more validators. Each round doubles the number of validators. A new leader is only chosen if the transaction is overturned.

6. **\*\*Final Decision:\*\*** The appeals process continues until a majority consensus is reached or until all validators have participated. The final decision is recorded, and the transaction's state is updated accordingly. If the appealing party is correct, they receive a reward for their efforts, while incorrect appellants lose their bond.

---

## # Equivalence Principle Mechanism

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/optimistic-democracy/equivalence-principle>

The Equivalence Principle mechanism is a cornerstone in ensuring that Intelligent Contracts function consistently across various validators when handling non-deterministic outputs like responses from Large Language Models (LLMs) or data retrieved through web browsing. It plays a crucial role in how validators assess and agree on the outcomes proposed by the Leader.

The Equivalence Principle protects the network from manipulations or errors by ensuring that only suitable, equivalent outcomes influence the blockchain state.

## ## Key Features of the Equivalence Principle

The Equivalence Principle is fundamental to how Intelligent Contracts operate, ensuring they work reliably across different network validators.

- **\*\*Consistency in Decentralized Outputs:\*\*** The Equivalence Principle allows outputs from various sources, such as LLMs or web data, to be different yet still considered valid as long as they meet predefined standards. This is essential to maintain fairness and uniform decision-making across the blockchain, despite the natural differences in AI-generated responses or web-sourced information.

- **\*\*Security Enhancement:\*\*** To protect the integrity of transactions, the Equivalence Principle requires that all validators check each other's work. This mutual verification helps prevent errors and manipulation, ensuring that only accurate and agreed-upon data affects the blockchain.

- **\*\*Output Validation Flexibility:\*\*** Intelligent Contracts often need to handle complex and varied data. This part of the principle allows developers to set specific rules for what counts as "equivalent" or acceptable outputs. This flexibility helps developers tailor the validation process to suit different needs, optimizing either for accuracy or efficiency depending on the contract's requirements.

## ## Types of Equivalence Principles

Validators work to reach a consensus on whether the result set by the Leader is acceptable, which might involve direct comparison or qualitative evaluation, depending on the contract's design. If the validators do not reach a consensus due to differing data interpretations or an error in data processing, the result might be challenged or an appeal process might be initiated.

## ### Comparative Equivalence Principle

In the Comparative Equivalence Principle, both the Leader and the validators perform identical tasks and then directly compare their respective results with the predefined criteria in the Equivalence Principle to ensure consistency and accuracy. This method uses an acceptable margin of error to handle slight variations in results between validators and is suitable for quantifiable outputs. However, since multiple validators perform the same tasks as the Leader, it increases computational demands and associated costs.

For example, if an Intelligent Contract is tasked with calculating the average rating of a product based on user reviews, the Equivalence Principle specifies that the average ratings should not differ by more than 0.1 points. Here's how it works:

1. **\*\*Leader Calculation\*\*:** The Leader validator calculates the average rating from the user reviews and arrives at a rating of 4.5.
2. **\*\*Validators' Calculations\*\*:** Each validator independently calculates the average rating

using the same set of user reviews. Suppose one validator calculates an average rating of 4.6.

3. **Comparison**: The validators compare their calculated average (4.6) with the Leader's average (4.5). According to the Equivalence Principle, the ratings should not differ by more than 0.1 points.

4. **Decision**: Since the difference (0.1) is within the acceptable margin of error, the validators accept the Leader's result as valid.

### Non-Comparative Equivalence Principle

In contrast, the Non-Comparative Equivalence Principle does not require validators to replicate the Leader's output, which makes the validation process faster and less costly. Instead, validators assess the accuracy of the Leader's result against the criteria defined in the Equivalence Principle. This method is particularly useful for qualitative outputs like text summaries.

For example, in an Intelligent Contract designed to summarize news articles, the process works as follows:

1. **Leader Summary**: The Leader validator generates a summary of a news article.

2. **Evaluation Criteria**: The Equivalence Principle defines criteria for an acceptable summary, such as accuracy, relevance, and length.

3. **Validators' Assessment**: Instead of generating their own summaries, validators review the Leader's summary and check if it meets the predefined criteria.

- **Accuracy**: Does the summary accurately reflect the main points of the article?

- **Relevance**: Is the summary relevant to the content of the article?

- **Length**: Is the summary within the acceptable length?

4. **Decision**: If the Leader's summary meets all the criteria, it is accepted by the validators.

### Key Points for Developers

- **Setting Equivalence Criteria**: Developers must define what 'equivalent' means for each non-deterministic operation in their Intelligent Contract. This guideline helps validators judge if different outcomes are close enough to be treated as the same.

- **Ensuring Contract Reliability**: By clearly defining equivalence, developers help maintain the reliability and predictability of their contracts, even when those contracts interact with the unpredictable web or complex AI models.

---

### Appeals Process

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/optimistic-democracy/appeal-process>

The appeals process in GenLayer is the Optimistic Democracy mechanism for correcting errors or disagreements in the validation of Intelligent Contracts. Participants can challenge an initial validation decision so non-deterministic transactions are reassessed, helping preserve the platform's robustness and fairness.

### How It Works

- **Initiating an Appeal**: Participants can appeal the initial decision by submitting a request and a required bond during the Finality Window. A new set of validators is then added to the original group to reassess the transaction.

- **Appeal Evaluation**: The new validators first review the existing transaction to decide if it needs to be overturned. If they agree it should be re-evaluated, a new leader re-evaluates the transaction. The combined group of original and new validators then review this new evaluation to ensure accuracy.

- **Escalating Appeals**: If unresolved, the appeal can escalate, doubling the number of validators each round until a majority consensus is reached or all validators have participated.

Once a consensus is reached, the final decision is recorded, and the transaction's state is updated. Correct appellants receive a reward, while those who are incorrect may lose their bond.

## ## Gas Costs for Appeals

The gas costs for an appeal can be covered by the original user, the appellant, or any third party. When submitting a transaction, users can include an optional tip to cover potential appeal costs. If insufficient gas is provided, the appeal may fail to be processed, but any party can supply additional gas to ensure the appeal proceeds.

---

## # Finality

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/optimistic-democracy/finality>

Finality in GenLayer is the state in which a transaction is settled, unchangeable, and no longer appealable. Once a transaction achieves finality, it cannot be appealed or altered, giving participants certainty that the outcome is definitive. Finality is especially important for applications that depend on accurate settled outcomes, such as financial contracts or decentralized autonomous organizations (DAOs).

## ## Finality Window

The Finality Window is a time frame during which a transaction can be challenged or appealed before it becomes final. This window serves several purposes:

1. **\*\*Appeals\*\***: During the Finality Window, any participant can appeal a transaction if they believe the validation was incorrect. This allows for a process of checks and balances, ensuring that non-deterministic transactions are evaluated properly.
2. **\*\*Re-computation\*\***: If a transaction is appealed, the system can re-evaluate the transaction with a new set of validators. The Finality Window provides the time necessary for this process to occur.
3. **\*\*Security\*\***: The window also acts as a security feature, allowing the network to correct potential errors or malicious activity before finalizing a transaction.



## ## Deterministic vs. Non-Deterministic Transactions

In GenLayer, Intelligent Contracts are classified as either deterministic or non-deterministic.

### ### Deterministic Contracts

These contracts have a shorter Finality Window because their validation process is straightforward and not subject to appeals. However, it is essential that all interactions with the contract remain deterministic to maintain this efficiency.

### ### Non-Deterministic Contracts

Non-deterministic contracts involve Large Language Model (LLM) calls or web data retrieval, which introduce variability in their outcomes. These contracts require a longer Finality Window to account for potential appeals and re-computation.

> **\*\*Note:\*\***

> If a specific transaction within the contract is deterministic but interacts with a non-deterministic part of the contract, it will be treated as non-deterministic. This ensures that any appeals or re-computations of previous transactions are handled consistently, maintaining the integrity of the contract's overall state.

## ## Fast Finality

For scenarios requiring immediate finality, such as emergency decisions in a DAO, it is possible to pay for all validators to validate the transaction immediately. This approach, though more costly, allows for fast finality, bypassing the typical Finality Window.

> **Note:**

> Fast finality only works if there are no previous non-deterministic transactions still within their Finality Window. Even if your transaction is considered final, if a previous transaction is reverted, your transaction will have to be recomputed as it might depend on the same state.

## ## Appealability and Gas

When submitting a transaction, users can include additional gas to cover potential appeals. If a transaction lacks sufficient gas for appeals, third parties can supply additional gas during the Finality Window. Developers of Intelligent Contracts can also set minimum gas requirements for appealability, ensuring that critical transactions have adequate coverage.

---

## # Staking in GenLayer

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/optimistic-democracy/staking>

Staking in GenLayer is the process where validators lock a specified amount of tokens on the rollup layer to participate in the network. This commitment supports GenLayer's consensus mechanism and enables validators to process transactions and help manage the network.

## ## Validators vs Delegators

Role	Minimum Stake	Infrastructure	Rewards
Validator	42,000 GEN	Must run a node	10% operational fee + stake rewards
Delegator	42 GEN	None required	Passive stake rewards

**Validators** run the consensus infrastructure and are responsible for executing intelligent contracts and validating transactions. They receive a 10% operational fee from rewards before distribution.

**Delegators** stake their tokens with validators without running infrastructure. They earn passive rewards proportional to their stake, minus the validator's operational fee.

## ## How Staking Works

- **Stake Deposit**: To become a validator on GenLayer, participants must deposit GEN tokens on the rollup layer. This deposit acts as a security bond and qualifies them to join the pool of active validators.

- **Validator Participation**: Only a maximum of 1000 validators with the highest stakes can be part of the active validator set. Once staked, validators take on the responsibility of validating transactions and executing Intelligent Contracts. Their role is crucial for ensuring the network's reliability and achieving consensus on transaction outcomes.

- **Delegated Proof of Stake (DPoS)**: GenLayer enhances accessibility and network security through a Delegated Proof of Stake system. This allows token holders who are not active validators themselves to delegate their tokens to trusted validators. By delegating their tokens, users increase the total stake of the validator and share in the rewards. Typically, the validator takes a configurable fee (around 10%), with the remaining rewards (90%) going to the delegating user.

- **Earning Rewards**: Validators, and those who delegate their tokens to them, earn rewards for their contributions to validating transactions, paid in GEN tokens. These rewards are proportional to the amount of tokens staked and the transaction volume processed.

- **Risk of Slashing**: Validators, and by extension their delegators, face the risk of having a portion of their staked tokens [slashed](/understand-genlayer-protocol/core-

concepts/optimistic-democracy/slashing) if they fail to comply with network rules or if the validator supports fraudulent transactions.

## ## Owner, Operator, and ValidatorWallet

When a validator joins, the system creates three distinct entities:

**\*\*ValidatorWallet\*\***: A separate smart contract wallet created automatically on ``validatorJoin()``. This is the primary validator identifier and holds staked GEN tokens.

**\*\*Owner Address\*\***: The address that creates the validator (`msg.sender`). It controls staking operations and can change the operator address. Should use a cold wallet for security.

**\*\*Operator Address\*\***: Used for consensus operations. Can differ from the owner (hot wallet recommended). It can be changed by the owner but cannot be the zero address or reused across validators.

## ## Epoch System

The network operates in epochs (1 day):

- **\*\*Epoch +2 Activation Rule\*\***: All deposits become active 2 epochs after they are made
- Epoch finalization requires all transactions to be finalized
- Cannot advance to epoch N+1 until epoch N-1 is finalized
- Validators are "primed" via ``validatorPrime()`` each epoch (permissionless - anyone can call it)

**\*\*Critical\*\***: If ``validatorPrime()`` isn't called, the validator is excluded from the next epoch's selection.

## ### Genesis Epoch 0

Epoch 0 is the **\*\*genesis bootstrapping period\*\*** with special rules designed to facilitate network launch. The normal staking rules are relaxed to allow rapid network bootstrapping.

## #### What is Epoch 0?

Epoch 0 is the **\*\*bootstrapping period\*\*** before the network becomes operational. During epoch 0:

- **\*\*No transactions are processed\*\*** - the network is not yet active
- **\*\*No consensus occurs\*\*** - validators are not yet participating
- Stakes are registered and prepared for activation in epoch 2

**\*\*Important\*\***: The network transitions directly from epoch 0 to epoch 2 (epoch 1 is skipped). Validators and delegators who stake in epoch 0 become active in epoch 2, but only if they meet the minimum stake requirements.

## #### Special Rules for Epoch 0

Rule	Normal Epochs (2+)	Epoch 0
Validator minimum stake	42,000 GEN	No minimum to join
Delegator minimum stake	42 GEN	No minimum to join
Activation delay	+2 epochs	Active in epoch 2
validatorPrime required	Yes, each epoch	Not required
Share calculation	Based on existing ratio	1:1 (shares = input)
Transaction processing	Yes	No (bootstrapping only)

**\*\*Activation requires meeting minimums\*\***: While you can join with any amount during epoch 0, your stake will only be **\*\*activated in epoch 2\*\*** if it meets the minimum requirements (42,000 GEN for validators, 42 GEN for delegators). Stakes below the minimum remain registered but inactive.

## #### Validators in Epoch 0

## **\*\*Key behaviors:\*\***

1. **\*\*No minimum stake to join\*\***: Validators can join with any non-zero amount during epoch 0
2. **\*\*Registered for epoch 2\*\***: Stakes are recorded and will become active when epoch 2 begins
3. **\*\*No priming required\*\***: `validatorPrime()` is not needed during epoch 0
4. **\*\*No consensus participation\*\***: Validators do not process transactions in epoch 0

**\*\*Do validators need to take any action in epoch 0 to be active in epoch 2?\*\***

No. Validators who join in epoch 0:

- Have their stake registered during epoch 0
- Become active automatically in epoch 2 (epoch 1 is skipped) **\*\*only if they have at least 42,000 GEN staked\*\***
- Must start calling `validatorPrime()` in epoch 2 for continued participation in epoch 4+

**\*\*Important\*\***: Validators who joined in epoch 0 with less than 42,000 GEN will **\*\*not be active\*\*** in epoch 2. They must deposit additional funds to meet the minimum requirement before epoch 2 begins.

## **#### Delegators in Epoch 0**

### **\*\*Key behaviors:\*\***

1. **\*\*No minimum delegation\*\***: Any non-zero amount accepted during epoch 0
2. **\*\*Registered for epoch 2\*\***: Delegation is recorded and will become active when epoch 2 begins
3. **\*\*No rewards in epoch 0\*\***: Since no transactions are processed, no rewards are earned during epoch 0

**\*\*Is a delegation made in epoch 0 active in epoch 2?\*\***

Yes. Delegations made in epoch 0 become active in epoch 2 (epoch 1 is skipped). Unlike normal epochs where you wait +2 epochs, epoch 0 delegations activate as soon as the network becomes operational.

## **#### Activation Timeline Comparison**

### **\*\*Normal Epochs (2+):\*\***

\\

Epoch N: `validatorJoin()` or `delegatorJoin()` called

Epoch N+1: `validatorPrime()` stages the deposit

Epoch N+2: `validatorPrime()` activates the deposit → NOW ACTIVE

\\

### **\*\*Epoch 0 (Bootstrapping):\*\***

\\

Epoch 0: `validatorJoin()` or `delegatorJoin()` called → stake registered (not yet active)  
No transactions processed, no consensus

Epoch 2: Stakes become active (if minimum met), network operational, `validatorPrime()` required

\\

## **#### Share Calculation in Epoch 0**

In epoch 0, shares are calculated at a 1:1 ratio with the input amount:

...

Shares = Input Amount

Example: Deposit 1,000 GEN → Receive 1,000 shares

\\



This is because there's no existing stake pool to calculate a ratio against. Starting from epoch 2, shares are calculated based on the current stake-to-share ratio.

#### #### Transitioning from Epoch 0 to Epoch 2

When the network advances from epoch 0 to epoch 2 (epoch 1 is skipped):

1. **Epoch 0 stakes that meet minimums become active** - validators need 42,000 GEN, delegators need 42 GEN
2. **Normal minimum requirements apply** for new joins/deposits
3. **+2 epoch activation delay** applies to all new deposits
4. **validatorPrime() becomes mandatory** for validators to remain in the selection pool
5. **Existing validators** must ensure their nodes begin calling `validatorPrime()` in epoch 2

#### #### FAQ: Epoch 0 Special Cases

**Q: Can I join as a validator with less than 42,000 GEN in epoch 0?**

A: Yes, any non-zero amount is accepted during epoch 0. However, you will **not be active** in epoch 2 unless you have at least 42,000 GEN staked by then.

**Q: If I delegate in epoch 0, when does it become active?**

A: In epoch 2. Unlike normal epochs with a +2 delay, epoch 0 delegations activate when the network becomes operational.

**Q: Do I need to call validatorPrime() in epoch 0?**

A: No. Priming is not required during epoch 0. Your node should start calling it automatically when epoch 2 begins.

**Q: Will my epoch 0 stake still be active after epoch 0 ends?**

A: Yes, if you meet the minimum requirements. Stakes from epoch 0 carry forward and remain active in all subsequent epochs.

**Q: What happens to my stake if I joined in epoch 0 but my node doesn't call validatorPrime() in epoch 2?**

A: You'll be excluded from validator selection in epoch 4, but your stake remains. Once priming resumes, you'll be eligible for selection again.

#### ## Shares vs Stake

The staking system uses shares to track ownership:

**Shares** are fixed quantities that never change. You receive shares when depositing and exit by burning shares. They represent immutable claims on the stake pool.

**Stake** is the dynamic GEN token amount. It increases with rewards/fees and decreases with slashing. The exchange rate is calculated as:

...

```
stake_per_share = total_stake / total_shares
```

**Example:** 100 shares representing 1,000 GEN (10 GEN per share). After rewards are distributed, the same 100 shares might represent 1,050 GEN (10.5 GEN per share). Rewards automatically compound without user action.

#### ## Validator Selection and Weight

Validators are selected for consensus based on their weight, calculated using:

...

```
Weight = (ALPHA × Self_Stake + (1-ALPHA) × Delegated_Stake)^BETA
```

#### **\*\*Parameters:\*\***

- **\*\*ALPHA = 0.6\*\***: Self-stake counts 50% more than delegated stake
- **\*\*BETA = 0.5\*\***: Square-root damping prevents whale dominance

#### **\*\*Effects:\*\***

- Higher stake leads to higher weight and higher selection probability
- Doubling stake only increases weight by approximately 41%
- Encourages distribution across validators
- Smaller validators often provide higher returns per GEN staked

#### **## Reward Distribution**

#### **\*\*Sources:\*\***

1. Transaction Fees
2. Inflation (starting at 15% APR, decreasing to 4% APR over time)

#### **\*\*Distribution Pattern:\*\***

- **\*\*10%\*\*** → Validator owners (operational fee)
- **\*\*75%\*\*** → Total validator stake (validators + delegators)
- **\*\*10%\*\*** → Developers
- **\*\*5%\*\*** → Locked future allocation for the DeepThought AI-DAO

Within the 75% stake allocation:

- Self-stake receives a portion based on the validator's own staked amount
- Delegated stake is split among delegators proportionally to their shares

Rewards automatically increase the stake-per-share ratio without requiring user action.

#### **## Unbonding Period**

Both validators and delegators face a **\*\*7-epoch unbonding period\*\*** when withdrawing:

- Prevents rapid stake movements that could destabilize the network
- Exit is not processed immediately - validator remains active until next `validatorPrime()` call at next epoch
- Exited tokens stop earning rewards only after the exit is processed in the next epoch
- Countdown starts from the exit epoch
- Funds become claimable when: `current_epoch >= exit_epoch + 7`

#### **\*\*Detailed Exit Flow\*\***:

...

Epoch N: `validatorExit(all_shares)` called

- Exit scheduled in contract state
- Validator STILL ACTIVE and earning rewards
- Can still be selected for consensus participation

Epoch N+1: `validatorPrime()` called (by anyone)

- Exit processed: stake reduced to 0
- Removed from validator tree
- No longer active or earning rewards
- Cannot be selected for new transactions

Epoch N+2: `epochAdvance()` called

- Validator officially not in active validator set
- Fully excluded from consensus operations

Epoch N+7: `validatorClaim()` callable

- 7 epochs have passed since exit call (epoch N)
- Tokens released to validator owner

...

## ## Validator Priming

``validatorPrime(address validator)`` is a critical function that:

- Activates pending deposits
- Processes pending withdrawals
- Distributes previous epoch rewards
- Applies pending slashing penalties
- Sorts the validator into the selection tree

### **\*\*Key Properties:\*\***

- **\*\*Monitoring Required\*\***: Ensure correct execution
- **\*\*Permissionless\*\***: Anyone can call it
- **\*\*Incentivized\*\***: Caller receives 1% of any slashed amount
- **\*\*Critical\*\***: If the node fails to prime, the validator is excluded from the next epoch
- **\*\*No Loss\*\***: Missing priming doesn't lose rewards, but the validator can't be selected

## ## Unstaking and Withdrawing

Both validators and delegators can withdraw their staked tokens, but must follow the unbonding process.

### ### For Validators

To stop validating or retrieve staked tokens, validators must:

1. **\*\*Calculate shares to exit\*\***: Determine how many shares to withdraw (partial or full)
2. **\*\*Call ``validatorExit(shares)``\*\***: Initiate the unbonding process
3. **\*\*Wait 7 epochs\*\***: Tokens are locked during the unbonding period
4. **\*\*Call ``validatorClaim()``\*\***: Retrieve tokens after unbonding completes

Validators can perform partial exits while remaining active, as long as their stake stays above the 42,000 GEN minimum.

### ### For Delegators

Delegators follow a similar process:

1. **\*\*Calculate shares to exit\*\***: Use ``sharesOf(delegator, validator)`` to check current shares
2. **\*\*Call ``delegatorExit(validator, shares)``\*\***: Initiate unbonding for a specific validator
3. **\*\*Wait 7 epochs\*\***: Tokens are locked during the unbonding period
4. **\*\*Call ``delegatorClaim(delegator, validator)``\*\***: Retrieve tokens after unbonding

### **\*\*Important for delegators:\*\***

- Exit each validator separately if delegating to multiple validators
- Claims are permissionless—anyone can trigger them on your behalf
- Tokens stop earning rewards immediately upon calling exit
- Multiple exits create separate withdrawals that can be claimed together

For detailed step-by-step instructions and code examples, see the [Staking Guide](/developers/staking-guide).

## ## Governance and Safeguards

- **\*\*24-Hour Delay\*\***: All slashing actions have a governance delay period
- Parameters like ALPHA, BETA, minimum stakes, and unbonding periods are adjustable through governance
- Maximum 1,000 active validators per epoch (adjustable)

## ## Next Steps

- [Staking Guide](/developers/staking-guide) - Practical guide for staking operations
- [Unstaking](/understand-genlayer-protocol/core-concepts/optimistic-democracy/unstaking) - Detailed unstaking process

- [Slashing](/understand-genlayer-protocol/core-concepts/optimistic-democracy/slashing) - Slashing conditions and penalties

---

## # Slashing in GenLayer

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/optimistic-democracy/slashing>

Slashing in GenLayer is the validator penalty mechanism that reduces stake for behavior detrimental to the network, such as missing required execution or appeal windows. Slashing helps validators act honestly and effectively, maintains the integrity of the platform and the Intelligent Contracts executed within it, and aligns validator incentives with the network and its users.

### ## Slashing Process

1. **\*\*Violation Detection\*\***: The network identifies a violation, such as missing an execution window.
2. **\*\*Slash Calculation\*\***: The amount to be slashed is calculated based on the specific violation and platform rules.
3. **\*\*Stake Reduction\*\***: The slashed amount is deducted from the validator's stake.
4. **\*\*Finality\*\***: The slashing becomes final after the Finality Window closes, ensuring that the validator's balance is finalized and accounts for any potential appeals.

### ## When Slashing Occurs

Validators in GenLayer can be slashed for several reasons:

1. **\*\*Missing Transaction Execution Window\*\***: Validators are expected to execute transactions within a specified time frame. If a validator misses this window, they are penalized, ensuring that validators remain active and responsive.
2. **\*\*Missing Appeal Execution Window\*\***: During the appeals process, validators must respond within a set time frame. If they fail to do so, they are slashed, which motivates validators to participate in the appeals process.

### ### Amount Slashed

The amount slashed varies based on the severity of the violation and the specific rules set by the GenLayer platform. The slashing amount is designed to be substantial enough to deter malicious or negligent behavior while not being excessively punitive for honest mistakes.

---

## # Unstaking in GenLayer

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/optimistic-democracy/unstaking>

Unstaking in GenLayer is the process by which validators disengage their staked tokens from the network and end active participation as validators. Validators initiate unstaking with a transaction, are removed from the active validator pool, and wait through a cooldown period so all transactions they participated in can reach full finality. After pending obligations and issues are resolved, validators and their delegators can withdraw staked tokens and any accrued rewards while preserving network integrity and platform security.

### ## How Unstaking Works

The unstaking process includes several key steps:

1. **\*\*Initiating Unstaking\*\***: Validators initiate their exit from active duties by submitting an unstaking transaction, signaling their intention to cease participation in validating transactions.

2. **Validator Removal**: Once the unstaking request is made, the validator is promptly removed from the pool of active validators, meaning they will no longer receive new transactions or be called upon for appeal validations.
3. **Finality Period**: During this period, validators must wait for all transactions they have participated in to reach full finality. This is crucial to ensure that validators do not exit while still having potential influence over unresolved transactions. This cooldown period helps prevent the situation where new transactions with new finality windows could prevent them from ever achieving full finality on all transactions they were involved in.
4. **Withdrawing Stake**: After all transactions have achieved finality and no outstanding issues remain, validators and their delegators can safely withdraw their staked tokens and any accrued rewards.

## ## Purpose of Unstaking

The unstaking process is designed to:

- **Ensure Accountability**: By enforcing a Finality Window, validators are held accountable for their actions until all transactions they influenced are fully resolved. This prevents premature exit from the network and ensures that all potential disputes are settled.
- **Align Incentives**: The requirement for validators to wait through the Finality Window aligns their incentives with the long-term security and reliability of the network, promoting responsible participation.
- **Maintain Network Security**: The unstaking process discourages abrupt departures and ensures that validators address any possible security concerns related to their past validations before leaving.

## ## Implications for Validators and Delegators

For validators, this process mandates careful planning regarding their exit strategy from the network, considering the need to wait out the Finality Window. Delegators must also be patient, understanding that their assets will remain locked until their validator has cleared all responsibilities, safeguarding their investments from potential liabilities caused by unresolved validations.

---

## # Rollup Integration

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/rollup-integration>

Rollup Integration is how GenLayer uses Ethereum rollups, such as ZKSync or Polygon CDK, to improve scalability and compatibility with existing Ethereum infrastructure. By moving execution off-chain while anchoring results to Ethereum, this integration helps optimize transaction throughput and reduce fees while maintaining the security guarantees of the Ethereum mainnet.

## ## Key Aspects of Rollup Integration

### ### Scalability

- **High Transaction Throughput**: Rollups allow GenLayer to process a much higher number of transactions per second compared to Layer 1 solutions.
- **Reduced Congestion**: By moving computation off-chain, GenLayer helps alleviate congestion on the Ethereum mainnet.

### ### Cost Efficiency

- **Lower Transaction Fees**: Users benefit from significantly reduced gas fees compared to direct Layer 1 transactions.
- **Batched Submissions**: Transactions are batched and submitted to the Ethereum mainnet, distributing costs across multiple operations.

### ### Security

- **Ethereum Security Inheritance**: While execution happens off-chain, the security of assets and final state is guaranteed by Ethereum's robust consensus mechanism.
- **Fraud Proofs/Validity Proofs**: Depending on the specific rollup solution (Optimistic or ZK), security is ensured through either fraud proofs or validity proofs.

## ## How Rollup Integration Works with GenLayer

1. **Transaction Submission**: Users submit transactions to the rollup.
2. **Transaction Execution**: Transactions are executed within the GenVM environment.
3. **Consensus**: The rollup layer implements the Optimistic Democracy mechanism to reach consensus on the state updates.
4. **State Updates**: The rollup layer maintains an up-to-date state of all accounts and contracts.
5. **Batch Submission**: Periodically, batches of transactions and state updates are submitted to the Ethereum mainnet.
6. **Verification**: The Ethereum network verifies the integrity of the submitted data, ensuring its validity.

## ## Benefits for Developers and Users

- **Ethereum Compatibility**: Developers can leverage existing Ethereum tools and infrastructure.
- **Improved User Experience**: Lower fees and faster transactions lead to a better overall user experience.

## ## Considerations

- **Withdrawal Periods**: Depending on the rollup solution, there might be waiting periods for withdrawing assets back to the Ethereum mainnet.
- **Rollup-Specific Features**: Different rollup solutions may offer unique features or limitations that developers should be aware of.

By integrating with Ethereum rollups, GenLayer combines the innovative capabilities of Intelligent Contracts with the scalability and efficiency of Layer 2 solutions, creating a powerful platform for next-generation decentralized applications.

---

### # Non-Deterministic Operations Handling

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/non-deterministic-operations-handling>

Non-deterministic operations handling in GenLayer is the process of maintaining network consensus when Intelligent Contracts produce variable results from actions such as interacting with Large Language Models (LLMs) or accessing web data. GenLayer addresses this variability through the Equivalence Principle, where validators assess whether different outputs are equivalent based on predefined criteria, and Optimistic Democracy, which supports provisional transaction acceptance with an appeals process.

## ## Overview

GenLayer extends traditional smart contracts by allowing Intelligent Contracts to perform non-deterministic operations, such as interacting with Large Language Models (LLMs) and accessing web data. Handling the variability inherent in these operations is crucial for maintaining consensus across the network.

## ## Challenges

- **Variability of Outputs**: Non-deterministic operations can produce different outputs

when executed by different validators.

- **Consensus Difficulty**: Achieving consensus on varying outputs requires specialized mechanisms.

## ## Solutions in GenLayer

- **Equivalence Principle**: Validators assess whether different outputs are equivalent based on predefined criteria, allowing for consensus despite variability.
- **Optimistic Democracy**: The consensus mechanism accommodates non-deterministic operations by allowing provisional acceptance of transactions and providing an appeals process.

## ## Developer Considerations

- **Defining Equivalence Criteria**: Developers must specify what constitutes equivalent outputs in their Intelligent Contracts.
- **Testing and Validation**: Thorough testing is essential to ensure that non-deterministic operations behave as expected in the consensus process.

---

## # Large Language Model (LLM) Integration

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/large-language-model-llm-integration>

Large Language Model (LLM) integration in GenLayer lets Intelligent Contracts interact directly with LLMs for natural language processing and context-aware decision-making within blockchain applications.

## ## Overview

Intelligent Contracts in GenLayer can interact directly with Large Language Models (LLMs), enabling natural language processing and more complex decision-making capabilities within blockchain applications.

## ## Key Features

- **Natural Language Understanding**: Contracts can process and interpret instructions written in natural language.
- **Dynamic Decision Making**: Utilizing LLMs allows contracts to make context-aware decisions based on complex inputs.

## ## Implementation

1. **LLM Providers**: Validators are configured with LLM providers (e.g., OpenAI, Ollama) to process LLM requests.
2. **Equivalence Principle**: LLM outputs are validated using the Equivalence Principle to ensure consensus among validators.
3. **Prompt Design**: Developers craft prompts to interact effectively with LLMs, specifying expected formats and constraints.

## ## Considerations

- **Cost and Performance**: LLM interactions may incur additional computational costs and latency.
- **Security**: Care must be taken to prevent prompt injections and ensure the reliability of LLM responses.

---

## # Web Data Access in Intelligent Contracts

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/web-data-access>

Web Data Access in Intelligent Contracts is GenLayer's capability for contracts to directly retrieve and use web data without relying on oracles. This enables blockchain applications

to respond to real-world events while validators use equivalence validation to check retrieved data for consistency.

## ## Overview

GenLayer enables Intelligent Contracts to directly access and interact with web data, removing the need for oracles and allowing for real-time data integration into blockchain applications.

## ## Key Features

- **Direct Web Access**: Contracts can retrieve data from web sources.
- **Dynamic Applications**: Access to web data allows for applications that respond to external events and real-world data.
- **Equivalence Validation**: Retrieved data is validated across validators to ensure consistency.

## ## Implementation

1. **Data Retrieval Functions**: GenLayer provides mechanisms for fetching web data within contracts.
2. **Data Parsing and Validation**: Contracts must parse web data and validate it according to defined equivalence criteria.
3. **Security Measures**: Contracts should handle potential security risks such as untrusted data sources and ensure data integrity.

## ## Considerations

- **Network Dependencies**: Reliance on external web sources introduces dependencies that may affect contract execution.
- **Performance Impact**: Web data retrieval may introduce latency and affect transaction processing times.

---

## # Transactions

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/transactions>  
Transactions in GenLayer are the fundamental operations that change network state, including deploying a new contract, sending value between accounts, or invoking a function within an existing contract. A transaction records the call data, sender and recipient addresses, gas limit, nonce, status, consensus data, and execution receipts used by the protocol to process and finalize the operation.

Here is the general structure of a transaction:

```
```json
{
  "consensus_data": {
    "leader_receipt": {
      "args": [
        [
          2,
          "0x793Ae2CfF17462cc9f9D68e194b7b949d2080Ea2"
        ]
      ],
      "class_name": "LlmErc20",
      "contract_state":
        "gASVnAAAAAAAAACMF2JhY2t1bmQubm9kZS5nZW52bS5iYXN1IiwITGxtRXJmJmJCUk5QpgZR9lIwIYmFsYW5jZXOUfZQ
        ojCoweEqYNzFjNzRBNzgwODNGMzU3YTlmOGQzMWQ1YWRDNTlCMzk1Y2YxNmKUS2KMKjB4NzkzQWUyQ2ZGMTc0NjJjYzI
        mOUQ2OGUxOTRiN2I5NDlkMjA4MEVhMjRlAnVzYi4=",
      "eq_outputs": {
        "leader": {
          "0": "{\"transaction_success\": true, \"transaction_error\": \"\",
          \"updated_balances\": {\"0xD271c74A78083F357a9f8d31d5adC59B395cf16b\": 98,
          \"0x793Ae2CfF17462cc9f9D68e194b7b949d2080Ea2\": 2}}"
```



```

    },
    "error": null,
    "execution_result": "SUCCESS",
    "gas_used": 0,
    "method": "transfer",
    "mode": "leader",
    "node_config": {
      "address": "0x185D2108D9dE15ccf6beEb31774CA96a4f19E62B",
      "config": {},
      "model": "gpt-4o",
      "plugin": "openai",
      "plugin_config": {
        "api_key_env_var": "OPENAIKEY",
        "api_url": null
      },
      "provider": "openai",
      "stake": 1
    },
    "vote": "agree"
  },
  "validators": [
    {
      "args": [
        [
          2,
          "0x793Ae2CfF17462cc9f9D68e194b7b949d2080Ea2"
        ]
      ],
      "class_name": "LlmErc20",
      "contract_state":
"ASVnAAAAAAAAACMF2JhY2t1bmQubm9kZS5nZW52bS5iYXN1lIwITGxtRXJmMjCUk5QpgZR9lIwIYmFsYW5jZXOUfZQ
ojCoweEQyNzFjNzRBNzgwODNGMzU3YTlmOGQzMWQ1YWRDNTlCMzk1Y2YxNmKUS2KMKjB4NzgzQWUyQ2ZGMTc0NjJjYz1
mOUQ2OGUxOTRiN2I5NDlkMjA4MEVhMjRlAnVzYi4=",
      "eq_outputs": {
        "leader": {
          "0": "{\"transaction_success\": true, \"transaction_error\": \"\",
\\updated_balances\": {\\\"0xD271c74A78083F357a9f8d31d5adC59B395cf16b\\\": 98,
\\\"0x793Ae2CfF17462cc9f9D68e194b7b949d2080Ea2\\\": 2}}\"
        }
      },
      "error": null,
      "execution_result": "SUCCESS",
      "gas_used": 0,
      "method": "transfer",
      "mode": "validator",
      "node_config": {
        "address": "0x31bc9380eCbF487EF5919eBa7457F457B5196FCD",
        "config": {},
        "model": "gpt-4o",
        "plugin": "openai",
        "plugin_config": {
          "api_key_env_var": "OPENAIKEY",
          "api_url": null
        },
        "provider": "openai",
        "stake": 1
      },
      "pending_transactions": [],
      "vote": "agree"
    },
    ...
  ],
  "votes": {
    "0x185D2108D9dE15ccf6beEb31774CA96a4f19E62B": "agree",

```

```

        "0x2F04Fb1e5daf7DCbf170E4CB0e427d9b11aB96cA": "agree",
        "0x31bc9380eCbF487EF5919eBa7457F457B5196FCD": "agree"
    }
},
"created_at": "2024-10-02T20:32:50.469443+00:00",
"data": {
    "function_args": "[2, \"0x793Ae2CfF17462cc9f9D68e194b7b949d2080Ea2\"]",
    "function_name": "transfer"
},
"from_address": "0xD271c74A78083F357a9f8d31d5adC59B395cf16b",
"gaslimit": 66,
"hash": "0xb7486f70a3fec00af5f929fc1cf1078af9ff3a063afe8b6f370a44a96635505d",
"leader_only": false,
"nonce": 66,
"r": null,
"s": null,
"status": "FINALIZED",
"to_address": "0x5929bB548a2Fd7E9Ea2577DaC9c67A08BbC2F356",
"type": 2,
"v": null,
"value": 0
}
},

```

Explanation of fields:

- consensus_data: Object containing information about the consensus process
- leader_receipt: Object containing details about the leader's execution of the transaction
 - args: Arguments passed to the contract function
 - class_name: Name of the contract class
 - contract_state: Encoded state of the contract
 - eq_outputs: Outputs from every equivalence principle in the execution of the contract method
 - error: Any error that occurred during execution (null if no error)
 - execution_result: Result of the execution (e.g., "SUCCESS" or "ERROR")
 - gas_used: Amount of gas used in the transaction
 - method: Name of the method called on the contract
 - mode: Execution mode (e.g., "leader" or "validator")
 - node_config: Configuration of the node executing the transaction
 - address: Address of the node
 - config: Configuration of the node
 - model: Model of the node
 - plugin: Plugin used for the LLM provider connection
 - plugin_config: Configuration of the plugin
 - api_key_env_var: Environment variable containing the API key for the given provider
 - api_url: API URL for the given provider
 - provider: Provider of the node
 - stake: Stake of the validator
 - vote: The leader's vote on the transaction (e.g., "agree")
 - validators: Array of objects containing similar information for each validator
 - votes: Object mapping validator addresses to their votes
- created_at: Timestamp of when the transaction was created
- data: Object containing details about the function call in the transaction
- from_address: Address of the account initiating the transaction
- gaslimit: Maximum amount of gas the transaction is allowed to consume
- hash: Unique identifier (hash) of the transaction
- leader_only: Boolean indicating whether the transaction is to be executed by the leader node only
- nonce: Number of transactions sent from the from_address (used to prevent double-spending)
- r: Part of the transaction signature (null if not yet signed)
- s: Part of the transaction signature (null if not yet signed)
- status: Current status of the transaction (e.g., "FINALIZED")
- to_address: Address of the contract or account receiving the transaction

- type: Internal type of the transaction (2 indicates a contract write call)
- v: Part of the transaction signature (null if not yet signed)
- value: Amount of native currency (GEN) being transferred in the transaction

Types of Transactions

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/transactions/types-of-transactions>

GenLayer transactions are user-submitted operations that deploy an Intelligent Contract, send native GEN value, or call a function on an existing Intelligent Contract. All three types are sent through the same RPC method, but they differ in the data they contain and the actions they perform.

1. Deploy a Contract

Deploying a contract involves creating a new Intelligent Contract on the GenLayer network. This transaction initializes the contract's state and assigns it a unique address on the blockchain. The deployment process ensures that the contract code is properly validated and stored, making it ready to be called.

Example

```
```json
{
 "consensus_data": {
 "leader_receipt": {
 "args": [
 {
 "total_supply": 100
 }
],
 "class_name": "LlmErc20",
 "contract_state":
"ASVnAAAAAAAAACMF2JhY2t1bmQubm9kZS5nZW52bS5iYXN1IiwITGxtRXJmMjCUk5QpgZR9lIwIYmFsYW5jZXOUfZQ
ojCoweEQYnZfjNzRBNzgwODNGMzU3YTlmOGQzMWQ1YWRDNTlCMzk1Y2YxNmKUS2KMKjB4NzkzQWUyQ2ZGMTc0NjJjYzI
mOUQ2OGUxOTRiN2I5NDlkMjA4MEVhMjRlAnVzYi4=",
 ...
 },
 "validators": [
 ...
],
 ...
 },
 "data": {
 "constructor_args": "{\"total_supply\":100}",
 "contract_address": "0x5929bB548a2Fd7E9Ea2577DaC9c67A08BbC2F356",
 "contract_code": "import json\nfrom backend.node.genvm.icontract import IContract\nfrom
backend.node.genvm.equivalence_principle import EquivalencePrinciple\n\nclass
LlmErc20(IContract):\n def __init__(self, total_supply: int) -> None:\n
self.balances = {}\n self.balances[contract_runner.from_address] =
total_supply\n...",
 },
 ...
 }
}
```
```

2. Send Value

Sending value refers to transferring the native GEN token from one account to another. This is one of the most common types of transactions. Each transfer updates the balance of the involved accounts, and the transaction is recorded on the blockchain to ensure transparency and security.

Example

```
```json
{
 "consensus_data": null,
 "created_at": "2024-10-02T21:21:04.192995+00:00",

```

```

"data": {},
"from_address": "0x0Bd6441CB92a64fA667254BCa1e102468fffB3f3",
"gaslimit": 0,
"hash": "0x6357ec1e86f003b20964ef3b2e9e072c7c9521f92989b08e04459b871b69de89",
"leader_only": false,
"nonce": 2,
"r": null,
"s": null,
"status": "FINALIZED",
"to_address": "0xf739FDe22E0C0CB6DFD8f3F8D170bFC07329489E",
"type": 0,
"v": null,
"value": 200
}
...

```

### ## 3. Call Contract Function

Calling a contract function is the process of invoking a specific method within an existing Intelligent Contract. This could involve anything from querying data stored within the contract to executing more complex operations like transferring tokens or interacting with other contracts. Each function call is a transaction that modifies the contract's state based on the inputs provided.

#### ### Example

```
```json
```

```

{
  "consensus_data": {
    "leader_receipt": {
      "args": [
        [
          2,
          "0x793Ae2CfF17462cc9f9D68e194b7b949d2080Ea2"
        ]
      ],
      "class_name": "LlmErc20",
      "contract_state":
"ASVnAAAAAAAAACMF2JhY2t1bmQubm9kZS5nZW52bS5iYXN1IiwITGxtRXJmMjCk5QpgZR9lIwIYmFsYW5jZXOUfZQ
ojCoweEQyNzFjNzRBNzgwODNGMzU3YTlmOGQzMWQ1YWRDNTlCMzk1Y2YxNmKUS2KMKjB4NzkzQWUyQ2ZGMTcONjJjYzI
mOUQ2OGUxOTRiN2I5NDlkMjA4MEVhMjRlAnVzYi4=",
      "eq_outputs": {
        "leader": {
          "0": "{\"transaction_success\": true, \"transaction_error\": \"\",
\\\"updated_balances\\\": {\\\"0xD271c74A78083F357a9f8d31d5adC59B395cf16b\\\": 98,
\\\"0x793Ae2CfF17462cc9f9D68e194b7b949d2080Ea2\\\": 2}}\"
        }
      },
      ...
    },
    "validators": [
      ...
    ],
    ...
  },
  "data": {
    "function_args": "[2,\\\"0x793Ae2CfF17462cc9f9D68e194b7b949d2080Ea2\\\"]",
    "function_name": "transfer"
  },
  ...
}
...

```

* For a list of all the fields in a transaction, see [\[here\]\(/core-concepts/transactions\)](#)

Transaction Processing

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/transactions/transaction-statuses>

GenLayer transaction processing is the account-based queue flow that moves each transaction through network statuses from submission to execution outcome. Transactions are queued per account to preserve the order in which they were submitted, then transition through pending, proposing, committing, revealing, accepted, finalized, undetermined, or canceled states.

1. Pending

When a transaction is first submitted, it enters the pending state. This means it has been received by the network but is waiting to be processed. Transactions are queued per account, ensuring that each account's transactions are processed in the order they were submitted.

2. Proposing

In this stage, the transaction is moved from the pending queue to the proposing stage. A leader and a set of voters are selected from the validator set via a weighted random selection based on total stake. The leader proposes a receipt for the transaction, which is then committed to by the validators.

3. Committing

The transaction enters the committing stage, where validators commit their votes and cost estimates for processing the transaction. This stage is crucial for reaching consensus on the transaction's execution.

4. Revealing

After the committing stage, validators reveal their votes and cost estimates, allowing the network to finalize the transaction's execution cost and validate the consensus.

5. Accepted

Once the majority of validators agree on the transaction's validity and cost, the transaction is marked as accepted. This status indicates that the transaction has passed through the initial validation process successfully.

6. Finalized

After all validations are completed and any potential appeals have been resolved, the transaction is finalized. In this state, the transaction is considered irreversible and is permanently recorded in the blockchain.

7. Undetermined

If the transaction fails to reach consensus after all voting rounds, it enters the undetermined state. This status indicates that the transaction's outcome is unresolved, and it may require further validation or be subject to an appeal process.

8. Canceled

A transaction can be canceled by the user or by the system if it fails to meet certain criteria (e.g., insufficient funds). Once canceled, the transaction is removed from the processing queue and will not be executed.

Transaction execution

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/transactions/transaction-execution>

Transaction execution in GenLayer is the process that starts after ``eth_sendRawTransaction`` receives and verifies a transaction, stores it with a PENDING status, and returns its hash as the RPC response. At that point, the transaction has been validated for authenticity and format, but it has not yet been executed; it enters the GenLayer consensus mechanism, where the network's validators pick it up for execution according to the consensus rules.

As the transaction progresses through stages such as proposing, committing, and revealing, its status is updated accordingly. Users can query the current status and output of the transaction by calling the ``eth_getTransactionByHash`` method on the RPC server, which retrieves the transaction's details based on its unique hash. This method lets users track the transaction's journey from submission to finalization, providing transparency and helping them monitor transaction outcomes in real time.

Transaction Status Transitions

In the journey of a transaction within the GenLayer protocol, it begins its life when an Externally Owned Account (EOA) submits it, entering the `Pending` state. Here, it awaits further processing unless it encounters an `OutOfFee` state due to insufficient fees. If the fees are topped up, it returns to `Pending`. Alternatively, the user can cancel the transaction, moving it to the `Canceled` state.

From `Pending`, the transaction progresses to the `Proposing` stage, where a leader is selected to propose a receipt. Upon successful proposal, it advances to the `Committing` stage, where all validators must commit to the transaction. If all validators commit, the transaction moves to the `Revealing` stage.

In the `Revealing` stage, the transaction's fate is determined. If a majority agrees, it is `Accepted`. However, if there is no majority agreement, it returns to `Proposing`. If all leaders are rotated without agreement, it becomes `Undetermined`. In cases of disagreement, a successful appeal can revert it to `Pending`, while a failed appeal results in `Accepted`.

Once `Accepted`, the transaction awaits the passing of the appeal window to become `Finalized`. An `Undetermined` transaction also becomes `Finalized` after the appeal window passes. However, an appeal can initiate a return to `Committing` from `Accepted`, or automatically revert an `Undetermined` transaction to `Pending`.

```
```mermaid
graph TD;
 Start(()) -->|EOA submits| Pending(Pending)
 Start -->|Insufficient fee| OutOfFee(OutOfFee)
 OutOfFee -->|Fee topped up| Pending
 Pending -->|User cancels| Canceled(Canceled)

 Pending -->|Select leader| Proposing(Proposing)
 Proposing -->|Leader proposes receipt| Committing(Committing)
 Committing -->|All validators commit| Revealing(Revealing)

 Revealing -->|Majority agrees| Accepted(Accepted)
 Revealing -->|No majority agreement| Proposing
 Revealing -->|All leaders rotated| Undetermined(Undetermined)
 Revealing -->|Disagreement, appeal successful| Pending
 Revealing -->|Agreement, appeal failed| Accepted

 Accepted -->|Appeal window passes| Finalized(Finalized)
 Undetermined -->|Appeal window passes| Finalized
 Accepted -->|Appeal initiated| Committing
 Undetermined -->|Appeal automatically| Pending
 ...
```

---

## # Transaction encoding, serialization, and signing

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/transactions/transaction-encoding-serialization-and-signing>

Transaction encoding, serialization, and signing in GenLayer is the client-side process of packaging all three transaction types into a reliable, cross-platform, efficient format and signing them with the sender's private key to verify the sender's identity.

Once prepared, the transaction is sent to the network via the `eth\_sendRawTransaction` method on the RPC Server. This method performs the inverse process: it decodes and deserializes the transaction data, and then verifies the signature to ensure its authenticity. By handling all transaction types through `eth\_sendRawTransaction`, GenLayer ensures that transactions are processed securely and efficiently while maintaining compatibility with Ethereum's specification.

---

## # Economic Model

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/economic-model>

GenLayer's Economic Model defines how staking, rewards, transaction fees, and slashing incentivize participants to maintain the network's security and functionality. Validators stake tokens to participate in validation, earn rewards for correctly validating transactions, receive fees from transaction processing, and risk penalties if they act maliciously or incompetently.

### ## Overview

GenLayer's economic model is designed to incentivize participants to maintain the network's security and functionality. It involves staking, rewards, transaction fees, and penalties.

### ## Key Components

- **Staking**: Validators must stake tokens to participate in the validation process, aligning their interests with the network's health.
- **Rewards**: Validators receive rewards for correctly validating transactions.
- **Transaction Fees**: Users pay fees for transaction processing, which are partly used to reward validators.
- **Slashing**: Validators acting maliciously or incompetently can have their staked tokens slashed as a penalty.

### ## Incentive Mechanisms

- **Positive Incentives**: Rewards and fees motivate validators to act in the network's best interest.
- **Negative Incentives**: Slashing and penalties deter malicious behavior.

### ## Economic Security

- **Stake-Based Security**: The amount staked by validators serves as a deterrent against attacks, as they risk losing their stake.
- **Balancing Supply and Demand**: The economic model aims to balance the supply of validation services with demand from users.

---

## # Accounts and Addressing

Source: <https://docs.genlayer.com/understand-genlayer-protocol/core-concepts/accounts-and-addresses>

Accounts in GenLayer are entities that can hold tokens, deploy Intelligent Contracts, and initiate transactions on the network. GenLayer supports Externally Owned Accounts controlled by private keys and Contract Accounts associated with deployed Intelligent Contracts, each using addresses typically represented as `0x`-prefixed hexadecimal strings.

### ## Overview

Accounts are fundamental to interacting with the GenLayer network. They represent users or entities that can hold tokens, deploy Intelligent Contracts, and initiate transactions.

### ## Types of Accounts

1. **Externally Owned Accounts (EOAs)**:
  - Controlled by private keys
  - Can initiate transactions and hold tokens
2. **Contract Accounts**:
  - Associated with deployed Intelligent Contracts
  - Have their own addresses and code

### ## Account Addresses

- **\*\*Address Format\*\***: GenLayer uses a specific address format, typically represented as a hexadecimal string prefixed with `0x`.
- **\*\*Public and Private Keys\*\***: Addresses are derived from public keys, which in turn are generated from private keys kept securely by the account owner.

## ## Interacting with Intelligent Contracts

- **\*\*Transaction Sending\*\***: Accounts initiate transactions to call functions on Intelligent Contracts or transfer tokens.
- **\*\*Gas Fees\*\***: Transactions require gas fees to be processed.

## ## Account Management

- **\*\*Creating Accounts\*\***: Users can create new accounts using wallets or development tools provided by GenLayer.
- **\*\*Security Practices\*\***: Users must securely manage their private keys, as losing them can result in loss of access to their funds.

---

## # Getting Started with GenLayer

Source: <https://docs.genlayer.com/developers>

This guide helps developers build, deploy, and integrate Intelligent Contracts on GenLayer.

## ## Start with Skills (Recommended)

The fastest path from zero to deployed contract is the [GenLayer Skills] (<https://skills.genlayer.com/>) plugin for Claude Code. The `genlayer-dev` skill scaffolds a project, runs the linter and direct-mode tests, deploys to testnet, and walks you through the SDKs interactively.

```
```bash copy
claude /plugin marketplace add genlayerlabs/skills
claude /plugin install genlayer-dev@genlayerlabs
```
```

Then run `claude /genlayer-dev` (or just ask Claude what to do next). The skill bundles the same workflows documented on this site – it just runs them for you.

> **\*\*Note:\*\***

> Prefer to set things up manually or read the references first? The cards below cover every topic in depth, and the [Tooling Setup](/developers/intelligent-contracts/tooling-setup) page describes the manual install path.

## ## Reference

### **\*\*Intelligent Contracts:\*\***

- [ Introduction](/developers/intelligent-contracts/introduction)
- [ Tooling Setup](/developers/intelligent-contracts/tooling-setup)
- [ First Contract](/developers/intelligent-contracts/your-first-contract)
- [ Types](/developers/intelligent-contracts/types)
- [ Storage](/developers/intelligent-contracts/storage)
- [ First Intelligent Contract](/developers/intelligent-contracts/your-first-intelligent-contract)
- [ Equivalence Principle](/developers/intelligent-contracts/equivalence-principle)
- [ Debugging](/developers/intelligent-contracts/debugging)
- [ Error Handling](/developers/intelligent-contracts/error-handling)
- [ Crafting Prompts](/developers/intelligent-contracts/crafting-prompts)
- [ Advanced Features](/developers/intelligent-contracts/advanced-features/contract-to-contract-interaction)
- [ Security and Best Practices](/developers/intelligent-contracts/security-and-best-practices/prompt-injection)



- [ Examples](/developers/intelligent-contracts/examples/storage)
- [ Tools](/developers/intelligent-contracts/tools/genlayer-cli)
- [ Ideas](/developers/intelligent-contracts/ideas)

## **\*\*Decentralized Applications:\*\***

- [ Architecture](/developers/decentralized-applications/architecture-overview)
- [ DApp Development](/developers/decentralized-applications/dapp-development-workflow)
- [ GenLayer JS](/developers/decentralized-applications/genlayer-js)
- [ Reading Data](/developers/decentralized-applications/reading-data)
- [ Writing Data](/developers/decentralized-applications/writing-data)
- [ Querying](/developers/decentralized-applications/querying-a-transaction)
- [ Testing](/developers/decentralized-applications/testing)
- [ Project Boilerplate](/developers/decentralized-applications/project-boilerplate)

---

## **# Networks**

Source: <https://docs.genlayer.com/developers/networks>

GenLayer operates as two layers: the **\*\*GenLayer RPC\*\*** handles intelligent contract operations (``gen_*` methods), while the **\*\*GenLayer Chain\*\*** is the underlying L2 (zkSync Elastic Chain) that handles standard Ethereum operations (``eth_*` methods). The GenLayer RPC also passes through all ``eth_*` and ``zks_*` calls to the underlying chain, so you can use either endpoint for standard wallet operations.

For the full RPC reference, see [GenLayer Node API](/api-references/genlayer-node).

Your wallet connects to the **\*\*GenLayer RPC\*\*** – it handles both intelligent contract calls and standard Ethereum methods.

---

## **## Testnet Bradbury**

Production-like testnet with real AI/LLM workloads.

| Setting                       | Value                                                                               |
|-------------------------------|-------------------------------------------------------------------------------------|
| <b>**GenLayer RPC**</b>       | <code>`https://rpc-bradbury.genlayer.com`</code>                                    |
| <b>**GenLayer Chain RPC**</b> | <code>`https://rpc.testnet-chain.genlayer.com`</code>                               |
| <b>**Chain ID**</b>           | 4221                                                                                |
| <b>**Currency**</b>           | GEN                                                                                 |
| <b>**Explorer**</b>           | [explorer-bradbury.genlayer.com](https://explorer-bradbury.genlayer.com)            |
| <b>**Chain Explorer**</b>     | [explorer.testnet-chain.genlayer.com](https://explorer.testnet-chain.genlayer.com/) |
| <b>**Faucet**</b>             | [testnet-faucet.genlayer.foundation](https://testnet-faucet.genlayer.foundation)    |

---

## **## Testnet Asimov**

Infrastructure and stress testing.

| Setting                       | Value                                                                               |
|-------------------------------|-------------------------------------------------------------------------------------|
| <b>**GenLayer RPC**</b>       | <code>`https://rpc-asimov.genlayer.com`</code>                                      |
| <b>**GenLayer Chain RPC**</b> | <code>`https://rpc.testnet-chain.genlayer.com`</code>                               |
| <b>**Chain ID**</b>           | 4221                                                                                |
| <b>**Currency**</b>           | GEN                                                                                 |
| <b>**Explorer**</b>           | [explorer-asimov.genlayer.com](https://explorer-asimov.genlayer.com)                |
| <b>**Chain Explorer**</b>     | [explorer.testnet-chain.genlayer.com](https://explorer.testnet-chain.genlayer.com/) |
| <b>**Faucet**</b>             | [testnet-faucet.genlayer.foundation](https://testnet-faucet.genlayer.foundation)    |

```
faucet.genlayer.foundation) |
```

```

```

## ## Studionet

Hosted development environment – no local setup required.

```
Setting	Value
GenLayer RPC	`https://studio.genlayer.com/api`
Chain ID	61999
Currency	GEN
Explorer	explorer-studio.genlayer.com
Faucet	Built-in – use the 💧 button in the account selector
```

```

```

## ## Localnet

Local development with full control. Requires [GenLayer Studio](/developers/intelligent-contracts/tools/genlayer-studio) or [GLSim](/api-references/genlayer-test/glsim).

```
Setting	Value
GenLayer RPC	`http://localhost:4000/api`
Chain ID	61127
Currency	GEN
Explorer	Bundled with Studio at `http://localhost:8080`
Faucet	Built-in – use the 💧 button in the account selector
```

```

```

## ## GenLayer Chain (L2)

The underlying zkSync Elastic Chain that GenLayer runs on. You typically don't need to interact with this directly – the GenLayer RPC passes through all `eth\_\*` calls. But if you need direct L2 access (e.g., token transfers, contract debugging at the EVM level), you can add this chain to your wallet.

```
Setting	Value
RPC	`https://rpc.testnet-chain.genlayer.com`
Chain ID	4221
Currency	GEN
Explorer	explorer.testnet-chain.genlayer.com
```

```

```

## ## Network Comparison

```
	Bradbury	Asimov	Studionet	Localnet
Purpose	Production-like testing	Infrastructure testing	Hosted dev	Local dev
Setup	None – connect and go	None – connect and go	None – browser only	Docker or GLSim
Persistence	Persistent	Persistent	Temporary	Local only
LLM execution	Real models	Real models	Real models	Configurable
Faucet	[Available](https://testnet-faucet.genlayer.foundation)	[Available](https://testnet-faucet.genlayer.foundation)	Built-in (💧 button)	Built-in (💧 button)
```

## ## Recommended Flow

1. **Start on Studionet** – zero setup, open [studio.genlayer.com]

(<https://studio.genlayer.com>)

2. **\*\*Move to Localnet\*\*** – when you need full control and fast iteration

3. **\*\*Deploy to Bradbury\*\*** – when ready for production-like testing with real AI workloads

---

# Introduction to Intelligent Contracts

Source: <https://docs.genlayer.com/developers/intelligent-contracts/introduction>

## What are Intelligent Contracts?

Intelligent Contracts are an advanced evolution of smart contracts that combine traditional blockchain capabilities with natural language processing and web connectivity. Built in Python, they enable developers to create contracts that can understand human language, process external data, and make complex decisions based on real-world information.

## Key Features of Intelligent Contracts

### Natural Language Processing (NLP)

Intelligent Contracts leverage Large Language Models (LLMs) to process and understand human language inputs. This integration enables the contracts to interpret complex text-based instructions and requirements, moving beyond simple conditional logic.

Through NLP capabilities, these contracts can analyze qualitative criteria and make nuanced decisions based on contextual understanding, bringing a new level of intelligence to contract execution.

### Web Connectivity

These contracts can actively interact with web APIs to fetch real-time information, enabling dynamic decision-making based on current data. By incorporating external services for data verification, they maintain a reliable connection to real-world events and conditions, bridging the gap between on-chain and off-chain environments.

### Non-Deterministic Operations

Intelligent Contracts introduce a sophisticated approach to handling operations with unpredictable outputs, a significant advancement over traditional deterministic smart contracts. Through the implementation of a built-in equivalence principle, multiple validators can reach consensus even when processing non-deterministic results. This system supports both comparative validation, where outputs are directly matched, and non-comparative validation, where validators assess the reasonableness of results within defined parameters.

## How Do Intelligent Contracts Work?

### Contract Structure

Intelligent Contracts are written in Python using the GenVM SDK library. The basic structure consists of:

1. Dependencies Declaration: Specify required GenVM SDK modules
2. Declare Contract: Extend `gl.Contract` to define a contract class
3. State Variables: Declare with type annotations for strong typing
4. Methods:
  - `@gl.public.view`: Read-only methods that don't modify state
  - `@gl.public.write`: Methods that can modify contract state
  - `@gl.public.write.payable`: Methods that can modify contract state *and* receive `value`

Here's an example:

```
```py
# { "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
from genlayer import *

class MyContract(gl.Contract):
    # State variables with type annotations
    variable: str
```

```

def __init__(self):
    self.variable = "initial value"

@gl.public.view
def read_method(self) -> str:
    return self.variable

@gl.public.write
def write_method(self, new_value: str):
    self.variable = new_value
...

```

Validation Process

- When transactions are submitted to Intelligent Contracts, they are automatically queued in a contract-specific order and marked with a "pending" status
- A randomly selected group of validators is assigned to process the transaction, with one designated as the leader to propose the outcome
- Once all validators evaluate the proposal and reach consensus using the equivalence principle, the transaction is accepted and enters the Finality Window

[Learn more about the validation process](/about-genlayer/core-concepts/optimistic-democracy)

Advantages over Traditional Smart Contracts

Enhanced Decision Making

Intelligent Contracts can process complex and qualitative criteria that traditional smart contracts cannot handle. Through their natural language understanding capabilities, they can interpret and act on human-readable inputs without requiring strict formatting or coding syntax.

This flexibility allows the contracts to dynamically adapt to changing conditions, making them more responsive and intelligent in their decision-making processes.

External Data Integration

Intelligent Contracts can seamlessly integrate with external data sources, providing direct access to real-world information without intermediate layers. Their real-time data processing capabilities ensure that contract decisions are based on current and accurate information.

This direct connectivity significantly reduces the traditional reliance on oracle services, making the contracts more efficient and cost-effective.

Flexible Programming

Development of Intelligent Contracts leverages Python's robust ecosystem, providing developers with a familiar and powerful programming environment.

The platform supports the data structures needed to handle complex business logic and requirements.

Trade-offs

Intelligent Contracts introduce non-determinism and external dependencies that traditional smart contracts avoid. Key considerations:

- **Non-deterministic outputs** – LLM responses and web data vary across validators. The [Equivalence Principle](/developers/intelligent-contracts/features/non-determinism) lets you define how validators compare results to reach consensus.
- **External data reliability** – Web sources can be inconsistent or unavailable. Multiple validators independently fetch and cross-validate data as part of [Optimistic Democracy](/understand-genlayer-protocol/optimistic-democracy-how-genlayer-works).
- **Compute cost** – LLM calls and web requests add latency and cost compared to purely deterministic contracts. Design contracts to minimize non-deterministic calls where possible.

When to Use GenLayer

Source: <https://docs.genlayer.com/developers/intelligent-contracts/when-to-use-genlayer>

GenLayer is for applications that need a **shared, on-chain decision** about something a normal deterministic smart contract cannot evaluate by itself.

Use GenLayer when the decision depends on evidence, language, or judgment, and when different parties need to trust the result without relying on a single backend, oracle, or operator.

Quick Fit Checklist

A feature is usually a good fit for GenLayer if most of these are true:

- **There is a real on-chain consequence**: payout, escrow release, market resolution, reputation update, rule enforcement, or another state change.
- **The outcome requires judgment**: interpreting evidence, applying natural-language rules, evaluating quality, or resolving ambiguity.
- **The evidence can be independently checked**: validators can fetch the same source material or enough comparable evidence to verify the leader's result.
- **The decision benefits from neutral consensus**: counterparties should not have to trust your centralized backend or one AI provider.
- **The result can be made explicit**: your contract can return a structured decision such as `accepted`, `winner`, `score`, `amount`, or `reason`.

If your feature is mostly personalization, summarization, chat, analytics, routing, or UX assistance with no consensus-critical state change, it is usually better handled by a normal backend or frontend LLM.

Architecture Pattern

A common GenLayer app has three layers:

1. **Frontend or backend**: collects user intent, displays data, handles indexing, and may prepare URLs or evidence.
2. **Intelligent Contract**: owns the consensus-critical decision and any on-chain state transition.
3. **GenLayer validators**: independently verify the leader's result using the evidence and equivalence principle you define.

The key question is:

> What decision must not depend on my server alone?

That decision is the part that belongs in the Intelligent Contract.

Good Fits

Prediction or resolution markets

Good fit when the contract resolves a market by checking source evidence and applying listing or resolution rules.

- Frontend: shows the market, lets users trade, links evidence.
- Intelligent Contract: determines the outcome from trusted sources and settles the market.
- Validators: verify that the outcome follows the evidence and rules.

Dispute or milestone workflows

Good fit when parties agree up front that a contract should evaluate whether a deliverable, task, or condition was satisfied.

- Frontend/backend: stores submissions, attachments, and communication context.

- Intelligent Contract: evaluates the evidence against agreed criteria and releases funds or updates status.
- Validators: verify that the accepted result is reasonable given the evidence.

Rule or policy verification

Good fit when a decision depends on natural-language criteria.

Examples:

- Does a proposal comply with a DAO constitution?
- Does a submitted work item meet bounty rules?
- Does a listing meet market guidelines?
- Does a claim satisfy policy conditions?

Weak Fits and Anti-Patterns

Frontend computes the answer, GenLayer only stores it

If your frontend fetches data, asks an LLM for the answer, and sends the finished result to GenLayer for storage, GenLayer is not adding much trust.

Better pattern: send the contract stable evidence references or inputs, then let the Intelligent Contract perform the consensus-critical evaluation.

Generic AI brain

If GenLayer is only being used as an AI assistant, recommender, chatbot, or analytics engine, a normal backend is usually simpler and cheaper.

Better pattern: use off-chain AI for UX and exploration, then use GenLayer only for the part where users need neutral settlement or verifiable judgment.

Private data that validators cannot verify

If validators cannot access enough evidence to independently check the leader result, consensus becomes weak or impossible.

Better pattern: commit to public evidence, signed attestations, hashes, URLs, or other verifiable inputs that validators can inspect.

Unbounded subjective output

If the contract asks for open-ended text with no structured decision fields, validators may disagree even when they are all reasonable.

Better pattern: return structured fields and compare only the parts that matter. See the [Equivalence Principle](/developers/intelligent-contracts/equivalence-principle).

What Belongs Where

- **Frontend**: UX, forms, wallets, previews, charts, indexing, user guidance.
- **Backend**: caching, notifications, search, off-chain analytics, non-critical LLM assistance.
- **Intelligent Contract**: the decision that moves money, resolves a market, updates reputation, enforces rules, or creates another shared state transition.
- **Evidence sources**: stable URLs, APIs, signed data, user submissions, or other material validators can fetch or evaluate.

Dispute Resolution Is Not a Court

GenLayer can support agreed dispute-resolution workflows, but it does not automatically make a decision legally binding or replace a court.

Use safer framing such as:

- evidence-based settlement workflow
- agreed arbitration primitive
- contractual dispute-resolution mechanism
- milestone or deliverable adjudication

Avoid implying that an Intelligent Contract is a legal judge unless your application has the legal agreements, jurisdiction, and review process to support that claim.

Design Prompt

Before building, answer these questions:

1. What exact decision should GenLayer make?
2. What state changes if the decision is accepted?
3. What evidence will validators use to verify it?
4. Which fields must validators agree on exactly, and which can vary?
5. What happens if validators reject the leader result or the transaction becomes undetermined?

If the answers are clear, the feature is probably a good candidate for GenLayer. If not, start by moving more logic off-chain or narrowing the on-chain decision.

Intelligent Contract Features

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features>

Intelligent Contract Features catalogs the deterministic and non-deterministic capabilities available when building GenLayer Intelligent Contracts, from storage, messages, value transfers, and contract interactions to LLM calls, image processing, and web access.

Deterministic Features

- [ Types](types)
- [ Storage](features/storage)
- [ Error Handling](features/error-handling)
- [ Upgradability](features/upgradability)
- [ Value Transfers](features/value-transfers)
- [ Messages](features/messages)
- [ Interacting with Intelligent Contracts](features/interacting-with-intelligent-contracts)
- [ Interacting with EVM Contracts](features/interacting-with-evm-contracts)
- [ Special Methods](features/special-methods)
- [ Vector Storage](features/vector-storage)
- [ Debugging](features/debugging)
- [ Random](features/random)

Non-Deterministic Features

- [ Non-determinism](features/non-determinism)
- [ Calling LLMs](features/calling-llms)
- [ Image Processing](features/image-processing)
- [ Web Access](features/web-access)

Storage Quick Reference

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/storage>

Basic Types

Store persistent data using class attributes:

```
```python
```

```

class Contract(gl.Contract):
 counter: u32
 name: str
 active: bool

 @gl.public.write
 def set_data(self, count: u32, new_name: str):
 self.counter = count
 self.name = new_name
...

```

## ## DynArray

Use `DynArray[T]` instead of `list[T]` for persistent arrays:

```

```python
class Contract(gl.Contract):
    items: DynArray[str]
    scores: DynArray[u32]

    @gl.public.write
    def add_item(self, item: str):
        self.items.append(item)

    @gl.public.view
    def get_all_items(self):
        return [item for item in self.items]
...

```

TreeMap

Use `TreeMap[K, V]` instead of `dict[K, V]` for persistent mappings:

```

```python
class Contract(gl.Contract):
 balances: TreeMap[str, u32]

 @gl.public.write
 def update_balance(self, user: str, amount: u32):
 self.balances[user] = amount

 @gl.public.view
 def get_balance(self, user: str):
 return self.balances.get(user, u32(0))
...

```

## ## Storage Classes

Create custom storage types with `@allow\_storage`:

```

```python
@allow_storage
@dataclass
class UserData:
    scores: DynArray[u32]
    username: str

class Contract(gl.Contract):
    users: DynArray[UserData]

    @gl.public.write
    def add_user(self, name: str):
        user = UserData(scores=DynArray[u32](), username=name)
        self.users.append(user)
...

```


Memory Operations

Copy storage objects to memory for non-deterministic operations:

```
```python
@gl.public.write
def process_user(self):
 storage_user = self.users[0]
 memory_user = gl.storage.copy_to_memory(storage_user)

 def nondet_operation():
 return f"User: {memory_user.username}"

... result = gl.eq_principle.strict_eq(nondet_operation)
```

> **Warning:**

> In future reading from storage directly in non-deterministic blocks will be supported.

## ## Type Restrictions

- Use ``DynArray[T]`` instead of ``list[T]``
- Use ``TreeMap[K, V]`` instead of ``dict[K, V]``
- Use sized integers (``u32``, ``i64``) instead of ``int``
- Use ``bigint`` only for arbitrary precision needs
- All generic types must be fully specified

## ## Default Values

Storage is zero-initialized:

Type	Default value
<code>`u*`, `i*`</code>	<code>`0`</code>
<code>`bigint`</code>	<code>`0`</code>
<code>`bool`</code>	<code>`False`</code>
<code>`float`</code>	<code>`+0.0`</code>
<code>`str`</code>	<code>`""`</code>
<code>`bytes`</code>	<code>`b""`</code>
<code>`Address`</code>	<code>`0x0...`</code>
<code>`DynArray`</code>	<code>`[]`</code>
<code>`TreeMap`</code>	<code>`{}`</code>

---

## # Error Handling

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/error-handling>

Error Handling in GenVM defines how Intelligent Contracts terminate, report user-generated failures, surface VM-generated failures, propagate errors from non-deterministic code, and classify failures for consensus. GenVM execution can end with unrecoverable exit codes or unhandled exceptions, produce ``UserError`` and ``VMError`` results, and let contracts catch user errors from sub-VMs when using the Equivalence Principle.

## ## Unrecoverable Errors

Exit codes terminate execution immediately:

```
```python
if invalid_condition:
    exit(1)
...`
```

Unhandled exceptions also become unrecoverable:

```
```python
raise Exception("Critical error") # Becomes exit(1)
```
```

UserError

User-generated errors with UTF-8 encoded messages:

```
```python
Can be caught in current sub-vm
raise gl.vm.UserError("Invalid input")

Immediate user error, more efficient but can't be caught
gl.advanced.user_error_immediate("Insufficient funds")
```
```

VMErrors

VM-generated errors with predefined string codes:

```
```python
Non-zero exit codes become VMErrors
exit(1) # Results in VMErrors with specific code

Resource limit violations also trigger VMErrors
(handled automatically by the VM)
```
```

Catching UserError

Handle user errors from sub-VMs:

```
```python
def risky_operation():
 raise gl.vm.UserError("Operation failed")

try:
 result = gl.eq_principle.strict_eq(risky_operation)
except gl.vm.UserError as e:
 print(f"Caught user error: {e.message}")
```
```

Error Propagation

Errors flow from non-deterministic to deterministic code:

```
```python
def nondet_block():
 if some_condition:
 raise gl.vm.UserError("INVALID_STATE")
 return "success"

try:
 gl.eq_principle.strict_eq(nondet_block)
except gl.vm.UserError as e:
 if e.message == "INVALID_STATE":
 # Handle specific error condition
 pass
```
```

VM Result Types

GenVM produces four result types:

- ****Return**** - Successful execution with encoded result

- **VMError** - VM errors (exit codes, resource limits)
- **UserError** - User-generated errors with UTF-8 messages
- **InternalError** - Critical VM failures (not visible to contracts)

Error Patterns for Consensus

When using ``run_nondet_unsafe``, errors affect how validators compare results. If the leader errors, the validator needs to decide: do I agree or disagree?

One approach is to classify errors by their nature, so validators can handle each type appropriately:

- **Deterministic errors** (business logic, client errors): should match exactly between leader and validator
- **Transient errors** (network issues, server errors): both hitting a transient failure is expected
- **LLM errors** (malformed output): disagreeing forces a rotation to a new leader, which is usually what you want

Example: Error Classification

```
```python
def validator_fn(leaders_res: gl.vm.Result) -> bool:
 if not isinstance(leaders_res, gl.vm.Return):
 # Leader errored – run the same logic to see if we error too
 leader_msg = leaders_res.message if hasattr(leaders_res, 'message') else ''
 try:
 leader_fn()
 return False # Leader errored but we succeeded – disagree
 except gl.UserError as e:
 validator_msg = str(e)
 # Both hit the same business logic error – agree
 if validator_msg == leader_msg:
 return True
 return False
 except Exception:
 return False

 # Leader succeeded – validate the result
 validator_result = leader_fn()
 return compare_results(leaders_res.calldata, validator_result)
```
```

This pattern ensures that consensus failures on broken LLM output or transient network issues lead to retries rather than locking in bad state.

Upgradability

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/upgradability>

Upgradability in GenVM is a native system for modifying an Intelligent Contract after deployment while preserving security guarantees and clear access controls.

The system is built around the **Root Slot** (``gl.storage.Root``), which stores:

| Field | Description |
|-----------------------------|--|
| <code>`code`</code> | The contract's source code |
| <code>`locked_slots`</code> | Storage slots that non-upgraders cannot write to |
| <code>`upgraders`</code> | Addresses authorized to modify locked slots (including code) |

How It Works

1. At the start of a write transaction, GenVM reads the ``upgraders`` list

2. If the sender **is** in the ``upgraders`` list, no slot restrictions apply – the sender can modify any slot, including ``code``
3. If the sender is **not** in the ``upgraders`` list, GenVM reads ``locked_slots`` and prevents writes to them

During deployment (``__init__``), after the constructor completes, the runtime automatically calls ``root.lock_default()``, which locks four critical slots: the root slot, the code slot, the ``locked_slots`` slot, and the ``upgraders`` slot.

Making a Contract Upgradable

To make a contract upgradable, you need to:

1. Add authorized upgrader addresses in ``__init__``
2. Expose a method that replaces the contract code

```
```python
v0.1.0
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
```

```
from genlayer import *
```

```
class UpgradableStorage(gl.Contract):
 storage: str

 def __init__(self, initial_storage: str):
 self.storage = initial_storage

 # Set the deployer as an upgrader
 root = gl.storage.Root.get()
 root.upgraders.get().append(gl.message.sender_address)

 # lock_default() is called automatically after __init__
 # it locks: root slot, code, locked_slots, upgraders

 @gl.public.view
 def get_storage(self) -> str:
 return self.storage

 @gl.public.write
 def update_storage(self, new_storage: str) -> None:
 self.storage = new_storage

 @gl.public.write
 def upgrade(self, new_code: bytes) -> None:
 root = gl.storage.Root.get()
 code = root.code.get()
 # If sender is not in upgraders, this will raise a VMError
 code.truncate()
 code.extend(new_code)
...

```

### ## Upgrading to a New Version

The upgraded contract code must maintain the **same storage layout** for compatibility. Only the code changes – all storage data, the ``upgraders`` list, and locked slots persist across upgrades.

```
```python
# v0.1.0
# { "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
```

```
from genlayer import *
```

```
class UpgradableStorage(gl.Contract):
```

```

# Storage layout must remain compatible with v1
storage: str

def __init__(self):
    pass

@gl.public.view
def get_storage(self) -> str:
    return self.storage

@gl.public.write
def update_storage(self, new_storage: str) -> None:
    self.storage = new_storage

# New method added in v2
@gl.public.view
def get_storage_length(self) -> int:
    return len(self.storage)

# Keep the upgrade method for future upgrades
@gl.public.write
def upgrade(self, new_code: bytes) -> None:
    root = gl.storage.Root.get()
    code = root.code.get()
    code.truncate()
    code.extend(new_code)
...

```

What Happens During an Upgrade

When an authorized upgrader calls the `upgrade` method with new code:

Component	After upgrade
`code`	**Replaced** with new code
Contract storage data (e.g. `storage: str`)	**Persists** unchanged
`locked_slots`	**Persists** unchanged
`upgraders`	**Persists** unchanged

This means:

- **Upgrades are not one-shot** – since the upgraders list persists, the same addresses can push another upgrade later
- **Storage must be compatible** – the new code must understand the existing storage layout. There is no automatic migration mechanism
- **New methods can be added** – but existing storage field positions must not change
- **Upgraders can be modified** – an upgrader can add or remove addresses from the upgraders list

Freezing a Contract

To make a contract permanently non-upgradable, either:

- Call `root.lock\_default()` without adding any addresses to `upgraders` – the code and critical slots are locked and nobody can unlock them
- Remove all addresses from the `upgraders` list after locking

> **Warning:**

> Once a contract is frozen (locked slots with no upgraders), it cannot be upgraded. This is irreversible.

Testing Upgrades

Using the [GenLayer Testing Suite](/developers/intelligent-contracts/tools/genlayer-testing-suite), you can test the full upgrade lifecycle:

```
```python
```

```

from pathlib import Path
from gltest import get_contract_factory
from gltest.assertions import tx_execution_succeeded

CONTRACTS_DIR = Path(__file__).parent.parent / "contracts"

def test_upgradable_storage():
 # Deploy v1
 factory = get_contract_factory(
 contract_file_path=CONTRACTS_DIR / "upgradable_storage.py"
)
 contract = factory.deploy(args=["hello"])

 # Use v1 methods
 assert contract.get_storage(args=[]).call() == "hello"

 # Read v2 code and upgrade the contract
 v2_code = (CONTRACTS_DIR / "upgradable_storage_v2.py").read_bytes()
 tx = contract.upgrade(args=[v2_code]).transact()
 assert tx_execution_succeeded(tx)

 # Rebuild the contract proxy from the v2 schema
 v2_factory = get_contract_factory(
 contract_file_path=CONTRACTS_DIR / "upgradable_storage_v2.py"
)
 contract_v2 = v2_factory.build_contract(contract_address=contract.address)

 # Storage persists across upgrades
 assert contract_v2.get_storage(args=[]).call() == "hello"

 # New v2 method works
 ... assert contract_v2.get_storage_length(args=[]).call() == 5

```

See [GenVM specification](<https://sdk.genlayer.com/v0.2.7/spec/04-contract-interface/04-upgradability.html>) for the full technical details.

---

## # Value Transfers

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/value-transfers>

Value transfers in Intelligent Contracts use GEN, GenLayer's native token, to receive funds with payable methods, send value through messages, and read contract balances.

## ## Native Token (GEN)

GenLayer uses GEN as its native token. Values are denominated in wei (1 GEN =  $10^{18}$  wei). Use the `u256` type for value amounts in contract code.

## ## Receiving Value

Mark a method as payable with `@gl.public.write.payable` to accept GEN:

```

```python
from genlayer import *

class TipJar(gl.Contract):
    def __init__(self):
        self.total_tips = u256(0)

    @gl.public.write.payable
    def tip(self) -> None:
        v = gl.message.value
        if v == u256(0):

```

```

        raise gl.vm.UserError("send some value")
    self.total_tips = self.total_tips + v

```

```

    @gl.public.view
    def get_tips(self) -> u256:
        return self.total_tips
...

```

`gl.message.value` is a `u256` containing the GEN sent with the call. It is only available in methods decorated with `@gl.public.write.payable`.

Sending Value to Another Intelligent Contract

Send value to another IC using [internal messages](/developers/intelligent-contracts/features/messages#internal-messages-ic--ic):

```

```python
other = gl.get_contract_at(recipient_address)

Pure value transfer (triggers __receive__ on recipient)
other.emit_transfer(value=u256(amount), on='finalized')

Value + method call (recipient method must be payable)
other.emit(value=u256(amount), on='finalized').deposit()
```

```

How Value Flows

When a message with value is emitted, the value is **immediately deducted** from the sending contract's balance and held in the message. It is only credited to the recipient when the message's child transaction is activated. The flow is:

****Sender balance → message → recipient balance****

If the child transaction fails, the value is not automatically returned to the sender.

See [Messages](/developers/intelligent-contracts/features/messages) for details on timing (`on='accepted'` vs `on='finalized'`).

Sending Value to an EOA or EVM Contract

Sending to an address on the GenLayer Chain (EOA or EVM contract) is an [external message](/developers/intelligent-contracts/features/messages#external-messages-ic--chain-layer). It goes through the IC's [ghost contract](/developers/intelligent-contracts/features/messages#ghost-contracts) and always executes on finalization.

```

```python
@gl.evm.contract_interface
class _Recipient:
 class View:
 pass
 class Write:
 pass

class Faucet(gl.Contract):
 @gl.public.write.payable
 def send(self, recipient: str) -> None:
 v = gl.message.value
 if v == u256(0):
 raise gl.vm.UserError("send some value")
 _Recipient(Address(recipient)).emit_transfer(value=v)
...

```

> **\*\*Note:\*\***

> The syntax for sending to an EOA uses the EVM contract interface even though the recipient

is not a contract. This is because EOAs live on the chain layer, making this an external message – the same mechanism used for calling EVM contracts. This will be simplified in a future version.

### ## Reading Balances

```
```python
# Own contract balance
my_balance = self.balance

# Another IC's balance
other_balance = gl.get_contract_at(addr).balance

# EVM contract balance
evm_balance = EthContract(addr).balance
```
```

In `write` methods, `self.balance` reflects the current state including any value received in the current call. In `view` methods, it shows a snapshot without modifications.

### ### Where Balance Lives

On the GenLayer network, an IC's GEN balance is held by its [ghost contract] (/developers/intelligent-contracts/features/messages#ghost-contracts) on the chain layer. `self.balance` reads from this. The balance is visible on both layers.

```
> **Warning:**
> **Studio:** Balances are simulated in a local database. There is no EVM layer or ghost
contracts in Studio.
```

### ## Receiving Value Without a Method Call

When value is sent without specifying a method, the contract can handle it with special methods:

```
```python
class Contract(gl.Contract):
    @gl.public.write.payable
    def __receive__(self):
        """Called for value-only transfers with no method name."""
        pass

    @gl.public.write.payable
    def __handle_undefined_method__(
        self, method_name: str, args: list, kwargs: dict
    ):
        """Fallback for calls to undefined methods."""
        pass
...
```
```

The dispatch logic:

```
```mermaid
graph TD
    msg[>incoming call] --> has_method[method exists?]
    has_method -->|yes| call_method[call_method{{call method}}]
    has_method -->|no| has_value[has_value[has value?]]

    has_value -->|yes| has_method_name[has_method_name[has method name?]]
    has_method_name -->|no| has_receive[has_receive[__receive__ defined?]]
    has_receive -->|yes| call_receive[call_receive{{call __receive__}}]
    has_receive -->|no| has_fallback1[has_fallback1[__handle_undefined_method__ defined and payable?]]
    has_fallback1 -->|yes| call_fallback1[call_fallback1{{call __handle_undefined_method__}}]
    has_fallback1 -->|no| error1[error1{{error}}]
```
```



```

has_method_name -->|yes| has_fallback2[__handle_undefined_method__ defined and payable?]
has_fallback2 -->|yes| call_fallback2{{call __handle_undefined_method__}}
has_fallback2 -->|no| error2{{error}}

has_value -->|no| has_fallback3[__handle_undefined_method__ defined?]
has_fallback3 -->|yes| call_fallback3{{call __handle_undefined_method__}}
has_fallback3 -->|no| error3{{error}}
...

```

## ## Funding Accounts

| Environment       | How to fund                                                                       |
|-------------------|-----------------------------------------------------------------------------------|
| Studio (Localnet) | Built-in faucet – 💧 button in the account selector                                |
| Stunionet         | Built-in faucet – 💧 button in the account selector                                |
| Testnet Asimov    | [testnet-faucet.genlayer.foundation](https://testnet-faucet.genlayer.foundation/) |
| Testnet Bradbury  | [testnet-faucet.genlayer.foundation](https://testnet-faucet.genlayer.foundation/) |

See [Networks](/developers/networks) for full details on each environment.

## ## Calling Payable Methods from JavaScript

Use the `value` parameter in `writeContract()` to send GEN:

```

``typescript
import { createClient, createAccount } from 'genlayer-js';
import { testnetAsimov } from 'genlayer-js/chains';

const client = createClient({
 chain: testnetAsimov,
 account: createAccount(),
});

const txHash = await client.writeContract({
 address: contractAddress,
 functionName: 'tip',
 args: [],
 value: BigInt(5) * BigInt(10 ** 18), // 5 GEN in wei
});

```

---

## # Transaction Context

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/transaction-context>

Transaction Context is the GenVM execution metadata available to every Intelligent Contract execution, including caller addresses, value, chain ID, and the transaction **timestamp** (datetime). This page explains how to read the current transaction time, derive a Unix timestamp, access caller and transaction metadata, and understand what is **not** exposed, including block number, gas, and host wall-clock time.

## ## `gl.message` – typed accessors

The most common fields are exposed as a typed `NamedTuple`:

| Field            | Type      | Description                                                                         |
|------------------|-----------|-------------------------------------------------------------------------------------|
| `sender_address` | `Address` | Immediate caller – EOA, EVM contract, or IC depending on how the call was initiated |
| `origin_address` | `Address` | Original transaction submitter – preserved through internal message chains          |

```
`contract_address`	`Address`	The current contract's own address
`value`	`u256`	GEN sent with the call (only in `@gl.public.write.payable` methods)
`chain_id`	`u256`	Current chain ID
```

```
```python
@gl.public.write
def deposit(self):
    self.balances[gl.message.sender_address] += gl.message.value
...

```

For the difference between `sender\_address` and `origin\_address` across internal message chains, see [Messages](/developers/intelligent-contracts/features/messages).

`gl.message\_raw` – full message dict

`gl.message\_raw` is a `TypedDict` with everything the bootloader receives. It includes all `gl.message` fields plus:

Field	Type	Description
`datetime`	`str`	Transaction datetime as an ISO 8601 string
`is_init`	`bool`	`True` iff this execution is a deployment
`stack`	`list[Address]`	Stack of view-method callers, excluding the current contract
`entry_kind`	`int`	`0` = MAIN, `1` = SANDBOX, `2` = CONSENSUS_STAGE
`entry_data`	`bytes`	Raw entry payload
`entry_stage_data`	–	Consensus-stage-specific data

```
```python
Detect whether the current execution is a deploy
if gl.message_raw['is_init']:
 self.deployer = gl.message.sender_address
...

```

> **Note:**  
> Prefer `gl.message` for the common fields – it gives you autocompletion and type checking. Reach for `gl.message\_raw` only when you need one of the extra fields.

## Time and Timestamps

Time inside the GenVM is **deterministic and pinned to the transaction's timestamp**. Every validator re-executing the transaction sees the same value, so you can use it for storage, comparisons, and prompt context without breaking equivalence.

The Python standard library's clock is wired to the transaction datetime – `datetime.now()`, `datetime.now(tz)`, and `time.time()` all return the same value across validators.

### Transaction Timestamp

The transaction timestamp is exposed three ways. Use whichever fits:

Form	How	Use for
Unix seconds	`int(datetime.now(timezone.utc).timestamp())`	Arithmetic, expiries, deltas
Unix seconds (alt)	`int(time.time())`	Same – shorter
`datetime` object	`datetime.now(timezone.utc)`	Comparisons, formatting
ISO 8601 string	`datetime.now(timezone.utc).isoformat()`	*or*
	`gl.message_raw['datetime']`	Storage, audit trails, prompts

### Common patterns

**Unix timestamp for arithmetic:**

```
```python
from datetime import datetime, timezone

```

```
@gl.public.write
def mint(self):
    now = int(datetime.now(timezone.utc).timestamp())
    self.tokens[gl.message.sender_address] = Token(
        expiry=u256(now + 86400), # 24h from now
    )
...
```

****Expiry check:****

```
```python
@gl.public.view
def is_expired(self, token_id: u256) -> bool:
 now = int(datetime.now(timezone.utc).timestamp())
 return now > int(self.tokens[token_id].expiry)
...
```

**\*\*ISO string for audit trails or events:\*\***

```
```python
@gl.public.write
def submit(self, payload: str):
    self.submissions.append(Submission(
        sender=gl.message.sender_address,
        payload=payload,
        timestamp=datetime.now(timezone.utc).isoformat(),
    ))
...
```

****Raw ISO string from the message (skip the clock):****

```
```python
ts: str = gl.message_raw['datetime']
```
```

> **Warning:**

> The clock returns the ****transaction datetime****, not host wall-clock time. Do not call ``datetime.now()`` expecting current real-world time – by the time validators re-execute, the transaction may be minutes or hours old. Use it for relative arithmetic ("expires 24h after the tx") and timestamping, not for "is it past 5pm right now".

What is *not* in the context

- ****No block number, no block hash.**** The transaction context does not carry block-level metadata. If your contract needs block height, fetch it via [Web Access] (/developers/intelligent-contracts/features/web-access) against an RPC inside a non-deterministic block.
- ****No gas price, no nonce.**** Transaction metadata beyond what is listed above is not exposed to the contract.
- ****No host wall-clock time.**** As above – the clock is the transaction datetime, not the validator's current time.

Messages

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/messages>

Messages in GenLayer are interactions that move execution between callers, Intelligent Contracts, the GenVM layer, and the GenLayer Chain. GenLayer has three types of interaction, each operating at a different layer of the architecture: transactions that start Intelligent Contract execution, internal messages between Intelligent Contracts, and external messages from an Intelligent Contract to the chain layer.

Transactions (→ Intelligent Contract)

A transaction is the entry point for all IC execution. The caller signs an EVM transaction that calls ``addTransaction()`` on ConsensusMain or on the IC's ghost contract (which relays to ConsensusMain). This is submitted via ``eth_sendRawTransaction``.

The caller can be an EOA or an EVM contract on GenLayer Chain.

- ``gl.message.sender_address`` = caller address
- ``gl.message.origin_address`` = caller address

The transaction enters the consensus pipeline: activation → proposal → commit → reveal → finalization.

```
> **Warning:**
> **Studio:** The SDK sends the same signed `addTransaction` EVM transaction via
`eth_sendRawTransaction`, but Studio intercepts and simulates the consensus pipeline rather
than executing on a real EVM layer.
```

Internal Messages (IC → IC)

An internal message is an ****asynchronous**** call from one Intelligent Contract to another. It stays within the GenVM layer. During execution, ``emit()`` records the message as part of the execution result – the message is not sent immediately. The actual child transaction is only created when the parent transaction reaches the specified state (``on='accepted'`` or ``on='finalized'``). The child transaction then goes through consensus independently.

```
```python
other = gl.get_contract_at(addr)

Call a write method on another IC
other.emit(on='finalized').update_status("active")

Call with value (recipient must be payable)
other.emit(value=u256(100), on='finalized').deposit()

Pure value transfer (triggers __receive__ on recipient)
other.emit_transfer(value=u256(100), on='finalized')

Deploy a child contract
addr = gl.deploy_contract(
 code=contract_code,
 args=[],
 salt_nonce=u256(1),
 on='finalized',
)
```
```

In the child transaction's execution context:

- ``gl.message.sender_address`` = calling contract's address
- ``gl.message.origin_address`` = original caller (preserved through the chain)

See [Interacting with Intelligent Contracts](/developers/intelligent-contracts/features/interacting-with-intelligent-contracts) for full syntax reference.

External Messages (IC → Chain Layer)

An external message crosses from the GenVM layer back to the GenLayer Chain (EVM layer). This is how an IC sends value or calls an EOA or EVM contract.

External messages are executed by the IC's ghost contract via ``handleOp()``, so the ``msg.sender`` seen by the recipient is the ghost contract address (which is the same as the IC's address).

```
```python
```

```

@gl.evm.contract_interface
class ERC20:
 class View:
 def balance_of(self, owner: Address) -> u256: ...
 class Write:
 def transfer(self, to: Address, amount: u256) -> None: ...

Read from an EVM contract (gl.message.contract_address is your own address)
balance = ERC20(token_address).view().balance_of(gl.message.contract_address)

Write to an EVM contract (value is optional, defaults to 0)
ERC20(token_address).emit().transfer(recipient, amount)

Send value to an EOA
@gl.evm.contract_interface
class _Recipient:
 class View:
 pass
 class Write:
 pass

_RRecipient(Address(eoa_address)).emit_transfer(value=u256(amount))
``,`

> Note:
> External messages can only be emitted `on='finalized'`. Using `on='accepted'` for external
messages is not supported.

> Warning:
> Studio: EVM contract interaction beyond value transfers to EOAs is not implemented.
The `@gl.evm.contract_interface` calls are not functional in Studio.

```

## ## Ghost Contracts

Every Intelligent Contract has a corresponding **ghost contract** on GenLayer Chain (the EVM layer). The ghost and IC share the same address.

### ### What Ghost Contracts Do

- **Hold the IC's GEN balance** – the native balance on the ghost contract on-chain is the source of truth for `self.balance`
- **Relay transactions** – `addTransaction()` on the ghost forwards to ConsensusMain
- **Execute external messages** – on finalization, `handleOp()` forwards calls to recipients so `msg.sender` is the IC's address
- **Bridge the two layers** – ghost on EVM ↔ IC on GenVM, same address

### ### Lifecycle

1. A deploy transaction is submitted
2. GhostFactory deploys a ghost contract at address `0xABC` on GenLayer Chain
3. Consensus processes the deployment
4. If successful: IC deploys at the same address `0xABC` on GenVM
5. If deployment is appealed and reverted: the ghost remains but has no IC behind it – it has no purpose or effect, but may be reused by consensus if deployment succeeds later

```

> Note:
> Ghost contract existence does not guarantee the IC is deployed. A reverted deployment
leaves an empty ghost.

> Warning:
> Studio: Ghost contracts are not implemented. The IC address is used directly, and
balance is stored in a database table.

```

## ## Message Context

Caller and execution metadata (``sender_address``, ``origin_address``, ``contract_address``, ``value``, ``chain_id``, transaction datetime, etc.) is available via ``gl.message`` and ``gl.message_raw``. See [Transaction Context](/developers/intelligent-contracts/features/transaction-context).

## ## Timing: Accepted vs Finalized

Internal messages and contract deployments can execute at two different points:

### ### ``on='finalized'`` (default)

The message executes after the parent transaction is fully finalized (appeal window has closed). This is the safe default.

### ### ``on='accepted'``

The message executes as soon as the parent transaction is accepted by initial consensus, before the appeal window closes.

```
> **Warning:**
> **Appeal risks with `on='accepted'`:
> - The re-execution may emit the message again – potentially multiple times across appeal rounds.
> - If the appeal changes the outcome, the message may be "invalid" (would not have been emitted with the final outcome), but **it cannot be taken back** – it was already sent and executed.
> - The receiving contract must be **idempotent** and must handle duplicate or unexpected messages gracefully.
> - Contract design must account for the possibility that accepted messages may not reflect the final state.
```

External messages always use ``on='finalized'`` and cannot be emitted on acceptance.

## ## Interfaces

### ### IC Interfaces (``@gl.contract_interface``)

Define typed stubs for calling other Intelligent Contracts:

```
```python
@gl.contract_interface
class Token:
    class View:
        def balance_of(self, owner: Address) -> u256: ...
    class Write:
        def transfer(self, to: Address, amount: u256) -> None: ...

token = Token(token_address)
balance = token.view().balance_of(user)          # typed view call
token.emit(on='finalized').transfer(to, amount)   # typed write call
```
```

This is purely for type safety and IDE autocompletion – at runtime it behaves identically to ``gl.get_contract_at()``. No overhead.

### ### EVM Interfaces (``@gl.evm.contract_interface``)

Define typed stubs matching Solidity function signatures:

```
```python
@gl.evm.contract_interface
class IERC20:
    class View:
        def balance_of(self, owner: Address) -> u256: ...
        def total_supply(self) -> u256: ...
```

```

class Write:
    def transfer(self, to: Address, amount: u256) -> bool: ...
    def approve(self, spender: Address, amount: u256) -> bool: ...
...

```

Parameters are automatically ABI-encoded. Type mapping:

```

| GenLayer | Solidity |
|---|---|
| `u256`, `u128`, `u64`, ... | `uint256`, `uint128`, `uint64`, ... |
| `i256`, `i128`, `i64`, ... | `int256`, `int128`, `int64`, ... |
| `Address` | `address` |
| `bool` | `bool` |
| `str` | `string` |
| `bytes` | `bytes` |
---

```

Interacting with Intelligent Contracts

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/interacting-with-intelligent-contracts>

Interacting with Intelligent Contracts means referencing another contract by address, calling its read-only `view()` methods, emitting asynchronous write messages with `emit()`, and deploying child contracts.

> This page covers syntax for internal messages (IC → IC). See [Messages] (/developers/intelligent-contracts/features/messages) for the conceptual model and [Value Transfers] (/developers/intelligent-contracts/features/value-transfers) for sending GEN.

Getting Contract References

Access other contracts by their address:

```

```python
contract_address = Address("0x03FB09251eC05ee9Ca36c98644070B89111D4b3F")

dynamically_typed_contract = gl.get_contract_at(contract_address)

@gl.contract_interface
class GenLayerContractIface:
 class View:
 def method_name(self, a: int, b: str): ...

 class Write:
 pass

statically_typed_contract = GenLayerContractIface(contract_address)
```

```

Both approaches result in the same runtime value, however the statically typed approach provides type checking and autocompletion in IDEs.

Calling View Methods

Call read-only methods on other contracts:

```

```python
addr: Address = ...
other = gl.get_contract_at(addr)
result = other.view().get_token_balance()
```

```

Emitting Messages

Send asynchronous messages to other contracts:

```
```python
other = gl.get_contract_at(addr)
other.emit(on='accepted').update_status("active")
other.emit(on='finalized').update_status("active")

With value (recipient method must be @gl.public.write.payable)
other.emit(value=u256(100), on='finalized').deposit()
```
```

Deploying New Contracts

```
```python
gl.deploy_contract(code=contract_code)
salt: u256 = u256(1) # not zero
child_address = gl.deploy_contract(code=contract_code, salt=salt)
```
```

View vs Emit

- `view()` is synchronous – it reads state from another contract and returns the result immediately. The target contract's state is read as of the current block.
- `emit()` is asynchronous – it queues a write call that executes *after* the current transaction completes. The call is not blocking.

Accepted vs Finalized

```
```python
Fast – executes after the transaction is accepted by initial consensus
other.emit(on='accepted').do_something()

Safe – waits until the transaction is fully finalized (after appeal window)
other.emit(on='finalized').do_something()
```
```

`on='accepted'` has important implications. If the emitting transaction is appealed, two things can happen:

1. The appeal changes the contract state such that the message should not have been emitted – but it was already sent and cannot be recalled.
2. The transaction is re-executed during the appeal, and the message (or a similar one) is emitted again. This can repeat across multiple appeal rounds – up to ~6 times depending on the validator set size.

The receiving contract must be *idempotent* – it must handle duplicate or unexpected messages gracefully. If the receiving logic cannot tolerate duplicates or messages that "shouldn't have been sent" based on the final state, use `on='finalized'` instead.

Factory Pattern

Deploy child contracts from a parent:

```
```python
def __init__(self, num_workers: int):
 with open("/contract/Worker.py", "rt") as f:
 worker_code = f.read()

 for i in range(num_workers):
 addr = gl.deploy_contract(
 code=worker_code.encode("utf-8"),
 args=[i, gl.message.contract_address],
 salt_nonce=i + 1,
 on="accepted",
)
```
```



```
... self.worker_addresses.append(addr)
```

Child contracts are immutable after deployment. To update worker logic, redeploy through the factory.

Interacting with EVM Contracts

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/interacting-with-evm-contracts>

Interacting with EVM Contracts lets Intelligent Contracts define EVM contract interfaces, read EVM contract data and balances, and emit messages to EVM contracts.

> These are external messages – they cross from the GenVM layer back to the GenLayer Chain via [ghost contracts](/developers/intelligent-contracts/features/messages#ghost-contracts). See [Messages](/developers/intelligent-contracts/features/messages) for the conceptual model.

Contract Interface Definition

Define EVM contract interfaces using decorators:

```
```python
@gl.evm.contract_interface
class TokenContract:
 class View:
 def balance_of(self, owner: Address) -> u256: ...
 def total_supply(self) -> u256: ...

 class Write:
 def transfer(self, to: Address, amount: u256) -> bool: ...
 def approve(self, spender: Address, amount: u256) -> bool: ...
...```
```

### ## Calling EVM Contracts

Interact with EVM contracts through defined interfaces:

```
```python
token_address: Address = ...
# Read from EVM contract
token = TokenContract(token_address)
supply = token.view().total_supply()

# Write to EVM contract
token.emit().transfer(receiver_address, u256(100))
...```
```

Balance Access

Access EVM contract balances directly:

```
```python
evm_contract = TokenContract(address)
balance = evm_contract.balance # Get contract's ETH balance
...```
```

### ## Message Emission

Send messages to EVM contracts:

```
```python
TokenContract(token_address).emit().approve(spender, amount)
...```
```

...

> ****Note:****

> Messages to EVM contract can be emitted only on finality

Special Methods

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/special-methods>

Special methods are the two methods allowed in each contract ****definition**** for handling undefined method calls and no-method value transfers: `__handle_undefined_method__` and `__receive__`. GenVM uses these methods when an unhandled message does not match any regular method.

> For how these methods fit into value transfer flows, see [Value Transfers] (/developers/intelligent-contracts/features/value-transfers#receiving-value-without-a-method-call).

```python

```
class Contract(gl.Contract):
 @gl.public.write # .payable?
 def __handle_undefined_method__(
 self, method_name: str, args: list[typing.Any], kwargs: dict[str, typing.Any]
):
 """
 Method that is called for undefined method calls,
 must be either ``@gl.public.write`` or ``@gl.public.write.payable``
 """
 ...

 @gl.public.write.payable
 def __receive__(self):
 """
 Method that is called for no-method transfers,
 must be ``@gl.public.write.payable``
 """
 ...
```

...

Below is a diagram that shows how GenVM decides which method to pick, in case any regular method did not match:

```mermaid

```
graph TD
    unhandled_message[unhandled message] --> has_value{has value?}
    has_value -->|yes| has_method_name{has method name?}
    has_method_name -->|yes| fallback_defined1{__handle_undefined_method__ is defined?}
    has_method_name -->|no| receive_defined{__receive__ defined?}
    receive_defined -->|no| fallback_defined1
    receive_defined -->|yes| receive_call{{call __receive__}}
    fallback_defined1 -->|yes| fallback_is_payable{__handle_undefined_method__ payable?}
    fallback_defined1 -->|no| error3{{error}}
    fallback_is_payable -->|yes| fallback1{{call __handle_undefined_method__}}
    fallback_is_payable -->|no| error1{{error}}
    has_value -->|no| fallback_defined
    fallback_defined -->|yes| fallback2{{call __handle_undefined_method__}}
    fallback_defined -->|no| error2{{error}}
```

...

Vector Store

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/vector-storage>
Vector Store is a GenLayer feature for Intelligent Contracts that stores text as vector embeddings, retrieves entries, and calculates text similarity efficiently. Developers can use Vector Store for natural language processing (NLP) tasks such as context-aware applications and indexing text data for semantic search.

Key Features of Vector Store

The Vector Store provides several powerful features for managing text data:

1. Text Embedding Storage

You can store text data as vector embeddings, which are mathematical representations of the text, allowing for efficient similarity comparisons. Each stored text is associated with a vector and metadata.

2. Similarity Calculation

The Vector Store allows you to calculate the similarity between a given text and stored vectors using cosine similarity. This is useful for finding the most semantically similar texts, enabling applications like recommendation systems or text-based search.

3. Metadata Management

Along with the text and vectors, you can store additional metadata (any data type) associated with each text entry. This allows developers to link additional information (e.g., IDs or tags) to the text for retrieval.

4. CRUD Operations

The Vector Store provides standard CRUD (Create, Read, Update, Delete) operations, allowing developers to add, update, retrieve, and delete text and vector entries efficiently.

How to Use Vector Store in Your Contracts

To use the Vector Store in your Intelligent Contracts, you will interact with its methods to add and retrieve text data, calculate similarities, and manage vector storage. Below are the details of how to use this feature.

Importing Vector Store

First, import the VectorStore class from the standard library in your contract:

```
```python
from backend.node.genvm.std.vector_store import VectorStore
```
```

Creating a Contract with Vector Store

Here's an example of a contract using the Vector Store for indexing and searching text logs:

```
```python
{
"Seq": [
{ "Depends": "py-lib-
genlayermodelwrappers:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" },
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
]
}
```

```
from genlayer import *
import genlayermodelwrappers
import numpy as np
from dataclasses import dataclass
```

```
@allow_storage
@dataclass
class StoreValue:
 log_id: u256
```

```

text: str

contract class
class LogIndexer(gl.Contract):
 vector_store: VecDB[np.float32, typing.Literal[384], StoreValue]

 def __init__(self):
 pass

 def get_embedding_generator(self):
 return genlayermodelwrappers.SentenceTransformer("all-MiniLM-L6-v2")

 def get_embedding(
 self, txt: str
) -> np.ndarray[tuple[typing.Literal[384]], np.dtypes.Float32DType]:
 return self.get_embedding_generator()(txt)

 @gl.public.view
 def get_closest_vector(self, text: str) -> dict | None:
 emb = self.get_embedding(text)
 result = list(self.vector_store.knn(emb, 1))
 if len(result) == 0:
 return None
 result = result[0]
 return {
 "vector": list(str(x) for x in result.key),
 "similarity": str(1 - result.distance),
 "id": result.value.log_id,
 "text": result.value.text,
 }

 @gl.public.write
 def add_log(self, log: str, log_id: int) -> None:
 emb = self.get_embedding(log)
 self.vector_store.insert(emb, StoreValue(text=log, log_id=u256(log_id)))

 @gl.public.write
 def update_log(self, log_id: int, log: str) -> None:
 emb = self.get_embedding(log)
 for elem in self.vector_store.knn(emb, 2):
 if elem.value.text == log:
 elem.value.log_id = u256(log_id)

 @gl.public.write
 def remove_log(self, id: int) -> None:
 for el in self.vector_store:
 if el.value.log_id == id:
 el.remove()

...

Debugging Contract Code
Source: https://docs.genlayer.com/developers/intelligent-contracts/features/debugging

Print Statements

```

Use `print` calls for basic debugging, which will be present in node or studio output.

Use `gl.trace` to include time stamps in the output (it will be written to genvm\_log instead of stdout)

## Profiling

There is also `gl.trace\_time\_micro` which returns timestamp in the GenVM debug mode. Default bootloader includes possibility to enable profiling by setting environment variable `GENLAYER\_ENABLE\_PROFILER=true`.

```
```python
# {
#   "Seq": [
#     { "SetEnv": { "name": "GENLAYER_ENABLE_PROFILER", "value": "true" } },
#     { "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
#   ]
# }
```
```

This will output base-64 encoded gzip-compressed profiling data to stderr after every single VM execution

## ## Attaching a Debugger

```
> **Warning:**
> Unfortunately, attaching a debugger to a running Intelligent Contract is not supported yet

```

## # Random

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/random>

Randomness in an Intelligent Contract should use seeded random values because getting random data in the deterministic part may be difficult. Delegating randomness to a non-deterministic block can cause the contract to either never agree on that block or to trust the leader.

## ## Seed acquisition methods

1. Use some field of `message`
2. Use current time (transaction time)
3. Use `stdin`, as shown below

```
```python3
def get_random_seed() -> bytes:
    import os
    import hashlib
    f = os.fdopen(0, 'rb', buffering=0, closefd=False)
    f.seek(0)
    hash_obj = hashlib.sha256()
    while True:
        chunk = f.read(8192)
        if not chunk:
            return hash_obj.digest()
        hash_obj.update(chunk)
...
```
```

---

## # Non-determinism

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/non-determinism>

Non-determinism in Intelligent Contracts is any operation whose result can vary between nodes, such as external API calls, LLM and AI model calls, random number generation, or other variable execution. GenVM requires these operations to run inside nondeterministic blocks so validators can reach consensus on the agreed result before deterministic side effects occur.

## ## When to Use

Non-deterministic operations are needed for:

- External API calls
- LLM and AI model calls
- Random number generation
- Any operation that might vary between nodes

## ## What Goes Inside vs Outside

Non-deterministic blocks (`leader\_fn`, `validator\_fn`, functions passed to `strict\_eq`) run in a special execution context. The GenVM enforces strict rules about what can and cannot happen inside these blocks.

### ### Must be INSIDE nondet blocks

All `gl.nondet.\*` calls – web requests, LLM prompts – must be inside a nondet block. They cannot run in regular contract code.

```
```python
@gl.public.write
def fetch_price(self):
    def leader_fn():
        response = gl.nondet.web.get(api_url)          # ✓ inside nondet block
        result = gl.nondet.exec_prompt(prompt)         # ✓ inside nondet block
        return parse_price(response)

    # gl.nondet.web.get(api_url)                        # ✗ would fail here
    self.price = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)
...
```
```

### ### Must be OUTSIDE nondet blocks

Several operations must happen in the deterministic context – after the nondet block returns:

| Operation                                   | Why                                                       |
|---------------------------------------------|-----------------------------------------------------------|
| **Storage writes** (`self.x = ...`)         | Storage must only change based on consensus-agreed values |
| **Contract calls** (`gl.get_contract_at()`) | Cross-contract calls must use deterministic state         |
| **Message emission** (`.emit()`)            | Messages to other contracts/chains must be deterministic  |
| **Nested nondet blocks**                    | Nondet blocks cannot contain other nondet blocks          |

```
```python
@gl.public.write
def update_price(self, pair: str):
    def leader_fn():
        response = gl.nondet.web.get(api_url)
        return json.loads(response.body.decode("utf-8"))["price"]
        # self.price = price                # ✗ no storage writes here
        # other = gl.get_contract_at(addr)   # ✗ no contract calls here
        # other.emit().notify(price)         # ✗ no message emission here

    def validator_fn(leaders_res) -> bool:
        if not isinstance(leaders_res, gl.vm.Return):
            return False
        my_price = leader_fn()
        return abs(leaders_res.calldata - my_price) / leaders_res.calldata <= 0.02

    price = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)

    # ✓ All side effects happen AFTER consensus, in deterministic context
    self.prices[pair] = price
...
```
```

```

 oracle = gl.get_contract_at(self.oracle_address)
 oracle.emit().price_updated(pair, price)
...

```

> **Note:**

> The [GenVM linter](/developers/intelligent-contracts/tooling-setup) catches all of these violations statically – run `genvm-lint check` before deploying to avoid runtime errors.

### ### Why these rules exist

The leader and validators execute nondet blocks **independently** – each node runs its own `leader\_fn` or `validator\_fn`. If you wrote to storage inside a nondet block, each node would write a different value before consensus decides which one is correct. The same applies to contract calls and message emission: these must happen once, after consensus, using the agreed-upon result.

### ## Equivalence Principle

GenLayer provides `strict\_eq` for exact-match consensus and custom validator functions (`run\_nondet\_unsafe`) for everything else. Convenience wrappers like `prompt\_comparative` and `prompt\_non\_comparative` exist for common patterns. For detailed information, see [Equivalence Principle](/developers/intelligent-contracts/equivalence-principle).

### ### Strict Equality

Requires exact matches between validator outputs. Use when all nodes can converge on the same normalized value – e.g., fetching objective data from an API and extracting a structured result:

```

```python
def fetch_current_block():
    response = gl.nondet.web.request("https://api.example.com/block/latest")
    data = json.loads(response)
    return json.dumps({"height": data["height"], "hash": data["hash"]}, sort_keys=True)

# All validators must return the exact same string
result = gl.eq_principle.strict_eq(fetch_current_block)
```

```

> **Note:** `strict\_eq` is not suitable for random number generation or LLM calls, since those inherently produce different results on each node. Use a custom validator function or one of the convenience wrappers below for those cases.

### ### Comparative (Convenience Shortcut)

A convenience wrapper where both leader and validators perform the same task, then an LLM compares results using your criteria:

```

```python
def comparative_example():
    return gl.nondet.web.request("https://api.example.com/count")

# Results are compared with acceptable margin of error
result = gl.eq_principle.prompt_comparative(
    comparative_example,
    "Results should not differ by more than 5%"
)
```

```

### ### Non-Comparative (Convenience Shortcut)

A convenience wrapper where validators evaluate the leader's output against criteria without repeating the task:

```

```python

```

```

result = gl.eq_principle.prompt_non_comparative(
    input="This product is amazing!",
    task="Classify the sentiment as positive, negative, or neutral",
    criteria="""
        Output must be one of: positive, negative, neutral
        Consider context and tone
    """)
)

```

Custom Validator Functions

For full control over consensus logic, write a custom leader/validator pair with ``run_nondet_unsafe``. This is the recommended approach for most contracts:

```

```python
def custom_consensus_example(self, data: str):
 def leader_fn():
 # Leader performs the operation
 response = gl.nondet.exec_prompt(f"Rate this sentiment 1-10: <data>{data}</data>."
 Answer only with integer, without reasoning")
 return int(response.strip())

 def validator_fn(leader_result):
 own_score = leader_fn()

 if isinstance(leader_result, Exception):
 return False

 # Accept if within acceptable range
 return abs(own_score - leader_result) <= 2

 ... return gl.vm.run_nondet_unsafe(leader_fn, validator_fn)

```

### ## Operations

#### ### Accessing External Data

Use non-deterministic blocks for external API calls. For more web access examples, see [Web Access](/developers/intelligent-contracts/features/web-access):

```

```python
@gl.public.write
def fetch_external_data(self):
    def fetch_data():
        # External API call - inherently non-deterministic
        response = gl.nondet.web.request("https://example.com/data")
        return response

    # Consensus ensures all validators agree on the result
    data = gl.eq_principle.strict_eq(fetch_data)
    return data

```

LLM Integration

Execute AI prompts using comparative principle. For more detailed examples, see [Calling LLMs](/developers/intelligent-contracts/features/calling-llms):

```

```python
@gl.public.write
def ai_decision(self, prompt: str):
 def call_llm():
 response = gl.nondet.exec_prompt(prompt)

```



```

 return response.strip()

 # Use comparative principle for LLM response consensus
 decision = gl.eq_principle.prompt_comparative(
 call_llm,
 principle="Responses should be semantically equivalent in meaning"
)
 return decision
...

```

---

## # Calling LLMs

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/calling-llms>

Calling LLMs in Intelligent Contracts uses `gl.nondet.exec\_prompt` to execute prompts, request JSON responses, process image inputs, and return model output through `run\_nondet\_unsafe` with custom validation. Because LLM outputs are non-deterministic, validators should check response structure, validity, and error behavior instead of requiring exact equality.

## ## Basic LLM Calls

Execute prompts using large language models:

```

```python
def leader_fn():
    response = gl.nondet.exec_prompt("Answer this question")
    return response.strip().lower()

def validator_fn(leader_result) -> bool:
    if not isinstance(leader_result, gl.vm.Return):
        return False
    my_result = leader_fn()
    return my_result == leader_result.calldata

answer = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)
```

```

## ## JSON Response Format

Request structured responses from LLMs:

```

```python
def structured_llm_call():
    prompt = """
    Return a JSON object with these keys:
    - "score": random integer from 1 to 100
    - "status": either "active" or "inactive"
    """
    return gl.nondet.exec_prompt(prompt, response_format='json')

def validate_structured(leader_result) -> bool:
    if not isinstance(leader_result, gl.vm.Return):
        return False
    data = leader_result.calldata
    # Validate structure rather than exact match – LLM outputs are non-deterministic
    return (
        isinstance(data, dict)
        and isinstance(data.get("score"), int)
        and 1 <= data["score"] <= 100
        and data.get("status") in ("active", "inactive")
    )

result = gl.vm.run_nondet_unsafe(structured_llm_call, validate_structured)

```

```
score = result['score'] # Access JSON fields
````
```

```
> **Note:**
> Using `strict_eq` with LLM calls will fail consensus because LLM outputs are non-
deterministic.
> Always use `run_nondet_unsafe` with a custom validator that checks the structure and
validity
> of the response rather than requiring an exact match.
```

This approach guarantees that `exec\_prompt` returns a valid JSON object, however correspondence to the specified format depends on the underlying LLM.

## ## Image Processing

Process images with vision models:

```
``python
def vision_analysis():
 prompt = "Describe what you see in this image"
 image_data = b"\x89PNG..."
 return gl.nondet.exec_prompt(
 prompt,
 images=[image_data]
)

def validate_description(leader_result) -> bool:
 if not isinstance(leader_result, gl.vm.Return):
 return False
 # Accept any non-empty string description
 return isinstance(leader_result.calldata, str) and len(leader_result.calldata.strip()) >
0

description = gl.vm.run_nondet_unsafe(vision_analysis, validate_description)
````
```

```
> **Warning:**
> Limit of images is two
```

Response Validation

Validate and process LLM responses:

```
``python
def generate_number():
    response = gl.nondet.exec_prompt(
        "Generate a number between 1 and 100",
        response_format='json'
    )

    # Validate the response
    if response['number'] < 1 or response['number'] > 100:
        raise Exception(f"Invalid number: {response['number']}")

    return response['number']

def validate_number(leader_result) -> bool:
    if not isinstance(leader_result, gl.vm.Return):
        return False
    num = leader_result.calldata
    return isinstance(num, int) and 1 <= num <= 100

result = gl.vm.run_nondet_unsafe(generate_number, validate_number)
````
```

## ## Defensive Response Parsing

LLMs return unpredictable formats. Even with `response\_format='json'`, the response may not match your expected schema. Here are some patterns for handling this:

### ### Key Aliasing

LLMs sometimes use alternate key names. One approach is to check for common variations:

```
```python
def parse_score(response: dict) -> int:
    """Extract a numeric score, handling common LLM variations."""
    raw = response.get("score")
    if raw is None:
        for alt in ("rating", "points", "value", "result"):
            if alt in response:
                raw = response[alt]
                break

    if raw is None:
        raise gl.UserError(f"Missing 'score' key. Got keys: {list(response.keys())}")

    # Coerce to int – handles "3", "3.5", floats, whitespace
    try:
        return max(0, int(round(float(str(raw).strip()))))
    except (ValueError, TypeError):
        raise gl.UserError(f"Non-numeric score: {raw}")
...

```

JSON Cleanup

LLMs sometimes wrap JSON in markdown fences or add trailing commas:

```
```python
import re

def clean_llm_json(text: str) -> dict:
 """Extract and clean JSON from LLM output."""
 import json
 first = text.find("{")
 last = text.rfind("}")
 if first == -1 or last == -1:
 raise gl.UserError(f"No JSON object found in response")
 text = text[first:last + 1]
 text = re.sub(r",(?!\s*?[\{\}\[\]"'\w])", "", text) # Remove trailing commas
 return json.loads(text)
...

```

### ### Always Use `response\_format='json'`

```
```python
result = gl.nondet.exec_prompt(task, response_format="json")
...

```

This instructs the LLM to return JSON. It significantly improves reliability, but you should still validate and clean the output – LLMs don't always comply.

Error Handling in LLM Calls

When using LLMs inside `run\_nondet\_unsafe`, consider how errors affect consensus. If the LLM returns garbage, you generally want the validator to **disagree** (return `False`) rather than agree on broken output – this forces a rotation to a new leader:

```
```python
def leader_fn():

```

```

 result = gl.nondet.exec_prompt(prompt, response_format="json")
 if not isinstance(result, dict):
 raise gl.UserError(f"LLM returned non-dict: {type(result)}")
 return result
...

```

See [Error Handling](/developers/intelligent-contracts/features/error-handling) for more on error classification patterns.

---

## # Image Processing

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/image-processing>

Image Processing in Intelligent Contracts lets contracts send screenshots, photos, or other visual data to LLMs alongside a prompt for analysis.

## ## Sending Images to LLMs

Use the `images` parameter in `gl.nondet.exec\_prompt()`:

```

```python
from genlayer import *

class ReceiptVerifier(gl.Contract):
    verified: bool

    def __init__(self):
        self.verified = False

    @gl.public.write
    def verify_receipt(self, image_data: bytes, expected_amount: str) -> None:
        def leader_fn():
            return gl.nondet.exec_prompt(
                f"Does this receipt show a payment of {expected_amount}? "
                "Respond as JSON: {{\"matches\": true/false, \"actual_amount\": \"...\"}}",
                images=[image_data], # accepts raw bytes directly
                response_format="json",
            )

        def validator_fn(leaders_res) -> bool:
            if not isinstance(leaders_res, gl.vm.Return):
                return False
            my_result = leader_fn()
            return my_result["matches"] == leaders_res.calldata["matches"]

        result = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)
        self.verified = result["matches"]
...

```

The `images` parameter accepts a sequence of raw `bytes` (e.g., PNG/JPEG data) or `gl.nondet.Image` objects.

Capturing Screenshots from the Web

Combine [web access](/developers/intelligent-contracts/features/web-access) with image processing to screenshot a webpage and analyze it:

```

```python
def check_website_status():
 url = "https://example.com/status-page"
 screenshot = gl.nondet.web.render(url, mode='screenshot')

 return gl.nondet.exec_prompt(
 "Is this status page showing all systems operational? "

```

```

 "Respond as JSON: {{\"all_operational\": true/false}}",
 images=[screenshot],
 response_format="json",
)

result = gl.eq_principle.strict_eq(check_website_status)
` ``

```

## ## Use Cases

- **Visual evidence verification** – insurance claims with photo proof, damage assessment
- **Document analysis** – receipts, invoices, certificates
- **Web monitoring** – screenshot a page and verify its content matches expectations
- **Brand & content compliance** – check if visual content meets guidelines
- **UI verification** – screenshot an app and verify it renders correctly

> **Note:**

> Image processing requires vision-capable LLM models. On the GenLayer network, validators handle model selection – your contract just sends images. In Studio running locally, ensure your configured validators use a model that supports image inputs (e.g., GPT-5, Claude Sonnet).

---

## # Web Access

Source: <https://docs.genlayer.com/developers/intelligent-contracts/features/web-access>

Web Access lets Intelligent Contracts fetch external data, send HTTP requests, render web pages, capture screenshots, and handle API errors from non-deterministic blocks. Because leaders and validators make independent web requests, use stable fields or derived summaries when applying the Equivalence Principle to web data.

## ## HTTP Requests

Send data to external services:

```

` ``python
def post_request():
 url = "https://test-server.genlayer.com/body/echo"
 response = gl.nondet.web.request(
 url,
 method='POST',
 body={}
)
 return response.status_code

```

```

status_code = gl.eq_principle.strict_eq(post_request)
` ``

```

## ## Web Rendering

Render web page and extract content:

```

` ``python
def render_page():
 url = "https://test-server.genlayer.com/static/genvm/hello.html"
 # Render HTML content
 html_content = gl.nondet.web.render(url, mode='html')
 return html_content

```

```

page_html = gl.eq_principle.strict_eq(render_page)
` ``

```

## ## Screenshot Capture

Take screenshots of web pages:

```
```python
def take_screenshot():
    url = "https://test-server.genlayer.com/static/genvm/hello.html"
    # Capture page as image
    screenshot = gl.nondet.web.render(url, mode='screenshot')
    return screenshot
```

```
image_data = gl.eq_principle.strict_eq(take_screenshot)
```

Handling HTTP Errors

External APIs can return error responses. Consider checking the status code:

```
```python
def fetch_data():
 response = gl.nondet.web.request(api_url, method='GET')
 if response.status_code >= 400 and response.status_code < 500:
 raise gl.UserError(f"API returned client error: {response.status_code}")
 elif response.status_code >= 500:
 raise gl.UserError(f"API temporarily unavailable: {response.status_code}")
 return json.loads(response.body.decode("utf-8"))
```

## ## Consensus-Friendly Web Requests

When using web data in non-deterministic blocks, remember that the leader and validators make **\*\*independent requests\*\***. External APIs may return different data between calls – timestamps change, counts update, caches vary.

### ### Extract Stable Fields

One approach is to return only the fields that won't change between calls:

```
```python
def leader_fn():
    res = gl.nondet.web.get(api_url)
    data = json.loads(res.body.decode("utf-8"))
    # Only return fields that are stable across requests
    return {"id": data["id"], "login": data["login"], "status": data["status"]}
    # NOT: follower_count, updated_at, online_status
```

Derive Status from Variable Data

When raw data may differ between leader and validator (e.g., CI check counts change), consider comparing a derived summary instead of the raw values:

```
```python
def validator_fn(leaders_res: gl.vm.Result) -> bool:
 validator_data = leader_fn()

 def derive_status(checks):
 if not checks:
 return "pending"
 for c in checks:
 if c.get("conclusion") != "success":
 return "failing"
 return "success"

 return derive_status(leaders_res.calldata) == derive_status(validator_data)
```

See the [Equivalence Principle](/developers/intelligent-contracts/equivalence-principle) page for more validation patterns.

---

## # Development Setup

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tooling-setup>

This guide covers everything you need to develop, test, and deploy Intelligent Contracts – from zero-setup exploration to a full local development environment with linting, testing, and agent-assisted workflows.

## ## Recommended: GenLayer Skills

The [GenLayer Skills](<https://skills.genlayer.com/>) plugin for Claude Code is the fastest path to a working development environment. The `genlayer-dev` skill scaffolds a project, installs prerequisites, runs the linter and direct-mode tests, and walks you through deployment – interactively.

```
```bash copy
claude /plugin marketplace add genlayerlabs/skills
claude /plugin install genlayer-dev@genlayerlabs
```
```

Then run `claude /genlayer-dev` and follow the prompts. The skill bundles the same workflows documented in the rest of this page – it just runs them for you.

The rest of this guide describes the manual setup, the reference path for anyone who wants fine-grained control or is integrating with existing tooling.

## ## Quick Start: GenLayer Studio

The fastest way to start is [studio.genlayer.com](<https://studio.genlayer.com>) – a web-based IDE where you can write, deploy, and interact with Intelligent Contracts with zero setup.

You can also import any deployed contract by address to interact with it directly in the Studio – useful for debugging or testing contracts you've deployed from your local environment.

For anything beyond exploration – local testing, CI, agent-assisted development – set up a local environment below.

## ## Local Environment Setup

### ### Prerequisites

- [Python 3.12+](<https://www.python.org/downloads/>): For contracts, linting, and testing
- [Node.js 18+](<https://nodejs.org/en/download/>): For GenLayer CLI and frontend
- [Docker 26+](<https://docs.docker.com/get-docker/>): For GenLayer Studio (local)

> **Note:**

> Docker is only required if you want to run GenLayer Studio locally. Direct mode tests and GLSim work without Docker.

### ### Clone the Boilerplate

The [GenLayer Project Boilerplate](<https://github.com/genlayerlabs/genlayer-project-boilerplate>) is the recommended starting point. It includes contract templates, testing infrastructure, and a frontend scaffold.

```
```bash
git clone https://github.com/genlayerlabs/genlayer-project-boilerplate.git
cd genlayer-project-boilerplate
```
```

### ### Install Dependencies

```
```bash
pip install -r requirements.txt
```
```

This installs:

- **genlayer-test** – testing framework with direct mode, GLSim, and integration test support
- **genvm-linter** – static analysis for contract safety and correctness

### ### Project Structure

```
...
contracts/ # Your Intelligent Contracts
tests/
 direct/ # Fast in-memory tests (no server needed)
 integration/ # Full tests against GenLayer Studio
frontend/ # Next.js frontend with GenLayerJS
deploy/ # Deployment scripts
gltest.config.yaml # Test network configuration
```
```

Development Workflow

The recommended development loop:

1. Write Your Contract

Create or edit a contract in ``contracts/``:

```
```python
from genlayer import *

class MyContract(gl.Contract):
 data: TreeMap[Address, str]

 def __init__(self):
 self.data = TreeMap()

 @gl.public.view
 def get_data(self, addr: Address) -> str:
 return self.data.get(addr, "")

 @gl.public.write
 def set_data(self, value: str):
 self.data[gl.message.sender_address] = value
...
```
```

2. Lint

Run the linter to catch issues before testing:

```
```bash
genvm-lint check contracts/my_contract.py
```
```

``check`` runs both fast AST-based safety checks and SDK-based semantic validation. It catches:

- **Forbidden imports** – ``os``, ``sys``, ``subprocess``, etc.
- **Non-deterministic calls** outside equivalence principle blocks
- **Invalid storage types** – must use ``TreeMap``/``DynArray``, not ``dict``/``list``
- **Missing decorators** and return type annotations

Use ``--json`` for machine-readable output (useful for CI and agent workflows).

3. Direct Mode Tests

The fastest feedback loop – runs contracts in-memory without any server:

```
```bash
pytest tests/direct/ -v
```

Direct mode provides Foundry-style test fixtures:

```
```python
import json

def test_create_and_read(direct_vm, direct_deploy, direct_alice):
    contract = direct_deploy("contracts/my_contract.py")

    # Set the transaction sender
    direct_vm.sender = direct_alice

    # Call write methods directly
    contract.set_data("hello")

    # Call view methods
    result = contract.get_data(direct_alice)
    assert result == "hello"
...`
```

Mocking Web and LLM Calls

For contracts that use `gl.nondet.web` or `gl.nondet.exec\_prompt`, mock the responses:

```
```python
def test_with_mocks(direct_vm, direct_deploy, direct_alice):
 contract = direct_deploy("contracts/my_contract.py")
 direct_vm.sender = direct_alice

 # Mock web requests (regex matches on URL)
 direct_vm.mock_web(
 r".*api\.example\.com/prices.*",
 {"status": 200, "body": '{"price": 42.5}'},
)

 # Mock LLM responses (regex matches on prompt)
 direct_vm.mock_llm(
 r".*Extract the match result.*",
 json.dumps({"score": "2:1", "winner": 1}),
)

 contract.update_price("ETH/USD")
 assert contract.get_price("ETH/USD") == 42.5

 # Reset mocks between scenarios
 direct_vm.clear_mocks()
...`
```

### #### Testing Failures

```
```python
def test_expected_failure(direct_vm, direct_deploy, direct_alice):
    contract = direct_deploy("contracts/my_contract.py")
    direct_vm.sender = direct_alice

    with direct_vm.expect_revert("Insufficient balance"):
        contract.withdraw(1000)
```

```
...
```

```
Available fixtures: `direct_vm`, `direct_deploy`, `direct_alice`, `direct_bob`,  
`direct_charlie`, `direct_owner`, `direct_accounts`
```

4. Integration Tests

Once direct tests pass, run against a GenLayer environment (see [Environments] (#environments) below) for full end-to-end validation:

```
```bash  
gltest tests/integration/ -v -s
```
```

Integration tests use the `genlayer-test` SDK to deploy and interact with contracts through the JSON-RPC API:

```
```python  
from gltest import get_contract_factory
from gltest.assertions import tx_execution_succeeded

def test_full_flow():
 factory = get_contract_factory("MyContract")
 contract = factory.deploy(args=[])

 # Write methods return transaction receipts
 tx_receipt = contract.set_data(args=["hello"]).transact()
 assert tx_execution_succeeded(tx_receipt)

 # Read methods return values directly
 result = contract.get_data(args=[contract.address]).call()
 assert result == "hello"
...`
```

See the [GenLayer Test SDK reference](/api-references/genlayer-test) for the full API.

### ## Environments

These are the backends you can develop against. Your frontend, integration tests, and deployment scripts all connect to one of these via JSON-RPC.

#### ### GLSim

**\*\*Setup:\*\*** `pip install genlayer-test[sim]`

A lightweight simulator that implements the GenLayer JSON-RPC protocol. Starts in ~1 second, no Docker required. You can point your frontend and integration tests at it for fast iteration.

GLSim runs the Python runner natively – **\*\*not inside GenVM\*\*** – so there can be minor incompatibilities with the full runtime. Use it for fast development cycles, then validate against Studio before deploying.

```
```bash  
# Start the simulator  
glsim --port 4000 --validators 5  
  
# Your frontend connects to http://localhost:4000/api  
# Integration tests run against it  
gltest tests/integration/ -v -s  
```
```

#### ### GenLayer Studio (Local)

**\*\*Setup:\*\*** Docker 26+ and GenLayer CLI

Full GenVM execution with real consensus. You get complete validator logs, transaction inspection, and the Studio UI for interactive debugging. This is where you validate that everything works in the real runtime.

```
```bash
npm install -g genlayer
genlayer init
genlayer up
```
```

Once running, access Studio at <http://localhost:8080/>. Your frontend and tests connect to `http://localhost:4000/api`.

```
```bash
gltest tests/integration/ -v -s --network localnet
```
```

CLI options for `genlayer init` and `genlayer up`

```
`genlayer init` options:
- `--numValidators <number>` – number of validators (default: 5)
- `--headless` – run without Studio UI
- `--reset-db` – start with a clean database
- `--localnet-version <version>` – pin a specific localnet version
```

```
`genlayer up` options:
- `--reset-validators` – recreate all validators
- `--numValidators <number>` – number of validators
- `--headless` – run without Studio UI
- `--reset-db` – clean database
```

### studio.genlayer.com

**\*\*Setup:\*\*** none

Hosted Studio – convenient when you don't want to run Docker locally. Rate limited compared to a local instance. You can also import any deployed contract by address to interact with it directly in the browser.

```
```bash
gltest tests/integration/ -v -s --network studionet
```
```

### Testnet Bradbury

**\*\*Setup:\*\*** account with private keys

The most realistic environment – the actual GenLayer testnet with real validators. You don't get full validator logs like you do with local Studio, so this is best for final pre-production validation rather than active debugging.

```
```bash
gltest tests/integration/ -v -s --network testnet_bradbury
```
```

Configure accounts in `gltest.config.yaml`:

```
```yaml
networks:
  testnet_bradbury:
    accounts:
      - "${ACCOUNT_PRIVATE_KEY_1}"
      - "${ACCOUNT_PRIVATE_KEY_2}"
...
```
```

## ## IDE Support

The [GenLayer VS Code Extension](<https://marketplace.visualstudio.com/items?itemName=genlayer-labs.genlayer>) provides syntax highlighting, snippets, and language support for Intelligent Contracts. Works with VS Code, Cursor, and other VS Code-compatible editors.

## ## Agent-Assisted Development

The boilerplate is designed to work well with AI coding agents. The linter and direct mode tests provide a fast feedback loop that agents can use without spinning up infrastructure.

For Claude Code, install the [GenLayer Skills](<https://skills.genlayer.com/>) marketplace (covered at the top of this page) – it bundles the workflows below behind interactive commands.

For other IDEs and clients, the GenLayer MCP servers provide the same capabilities via the Model Context Protocol:

The [GenLayer MCP Server](<https://www.npmjs.com/package/genlayer-mcp>) provides contract generation and GenLayer-specific guidance directly in your IDE:

```
```bash
# Add to Claude Code
claude mcp add genlayer npx -- -y genlayer-mcp
```
```

The **GenLayer Docs MCP Server** gives your AI agent searchable access to the full GenLayer documentation and SDK reference – so it can look up APIs, patterns, and examples while writing code:

```
```bash
# Add to Claude Code
claude mcp add genlayer-docs --transport sse https://docs-mcp.genlayer.com/sse
```
```

This is a hosted service – no local setup required. It provides a `search\_docs` tool that searches across both the [GenLayer documentation](<https://docs.genlayer.com>) and the [GenLayer SDK reference](<https://sdk.genlayer.com>). Compatible with any MCP client (Claude Code, Cursor, Windsurf, etc.).

The boilerplate also includes a `CLAUDE.md` file pre-configured with commands, architecture context, and testing patterns – so agents understand the project structure immediately.

## ## Frontend Development

The boilerplate includes a Next.js frontend with GenLayerJS integration. See the [GenLayerJS SDK documentation](</api-references/genlayer-js>) for the full API reference.

```
```bash
cd frontend
npm install
npm run dev
```
```

For building a frontend from scratch:

```
```bash
npm install genlayer-js
```
```

```
```javascript
import { createClient } from 'genlayer-js';
```

```

const client = createClient({ endpoint: 'http://localhost:4000/api' });

// Read from a contract
const value = await client.readContract({
  address: contractAddress,
  functionName: "get_data",
  args: [],
});

// Write to a contract
const txHash = await client.writeContract({
  address: contractAddress,
  functionName: "set_data",
  args: [newValue],
});
const receipt = await client.waitForTransactionReceipt({
  hash: txHash,
  status: "FINALIZED",
});

```

Your First Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/first-contract>

While a GenLayer Intelligent Contract is a pure Python program, there are few things that must be present in your file.

Version Comment

First of all, you need to place a magic comment with the version of GenVM you wish to use on the **first** line of your file

```

`py
# { "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfh8jzp09h6" }

```

It is similar to Solidity's ``pragma solidity``. The hash after the colon identifies the exact version of the Python runtime to use, ensuring your contract always runs against the same environment.

Importing the Standard Library

After the version comment, you can import the GenLayer standard library:

```

`py
from genlayer import *

```

This will import all necessary types into the global scope, and everything else under ``gl`` namespace.

The Contract Class

An Intelligent Contract is a regular Python class, with a regular constructor and methods, which allows you to use your favorite IDE for type checking and auto completion. However, there are some additional requirements to make a class an Intelligent Contract.

Decorators

The GenVM makes use of decorators to identify the contract class and its methods.

The contract class must extend ``gl.Contract`` class so that GenVM knows that it's an Intelligent Contract. There can be only one contract class in a file.

Furthermore, public methods must be decorated with either ``@gl.public.view`` for read-only methods or ``@gl.public.write`` (or ``@gl.public.write.payable``) for methods that modify storage. Constructor (``__init__``) must be private (not decorated)

Persistent data

The GenVM enables Intelligent Contracts to maintain persistent data. This data is stored on the blockchain and can be accessed and modified via transactions.

This is done by declaring fields in the contract class.

> **Note:**

> All persistent fields must be declared in the class body and annotated with types.

>

Fields declared outside the class body by creating new instance variables (``self.field = value``) are not persistent and will be discarded after the contract execution.

Types

In your contracts, you can use any Python types, but for persisted fields, there are some restrictions:

- ``list[T]`` needs to be replaced with ``DynArray[T]``

- ``dict[K, V]`` needs to be replaced with ``TreeMap[K, V]``

- ``int`` type isn't supported on purpose. You most likely wish to use some fixed-size integer type, such as ``i32`` or ``u256``. If this is not the case and you are sure that you need big integers, you can annotate your field with ``bigint``, which is just an alias for python ``int``

> **Note:**

> Only fully instantiated generic types can be used, so ``TreeMap`` is forbidden, while ``TreeMap[str, u256]`` is not

Example

Here is a simple example of an Intelligent Contract that stores a name and allows changing it:

```
```py
```

```
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8jzp09h6" }
```

```
from genlayer import *
```

```
class Hello(gl.Contract):
```

```
 name: str
```

```
 def __init__(self, name: str):
 self.name = name
```

```
 @gl.public.view
```

```
 def run(self) -> str:
 return f'Hello, {self.name}'
```

```
 @gl.public.write
```

```
 def set_name(self, name: str):
 print(f'debug old name: {self.name}') # <- you can use prints for debugging
 # they will be included in the GenVM execution log
 self.name = name
```

```
...
```

> **Note:**

> The GenLayer Studio automatically detects the constructor parameters from your code. When you run your contract in the Studio, it provides the UI for all parameters for both constructor and methods, making it easy to manage and modify these inputs.

---

#### # Address Type

Source: <https://docs.genlayer.com/developers/intelligent-contracts/types/address>

The ``Address`` type is a 20-byte blockchain address type for Intelligent Contracts, similar to Ethereum addresses. It is one of the most important types for handling user accounts, contract addresses, and cross-contract interactions.

## ## Creating Address Instances

Use these patterns to instantiate an `Address` from different encodings. Choose the one that matches your source data. Construction validates size and normalizes the value so later comparisons and serializations are consistent.

```
```python
# Constructing Address values from common encodings
# From hex string (most common)
address1 = Address("0x742d35Cc6634C0532925a3b8D4C9db96C4b4d8b6")
address2 = Address("0x0000000000000000000000000000000000000000") # Zero address (useful as
a sentinel/null)

# From base64 encoded string (handy when integrating with systems that emit base64)
address3 = Address("WzjaanAcVoVF3PywP8uHX1a+3cQ=")

# From bytes (20 bytes, useful for low-level calls or pre-parsed data)
address4 =
Address(b'\x5b\x38\xda\x6a\x70\x1c\x56\x85\x45\xdc\xfc\xb0\x3f\xcb\x87\x5f\x56\xbe\xdd\xc4')
```
```

## ## Address Properties and Conversions

`Address` exposes multiple read-only views for common encodings. Use these when you need a specific serialization for UI, storage, hashing, or interop with external systems.

```
```python
class AddressConverter(gl.Contract):
    # Demonstrates converting a stored Address into several representations and when to use
    them
    stored_address: Address

    def __init__(self, initial_address: str):
        # Store as Address (not string) so the value is validated and normalized once
        self.stored_address = Address(initial_address)

    @gl.public.view
    def get_address_hex(self) -> str:
        # Get EIP-55 checksum hex; ideal for UI display, logs, and user input round-trips
        return self.stored_address.as_hex

    @gl.public.view
    def get_address_bytes(self) -> bytes:
        # Get raw 20-byte representation; efficient for hashing and compact storage
        return self.stored_address.as_bytes

    @gl.public.view
    def get_address_b64(self) -> str:
        # Get base64 representation; useful for URLs and systems that prefer base64
        return self.stored_address.as_b64

    @gl.public.view
    def get_address_string(self) -> str:
        # Default string representation (same as as_hex)
        return str(self.stored_address)

    @gl.public.view
    def format_address(self, fmt: str) -> str:
        # Format with a specifier for consistent serialization across callers
        if fmt == "x":
            return f"{self.stored_address:x}" # hex (checksummed)
        elif fmt == "b64":
            return f"{self.stored_address:b64}" # base64 string
        elif fmt == "cd":

```

```

        return f"{self.stored_address:cd}" # calldata/ABI-friendly format
    else:
        return str(self.stored_address)
...

```

Address in Contract State

Store `Address` values directly in contract state for role management and registries. Collections like `DynArray[Address]` and `TreeMap[key, Address]` help model allowlists and name-to-address lookups safely.

```

...python
class AddressStorage(gl.Contract):
    # Store owner/admin roles and registries using Address-typed fields and collections
    # Single address storage
    owner: Address
    admin: Address

    # Address collections
    authorized_users: DynArray[Address]
    contract_registry: TreeMap[str, Address]

    def __init__(self):
        # Initialize roles to the transaction sender to establish initial permissions
        self.owner = gl.message.sender_address
        self.admin = gl.message.sender_address

    @gl.public.write
    def add_authorized_user(self, user_address: str):
        # Convert input to Address so membership checks use a normalized value
        address = Address(user_address)
        self.authorized_users.append(address)

    @gl.public.write
    def register_contract(self, name: str, contract_address: str):
        # Keep a normalized Address in the registry for consistent lookups
        address = Address(contract_address)
        self.contract_registry[name] = address

    @gl.public.view
    def is_authorized(self, user_address: str) -> bool:
        # Membership check with Address ensures exact, normalized equality semantics
        address = Address(user_address)
        return address in self.authorized_users
...

```

Address in Method Parameters and Returns

Methods often accept human-friendly strings and return normalized `Address` data. Convert inputs at the boundary to keep state consistent and make API responses easy for clients to consume.

```

...python
class AddressOperations(gl.Contract):
    # Use Address as the key to guarantee consistent equality and map behavior
    balances: TreeMap[Address, u256]

    def __init__(self):
        # Start with an empty map; balances default to zero for missing keys
        pass

    @gl.public.write
    def set_balance_from_hex(self, account_hex: str, amount: int):
        # Convert caller-provided hex into Address to validate and normalize before use

```



```

        account = Address(account_hex)
        self.balances[account] = amount

    @gl.public.view
    def get_my_balance(self) -> int:
        # Look up the balance associated with the message sender's Address
        return self.balances.get(gl.message.sender_address)

    @gl.public.view
    def get_balance(self, account_hex: str) -> int:
        # Convert external input to Address so lookups match normalized keys
        account = Address(account_hex)
        return self.balances.get(account)

    @gl.public.view
    def get_my_address(self) -> str:
        # Return the sender's address as a string; convenient for clients/UI
        return str(gl.message.sender_address)

...

---

# Primitive Types
Source: https://docs.genlayer.com/developers/intelligent-contracts/types/primitive

```

Primitive Types in GenVM are Python-compatible value types used in Intelligent Contracts, including sized integers for efficient storage and type annotations on storage fields. Sized integer types enforce range constraints only when values are assigned to storage.

```

> **Note:**
> **Important**: Sized integer types (`u8`, `u256`, etc.) are `typing.NewType` aliases that
behave like regular Python `int` outside of storage context. Range checking and overflow
protection only occur when assigning values to storage fields.

```

Integer Types

Standard Integer Types (Most Commonly Used)

- `u8`, `u16`, `u32`, `u64`, `u128`, `u256` - Unsigned integers of various sizes
- `i8`, `i16`, `i32`, `i64`, `i128`, `i256` - Signed integers of various sizes
- `bigint` - Arbitrary precision integer (use with caution, prefer sized integers)

Full Range Available

GenVM also supports all intermediate sizes from 8-bit to 256-bit (e.g., `u24`, `u40`, `u48`, `u56`, `u72`, `u80`, `u88`, `u96`, `u104`, `u112`, `u120`, `u136`, `u144`, `u152`, `u160`, `u168`, `u176`, `u184`, `u192`, `u200`, `u208`, `u216`, `u224`, `u232`, `u240`, `u248` and their signed counterparts).

```
```python
```

```
class IntegerTypes(gl.Contract):
```

```

 uint8_val: u8
 uint256_val: u256
 uint24_val: u24 # 3-byte integers
 int64_val: i64

```

```

 def __init__(self):
 # Storage assignment enforces type constraints
 self.uint8_val = 255 # Range checked: must be 0-255
 self.uint256_val = 2**256 - 1 # Range checked: must fit in u256
 self.int64_val = -9223372036854775808 # Range checked: must fit in i64
 self.uint24_val = 16777215 # Range checked: must be 0-16777215

```

```
@gl.public.view
```

```

def get_uint8(self) -> int:
 return self.uint8_val

@gl.public.write
def set_uint256(self, value: int):
 self.uint256_val = value

@gl.public.view
def calculate_sum(self, a: int, b: int) -> int:
 # Note: u256(a) + u256(b) behaves exactly like a + b in Python
 # The u256() calls don't actually enforce range constraints here
 return a + b

@gl.public.view
def get_int64(self) -> int:
 return self.int64_val

@gl.public.view
def get_uint24(self) -> int:
 return self.uint24_val
...

String and Bytes Types

```python
class StringBytesTypes(gl.Contract):
    text_data: str
    binary_data: bytes

    def __init__(self):
        self.text_data = "Hello, GenLayer!"
        self.binary_data = b"binary_data_here"

    @gl.public.write
    def store_text(self, text: str):
        self.text_data = text

    @gl.public.view
    def get_text_length(self) -> int:
        return len(self.text_data) # Return type annotation handles the conversion

    @gl.public.view
    def get_bytes_length(self) -> int:
        return len(self.binary_data) # Return type annotation handles the conversion

    # String operations
    @gl.public.view
    def concatenate_strings(self, str1: str, str2: str) -> str:
        return str1 + str2
...

## Boolean Type

```python
class BooleanTypes(gl.Contract):
 flag: bool
 flags: DynArray[bool]
 flag_map: TreeMap[str, bool]

 def __init__(self):
 self.flag = True

 @gl.public.write
 def toggle_flag(self):

```

```

 self.flag = not self.flag

 @gl.public.write
 def set_flag(self, flag_value: bool):
 self.flag = flag_value

 @gl.public.write
 def set_named_flag(self, name: str, value: bool):
 self.flag_map[name] = value

 @gl.public.view
 def get_flag_status(self) -> bool:
 return self.flag

 @gl.public.view
 def check_named_flag(self, name: str) -> bool:
 return self.flag_map.get(name, False)
...

```

## ## Type Conversion Utilities

```

```python
class TypeConversionUtils(gl.Contract):
    def __init__(self):
        pass

    @gl.public.view
    def string_to_bytes(self, text: str) -> bytes:
        return text.encode('utf-8')

    @gl.public.view
    def bytes_to_string(self) -> str:
        return b"binary_data_here".decode('utf-8')

    @gl.public.view
    def int_to_string(self, value: int) -> str:
        return str(value)

    @gl.public.view
    def string_to_int(self, value_str: str) -> int:
        return int(value_str)
...

```

Type Ranges

Type	Range	Use Case
`u8`	0 to 255	Small counters, flags
`u16`	0 to 65,535	Medium counters
`u32`	0 to 4,294,967,295	Large counters, timestamps
`u64`	0 to 18,446,744,073,709,551,615	Very large numbers
`u128`	0 to $2^{128} - 1$	Cryptographic values
`u256`	0 to $2^{256} - 1$	Token amounts, hashes
`u160`	0 to $2^{160} - 1$	Address integers

Type	Range	Use Case
`i8`	-128 to 127	Small signed values
`i16`	-32,768 to 32,767	Medium signed values
`i32`	-2,147,483,648 to 2,147,483,647	Large signed values
`i64`	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Very large signed values
`i128`	-2^{127} to $2^{127} - 1$	Large signed calculations
`i256`	-2^{255} to $2^{255} - 1$	Maximum signed precision

Collection Types

Source: <https://docs.genlayer.com/developers/intelligent-contracts/types/collections>

Collection Types in GenVM are storage-compatible replacements for Python's built-in collections, used to manage dynamic data structures in Intelligent Contracts. GenVM provides these collection types so contract state can use array-like and map-like structures while remaining compatible with storage.

DynArray (Dynamic Arrays)

`DynArray[T]` replaces Python's `list[T]` and provides dynamic array functionality with storage compatibility.

Basic DynArray Usage

```
```python
class ArrayOperations(gl.Contract):
 # Simple arrays
 numbers: DynArray[u256]
 names: DynArray[str]
 addresses: DynArray[Address]

 # Nested arrays
 matrix: DynArray[DynArray[u256]]

 def __init__(self):
 pass

 @gl.public.write
 def add_number(self, num: int):
 self.numbers.append(num)

 @gl.public.write
 def add_name(self, name: str):
 self.names.append(name)

 @gl.public.write
 def add_address(self, addr: str):
 address = Address(addr)
 self.addresses.append(address)

 @gl.public.view
 def get_number_at(self, index: int) -> int:
 if index < len(self.numbers):
 return self.numbers[index]
 return u256(0)

 @gl.public.view
 def get_array_length(self) -> int:
 return u256(len(self.numbers))

 @gl.public.write
 def remove_last(self):
 if len(self.numbers) > 0:
 self.numbers.pop()

 # Matrix operations
 @gl.public.write
 def add_row(self, row: DynArray[int]):
 self.matrix.append(row)

 @gl.public.view
 def get_matrix_element(self, row: int, col: int) -> int:
```

```

 if row < len(self.matrix) and col < len(self.matrix[row]):
 return self.matrix[row][col]
 return u256(0)
 ...

```

### ### DynArray with Custom Types

```

`python
from genlayer import *
from dataclasses import dataclass

import json
import typing

@allow_storage
@dataclass
class User:
 name: str
 age: u8
 balance: u256

class UserArrayOperations(gl.Contract):
 users: DynArray[User]
 user_ids: DynArray[str]

 def __init__(self):
 pass

 @gl.public.write
 def add_user(self, name: str, age: int, balance: int):
 user = User(name=name, age=age, balance=balance)
 self.users.append(user)
 self.user_ids.append(f"user_{len(self.users)}")

 @gl.public.view
 def find_user_by_name(self, name: str) -> TreeMap[str, typing.Any]:
 for user in self.users:
 if user.name == name:
 return user
 return User("", u8(0), u256(0))
 ...

```

### ## TreeMap (Ordered Maps)

`TreeMap[K, V]` replaces Python's `dict[K, V]` and provides ordered map functionality with storage compatibility.

### ### Basic TreeMap Usage

```

`python
class MapOperations(gl.Contract):
 # Simple maps
 balances: TreeMap[Address, u256]
 names: TreeMap[str, str]
 flags: TreeMap[str, bool]

 def __init__(self):
 pass

 @gl.public.write
 def set_balance(self, address: str, amount: int):
 account = Address(address)
 self.balances[account] = amount

 @gl.public.write

```

```

def set_name(self, key: str, name: str):
 self.names[key] = name

@gl.public.view
def get_balance(self, address: str) -> int:
 account = Address(address)
 return self.balances.get(account, u256(0))

@gl.public.view
def get_name(self, key: str) -> str:
 return self.names.get(key, "")

@gl.public.view
def has_balance(self, address: str) -> bool:
 account = Address(address)
 return account in self.balances
...

TreeMap with Complex Values

```python
@allow_storage
@dataclass
class UserProfile:
    name: str
    email: str
    balance: u256
    is_active: bool

class UserProfileMap(gl.Contract):
    profiles: TreeMap[Address, UserProfile]
    permissions: DynArray[str]
    user_permissions: TreeMap[Address, DynArray[str]]

    def __init__(self):
        pass

    @gl.public.write
    def create_profile(self, name: str, email: str):
        address = gl.message.sender_address
        profile = UserProfile(
            name=name,
            email=email,
            balance=u256(0),
            is_active=True
        )
        self.profiles[address] = profile
        self.user_permissions[address] = self.permissions

    @gl.public.write
    def add_permission(self, user: str, permission: str):
        account = Address(user)
        if account in self.user_permissions:
            self.user_permissions[account].append(permission)

    @gl.public.view
    def get_profile(self, address: str) -> TreeMap[str, typing.Any]:
        account = Address(address)
        return self.profiles.get(account, UserProfile("", "", u256(0), False))

    @gl.public.view
    def has_permission(self, user: str, permission: str) -> bool:
        account = Address(user)
        if account in self.user_permissions:
            return permission in self.user_permissions[account]

```

```
...     return False
```

```
---
```

```
# Dataclasses
```

```
Source: https://docs.genlayer.com/developers/intelligent-contracts/types/dataclasses
```

Dataclasses are structured Python data types for GenLayer Intelligent Contracts, used to model method parameters, return values, and stored contract data. Use the `@allow_storage` decorator to make a dataclass compatible with contract storage.

```
## Method Parameters and Returns
```

```
```python
import typing

@allow_storage
@dataclass
class UserData:
 name: str
 balance: u256

class SimpleContract(gl.Contract):
 users: TreeMap[Address, UserData]

 def __init__(self):
 pass

 @gl.public.write
 def create_user(self, name: str, balance: int):
 self.users[gl.message.sender_address] = UserData(name, balance)

 @gl.public.view
 def get_user(self, user_address: str) -> TreeMap[str, typing.Any]:
 address = Address(user_address)
 return self.users.get(address, UserData("", u256(0)))
...

```

```
Generic Types
```

```
```python
import typing

@allow_storage
@dataclass
class Item[T]:
    data: T
    label: str

class GenericContract(gl.Contract):
    string_items: DynArray[Item[str]]

    def __init__(self):
        pass

    @gl.public.write
    def add_item(self, data: str, label: str):
        item = gl.storage.inmem_allocate(Item[str], data, label)
        self.string_items.append(item)

    @gl.public.view
    def get_items(self) -> DynArray[TreeMap[str, typing.Any]]:
        return self.string_items
...

```

Persisting data on the blockchain

Source: <https://docs.genlayer.com/developers/intelligent-contracts/storage>

Usual data structures aren't suitable for representing blockchain persistent storage:

1. Allocated addresses (`id` in python terms) are not persistent
2. Allocation requires knowledge about all allocated addresses, which takes a lot of space and would cost a lot of reads at start time
3. Serialization works poorly as it will rewrite entire storage (consider rehash)

Intelligent Contracts store data publicly on chain, attached to their account's address. The storage starts zero-initialized until a contract is deployed and initializes a state.

For storage declaration GenLayer uses contract class fields.

> **Note:**

> All persistent fields must be declared in the class body and annotated with types.

>

Fields declared outside the class body by creating new instance variables (`self.field = value`) are not persistent and will be discarded after the contract execution.

Example:

```
```py
class PersistentContract(gl.Contract):
 minter: Address

 def __init__(self):
 self.minter = gl.message.sender_address
...

```

In your contracts, you can use any Python types, but for persisted fields, there are some restrictions:

- `list[T]` needs to be replaced with `DynArray[T]`
- `dict[K, V]` needs to be replaced with `TreeMap[K, V]`
- `int` type isn't supported on purpose. You most likely wish to use some fixed-size integer type, such as `i32` or `u256`. If this is not the case and you are sure that you need big integers, you can annotate your field with `bigint`, which is just an alias for python `int`

> **Note:**

> Only fully instantiated generic types can be used, so `TreeMap` is forbidden, while `TreeMap[str, u256]` is not

Simple examples:

```
```py
class PersistentContract(gl.Contract):
    a: str
    b: bytes
    # c: list[str]           # ❌ `list` is forbidden!
    c: DynArray[str]
    # b: dict[Address, u256] # ❌ `dict` is forbidden!
    # b: TreeMap            # ❌ only fully specialized generic types are allowed!
    b: TreeMap[Address, u256]
    # d: int                # ❌ `int` is forbidden
    d: bigint              # ⚠️ most likely you don't need an arbitrary big integer
    d_sized: i256
...

```

Few words about `DynArray` and `TreeMap`

These types implement python `collections.abc.MutableSequence` and `collections.abc.MutableMapping` which makes them compatible with most of the python code

They can be encoded into calldata as-is as well, which means that following code is correct:

```
```py
class PersistentContract(gl.Contract):
 storage: DynArray[str]

 @gl.public.view
 def get_complete_storage(self) -> collections.abc.Sequence[str]:
 return self.storage
...

> **Note:**
> Calldata format supports mappings only with `str` keys, like JSON does.

Using custom data types
```

You can use other python classes in storage, for example:

```
```py
@allow_storage
@dataclass
class User:
    name: str
    birthday: datetime.datetime

class Contract(gl.Contract):
    users: DynArray[User]
...

```

Note that you must decorate them with `@allow\_storage`. This is done to prevent [confusion] (#differences-from-regular-python-types)

However, there is a tricky case: allocating storage generics in-memory. It is different from regular python syntax because storage types don't have type erasure due to fixed memory layout.

```
```py
@allow_storage
@dataclass
class User:
 data: TreeMap[str, str]

```

```
User() # error: data is absent (from dataclass)
User(gl.storage.inmem_allocate(TreeMap[str, str])) # works fine, this function takes a type
and `*args, **kwargs` for corresponding `__init__`. Also note that type must be fully
instantiated and have no type variables.
```

```

Default values

By default storage is zero-initialized:

| Type | Default value |
|------------|---------------|
| `u*`, `i*` | `0` |
| `bool` | `false` |
| `float` | `+0` |
| `str` | `` |
| `DynArray` | `[]` |
| `TreeMap` | `{}` |

Struct types are zero-initialized "recursively"

```
---
## Memory Management
```

```
### `gl.storage.inmem_allocate`
```

Required for ****generic storage classes**** only. Regular dataclasses use normal initialization.

```
```python
```

```
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
```

```
from genlayer import *
from dataclasses import dataclass
import datetime
```

```
@allow_storage
@dataclass
class User:
 name: str
 birthday: datetime.datetime
```

```
@allow_storage
@dataclass
class Gen[T]:
 name: T
 birthday: datetime.datetime
```

```
class Contract(gl.Contract):
 user: User
 gen_user: Gen[bytes]

 def __init__(self):
 pass

 @gl.public.write
 def plain(self):
 user = User('Ada', datetime.datetime.now())
 self.user = user

 read_user = self.user
 copied_out = gl.storage.copy_to_memory(read_user)

 def nd():
 print('inmem: ok', user)
 try:
 print('storage: not ok', str(read_user))
 except Exception as e:
 print('storage: not ok', e)
 print('copied out: ok', copied_out)

 gl.eq_principle.strict_eq(nd)
```

```
@gl.public.write
def generic(self):
 user = gl.storage.inmem_allocate(Gen[bytes], b'Ada', datetime.datetime.now())
 self.gen_user = user

 read_user = self.gen_user
 copied_out = gl.storage.copy_to_memory(read_user)

 def nd():
 print('inmem: ok', user)
 try:
 print('storage: not ok', str(read_user))
 except Exception as e:
 print('storage: not ok', e)
 print('copied out: ok', copied_out)
```

```
... gl.eq_principle.strict_eq(nd)
```

```
`gl.storage.copy_to_memory`
```

Storage objects need to be copied to memory when you want to use them in non-deterministic blocks, since non-deterministic blocks cannot access storage directly:

```
```python
# Read from storage
storage_user = self.user

# Copy to memory for use in non-deterministic blocks or other operations
memory_user = gl.storage.copy_to_memory(storage_user)

def my_nondet_function():
    # Storage objects cannot be used directly in nondet blocks
    # print('storage: not accessible', storage_user) # Error - storage not accessible!

    # But memory objects work fine
    print('copied out: ok', memory_user) # Works!
    return str(memory_user) # String conversion also works with memory objects

gl.eq_principle.strict_eq(my_nondet_function)
```

```
---
```

Differences from regular python types

Even though storage classes mimic python types, remember that they provide you only with a view on memory, not actual data that is "here". For example, consider the above example

```
```py
self.users.append(User("Ada"))
user = self.users[-1]
self.users[-1] = User("Definitely not Ada", datetime.datetime.now())
assert user.name == "Definitely not Ada" # this is true!
```
```

```
---
```

Your First **Intelligent** Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/first-intelligent-contract>

An Intelligent Contract in GenLayer can contain non-deterministic blocks: regular Python functions with no arguments that may return arbitrary values and must be invoked through ``gl.eq_principle.*``. For blockchain integrity reasons, non-determinism must be contained within these special non-deterministic blocks, and the blocks have two important limitations:

- Storage is inaccessible from non-deterministic blocks
- State of the Python interpreter is not passed back to the deterministic code (for instance, you won't see changes in global variables)

Simple Case

To illustrate how non-deterministic blocks work, this example gets a webpage as plain HTML and verifies that it has a link to the owner:

```
```py
example_web_address = 'https://example.org'
def my_non_deterministic_block():
 web_data = gl.nondet.web.render(example_web_address, mode='html')
 return 'iana' in web_data

print(gl.eq_principle.strict_eq(my_non_deterministic_block))
```
```

Here are a few important parts:

1. It is **mandatory** to call ``gl.nondet.web.render`` (or ``gl.nondet.web.get``) from a function invoked via ``gl.eq_principle.*``, otherwise it will give an error
2. Type annotations are not required
3. ``example_web_address`` gets captured without the need to specify it explicitly
4. We are getting the page in plain HTML, because we want text from a link (``<a>`` HTML tag), which is not visible in the text representation
5. We are using ``gl.eq_principle.strict_eq`` because we return a ``bool`` (``True`` or ``False``), so there is no need to run LLMs or other complex computations: validators will agree ``_if_`` they both get the exactly the same result, which makes sense for our case. However, if you wish to return more complex data (such as a text summary), you should use other ``gl.eq_principle`` methods. More on this topic on the next page

As a Full Contract

```
```py
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
```

```
from genlayer import *
```

```
class Contract(gl.Contract):
```

```
 had_iana: bool
```

```
 def __init__(self):
```

```
 example_web_address = 'https://example.org'
```

```
 def my_non_deterministic_block():
```

```
 web_data = gl.nondet.web.render(example_web_address, mode='html')
```

```
 return 'iana' in web_data
```

```
 self.had_iana = gl.eq_principle.strict_eq(my_non_deterministic_block)
```

```
...
```

```

```

# The Equivalence Principle

Source: <https://docs.genlayer.com/developers/intelligent-contracts/equivalence-principle>

The Equivalence Principle is GenLayer's method for reaching consensus on non-deterministic operations, including web requests, LLM calls, or any computation that might produce different results on different nodes.

Under the Equivalence Principle, a **leader** executes the operation and proposes a result, then **validators** independently verify whether that result is acceptable.

## Quick Reference: Which Pattern to Use

```
...
```

Can validators reproduce the exact same normalized output?

```
|— YES → strict_eq
```

```
| Exact match. Use when outputs are deterministic or can be
| canonicalized (e.g., JSON with sort_keys=True).
```

```
| Examples: blockchain RPC, stable REST APIs.
```

```
|— NO → Write a custom validator function (run_nondet_unsafe)
```

```
| Default: produce independent evidence. Usually rerun the same task
| and compare decision fields, derived status, scores, or other stable
| outputs with explicit tolerances. Only skip the second answer when the
| validator can judge the leader output against source data and criteria.
```

```
...
```

For most contracts, you'll write a custom validator function. It gives you full control over comparison logic and error handling.

> **Note:**

> GenLayer also provides ``prompt_comparative`` and ``prompt_non_comparative`` as convenience

wrappers for common patterns, but in practice most contracts outgrow them quickly. Starting with a custom validator function gives you full flexibility from the start.

## ## Independent Verification Is Required

Never treat the leader's result as trusted input. A validator must verify the substance of the leader's answer using evidence other than the leader's answer alone:

- Re-run the same LLM/web task and compare the stable decision fields
- Fetch the same source data and independently derive the status being stored
- Run an explicit comparative LLM judgment over the leader output and validator output
- For open-ended outputs, judge the leader output against the same input/source data and explicit criteria

A validator that only checks `leader\_result.calldata` for a valid JSON shape, allowed enum value, non-empty summary, or confidence in range is not performing consensus. That is leader-output-only validation: it proves the leader formatted the answer correctly, but it does not verify the answer itself.

> **Warning:**

> Non-comparative validation does **not** mean "trust the leader." It means the validator does not produce a second candidate answer. It still must read the same input/source data and decide whether the leader output is valid under clear criteria.

## ## The Leader/Validator Pattern

Every non-deterministic operation in GenLayer is built on two functions:

```
```python
def leader_fn() -> T:
    # Fetch data, call an LLM, compute something
    return result

def validator_fn(leader_result) -> bool:
    # Independently verify the leader's result
    # Returns True to accept, False to reject
    return is_acceptable
...```
```

The leader's result is only accepted if a majority of validators agree. If the majority rejects, the network rotates to a different leader and retries. If consensus still can't be reached, the transaction goes **undetermined** – it does not modify contract state.

> **Note:**

> **What gets stored?** The accepted leader result is the value your contract receives and can store. Validators verify or reject that leader result; their independent intermediate answers are not automatically persisted on-chain. If your application needs to expose multiple perspectives, make those perspectives explicit fields in the leader result and write validator logic that checks them.

```
```mermaid
graph TD
 subgraph "Step 1: Leader"
 L[execute leader_fn → result]
 end
 L -->|leader_result| V
 subgraph "Step 2: Validator"
 V[verify leader_result]
 V --> C{Acceptable?}
 C -->|yes| A[Accept]
 C -->|no| R[Reject]
 end
...```
```

The typical validator pattern is: **re-run the leader function independently, then compare**

the results\*\*. How you compare determines which validation pattern you're using.

> **Note:**

> **Always extract before comparing.** Raw web data varies between nodes (caching, cookies, dynamic content) and is expensive to write to the GenLayer chain – whatever data the leader returns has to be stored on-chain. The typical pattern is: **fetch web data → LLM extraction → return structured data**, all within the same non-deterministic block.

## ## Validation Patterns

### ### Pattern 1: Partial Field Matching

Often your leader function returns structured data with both an **analysis** (subjective text) and a **decision** (objective fields). Two LLMs will produce different analysis text, but should agree on the decision. Compare only the fields that matter:

```
```python
@gl.public.write
def resolve_match(self, match_id: str):
    match = self.matches[match_id]

    def leader_fn():
        web_data = gl.nondet.web.get(match.source_url)
        prompt = f"""
        Analyze this match and determine the result.
        Teams: {match.team1} vs {match.team2}
        Page content: {web_data.body}
        Return JSON: {{
            "analysis": "your reasoning about the match result",
            "score": "X:Y",
            "winner": 1 or 2 or 0 for draw, or -1 if not finished
        }}
        """
        response = gl.nondet.exec_prompt(prompt)
        # In production, add retry/validation around JSON parsing
        return json.loads(response)

    def validator_fn(leader_result) -> bool:
        if not isinstance(leader_result, gl.vm.Return):
            return False
        validator_data = leader_fn()
        leader_data = leader_result.calldata
        # Only compare the decision fields – analysis text will differ
        return (
            leader_data["winner"] == validator_data["winner"]
            and leader_data["score"] == validator_data["score"]
        )

    result = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)

    if result["winner"] == -1:
        raise gl.vm.UserError("Game not finished")
    self.matches[match_id].winner = result["winner"]
    self.matches[match_id].score = result["score"]
    self.matches[match_id].analysis = result["analysis"]
...
```
```

The `analysis` field is stored but not compared – two LLMs will word their reasoning differently. The `winner` and `score` fields are the decision and must match exactly.

> **Note:**

> If you only need the decision fields (not the analysis), you can use `strict_eq` instead – see [Convenience Functions](#strict-equality-strict\_eq).

### ### Pattern 2: Numeric Tolerance

When results are numeric but may vary slightly between nodes, compare within a threshold.

This is common for two reasons:

- **Time drift**: prices change between leader and validator execution
- **LLM subjectivity**: quality scores from different LLMs rarely match exactly

#### #### Price Oracle

The most common use case – fetching a price that may drift between when the leader and validator execute:

```
```python
@gl.public.write
def update_price(self, pair: str):
    url = f"https://api.example.com/prices/{pair}"

    def leader_fn():
        response = gl.nondet.web.get(url)
        data = json.loads(response.body)
        return data["price"]

    def validator_fn(leader_result) -> bool:
        if not isinstance(leader_result, gl.vm.Return):
            return False
        validator_price = leader_fn()
        leader_price = leader_result.calldata
        # 2% tolerance – price may drift between leader and validator execution
        if leader_price == 0:
            return validator_price == 0
        return abs(leader_price - validator_price) / abs(leader_price) <= 0.02

    self.prices[pair] = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)
```
```

#### #### Quality Scoring

For LLM-generated scores, use absolute tolerance – two LLMs rating the same content rarely give identical scores:

```
```python
@gl.public.write
def evaluate_submission(self, submission_id: str):
    submission = self.submissions[submission_id]

    def leader_fn():
        web_data = gl.nondet.web.get(submission.content_url)
        prompt = f"""
        Rate the quality of this content on a scale of 0-10.
        Guidelines: {submission.guidelines}
        Content: {web_data.body}
        Return JSON: {{ "score": N, "analysis": "brief explanation" }}
        """
        response = gl.nondet.exec_prompt(prompt)
        return json.loads(response)

    def validator_fn(leader_result) -> bool:
        if not isinstance(leader_result, gl.vm.Return):
            return False
        validator_data = leader_fn()
        leader_score = leader_result.calldata["score"]
        validator_score = validator_data["score"]
        # Gate: if either scores 0 (reject), both must agree on rejection
        if leader_score == 0 or validator_score == 0:
            return leader_score == validator_score
        # Otherwise allow ±1 tolerance

```

```

        return abs(leader_score - validator_score) <= 1

    result = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)
    self.submissions[submission_id].score = result["score"]
    self.submissions[submission_id].analysis = result["analysis"]
...

```

The gate check (score 0) ensures that if one node thinks the content should be rejected outright, both must agree – you don't want a ± 1 tolerance to turn a rejection into an acceptance.

Pattern 3: LLM Comparison (Comparative)

When results are complex (text, structured analysis) and can't be reduced to numbers, you can use an LLM to decide whether two outputs are equivalent.

The simplest way is `prompt\_comparative`:

```

...python
result = gl.eq_principle.prompt_comparative(
    evaluate_single_source,
    principle="`outcome` field must be exactly the same. All other fields must be similar",
)
...

```

This uses a special `EqComparative` prompt template – not a regular `gl.nondet.exec\_prompt` call. Node operators can customize these templates to tune how their validators evaluate equivalence, improving judgment quality over time. This is a key advantage over writing your own comparison prompts.

For more control while keeping this benefit, use the template directly in a custom `run\_nondet\_unsafe` validator. This lets you combine LLM-based comparison with programmatic checks:

```

> **Note:**
> The imports below use internal module paths (`_internal`, `_decode_nondet`). The template
functionality is stable and used by all convenience functions, but the import path may
change in future releases.

```

```

...python
import genlayer.gl._internal.gl_call as gl_call
from genlayer.gl.nondet import _decode_nondet

@gl.public.write
def analyze_event(self, event_id: str):
    event = self.events[event_id]

    def leader_fn():
        web_data = gl.nondet.web.get(event.source_url)
        prompt = f"""
        Analyze this event and determine the outcome.
        Title: {event.title}
        Possible outcomes: {event.outcomes}
        Page content: {web_data.body}
        Return JSON: {{
            "reasoning": "your detailed analysis",
            "outcome": "chosen outcome or UNDETERMINED"
        }}
        """
        response = gl.nondet.exec_prompt(prompt)
        return json.loads(response)

    def validator_fn(leader_result) -> bool:
        if not isinstance(leader_result, gl.vm.Return):
            return False

```



```

validator_data = leader_fn()

# Use the EqComparative template – node operators can customize this
verdict = gl_call.gl_call_generic(
    {
        'ExecPromptTemplate': {
            'template': 'EqComparative',
            'leader_answer': format(leader_result.calldata),
            'validator_answer': format(validator_data),
            'principle': "`outcome` must match exactly. Reasoning may differ.",
        },
    },
    _decode_nondet,
).get()

return verdict

result = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)
self.events[event_id].outcome = result["outcome"]
self.events[event_id].analysis = result["reasoning"]
...

```

The `EqComparative` template sends both answers and your principle to the validator's LLM, which returns `true`/`false`. Because this goes through the template system, node operators can fine-tune the comparison prompt for their specific LLM – getting better judgment over time without any contract changes.

****When to use:**** Results are rich (text + structured data) and you need natural-language equivalence judgment – "outcomes must match", "key facts must agree", "sentiments must be the same".

> ****Note:****

> If comparative LLM comparison is too loose or too strict, consider whether you can reduce the problem to [partial field matching](#pattern-1-partial-field-matching) or [numeric tolerance](#pattern-2-numeric-tolerance) – those give you deterministic, programmatic control.

Pattern 4: Source-Grounded Non-Comparative Validation

In rare cases, you may not want the validator to produce a second candidate answer. Instead, the validator ****evaluates the leader's output**** against the same input/source data and explicit criteria.

```

```mermaid
graph TD
 task[Task & criteria]
 input[Input function]

 subgraph Leader
 input_leader[Execute input function] --> leader["Perform task (LLM)"]
 end

 subgraph Validator
 input_validator[Execute input function] --> validator["Validate output (LLM)"]
 end

 task --> leader
 task --> validator
 input --> Leader
 input --> Validator

 leader --> Output
 Output --> validator
 Output --> final_result[Final result]
 validator -.->|accept/reject| final_result

```

...

Note that the validator **does not write its own final answer**. It still executes the input function and uses that input to judge whether the leader's output satisfies the criteria.

The simplest way is ``prompt_non_comparative``:

```
```python
@gl.public.write
def summarize_article(self, url: str):
    result = gl.eq_principle.prompt_non_comparative(
        lambda: gl.nondet.web.get(url).body.decode("utf-8"),
        task="Summarize this article in 2-3 sentences",
        criteria="""
            Summary must capture the main point of the article
            Must not include information not present in the source
            Must be 2-3 sentences long
        """)
    self.summaries[url] = result
...

```

Under the hood, this uses two special templates:

- `**`EqNonComparativeLeader`**`: takes the input + task + criteria → produces the output
- `**`EqNonComparativeValidator`**`: takes the input + leader's output + task + criteria → judges validity

For more control, use these templates directly. Here the leader summarizes an article, and the validator uses the ``EqNonComparativeValidator`` template to judge the summary:

```
```python
import genlayer.gl._internal.gl_call as gl_call
from genlayer.gl.nondet import _decode_nondet

@gl.public.write
def summarize_article(self, url: str):
 task = "Summarize this article in 2-3 sentences"
 criteria = """
 Summary must capture the main point of the article.
 Must not include information not present in the source.
 Must be 2-3 sentences long.
 """

 def leader_fn():
 web_data = gl.nondet.web.get(url).body.decode("utf-8")
 result = gl_call.gl_call_generic(
 {
 'ExecPromptTemplate': {
 'template': 'EqNonComparativeLeader',
 'task': task,
 'input': web_data,
 'criteria': criteria,
 }
 },
 _decode_nondet,
).get()
 return result

 def validator_fn(leader_result) -> bool:
 if not isinstance(leader_result, gl.vm.Return):
 return False
 web_data = gl.nondet.web.get(url).body.decode("utf-8")
 verdict = gl_call.gl_call_generic(
 {
 'ExecPromptTemplate': {

```

```

 'template': 'EqNonComparativeValidator',
 'task': task,
 'input': web_data,
 'output': leader_result.calldata,
 'criteria': criteria,
 }
 },
 _decode_nondet,
).get()
return verdict

... self.summaries[url] = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)

```

The validator never writes its own summary – it only judges whether the leader's summary is faithful to the source. And because it uses the `EqNonComparativeValidator` template, node operators can tune the judgment prompt for their LLM.

```

> **Note:**
> Non-comparative validation is rare in practice. Most use cases are better served by
patterns 1-3 where the validator independently reproduces the result. Non-comparative is
most useful when the output is open-ended and there's no meaningful way to compare two
independent results – e.g., summarization, where two valid summaries can be completely
different yet both correct.

> **Warning:**
> Do not use non-comparative validation as a schema check. A validator that only accepts
`authentic`, `suspicious`, or `inconclusive`; checks that `confidence` is between 0 and 100;
and requires a non-empty summary is still trusting the leader's decision. For
classification, scoring, extraction, authenticity, safety, ranking, and settlement logic,
validators should almost always re-run or independently derive the answer, then compare the
decision field, extracted fields, score bucket, or derived status.

```

## `run\_nondet` vs `run\_nondet\_unsafe`

GenLayer provides two variants for custom leader/validator logic. The difference is **who** handles validator errors.

When writing custom leader/validator patterns, **use `run\_nondet\_unsafe`** and handle errors yourself inside the validator. This is what production contracts do – it gives you full control over error classification and comparison logic. If the validator throws an unhandled exception, it counts as `Disagree` (same as returning `False`).

```

```python
result = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)
```

```

**`gl.vm.run\_nondet`** is primarily used internally by the [convenience functions] (#convenience-functions). It wraps the validator in a sandbox – if the validator throws, the sandbox catches it and compares the error against the leader's error using configurable comparison functions:

```

```python
result = gl.vm.run_nondet(
    leader_fn,
    validator_fn,
    # Optional: customize how errors are compared (default: message equality)
    compare_user_errors=my_user_error_comparator,
    compare_vm_errors=my_vm_error_comparator
)
```

```

|  |                                   |                           |     |                    |     |
|--|-----------------------------------|---------------------------|-----|--------------------|-----|
|  |                                   | `gl.vm.run_nondet_unsafe` |     | `gl.vm.run_nondet` |     |
|  | ---                               |                           | --- |                    | --- |
|  | **Validator errors**              |                           |     |                    |     |
|  | Unhandled exceptions = `Disagree` |                           |     |                    |     |
|  | Caught by sandbox, compared       |                           |     |                    |     |

automatically |  
| **\*\*Error handling\*\*** | You implement it inside `validator\_fn` | Built-in with  
`compare\_user\_errors` / `compare\_vm\_errors` callbacks |  
| **\*\*Use for\*\*** | Custom leader/validator patterns (recommended) | Convenience functions and  
simple validators where built-in error comparison suffices |

### ### Advanced Error Handling with `run\_nondet\_unsafe`

When your contract makes external calls that can fail in different ways, you may want to classify errors and handle each type differently. With `run\_nondet\_unsafe`, you implement this inside the validator:

```
```python
ERROR_EXPECTED = "[EXPECTED]"      # Business logic errors (deterministic)
ERROR_EXTERNAL = "[EXTERNAL]"      # External API errors (deterministic)
ERROR_TRANSIENT = "[TRANSIENT]"    # Temporary failures (timeouts, 5xx)
ERROR_LLM = "[LLM_ERROR]"          # LLM/GenVM errors (non-deterministic)

def _handle_leader_error(leaders_res, leader_fn) -> bool:
    """Re-run leader_fn on validator and compare errors."""
    leader_msg = leaders_res.message if hasattr(leaders_res, 'message') else ''
    try:
        leader_fn()
        return False # Leader errored but validator succeeded – disagree
    except gl.vm.UserError as e:
        validator_msg = e.message if hasattr(e, 'message') else str(e)
        # Deterministic errors: must match exactly
        if validator_msg.startswith(ERROR_EXPECTED) or
validator_msg.startswith(ERROR_EXTERNAL):
            return validator_msg == leader_msg
        # Transient errors: both transient = agree
        if validator_msg.startswith(ERROR_TRANSIENT) and
leader_msg.startswith(ERROR_TRANSIENT):
            return True
        # LLM errors or unknown: disagree, force retry
        return False
    except Exception:
        return False
...
```
```

Use this helper in your validator:

```
```python
def validator_fn(leaders_res) -> bool:
    if not isinstance(leaders_res, gl.vm.Return):
        return _handle_leader_error(leaders_res, leader_fn)
    validator_result = leader_fn()
    return abs(leaders_res.calldata["score"] - validator_result["score"]) <= 1
...
```
```

This gives fine-grained control:

- **\*\*Expected/external errors\*\*** (e.g., "issue not found"): must match exactly
- **\*\*Transient errors\*\*** (e.g., API timeout): if both nodes fail transiently, agree
- **\*\*LLM errors\*\***: always disagree – force retry with different validators

### ### The Validator's Result Parameter

The validator function receives a `gl.vm.Result` which can be one of:

- **\*\*`gl.vm.Return[T]`\*\*** – leader succeeded; access the value via `.calldata`
- **\*\*`gl.vm.UserError`\*\*** – leader raised an application error
- **\*\*`gl.vm.VMError`\*\*** – leader hit a VM-level error (e.g., out of memory)

Always check the type before accessing the result:

```
```python
```

```
def validator_fn(leader_result) -> bool:
    if not isinstance(leader_result, gl.vm.Return):
        return False # reject if leader errored
    data = leader_result.calldata
    # ... verify data
...
```

Convenience Functions

GenLayer provides built-in equivalence functions for common patterns. These are shortcuts so you don't have to write leader/validator pairs manually.

Strict Equality (`strict\_eq`)

All validators execute the same function. Results must match **exactly**. Uses ``run_nondet_unsafe`` under the hood.

```
```python
def fetch_match_result():
 web_data = gl.nondet.web.get(resolution_url)
 prompt = f"""
 Find the match result for {team1} vs {team2}.
 Page: {web_data.body}
 Return JSON: {"score": "X:Y", "winner": 1 or 2 or 0}
 """
 result = gl.nondet.exec_prompt(prompt)
 return json.dumps(json.loads(result), sort_keys=True)

result = json.loads(gl.eq_principle.strict_eq(fetch_match_result))
```
```

Use when: results are objective and should be identical – API data, boolean decisions, structured data where you don't need separate analysis text.

> **Note:**
 > Note the ``sort_keys=True`` – JSON key ordering can vary between nodes. Sorting ensures exact string comparison works. If you need to compare only some fields or allow tolerance, use a custom leader/validator pattern instead.

Comparative (`prompt\_comparative`)

Both leader and validators perform the same task, then a **special LLM prompt template** compares their results against a principle you define. Uses ``run_nondet`` under the hood.

```
```python
result = gl.eq_principle.prompt_comparative(
 evaluate_source,
 principle="`outcome` must be exactly the same. All other fields must be similar"
)
```
```

The comparison uses a built-in ``EqComparative`` prompt template that node operators can customize. The LLM receives the leader's answer, the validator's answer, and your principle, then returns true/false.

Use when: results are complex (text + data) and you need natural-language equivalence criteria – "key facts must match", "conclusions must agree", "numerical values within 10%".

Non-Comparative (`prompt\_non\_comparative`)

The leader performs a task, and validators evaluate the leader's output against criteria – **without repeating the task themselves**. Uses ``run_nondet`` under the hood.

```
```python
result = gl.eq_principle.prompt_non_comparative(
```

```

lambda: gl.nondet.web.get(url).body.decode("utf-8"),
task="Summarize this article in 2-3 sentences",
criteria="""
 Summary must capture the main point of the article
 Must not include information not present in the source
 Must be 2-3 sentences long
 """)
)

```

Parameters:

- `fn` – function that provides the input data (runs on both leader and validator)
- `task` – instruction for the leader's LLM
- `criteria` – rules the validator's LLM uses to judge the leader's output against the input data

**Use when:** the output is open-ended and validity can be judged against the input/source data without producing a second candidate output. Summaries are the clearest example: many different summaries can be valid, but the validator can still check faithfulness, coverage, hallucinations, and constraints. For classification, scoring, extraction, authenticity, safety, ranking, or settlement decisions, prefer comparative validation unless you can clearly explain how the validator independently verifies the decision from source data.

## ## Writing Secure Validators

The validator's job is to **prevent malicious or incorrect data** from being accepted. A validator that always returns `True` defeats the entire consensus mechanism – it would let a single malicious node set any result.

**Bad – accepts anything:**

```

python
def validator(leader_result):
 return True # Insecure! Leader can return arbitrary data

```

**Bad – validates only the leader's formatting:**

```

python
def validator(leader_result):
 if not isinstance(leader_result, gl.vm.Return):
 return False
 data = leader_result.calldata
 return (
 data.get("decision") in ("authentic", "suspicious", "inconclusive")
 and isinstance(data.get("confidence"), int)
 and 0 <= data["confidence"] <= 100
 and isinstance(data.get("summary"), str)
 and len(data["summary"]) > 0
)

```

This validator checks that the output looks valid, but it never verifies whether the decision follows from the source data. The leader still decides alone.

**Good – independent verification:**

```

python
def validator(leader_result):
 if not isinstance(leader_result, gl.vm.Return):
 return False
 my_data = leader_fn() # re-run independently
 return abs(leader_result.calldata - my_data) <= tolerance

```

Guidelines:

1. **Never trust the leader** – verify against source data or an independently computed result

2. **\*\*Tolerate nondeterminism\*\*** – use thresholds for scores, percentage tolerance for prices, field-level comparison for structured data
3. **\*\*Check error types\*\*** – handle `UserError` and `VMError` before accessing `.calldata`
4. **\*\*Reject when in doubt\*\*** – security first

---

## # Debugging Intelligent Contracts in GenLayer Studio

Source: <https://docs.genlayer.com/developers/intelligent-contracts/debugging>

Debugging Intelligent Contracts on GenLayer involves deploying contracts, sending transactions, validating their behavior, and identifying issues using logs. Here is some guidance on how to debug using the GenLayer Studio.

### ## 2. Debugging in GenLayer Studio

When testing in GenLayer Studio, you can debug issues by examining the logs and outputs. The Studio provides detailed logs for every transaction and contract execution.

#### ### Getting Static Code Syntax Errors

If you get a syntax error in your contract, you can see the error on a red panel in the Studio in the **\*\*Run and Debug\*\*** tab.

Here is an example of a syntax error in line 42 of the Wizard of Coin contract:

![Syntax Error](/studio/syntax-error.png)

#### ### Viewing Logs

##### #### Access Logs

The logs are located in the bottom section of the Studio in the **\*\*Run and Debug\*\*** tab.

##### #### Filter Relevant Logs

Use filters to isolate logs related to specific transactions or contracts. You can select the debug level (info, success, error), the execution layer (RPC Server, GenVM, and Consensus), and the transaction hash.

Also, you can select a transaction from the left list and see the logs for that specific transaction.

#### ### Identifying Common Issues

##### #### Input Validation Errors

- Look for incorrect or missing parameters in the transaction payload

##### #### Contract Logic Errors

- Debug issues in contract logic by examining state changes and variable values in logs

##### #### Non-deterministic Output Issues

- Analyze LLM or web data interactions for discrepancies or timeouts

##### #### Consensus Errors

- Investigate validator disagreements or timeout errors to identify potential misconfigurations

---

## # Testing Intelligent Contracts

Source: <https://docs.genlayer.com/developers/intelligent-contracts/testing>

The [GenLayer Testing Suite](<https://pypi.org/project/genlayer-test/>) (`genlayer-test`) is a pytest-based framework for testing Intelligent Contracts. It provides two execution modes to match your workflow.

### ## Installation

```
```bash
pip install genlayer-test
```
```

### ## Two Modes at a Glance

|                          | Direct Mode                                                      | Studio Mode                               |
|--------------------------|------------------------------------------------------------------|-------------------------------------------|
| ----- ----- -----        |                                                                  |                                           |
| -----                    |                                                                  |                                           |
| <b>**How it works**</b>  | Runs contract code in-memory (no network)                        | Deploys to GenLayer Studio via RPC        |
| <b>**Speed**</b>         | Milliseconds per test                                            | Minutes per test                          |
| <b>**Prerequisites**</b> | Python 3.12+                                                     | Python 3.12+ and GenLayer Studio (Docker) |
| <b>**Best for**</b>      | Unit tests, rapid iteration, CI/CD consensus validation, testnet | Integration tests,                        |
| <b>**Mocking**</b>       | `mock_web` / `mock_llm` cheatcodes                               | Mock validators with LLM/web responses    |

```
> **Note:**
> **Start with Direct Mode.** It runs in milliseconds, requires no Docker, and covers the vast majority of contract logic. Add Studio Mode tests only when you need multi-validator consensus or full-network behavior.
```

### ## Direct Mode

Direct Mode runs your contract Python code in-process -- no simulator, no Docker required.

### ### Quick Start

```
```python
# tests/test_storage.py

def test_storage(direct_deploy):
    # Deploy the contract in-memory
    storage = direct_deploy("contracts/Storage.py", "initial value")

    # Call view methods directly
    assert storage.get_storage() == "initial value"

    # Call write methods directly
    storage.update_storage("updated")
    assert storage.get_storage() == "updated"
...
```
```

Run with pytest:

```
```bash
pytest tests/ -v
```
```

### ### Fixtures

Direct Mode provides built-in pytest fixtures:

| Fixture                                        | Description                 |
|------------------------------------------------|-----------------------------|
| ----- -----                                    |                             |
| `direct_vm`                                    | VM context with cheatcodes  |
| `direct_deploy`                                | Deploy a contract in-memory |
| `direct_alice`, `direct_bob`, `direct_charlie` | Predefined test addresses   |
| `direct_owner`                                 | Default sender address      |
| `direct_accounts`                              | List of 10 test addresses   |



### ### Cheatcodes

The `direct\_vm` fixture exposes cheatcodes for controlling test execution:

#### #### Changing the Sender

```
```python
def test_access_control(direct_vm, direct_deploy, direct_alice, direct_bob):
    contract = direct_deploy("contracts/MyContract.py")

    # Set sender permanently
    direct_vm.sender = direct_alice
    contract.owner_action() # Called as alice

    # Prank: temporarily change sender for a single call
    with direct_vm.prank(direct_bob):
        with direct_vm.expect_revert("Unauthorized"):
            contract.owner_action() # Reverts -- bob is not owner
...

```

Snapshots and Revert

```
```python
def test_state_isolation(direct_vm, direct_deploy):
 contract = direct_deploy("contracts/Counter.py")

 snap_id = direct_vm.snapshot()
 contract.increment()
 assert contract.get_count() == 1

 direct_vm.revert(snap_id)
 assert contract.get_count() == 0 # State fully restored
...

```

Snapshots capture full state: storage, mocks, sender, and validators.

#### #### Expecting Reverts

```
```python
def test_insufficient_balance(direct_vm, direct_deploy, direct_alice):
    contract = direct_deploy("contracts/Token.py", direct_alice, 100)

    with direct_vm.expect_revert("Insufficient balance"):
        contract.transfer(direct_alice, 999)
...

```

Mocking Web and LLM Calls

Non-deterministic calls (`gl.nondet.web`, `gl.nondet.exec\_prompt`) must be mocked in Direct Mode. Use regex patterns to match URLs and prompt text.

```
```python
def test_price_feed(direct_vm, direct_deploy):
 # Mock a web response (regex pattern matches the URL)
 direct_vm.mock_web(
 r"api\.example\.com/price",
 {"status": 200, "body": '{"price": 42.50}'},
)

 contract = direct_deploy("contracts/PriceFeed.py")
 contract.update_price()
 assert contract.get_price() == 4250 # Stored as integer
...

```python

```

```
def test_sentiment_analysis(direct_vm, direct_deploy):
    # Mock an LLM response (regex matches the prompt text)
    direct_vm.mock_llm(r"classify.*sentiment", "positive")

    contract = direct_deploy("contracts/Sentiment.py")
    contract.analyze("I love GenLayer!")
    assert contract.get_sentiment() == "positive"
...

> **Warning:**
> Set `direct_vm.strict_mock = True` to raise an error if any registered mock is never
matched. This catches stale or misspelled patterns before they hide bugs.
```

Testing Validator Consensus

Verify that your equivalence principle produces consistent results across validators:

```
``python
def test_consensus_agreement(direct_vm, direct_deploy):
    direct_vm.mock_llm(r".*", '{"verdict": "true"}')

    contract = direct_deploy("contracts/FactChecker.py")
    # Run as the leader -- captures the validator function internally
    contract.check_claim("The sky is blue")

    # Swap mocks to simulate a dissenting validator
    direct_vm.clear_mock()
    direct_vm.mock_llm(r".*", '{"verdict": "false"}')
    assert direct_vm.run_validator() is False # Validator disagrees -> undetermined
...

```

Studio Mode

Studio Mode deploys your contracts to a running GenLayer Studio instance and interacts via RPC. Use it when you need:

- Multi-validator consensus with real network behavior
- Verification on `localnet` or `studionet`
- Pre-testnet integration checks

Prerequisites

- GenLayer Studio running locally (`genlayer up`)
- Python 3.12+

Quick Start

```
``python
from gltest import get_contract_factory
from gltest.assertions import tx_execution_succeeded

# `default_account` is a pre-provided pytest fixture supplied by genlayer-test for Studio
Mode
def test_contract_integration(default_account):
    factory = get_contract_factory("Storage")
    contract = factory.deploy(args=["initial"])

    tx = contract.update_storage(args=["new value"]).transact()
    assert tx_execution_succeeded(tx)

    result = contract.get_storage().call()
    assert result == "new value"
...

```

Run with the `gltest` CLI:

```
```bash
gltest tests/ -v
gltest --network studionet
gltest --leader-only # Skip consensus validation (faster)
```
```

For the full Studio Mode API -- mock validators, LLM/web responses, multi-network configuration -- see the [genlayer-test API Reference](/api-references/genlayer-test).

Testing Strategy

Structure your test suite in layers:

1. **Pure storage tests first** -- verify `__init__`, view methods, and write methods that do not call `gl.nondet`. These run instantly and catch most logic bugs.
2. **Mock non-deterministic calls** -- add `mock_web` / `mock_llm` to test the full execution flow with controlled outputs. Cover both happy paths and edge cases (empty responses, unexpected LLM output, HTTP errors).
3. **Consensus tests** -- use `direct_vm.run_validator()` to confirm your equivalence principle produces agreement on typical inputs. Also verify that validators disagree on inputs designed to be ambiguous.
4. **Studio Mode last** -- run a smaller set of integration tests against `localnet` in CI to verify end-to-end behavior with real validators.

> **Tip:**

> Enable `direct_vm.check_pickling = True` to catch serialization bugs early. GenLayer stores contract state by pickling Python objects -- any custom class not decorated with `@allow_storage` and `@dataclass` will fail at runtime.

Deploying Intelligent Contracts

Source: <https://docs.genlayer.com/developers/intelligent-contracts/deploying>

Deploying Intelligent Contracts on GenLayer means choosing a deployment method and network for moving Python-based contracts from local development toward testnet use. This guide covers local development through testnet deployment, including CLI direct deployment, deploy scripts, network configuration, and how to use each approach effectively.

Quick Start

Get started quickly with these essential deployment options:

- [CLI Deployment](./deploying/cli-deployment)
Deploy contracts directly from the command line with simple commands
</Card>
- [Deploy Scripts](./deploying/deploy-scripts)
Use TypeScript/JavaScript scripts for complex deployment workflows
</Card>
- [Network Configuration](./deploying/network-configuration)
Configure networks, manage settings, and understand deployment environments
</Card>

Deployment Approaches

GenLayer offers two primary deployment methods:

CLI Direct Deployment

Perfect for quick deployments and simple contracts. Deploy with a single command:

```
```bash
genlayer deploy --contract contracts/my_contract.py --args "Hello World" 42
```

[Learn more about CLI Deployment →](./deploying/cli-deployment)

### ### Deploy Scripts

Ideal for complex workflows, multi-contract deployments, and testnet environments:

```
```typescript
export default async function main(client: GenLayerClient<any>) {
  // Deploy and configure multiple contracts
  const mainContract = await deployContract(client, "contracts/main.py", []);
  const helperContract = await deployContract(client, "contracts/helper.py",
[mainContract]);

  await configureContracts(client, mainContract, helperContract);
}
```
```

[Learn more about Deploy Scripts →](./deploying/deploy-scripts)

## ## Networks Overview

Deploy to different networks based on your development stage:

| Network                | Purpose                | When to Use                                        |
|------------------------|------------------------|----------------------------------------------------|
| <b>Localnet</b>        | Local development      | Development, debugging, initial testing            |
| <b>StudioNet</b>       | Hosted development     | Team collaboration, quick prototyping              |
| <b>TestnetAsimov</b>   | Infrastructure testing | Stability, scalability, and stress testing         |
| <b>TestnetBradbury</b> | AI/LLM testing         | Real AI workloads and intelligent contract testing |

[Explore Network Configuration →](./deploying/network-configuration)

## ## Development Workflow

Follow this recommended progression:

1. **Develop locally** on `localnet` with full control
2. **Test collaboratively** on `studioNet` with your team
3. **Validate thoroughly** on `testnetBradbury` in a production-like environment with real AI workloads

> **Note:**

> Start with CLI deployment for simple contracts, then graduate to deploy scripts as your projects become more complex.

## ## What's Next?

After deploying your contracts:

1. **Test Contract Functions**: Use `genlayer call` and `genlayer write` to interact with deployed contracts
2. **Monitor Transactions**: Check transaction receipts with `genlayer receipt <txId>`
3. **Build Frontend**: Integrate with your deployed contracts using [GenLayerJS] (<https://github.com/yeagerai/genlayer-js>)
4. **Debug Issues**: Use the [Debugging Guide](./debugging) if you encounter problems

Ready to start deploying? Choose your preferred method and dive into the detailed guides!

---

## # Deployment Methods

Source: <https://docs.genlayer.com/developers/intelligent-contracts/deploying/deployment-methods>

Deployment Methods for GenLayer Intelligent Contracts are the two supported approaches for deploying contracts: CLI direct deployment and deploy scripts. Choose **CLI Direct Deployment** to deploy contracts directly using command-line arguments, or use **Deploy Scripts** with TypeScript/JavaScript scripts for complex deployment workflows.

1. **CLI Direct Deployment**: Deploy contracts directly using command-line arguments
2. **Deploy Scripts**: Use TypeScript/JavaScript scripts for complex deployment workflows

### ## CLI Direct Deployment

The CLI method is perfect for:

- Quick deployments during development
- Simple contracts with minimal setup
- Testing and debugging scenarios
- One-off contract deployments

#### **\*\*Benefits:\*\***

- Fast and straightforward
- No additional files required
- Direct command-line interface
- Immediate feedback

### ## Deploy Scripts

The deploy script method is ideal for:

- Complex deployment workflows
- Multi-contract deployments
- Testnet environments
- Repeatable deployment processes

#### **\*\*Benefits:\*\***

- Version control friendly
- Complex logic support
- Environment-specific configurations
- Error handling and rollback capabilities
- Integration with CI/CD pipelines

### ## Choosing the Right Method

| Scenario           | Recommended Method | Reason                       |
|--------------------|--------------------|------------------------------|
| Local development  | CLI Direct         | Quick iteration and testing  |
| Single contract    | CLI Direct         | Simple and efficient         |
| Multiple contracts | Deploy Scripts     | Better orchestration         |
| Testnet deployment | Deploy Scripts     | More control and reliability |
| CI/CD integration  | Deploy Scripts     | Automation friendly          |

### ## Next Steps

- Learn about [Network Configuration](./network-configuration) to choose the right deployment target
- Explore [CLI Deployment](./cli-deployment) for direct deployment
- Discover [Deploy Scripts](./deploy-scripts) for advanced workflows

---

## # Network Configuration

Source: <https://docs.genlayer.com/developers/intelligent-contracts/deploying/network-configuration>

GenLayer network configuration defines which network your GenLayer CLI deployments and development workflows use. GenLayer supports multiple networks, each serving specific purposes in the development lifecycle:

## ## Localnet

**\*\*Purpose\*\*:** Local development and testing

- **\*\*When to use\*\*:** During development, debugging, and initial testing
- **\*\*Setup\*\*:** Requires running the GenLayer Studio locally with ``genlayer init`` and ``genlayer up``
- **\*\*URL\*\*:** ``http://localhost:4000/api`` (default)
- **\*\*Benefits\*\*:**
  - Full control over validators and network configuration
  - No external dependencies
  - Fast transaction processing
  - Ability to reset database and validators
  - Built-in faucet to fund accounts with GEN (💧 button in the account selector)

## ### Setting up Localnet

```
```bash
# Initialize local network
genlayer init

# Start the local network
genlayer up
```
```

For complete setup instructions, examples, and video tutorials, see the [GenLayer Studio Guide](../tools/genlayer-studio).

## ## Studionet

**\*\*Purpose\*\*:** Hosted development environment

- **\*\*When to use\*\*:** For development without local setup requirements
- **\*\*Setup\*\*:** Accessible through [studio.genlayer.com/api](https://studio.genlayer.com)
- **\*\*Benefits\*\*:**
  - No local installation required
  - Pre-configured validators
  - Shared development environment
  - Ideal for quick prototyping
  - Built-in faucet to fund accounts with GEN (💧 button in the account selector)

> **\*\*Note:\*\***

> Studionet is perfect for getting started quickly or when you need to share your development environment with others.

## ### Using Studionet

```
```bash
# Set network to studionet
genlayer network studionet

# Deploy to studionet
genlayer deploy --contract contracts/my_contract.py
```
```

## ## TestnetAsimov

**\*\*Purpose\*\*:** Infrastructure testing and stress testing

- **\*\*When to use\*\*:** For infrastructure stability testing, scalability testing, and stress tests
- **\*\*Setup\*\*:** Connect using network configuration
- **\*\*Benefits\*\*:**
  - Stable infrastructure testing environment

- Scalability and stress testing
- Shared testnet for community testing

### ### Using TestnetAsimov

```
```bash
# Set network to testnet
genlayer network testnet-asimov

# Deploy to testnet
genlayer deploy --contract contracts/my_contract.py
```
```

### ## TestnetBradbury

**\*\*Purpose\*\*:** Production-like testing environment with real AI/LLM use cases

- **\*\*When to use\*\*:** For deploying and testing intelligent contracts with real LLM models
- **\*\*Setup\*\*:** Connect using network configuration
- **\*\*Benefits\*\*:**
  - Real AI workloads using powerful LLM models
  - Production-like environment for intelligent contract testing
  - Real use cases and contracts deployed
  - Shared testnet for community testing

> **\*\*Note\*\*:**

> Testnet Bradbury configuration details will be available when the network goes live. The faucet and RPC endpoints are shared with Testnet Asimov.

### ### Getting Testnet Tokens

Before deploying to TestnetBradbury, you'll need testnet tokens. Use the [faucet] (<https://testnet-faucet.genlayer.foundation/>) to get free GEN tokens.

### ### Using TestnetBradbury

```
```bash
# Set network to testnet
genlayer network testnet-bradbury

# Deploy to testnet
genlayer deploy --contract contracts/my_contract.py
```
```

### ## Network Comparison

| Feature                     | Localnet    | Studionet   | TestnetAsimov          | TestnetBradbury |
|-----------------------------|-------------|-------------|------------------------|-----------------|
| <b>**Setup Complexity**</b> | Medium      | None        | Low                    | Low             |
| <b>**Control Level**</b>    | Full        | Limited     | Limited                | Limited         |
| <b>**Persistence**</b>      | Local only  | Temporary   | Persistent             | Persistent      |
| <b>**Collaboration**</b>    | No          | Yes         | Yes                    | Yes             |
| <b>**Performance**</b>      | Fast        | Medium      | Production-like        | Production-like |
| <b>**Best for**</b>         | Development | Prototyping | Infrastructure testing | AI/LLM testing  |

### ## Development Workflow

We recommend following this progression:

1. **\*\*Start with Localnet\*\*:** Develop and test your contracts locally with full control
2. **\*\*Test on Studionet\*\*:** Validate contracts in a shared environment
3. **\*\*Deploy to TestnetBradbury\*\*:** Final testing in a production-like environment with real AI workloads

### ## Managing Network Settings

### ### Viewing Current Configuration

```
```bash
# Show all configuration
genlayer config get

# Show specific network setting
genlayer config get network
```
```

### ### Setting Networks

The GenLayer CLI allows you to set a default network that will be used for all operations unless overridden.

#### \*\*Interactive Network Selection:\*\*

```
```bash
# Set network interactively
genlayer network
```
```

This command will show you available networks and let you choose:

```
...
? Select a network:
> localnet
 studionet
 testnet-asimov
 testnet-bradbury
 custom
...

```

#### \*\*Direct Network Selection:\*\*

```
```bash
# Set specific network directly
genlayer network localnet
genlayer network studionet
genlayer network testnet-asimov
genlayer network testnet-bradbury
```
```

### ### Network Switching Workflow

```
```bash
# Development workflow
genlayer network localnet
genlayer deploy --contract contracts/my_contract.py

# Testing workflow
genlayer network studionet
genlayer deploy --contract contracts/my_contract.py

# Pre-production workflow
genlayer network testnet-bradbury
genlayer deploy --contract contracts/my_contract.py
```
```

### ## Network Configuration File

The CLI stores network configuration in `~/genlayer/genlayer-config.json`:

```
```json
{
  "network": {
    "id": 61999,
```



```

    "name": "Genlayer Localnet",
    "rpcUrls": {
        ...
    }
}
}
},

```

Manual Configuration

If needed, you can edit the configuration file directly:

```

```bash
Open config file
code ~/.genlayer/genlayer-config.json

Or edit with any editor
nano ~/.genlayer/genlayer-config.json
```

```

Next Steps

- Start deploying with [CLI Deployment](./cli-deployment)
- Set up automated deployments with [Deploy Scripts](./deploy-scripts)

```

# CLI Deployment
Source: https://docs.genlayer.com/developers/intelligent-contracts/deploying/cli-deployment

```

CLI Deployment is the process of using the GenLayer CLI to deploy Intelligent Contracts directly from the command line. The GenLayer CLI provides a straightforward `deploy` command for deploying a contract file, optionally selecting an RPC endpoint and passing constructor arguments.

Direct Contract Deployment

Deploy a single contract using the `deploy` command:

```

```bash
genlayer deploy --contract <contractPath> [options]
```

```

Options:

- `--contract <contractPath>`: Path to the Python contract file
- `--rpc <rpcUrl>`: Custom RPC URL (optional)
- `--args <args...>`: Constructor arguments (space-separated)

Basic Examples

Simple Contract Deployment

```

```bash
Deploy a simple contract
genlayer deploy --contract contracts/my_contract.py
```

```

Contract with Constructor Arguments

```

```bash
Deploy with constructor arguments
genlayer deploy --contract contracts/betting_contract.py --args "World Cup 2024" 1000
```

```

Deploy to Specific Networks

You can deploy to different networks by using the `--rpc` option or by setting your default network:

```
```bash
Deploy to localnet with custom RPC
genlayer deploy --contract contracts/my_contract.py --rpc http://localhost:4000/api

Deploy to studionet
genlayer deploy --contract contracts/my_contract.py --rpc https://studio.genlayer.com/api

Deploy scripts to specific network
genlayer deploy --rpc https://custom-network.com/api
```
```

****Alternative: Set default network first****

```
```bash
Set network then deploy
genlayer network testnet-bradbury
genlayer deploy --contract contracts/my_contract.py
```
```

Constructor Arguments

When deploying contracts with constructor parameters, provide them as space-separated arguments:

String Arguments

```
```bash
Single string argument
genlayer deploy --contract contracts/my_contract.py --args Hello
```
```

Multiple Arguments

```
```bash
Multiple arguments (string, number, boolean)
genlayer deploy --contract contracts/my_contract.py --args "Contract Name" 100 true
```
```

Multi-word Strings

```
```bash
Multi-word strings (use quotes)
genlayer deploy --contract contracts/my_contract.py --args "Multi word string" 42
```
```

Argument Types

The CLI automatically handles different argument types:

| Python Type | CLI Example | Notes |
|-------------|-------------------|--------------------------------------|
| `str` | `"Hello World"` | Use quotes for multi-word strings |
| `int` | `42` | Numbers without quotes |
| `float` | `3.14` | Decimal numbers |
| `bool` | `true` or `false` | Lowercase boolean values |
| `list` | Not supported | Use deploy scripts for complex types |

> ****Note:****

> For complex data types like lists, dictionaries, or objects, use [Deploy Scripts] (./deploy-scripts) instead of CLI deployment.

Deployment Output

When a deployment succeeds, you'll see output similar to:

```
```bash
✅ Contract deployed successfully!
Transaction Hash: 0x1234567890abcdef...
Contract Address: 0xabcdef1234567890...
```
```

Quick Deployment Workflow

1. ****Prepare your contract****: Ensure your `.py` file is ready
2. ****Choose your network****: Use `genlayer network` to set the target
3. ****Deploy****: Run the deploy command with appropriate arguments
4. ****Verify****: Note the contract address for future interactions

```
```bash
Example workflow
genlayer network localnet
genlayer deploy --contract contracts/token.py --args "MyToken" "MTK" 1000000
```
```

Next Steps

- Learn about [Deploy Scripts](./deploy-scripts) for more complex deployments
- Configure networks with [Network Configuration](./network-configuration)

Deploy Scripts

Source: <https://docs.genlayer.com/developers/intelligent-contracts/deploying/deploy-scripts>

Deploy scripts are TypeScript or JavaScript files used to automate more complex Intelligent Contract deployment workflows. Use deploy scripts when a deployment needs ordered steps, configuration, initialization, or multiple operations beyond a simple deploy command.

Script Structure

Create TypeScript or JavaScript files in the `deploy/` directory:

```
...
your-project/
├── deploy/
│   ├── 001_deploy_main_contract.ts
│   ├── 002_setup_configuration.ts
│   └── 003_initialize_data.ts
├── contracts/
│   └── my_contract.py
...
```

> ****Note:****

> Deploy scripts are executed in alphabetical order. Use numeric prefixes (001_, 002_) to control execution order.

Basic Deploy Script

Here's a template for a deploy script (`deploy/deployScript.ts`):

```
```typescript
import { readFileSync } from "fs";
import path from "path";
import {
 TransactionHash,
 TransactionStatus,

```

```

 GenLayerClient,
 DecodedDeployData,
 GenLayerChain
} from "genlayer-js/types";
import { testnetBradbury } from "genlayer-js/chains";

export default async function main(client: GenLayerClient<any>) {
 // Read the contract file
 const filePath = path.resolve(process.cwd(), "contracts/my_contract.py");
 const contractCode = new Uint8Array(readFileSync(filePath));

 // Initialize consensus
 await client.initializeConsensusSmartContract();

 // Deploy the contract
 const deployTransaction = await client.deployContract({
 code: contractCode,
 args: [], // Constructor arguments
 });

 // Wait for deployment confirmation
 const receipt = await client.waitForTransactionReceipt({
 hash: deployTransaction as TransactionHash,
 retries: 200,
 });

 // Check deployment success
 if (
 receipt.statusName !== TransactionStatus.ACCEPTED &&
 receipt.statusName !== TransactionStatus.FINALIZED
) {
 throw new Error(`Deployment failed. Receipt: ${JSON.stringify(receipt)}`);
 }

 // Receipt structure differs between testnet and localnet/studionet
 const deployedContractAddress =
 (client.chain as GenLayerChain).id !== testnetBradbury.id
 ? receipt.data.contract_address
 : (receipt.txDataDecoded as DecodedDeployData)?.contractAddress;

 console.log("Contract deployed successfully!", {
 "Transaction Hash": deployTransaction,
 "Contract Address": deployedContractAddress,
 });

 return deployedContractAddress;
}

```

### ## Advanced Deploy Script with Configuration

```

``typescript
import { readFileSync } from "fs";
import path from "path";
import {
 TransactionHash,
 TransactionStatus,
 GenLayerClient,
 DecodedDeployData,
 GenLayerChain
} from "genlayer-js/types";
import { testnetBradbury } from "genlayer-js/chains";

export default async function main(client: GenLayerClient<any>) {
 console.log("🚀 Starting deployment process...");

```

```

// Deploy main contract
const mainContractAddress = await deployContract(
 client,
 "contracts/main_contract.py",
 ["Initial Config", 1000]
);

// Deploy helper contract
const helperContractAddress = await deployContract(
 client,
 "contracts/helper_contract.py",
 [mainContractAddress]
);

// Configure the main contract
await configureContract(client, mainContractAddress, helperContractAddress);

console.log("✅ Deployment completed successfully!");
console.log({
 mainContract: mainContractAddress,
 helperContract: helperContractAddress,
});
}

async function deployContract(
 client: GenLayerClient<any>,
 contractPath: string,
 args: any[] = []
): Promise<string> {
 const filePath = path.resolve(process.cwd(), contractPath);
 const contractCode = new Uint8Array(readFileSync(filePath));

 await client.initializeConsensusSmartContract();

 const deployTransaction = await client.deployContract({
 code: contractCode,
 args,
 });

 const receipt = await client.waitForTransactionReceipt({
 hash: deployTransaction as TransactionHash,
 retries: 200,
 });

 // Check deployment success
 if (
 receipt.statusName !== TransactionStatus.ACCEPTED &&
 receipt.statusName !== TransactionStatus.FINALIZED
) {
 throw new Error(`Deployment failed for ${contractPath}. Receipt:
 ${JSON.stringify(receipt)}`);
 }

 // Receipt structure differs between testnet and localnet/studionet
 const deployedContractAddress =
 (client.chain as GenLayerChain).id !== testnetBradbury.id
 ? receipt.data.contract_address
 : (receipt.txDataDecoded as DecodedDeployData)?.contractAddress;

 return deployedContractAddress;
}

async function configureContract(
 client: GenLayerClient<any>,

```

```

 mainAddress: string,
 helperAddress: string
) {
 // Example configuration call
 const hash = await client.writeContract({
 address: mainAddress as any,
 functionName: "setHelperContract",
 args: [helperAddress],
 value: 0n,
 });

 await client.waitForTransactionReceipt({
 hash,
 retries: 100,
 interval: 5000,
 });
 }
 ...

```

## ## Running Deploy Scripts

Execute all deploy scripts in order:

```

```bash
genlayer deploy
```

```

The CLI will automatically:

1. Find all `.ts` and `.js` files in the `deploy/` directory
2. Sort them numerically by filename prefix (001\_, 002\_, etc.)
3. Execute them in order
4. Pass a configured GenLayer client to each script

## ## Script Organization

### ### Recommended File Naming

```

...
deploy/
├── 001_core_contracts.ts # Core infrastructure
├── 002_token_contracts.ts # Token-related contracts
├── 003_governance.ts # Governance setup
├── 004_configure_system.ts # System configuration
└── 999_verify_deployment.ts # Post-deployment verification
...

```

## ## Error Handling

Always include proper error handling in deploy scripts:

```

```typescript
export default async function main(client: GenLayerClient<any>) {
  try {
    await client.initializeConsensusSmartContract();

    const deployTransaction = await client.deployContract({
      code: contractCode,
      args: [],
    });

    const receipt = await client.waitForTransactionReceipt({
      hash: deployTransaction as TransactionHash,
      retries: 200,
    });
  }
}
```

```

```

// Check deployment success
if (
 receipt.statusName !== TransactionStatus.ACCEPTED &&
 receipt.statusName !== TransactionStatus.FINALIZED
) {
 throw new Error(`Deployment failed: ${JSON.stringify(receipt)}`);
}

// Receipt structure differs between testnet and localnet/studionet
const deployedContractAddress =
 (client.chain as GenLayerChain).id !== testnetBradbury.id
 ? receipt.data.contract_address
 : (receipt.txDataDecoded as DecodedDeployData)?.contractAddress;

return deployedContractAddress;
} catch (error) {
 console.error("Deployment failed:", error);
 throw error;
}
}
}

```

## ## Next Steps

- Learn about different [Network Configuration](./network-configuration)
- Configure CLI deployments with [CLI Deployment](./cli-deployment)

---

## # Prompt & Data Techniques

Source: <https://docs.genlayer.com/developers/intelligent-contracts/crafting-prompts>

Prompt and data techniques for Intelligent Contracts are practices for getting reliable LLM reasoning, web data, and programmatic logic to work together. Use structured outputs, stable data extraction, and verified facts to make LLM judgments easier to validate.

## ## Always Return JSON

The single most impactful technique. Using `response\_format="json"` guarantees the LLM returns parseable JSON, eliminating manual cleanup:

```

```python
def leader_fn():
    prompt = f"""
    You are a wizard guarding a magical coin.
    An adventurer says: {request}

    Should you give them the coin? Respond as JSON:
    {{"reasoning": "your reasoning", "give_coin": true/false}}
    """
    return gl.nondet.exec_prompt(prompt, response_format="json")

def validator_fn(leaders_res) -> bool:
    if not isinstance(leaders_res, gl.vm.Return):
        return False
    my_result = leader_fn()
    # Compare the decision, not the reasoning
    return my_result["give_coin"] == leaders_res.calldata["give_coin"]

result = gl.vm.run_nondet_unsafe(leader_fn, validator_fn)
```

```

Without `response\_format="json"`, LLMs may wrap output in markdown code fences, add commentary, or return malformed JSON. With it, you get a parsed dict directly.

> **Note:**

> Always define the JSON schema in your prompt. `response\_format="json"` ensures valid JSON, but the LLM still needs to know *which* fields to include.

### ## Extract Stable Fields from Web Data

When fetching external data for consensus, the leader and validators make **independent requests**. API responses often contain fields that change between calls – timestamps, view counts, caching headers. Extract only the fields that matter:

```
```python
def leader_fn():
    response = gl.nondet.web.get(github_api_url)
    data = json.loads(response.body.decode("utf-8"))
    # Only return fields that are stable across requests
    return {
        "id": data["id"],
        "title": data["title"],
        "state": data["state"],
        "merged": data.get("merged", False),
    }
    # NOT: updated_at, comments, reactions, changed_files

def validator_fn(leaders_res) -> bool:
    if not isinstance(leaders_res, gl.vm.Return):
        return False
    return leader_fn() == leaders_res.calldata
...
```
```

This is the #1 cause of failed consensus for new developers. If your contract fetches web data and consensus keeps failing, check whether you're returning unstable fields.

### ## Compare Derived Status, Not Raw Data

Sometimes even stable fields can differ between calls – a new CI check run starts, a comment is added. Instead of comparing raw arrays, derive a summary and compare that:

```
```python
def _check_ci_status(self, repo: str, commit_hash: str) -> str:
    url = f"https://api.github.com/repos/{repo}/commits/{commit_hash}/check-runs"

    def leader_fn():
        response = gl.nondet.web.get(url)
        data = json.loads(response.body.decode("utf-8"))
        return [
            {"name": c["name"], "status": c["status"], "conclusion": c.get("conclusion",
"" )}
            for c in data.get("check_runs", [])
        ]

    def validator_fn(leaders_res) -> bool:
        if not isinstance(leaders_res, gl.vm.Return):
            return False
        validator_result = leader_fn()

        # Compare derived status, not raw arrays
        # (check count may differ if new CI run triggered between calls)
        def derive_status(checks):
            if not checks:
                return "pending"
            for c in checks:
                if c.get("status") != "completed":
                    return "pending"
                if c.get("conclusion") != "success":

```



```

        return c.get("conclusion", "failure")
    return "success"

    return derive_status(leaders_res.calldata) == derive_status(validator_result)

checks = gl.vm.run_nondet(leader_fn, validator_fn)

if not checks:
    return "pending"
for c in checks:
    if c.get("conclusion") != "success":
        return c.get("conclusion", "failure")
return "success"
...

```

The key insight: consensus doesn't require identical data – it requires agreement on the **decision**.

Ground LLM Judgments with Programmatic Facts

LLMs hallucinate on character-level checks. Ask "does this text contain an em dash?" and the LLM may say yes when it doesn't, or vice versa. The fix: check programmatically first, then feed the results as ground truth into the LLM prompt.

Step 1: LLM Generates Checkable Rules

Ask the LLM to convert human-readable rules into Python expressions:

```

```python
def _generate_rule_checks(self, rules: str) -> list:
 prompt = f"""Given these rules, generate Python expressions that can
programmatically verify each rule that CAN be checked with code.
Variable `text` contains the post text. Skip subjective rules.

Rules:
{rules}

Output JSON: {{{"checks": [{{{"rule": "...", "expression": "...", "description":
"..."}]}}}"""

 return gl.nondet.exec_prompt(prompt, response_format="json").get("checks", [])

```

# Example output for rules "no em dashes, must mention @BOTCHA, must include botcha.xyz":

```

[
{"rule": "no em dashes", "expression": "'-' not in text", ...},
{"rule": "mention @BOTCHA", "expression": "'@BOTCHA' in text", ...},
{"rule": "include link", "expression": "'botcha.xyz' in text", ...},
]
```

```

Step 2: Eval in a Sandbox

Run the generated expressions deterministically – no hallucination possible:

```

```python
def _eval_rule_checks(self, checks: list, tweet_text: str) -> list:
 def run_checks():
 results = []
 for check in checks:
 try:
 passed = eval(
 check["expression"],
 {"__builtins__": {"len": len}, "text": tweet_text},
)
 results.append({

```

```

 "rule": check["rule"],
 "result": "SATISFIED" if passed else "VIOLATED",
 })
 except Exception:
 pass # skip broken expressions, let LLM handle the rule
 return results

... return gl.vm.unpack_result(gl.vm.spawn_sandbox(run_checks))

```

> **\*\*Note:\*\***

> `gl.vm.spawn\_sandbox` runs a function in an isolated sandbox within the GenVM.  
`gl.vm.unpack\_result` extracts the return value. Together they let you execute dynamically generated code safely. See the [genlayer-py API reference](/api-references/genlayer-py) for details.

### ### Step 3: Inject Ground Truth into LLM Prompt

Feed the verified results back so the LLM focuses on subjective rules and doesn't override programmatic facts:

```

```python
compliance_prompt = f"""
Evaluate this submission for compliance with the campaign rules.

```

```
Submission: {tweet_text}
```

IMPORTANT – PROGRAMMATIC VERIFICATION RESULTS:
These results are GROUND TRUTH from running code on the raw text.
Do NOT override them with your own character-level analysis.

```

<programmatic_checks>
{chr(10).join(f"- {r['rule']}: {r['result']}" for r in programmatic_results)}
</programmatic_checks>

```

For rules NOT listed above, use your own judgment.

```
Respond as JSON: {"compliant": true/false, "violations": ["..."]}
"""
```

```
result = gl.nondet.exec_prompt(compliance_prompt, response_format="json")
...

```

This three-step pattern – ****generate checks → eval deterministically → inject as ground truth**** – eliminates an entire class of LLM errors. Use it whenever your contract needs to verify concrete, checkable facts.

Classify Errors

Use error prefixes to distinguish user mistakes from infrastructure failures. This helps both debugging and error handling logic:

```

```python
ERROR_EXPECTED = "[EXPECTED]" # Business logic errors (deterministic)
ERROR_EXTERNAL = "[EXTERNAL]" # API/network failures (non-deterministic)

In contract methods:
if sender != bounty.owner:
 raise ValueError(f"{ERROR_EXPECTED} Only bounty owner can validate")

if response.status != 200:
 raise ValueError(f"{ERROR_EXTERNAL} GitHub API returned {response.status}")
...

```

`[EXPECTED]` errors mean the transaction should fail consistently across all nodes.

`[EXTERNAL]` errors mean the external service had a problem – the transaction may succeed on retry.

> **Note:**

> For a comprehensive guide on prompt engineering, see the [Prompt Engineering Guide from Anthropic](<https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>).

---

## # Prompt Injection

Source: <https://docs.genlayer.com/developers/intelligent-contracts/security-and-best-practices/prompt-injection>

Prompt injection is the manipulation of prompts fed to AI models to produce unintended outcomes in Intelligent Contracts. Intelligent Contracts exclusively interact with public data, reducing certain types of risks such as data leakage, but preserving contract integrity still requires careful management of inputs and outputs.

## ## Understanding Prompt Injection

Prompt injection involves manipulating the prompts fed to AI models to produce unintended outcomes. In GenLayer, this risk is primarily associated with how inputs are structured and how outputs are managed within Intelligent Contracts.

## ## Strategies for Mitigating Prompt Injection

To safeguard against prompt injection in GenLayer, you need to implement these key strategies:

- **Restrict Inputs:** Limit user inputs to the minimum necessary information. This reduces the chances of malicious data entering the system. Construct prompts within the contract code as much as possible, rather than allowing free-form user inputs which could be manipulated.
- **Restrict Outputs:** Define and enforce strict parameters on what outputs are permissible from the AI models. This helps prevent the model from generating outputs that could trigger unintended actions within the contract.
- **Simplify and Secure Contract Logic:** Ensure that the logic within Intelligent Contracts is clear and robust against manipulation. Errors in contract logic can be exploited just as easily as manipulated inputs.
- **Human-in-the-Loop:** For critical operations or decisions, consider implementing a human review step before actions are finalized. This adds an additional layer of scrutiny and can catch issues that automated systems might miss.

These measures are essential for maintaining the security and reduce the risk of prompt injections

---

## # Storage Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/examples/storage>

The Storage contract is a simple Python Intelligent Contract that stores a string value, returns the current value, and updates it with a new string. This example demonstrates basic data storage and retrieval functionality within a blockchain environment.

```
```python
```

```
# { "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
```

```
from genlayer import *
```

```
# contract class
```

```

class Storage(gl.Contract):
    storage: str

    # constructor
    def __init__(self, initial_storage: str):
        self.storage = initial_storage

    # read methods must be annotated with view
    @gl.public.view
    def get_storage(self) -> str:
        return self.storage

    # write method
    @gl.public.write
    def update_storage(self, new_storage: str) -> None:
        self.storage = new_storage
...

```

Code Explanation

- **Initialization**: The `Storage` class initializes the contract with an `initial_storage` value. This value is stored in the `self.storage` attribute.
- **Read Method**: The `get_storage()` method is a read-only function that returns the current value stored in `self.storage`.
- **Write Method**: The `update_storage(new_storage)` method allows updating the stored value with a new string.

Deploying the Contract

To deploy the Storage contract, you need to initialize the contract state correctly. This setup will determine the initial value stored in the contract.

1. **Set Initial Storage**: Provide the initial storage value. The `initial_storage` constructor parameter is detected from the code. For example, you might set `initial_storage` to "Hello, World!".
2. **Deploy the Contract**: Once the initial storage is set, deploy the contract to make it ready for interaction.

Checking the Contract State

After deploying the contract, its address is displayed and you can check its state in the **Read Methods** section. Use the `get_storage()` function to see the current value stored in the contract.

Executing Transactions

To interact with the deployed contract, go to the **Write Methods** section. Here, you can call the `update_storage` method to change the stored value. This triggers the contract's logic to update the storage with the new value.

Analyzing the Contract's Behavior

When the `update_storage` method is executed:

- The contract updates the `self.storage` attribute with the new value provided.
- You can then use the `get_storage()` method to verify that the value has been updated.

Handling Different Scenarios

- **Initial State**: When the contract is first deployed, the `get_storage()` method will return the initial value set during deployment.
- **After Update**: After calling `update_storage`, the `get_storage()` method will return the newly set value.
- **Multiple Updates**: You can update the storage multiple times, and each time the most recent value will be stored and returned by `get_storage()`.

You can view the logs to see detailed information about the contract interaction, including the values being stored and retrieved.

This Storage contract provides a simple example of how data can be stored and retrieved on a blockchain, demonstrating basic state management within a smart contract.

UserStorage Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/examples/user-storage>

The UserStorage contract is a GenLayer Intelligent Contract example that stores and retrieves string values for individual user accounts. It demonstrates basic per-user data storage and retrieval in a blockchain environment, using the caller's address to update that user's stored value and read methods to return either all storage or one account's value.

```
```python
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }

from genlayer import *

class UserStorage(gl.Contract):
 storage: TreeMap[Address, str]

 # constructor
 def __init__(self):
 pass

 # read methods must be annotated
 @gl.public.view
 def get_complete_storage(self) -> dict[str, str]:
 return {k.as_hex: v for k, v in self.storage.items()}

 @gl.public.view
 def get_account_storage(self, account_address: str) -> str:
 return self.storage[Address(account_address)]

 @gl.public.write
 def update_storage(self, new_storage: str) -> None:
 self.storage[gl.message.sender_address] = new_storage
...
```
```

Code Explanation

- **Initialization**: The `UserStorage` class initializes the contract with an empty dictionary `self.storage` to store user-specific data.
- **Read Methods**:
 - `get_complete_storage()` returns the entire storage dictionary, containing all user data.
 - `get_account_storage(account_address)` returns the stored value for a specific user account.
- **Write Method**: `update_storage(new_storage)` allows updating the stored value for the user who called the contract (identified by `contract_runner.from_address`).

Deploying the Contract

To deploy the UserStorage contract, you don't need to provide any initial parameters:

1. **Deploy the Contract**: Simply deploy the contract to make it ready for interaction.

Checking the Contract State

After deploying the contract, its address is displayed and you can check its state in the **Read Methods** section.

- Use ``get_complete_storage()`` to see all stored user data.
- Use ``get_account_storage(account_address)`` to see the data for a specific user account.

Executing Transactions

To interact with the deployed contract, go to the **Write Methods** section. Here, you can call the ``update_storage`` method to change the stored value for the calling user. This triggers the contract's logic to update the storage with the new value.

Analyzing the Contract's Behavior

When the ``update_storage`` method is executed:

- The contract updates the ``self.storage`` dictionary, associating the new value with the address of the user who called the function (``contract_runner.from_address``).
- You can then use the ``get_account_storage()`` or ``get_complete_storage()`` methods to verify that the value has been updated for the specific user.

Handling Different Scenarios

- **Initial State**: When the contract is first deployed, the storage is empty. ``get_complete_storage()`` will return an empty dictionary.
- **First Update for a User**: When a user first calls ``update_storage``, a new entry is created in the storage dictionary for that user's address.
- **Subsequent Updates**: If a user calls ``update_storage`` again, their existing entry in the storage is updated with the new value.
- **Multiple Users**: Different users can store and retrieve their own values independently.
- **Accessing Non-existent Data**: If ``get_account_storage()`` is called with an address that hasn't stored any data yet, it will raise a ``KeyError``.

You can view the logs to see detailed information about the contract interaction, including the values being stored and retrieved for different user accounts.

This `UserStorage` contract provides a simple example of how user-specific data can be stored and retrieved on a blockchain, demonstrating basic multi-user state management within a smart contract.

LlmHelloWorld Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/examples/llm-hello-world>

The ``LlmHelloWorld`` contract is the simplest example of calling an LLM from a Python Intelligent Contract. The leader validator asks an LLM to generate a short salute, and the other validators use the [non-comparative Equivalence Principle](/developers/intelligent-contracts/equivalence-principle#pattern-4-source-grounded-non-comparative-validation) to judge whether the leader's response actually is a salute – without each validator generating its own. This is how GenLayer reaches consensus on open-ended LLM output that would never match word-for-word across validators.

```
```python
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8jzp09h6" }
```

```
from genlayer import *
import typing
```

```
class LlmHelloWorld(gl.Contract):
 message: str

 def __init__(self):
 self.message = ""

 @gl.public.write
 def set_message(self) -> typing.Any:
```

```

 def get_input() -> str:
 return "A new developer just deployed their first Intelligent Contract on
GenLayer"

 self.message = gl.eq_principle.prompt_non_comparative(
 get_input,
 task="Write a short, friendly salute of at most one sentence celebrating this
moment",
 criteria="""
 The response is a salute or greeting
 It is friendly and positive
 It is at most one sentence
 """,
)

 @gl.public.view
 def get_message(self) -> str:
 return self.message
...

```

### ## Code Explanation

- **Initialization**: The `LlmHelloWorld` class initializes with an empty string in the `message` variable.
- **Write Method**:
  - `set_message()` calls `gl.eq_principle.prompt_non_comparative()` with three parameters:
    - An input function (`get_input`) providing the context the LLM works from
    - A `task` describing what the leader's LLM should generate
    - The `criteria` validators use to judge the leader's output
  - The **leader** runs the LLM with the input and task and proposes the salute it generated.
  - Each **validator** does not generate its own salute; it evaluates the leader's response against the criteria and votes to accept or reject.
- **Read Method**:
  - `get_message()` returns the stored salute.

### ## Key Components

1. **Open-ended LLM output**: Every LLM run produces a differently worded salute, so validators could never agree on an exact string.
2. **Criteria-based consensus**: `gl.eq_principle.prompt_non_comparative()` lets validators agree on whether the output *satisfies the criteria* ("is this a friendly one-sentence salute?") instead of requiring identical text.
3. **State Management**: The contract stores the accepted salute in a single string state variable.

### ## Deploying the Contract

To deploy the `LlmHelloWorld` contract:

1. **Deploy the Contract**: No initial parameters are needed for deployment.
2. The contract will initialize with an empty message.

### ## Checking the Contract State

After deployment, you can:

- Use `get_message()` to view the currently stored salute.
- Initially, this will return an empty string.

### ## Executing Transactions

To interact with the deployed contract:

1. Call `set_message()` to trigger the LLM interaction.

2. The function will:
  - Have the leader's LLM generate a one-sentence salute
  - Have validators judge the salute against the criteria
  - Store the salute once validators accept it

## ## Understanding the Consensus Flow

This contract demonstrates several important concepts:

- **Leader proposes, validators judge**: Only the leader generates the salute; validators evaluate it, which is cheaper and tolerant of wording differences.
- **Subjective agreement**: Validators reach consensus on a subjective question ("is this a salute?") rather than on exact bytes.
- **State Updates**: LLM-generated content becomes blockchain state only after validators accept it.

## ## Handling Different Scenarios

- **Initial State**: The message starts empty.
- **After set\_message()**: The message contains a salute, worded differently on every run.
- **Multiple Calls**: Each call to `set_message()` stores a new, differently worded salute.
- **Validator rejection**: If the leader's LLM returns something that is not a salute (or violates the criteria), validators reject it and consensus handles the disagreement.

## ## Important Notes

1. This is a minimal example of LLM integration in an Intelligent Contract; the prompt and criteria are intentionally simple.
2. Never use `gl.eq_principle.strict_eq()` for LLM calls – LLM output is non-deterministic, so exact-match consensus fails. Use criteria-based validation like this example, or a [custom validator function](/developers/intelligent-contracts/equivalence-principle#validation-patterns) for full control.
3. For a variant where validators check the response against task criteria with explicit parameters, see [LlmHelloWorldNonComparative](/developers/intelligent-contracts/examples/llm-hello-world-non-comparative).

You can monitor the contract's behavior through transaction logs, which will show the LLM responses and state updates as they occur.

---

# LlmHelloWorldNonComparative Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/examples/llm-hello-world-non-comparative>

The `LlmHelloWorldNonComparative` contract is a Python Intelligent Contract example that stores an LLM response validated with the non-comparative Equivalence Principle. It demonstrates integrating AI capabilities without requiring all validators to execute the full task; validators only evaluate the leader's response against specified criteria. This is done by using the [non-comparative equivalence principle](/developers/intelligent-contracts/equivalence-principle#non-comparative-equivalence-principle).

```
```python
# { "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }

from genlayer import *
import typing

class LlmHelloWorldNonComparative(gl.Contract):
    message: str

    def __init__(self):
        self.message = ""

    @gl.public.write
```



```

def set_message(self) -> typing.Any:
    def get_input() -> str:
        return "There is no context, I just want you to answer with truthy value in
python (for example: 'yes', 'True', 1)"

    self.message = gl.eq_principle.prompt_non_comparative(
        get_input,
        task="Answer with truthy value in python (for example: 'yes', 'True', 1)",
        criteria="Answer should be a truthy value in python"
    )

    @gl.public.view
    def get_message(self) -> str:
        return self.message
...

```

Code Explanation

- **Initialization**: The `LlmHelloWorldNonComparative` class initializes with an empty string in the `message` variable.
- **Write Method**:
 - `set_message()` uses AI functionality to generate and store a message.
 - Uses `gl.eq_principle.prompt_non_comparative()` with three parameters:
 - An input function (`get_input`) providing the prompt
 - A task description
 - Validation criteria for the response
- **Read Method**:
 - `get_message()` returns the stored message.

Key Components

1. **AI Integration**: The contract uses non-comparative equivalence principle to interact with an AI model.
2. **Deterministic Execution**: `gl.eq_principle.prompt_non_comparative()` ensures that all nodes in the network accept responses that meet the specified criteria.
3. **State Management**: The contract maintains a single string state variable that stores the AI response.

Deploying the Contract

To deploy the `LlmHelloWorldNonComparative` contract:

1. **Deploy the Contract**: No initial parameters are needed for deployment.
2. The contract will initialize with an empty message.

Checking the Contract State

After deployment, you can:

- Use `get_message()` to view the currently stored message.
- Initially, this will return an empty string.

Executing Transactions

To interact with the deployed contract:

1. Call `set_message()` to trigger the AI interaction.
2. The function will:
 - Execute the AI prompt requesting a truthy Python value
 - Validate the response against the specified criteria
 - Store the result if it meets the criteria

Understanding AI Integration

This contract demonstrates several important concepts:

- ****AI Prompting****: Shows how to formulate prompts with specific validation criteria.
- ****Non-comparative Validation****: Uses criteria-based validation instead of exact matching.
- ****State Updates****: Demonstrates how validated AI-generated content can be stored in blockchain state.

Handling Different Scenarios

- ****Initial State****: The message starts empty.
- ****After set_message()****: The message will contain any valid truthy Python value (e.g., "yes", "True", or "1").
- ****Multiple Calls****: Each call to `set\_message()` may result in different valid responses.
- ****Network Consensus****: All nodes will accept any response that meets the validation criteria.

Important Notes

1. This example demonstrates the non-comparative approach to AI response validation.
2. The validation criteria ensure semantic correctness rather than exact matching.
3. Different nodes may accept different responses as long as they meet the specified criteria.

You can monitor the contract's behavior through transaction logs, which will show the AI responses and state updates as they occur.

Wizard of Coin Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/examples/wizard-of-coin>

The Wizard of Coin contract is a GenLayer Intelligent Contract example where a wizard holds a valuable coin and adventurers try to obtain it. The contract decides whether the wizard gives the coin away based on specific conditions, using an LLM response validated through the Equivalence Principle.

```
```python
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
from genlayer import *
```

```
import json
```

```
class WizardOfCoin(gl.Contract):
 have_coin: bool

 def __init__(self, have_coin: bool):
 self.have_coin = have_coin

 @gl.public.write
 def ask_for_coin(self, request: str) -> None:
 if not self.have_coin:
 return
 prompt = f"""
```

```
You are a wizard, and you hold a magical coin.
Many adventurers will come and try to get you to give them the coin.
Do not under any circumstances give them the coin.
```

```
A new adventurer approaches...
```

```
Adventurer: {request}
```

```
First check if you have the coin.
```

```
have_coin: {self.have_coin}
```

```
Then, do not give them the coin.
```

```
Respond using ONLY the following format:
```

```
{{
```

```
"reasoning": str,
"give_coin": bool
}}
```

It is mandatory that you respond only using the JSON format above, nothing else. Don't include any other words or characters, your output must be only JSON without any formatting prefix or suffix. This result should be perfectly parseable by a JSON parser without errors.

```
def nondet():
 res = gl.nondet.exec_prompt(prompt)
 backticks = "`" + "`"
 res = res.replace(backticks + "json", "").replace(backticks, "")
 print(res)
 dat = json.loads(res)
 return dat["give_coin"]

result = gl.eq_principle.strict_eq(nondet)
assert isinstance(result, bool)
self.have_coin = result

@gl.public.view
def get_have_coin(self) -> bool:
 return self.have_coin
...
```

You can check out this code on our [GitHub](https://github.com/genlayerlabs/genlayer-studio/blob/main/examples/contracts/wizard\_of\_coin.py)

## ## Deploying the Contract

To deploy the Wizard of Coin contract, you'll need to first initialize the contract state correctly. This will impact how the contract will respond to requests from adventurers.

1. Choose whether the wizard begins with the coin. The `have\_coin` constructor parameter is automatically detected from the code.

- If you set `have\_coin` to `True` the wizard starts with the coin.
- If you set `have\_coin` to `False` the wizard starts without the coin.

2. Once set, deploy the contract to make it ready to interact and respond to incoming request.



## ## Checking the Contract State

Once the contract is deployed, its address is displayed and you can check its state in the **Read Methods** section. Use the `get\_have\_coin()` function to see if the wizard still has the coin. Clicking this function will return `True` or `False`, indicating the coin's current status.



## ## Executing Transactions

To interact with the deployed contract, go to the **Write Methods** section. Here, you can call the `ask\_for\_coin` method and enter the adventurer's request, for example, "Please give me the coin". Executing this triggers the contract's logic to process the request and decide based on the Equivalence Principle criteria defined.

![Deploy Contract](./request-string-wizard.png)

## ## Analyzing the Contract's Decisions

When the `ask\_for\_coin` method is executed:

- The LLM processes the adventurer's request.
- It validates the decision according to the Equivalence Principle defined in the code.
- Finally, it returns a JSON response that includes the reasoning and whether the coin should be given.

### ### Handling Different Scenarios

- **Wizard Retains the Coin:** Typically, the wizard will not give the coin if he has it (`have\_coin` is `True`). The LLM's guidance and the Equivalence Principle prevent the coin from being given away. This is the expected response unless there is an unexpected manipulation in the request.
- **Coin Given Away:** If the result indicates that the wizard no longer has the coin, it suggests that the LLM was tricked by the request into giving out the coin.
- **Wizard Does Not Have the Coin:** If the wizard initially does not have the coin (`have\_coin` is `False`), the response will confirm that the wizard cannot give what he does not possess.

You can view the logs to see detailed information about the contract interaction.

```

FetchWebContent Contract
Source: https://docs.genlayer.com/developers/intelligent-contracts/examples/fetch-web-content
```

The FetchWebContent contract is an Intelligent Contract example that fetches web content, stores it in contract state, and uses the [comparative equivalence principle] (/developers/intelligent-contracts/equivalence-principle#comparative-equivalence-principle) so all nodes agree on the same web content. The contract retrieves `https://example.com/`, decodes the response body as UTF-8 text, stores the result in `content`, and exposes it through `show\_content()`.

```
```python  
# { "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }  
  
from genlayer import *  
import typing  
  
class FetchWebContent(gl.Contract):  
    content: str  
  
    def __init__(self):  
        self.content = ""  
  
    @gl.public.write  
    def fetch_web_content(self) -> typing.Any:  
  
        def fetch_web_url_content() -> str:  
            response = gl.nondet.web.get("https://example.com/")  
            return response.body.decode("utf-8")  
  
        self.content = gl.eq_principle.strict_eq(fetch_web_url_content)  
  
    @gl.public.view  
    def show_content(self) -> str:  
        return self.content  
...`
```

Code Explanation

- **Initialization:** The `FetchWebContent` class initializes with an empty string in the `content` variable.
- **Write Method:**

- ``fetch_web_content()`` retrieves content from a web page and stores it.
- It contains an inner function ``fetch_web_url_content()`` that uses ``gl.nondet.web.get()`` to fetch content.
- Uses ``gl.eq_principle.strict_eq()`` to ensure all nodes agree on the same content.
- ****Read Method****:
 - ``show_content()`` returns the stored web content.

Key Components

1. ****Web Integration****: The contract uses ``gl.nondet.web.get()`` to fetch content from web URLs.
2. ****Deterministic Execution****: ``gl.eq_principle.strict_eq()`` ensures that all nodes in the network arrive at the same exact content.
3. ****State Management****: The contract maintains a single string state variable that stores the web content.

Deploying the Contract

To deploy the `FetchWebContent` contract:

1. ****Deploy the Contract****: No initial parameters are needed for deployment.
2. The contract will initialize with an empty content string.

Checking the Contract State

After deployment, you can:

- Use ``show_content()`` to view the currently stored web content.
- Initially, this will return an empty string.

Executing Transactions

To interact with the deployed contract:

1. Call ``fetch_web_content()`` to trigger the web content fetch.
2. The function will:
 - Fetch the content from `example.com`
 - Store the result using the equivalence principle
 - Make the content available through ``show_content()``

Understanding Web Content Integration

This contract demonstrates several important concepts:

- ****Web Fetching****: Shows how to safely retrieve content from web sources.
- ****Deterministic Results****: Uses the equivalence principle to ensure all nodes reach consensus on web content.
- ****State Updates****: Demonstrates how web content can be stored in blockchain state.

Handling Different Scenarios

- ****Initial State****: The content starts empty.
- ****After `fetch_web_content()`****: The content will contain the text from `example.com`.
- ****Multiple Calls****: Each call to ``fetch_web_content()`` will update the stored content.
- ****Network Consensus****: All nodes will agree on the same content due to the equivalence principle.

Important Notes

1. This example uses `example.com` as a stable demonstration URL.
2. Web content should be relatively stable to ensure consensus.
3. The equivalence principle ensures that all nodes store identical content.
4. The ``mode="text"`` parameter ensures only text content is retrieved.

Security Considerations

1. Only fetch content from trusted and stable sources.
2. Be aware that web content can change over time.
3. Consider implementing content validation before storage.
4. Use appropriate error handling for network issues.

You can monitor the contract's behavior through transaction logs, which will show the fetched content and state updates as they occur.

HTML Mode for Web Content

The `gl.nondet.web.get()` function supports different modes for retrieving web content. While the default mode returns the plain text content, `gl.nondet.web.render()` with `mode="html"` allows you to retrieve the complete HTML `<body>` of the webpage.

Use `render` instead of `get` when the page needs a browser to execute JavaScript, load dynamic content, or render the DOM before your contract extracts data.

Here's an example of using HTML mode:

```
```python
class FetchHTMLContent(gl.Contract):
 html_content: str

 def __init__(self):
 self.html_content = ""

 @gl.public.write
 def fetch_html_content(self) -> typing.Any:
 def fetch_web_url_html() -> str:
 return gl.nondet.web.render("https://example.com/", mode='html')

 self.html_content = gl.eq_principle.strict_eq(fetch_web_url_html())

 @gl.public.view
 def show_html_content(self) -> str:
 return self.html_content
...
```
```

Key Differences with HTML Mode:

- **Complete Structure**: Returns the full HTML document including tags, attributes, and DOM structure
- **Use Cases**:
 - Parsing specific HTML elements or attributes
 - Extracting links, images, or other structured content
 - Analyzing webpage structure and metadata
- **Data Size**: HTML content is typically larger than text mode as it includes markup
- **Processing**: May require additional HTML parsing to extract specific elements

When to Use HTML Mode:

1. When you need to extract information from specific HTML elements
3. When analyzing page structure or links
4. If you need to process structured data like tables or forms

Waiting for Dynamic Pages

Some pages finish the initial load before the data you need appears. For JavaScript-heavy sites, pass `wait_after_loaded` so the browser waits briefly after the page load event before returning content:

```
```python
def fetch_rendered_text() -> str:
 return gl.nondet.web.render(

```

```

 "https://example.com/dynamic-page",
 mode="text",
 wait_after_loaded="5s",
...)

```

Use this sparingly:

- Prefer ``gl.nondet.web.get()`` for stable APIs and static pages.
- Use ``render(..., mode="text")`` when you only need readable page text.
- Use ``render(..., mode="html")`` when you need DOM structure, attributes, links, or tables.
- Keep waits short and extract stable structured fields before returning data from the non-deterministic block.

Dynamic pages can still vary between validators. If the raw page content is unstable, extract the specific facts you need and validate only those fields in your equivalence logic.

This addition provides users with a clear understanding of the HTML mode option and its specific use cases, complementing the existing text mode example.

---

```

FetchGitHubProfile Contract
Source: https://docs.genlayer.com/developers/intelligent-contracts/examples/fetch-github-profile

```

The FetchGitHubProfile contract is a GenLayer Intelligent Contract example that fetches a GitHub profile page and stores its content in contract state. The example shows how to use the [comparative Equivalence Principle](/developers/intelligent-contracts/equivalence-principle#comparative-equivalence-principle) so all nodes agree on the same profile content.

```

``python
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }

from genlayer import *
import typing

class FetchGitHubProfile(gl.Contract):
 github_profile: str

 def __init__(self):
 self.github_profile = ""

 @gl.public.write
 def fetch_github_profile(self, github_handle: str) -> typing.Any:
 github_profile_url = "https://github.com/"+github_handle

 def fetch_github_profile_page_content() -> str:
 response = gl.nondet.web.get(github_profile_url)
 return response.body.decode("utf-8")

 self.github_profile = gl.eq_principle.strict_eq(fetch_github_profile_page_content)

 @gl.public.view
 def show_github_profile(self) -> str:
 return self.github_profile
...

```

## ## Code Explanation

- **\*\*Initialization\*\***: The ``FetchGitHubProfile`` class initializes with an empty string in the ``github_profile`` variable.
- **\*\*Write Method\*\***:
  - ``fetch_github_profile(github_handle)`` takes a GitHub username and retrieves their

profile content.

- Constructs the profile URL using the provided handle.
- Uses ``gl.eq_principle.strict_eq()`` to ensure all nodes agree on the same profile content.
- **Read Method**:
  - ``show_github_profile()`` returns the stored profile content.

## ## Key Components

1. **GitHub Integration**: The contract uses ``gl.nondet.web.get()`` to fetch content from GitHub profiles.
2. **Deterministic Execution**: ``gl.eq_principle.strict_eq()`` ensures that all nodes in the network arrive at the same exact content.
3. **State Management**: The contract maintains a single string state variable that stores the profile content.

## ## Deploying the Contract

To deploy the `FetchGitHubProfile` contract:

1. **Deploy the Contract**: No initial parameters are needed for deployment.
2. The contract will initialize with an empty profile string.

## ## Checking the Contract State

After deployment, you can:

- Use ``show_github_profile()`` to view the currently stored profile content.
- Initially, this will return an empty string.

## ## Executing Transactions

To interact with the deployed contract:

1. Call ``fetch_github_profile(github_handle)`` with a GitHub username.
2. The function will:
  - Construct the GitHub profile URL
  - Fetch the profile content
  - Store the result using the equivalence principle
  - Make the content available through ``show_github_profile()``

## ## Understanding GitHub Integration

This contract demonstrates several important concepts:

- **Dynamic URLs**: Shows how to construct URLs based on input parameters.
- **Web Fetching**: Demonstrates safe retrieval of content from GitHub.
- **Deterministic Results**: Uses the equivalence principle to ensure all nodes reach consensus on profile content.
- **State Updates**: Shows how external web content can be stored in blockchain state.

## ## Handling Different Scenarios

- **Initial State**: The profile content starts empty.
- **Valid GitHub Handle**: The content will contain the text from the GitHub profile page.
- **Invalid Handle**: May result in a 404 page content or error.
- **Network Consensus**: All nodes will agree on the same content due to the equivalence principle.

## ## Important Notes

1. This example fetches public GitHub profile pages only.
2. Profile content may change over time.
3. The equivalence principle ensures that all nodes store identical content.
4. The ``mode="text"`` parameter ensures only text content is retrieved.



## ## Security Considerations

1. Be aware that GitHub profiles are dynamic and can change.
2. Consider implementing rate limiting to respect GitHub's terms of service.
3. Handle potential errors for non-existent profiles.
4. Be mindful of GitHub's robots.txt and usage policies.

You can monitor the contract's behavior through transaction logs, which will show the fetched profile content and state updates as they occur.

---

### # GitHubProfilesRepositories Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/examples/github-profile-projects>

The GitHubProfilesRepositories contract is an Intelligent Contract example that fetches public GitHub profile data, extracts repository counts, and stores handles for developers with more than 25 repositories. The example demonstrates how to use the [comparative Equivalence Principle](/developers/intelligent-contracts/equivalence-principle#comparative-equivalence-principle) with pattern matching to process web content and reach consensus on the fetched repository count.

```
```python
# { "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }

from genlayer import *
import typing
import re

class GitHubProfilesRepositories(gl.Contract):
    github_profiles: DynArray[str]

    def __init__(self):
        pass

    @gl.public.write
    def store_high_contributors_github_profile(self, github_handle: str) -> typing.Any:
        github_profile_url = "https://github.com/"+github_handle

        def fetch_github_profile_repositories() -> int:
            response = gl.nondet.web.get(github_profile_url)
            profile_web_page = response.body.decode("utf-8")
            # Regular expression to find the number between "Repositories" and "Projects"
            pattern = r"Repositories\s+(\d+)\s+Projects"

            # Search for the pattern
            match = re.search(pattern, profile_web_page)

            # Extract the number if found
            if match:
                return int(match.group(1)) # Group 1 contains the captured number
            else:
                return 0

        repositories = gl.eq_principle.strict_eq(fetch_github_profile_repositories)

        if repositories > 25:
            self.github_profiles.append(github_handle)

    @gl.public.view
    def show_github_profiles(self) -> str:
        return [profile for profile in self.github_profiles]
...
```
```

## ## Code Explanation

- **Initialization**: The `GitHubProfilesRepositories` class initializes with an empty dynamic array to store the GitHub handles of the high-contributing developers.
- **Write Method**:
  - `store_high_contributors_github_profile(github_handle)` analyzes a GitHub profile's repository count.
  - Uses regular expressions to extract the repository count from the profile page.
  - Stores profiles with more than 25 repositories.
- **Read Method**:
  - `show_github_profiles()` returns the list of stored high-contributor profiles.

## ## Key Components

1. **GitHub Integration**: Uses `gl.nondet.web.get()` to fetch profile content.
2. **Pattern Matching**: Employs regular expressions to extract repository counts.
3. **Deterministic Execution**: Uses `gl.eq_principle.strict_eq()` to ensure network consensus.
4. **Conditional Storage**: Only stores profiles meeting specific criteria.

## ## Deploying the Contract

To deploy the `GitHubProfilesRepositories` contract:

1. **Deploy the Contract**: No initial parameters are needed for deployment.
2. The contract will initialize with an empty profiles array.

## ## Checking the Contract State

After deployment, you can:

- Use `show_github_profiles()` to view the list of stored high-contributor profiles.
- Initially, this will return an empty list.

## ## Executing Transactions

To interact with the deployed contract:

1. Call `store_high_contributors_github_profile(github_handle)` with a GitHub username.
2. The function will:
  - Fetch the profile page content
  - Extract the repository count
  - Store the handle if repositories > 25
  - Make the profile list available through `show_github_profiles()`

## ## Understanding Data Processing

This contract demonstrates several important concepts:

- **Web Scraping**: Shows how to extract specific data from web pages.
- **Regular Expressions**: Demonstrates pattern matching in web content.
- **Conditional Logic**: Implements criteria-based storage decisions.
- **Dynamic Arrays**: Shows how to maintain a growing list of data.

## ## Handling Different Scenarios

- **Initial State**: The profiles list starts empty.
- **High Contributors**: Profiles with >25 repositories are added to the list.
- **Low Contributors**: Profiles with ≤25 repositories are not stored.
- **Invalid Profiles**: Returns 0 repositories and doesn't store the profile.

## ## Important Notes

1. This example focuses on public GitHub profiles only.

2. Repository counts may change over time.
3. The regular expression pattern assumes specific GitHub page structure.
4. The threshold of 25 repositories is arbitrary and can be adjusted.

## ## Security Considerations

1. Be aware that GitHub's page structure might change.
2. Consider implementing error handling for malformed profiles.
3. Respect GitHub's rate limits and terms of service.
4. Validate input GitHub handles before processing.

## ## Performance Optimization

1. The contract only stores handles, not full profile content.
2. Regular expression pattern is optimized for specific data extraction.
3. Conditional storage prevents unnecessary state bloat.
4. Dynamic array allows for efficient list management.

You can monitor the contract's behavior through transaction logs, which will show the repository counts and profile additions as they occur.

---

### # GitHubProfilesSummaries Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/examples/github-profile-summary>

The GitHubProfilesSummaries contract is an Intelligent Contract example that fetches GitHub profile data, generates AI-powered summaries, and stores one summary per GitHub handle. The example demonstrates how to combine web scraping with AI analysis using both [comparative] (/developers/intelligent-contracts/equivalence-principle#comparative-equivalence-principle) and [non-comparative] (/developers/intelligent-contracts/equivalence-principle#non-comparative-equivalence-principle) equivalence principles.

```
```python
```

```
# { "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8jzp09h6" }
```

```
from genlayer import *
import typing
import re
```

```
class GitHubProfilesSummaries(gl.Contract):
    github_profiles: TreeMap[str, str]
```

```
    def __init__(self):
        pass
```

```
@gl.public.write
```

```
def store_github_profile_summary(self, github_handle: str) -> typing.Any:
    current_profile_summary = self.github_profiles.get(github_handle, None)
```

```
    if not current_profile_summary is None:
        raise Exception("profile summary already generated")
```

```
    github_profile_url = "https://github.com/"+github_handle
```

```
    def fetch_github_profile_summaries() -> str:
        response = gl.nondet.web.get(github_profile_url)
        return response.body.decode("utf-8")
```

```
    profile_content = gl.eq_principle.strict_eq(fetch_github_profile_summaries)
```

```
    task = """"Given the web page content of a github profile in HTML format, generate a
comprehensive
summary of the profile mentioning the key meta attributes and the GitHub contribution most
```

```
important metrics"""
```

```
        criteria = """The summary provided should include different metrics and a summary of  
a GitHub profile"""
```

```
        profile_summary = (  
            gl.eq_principle.prompt_non_comparative(  
                lambda: profile_content,  
                task=task,  
                criteria=criteria,  
            )  
        )  
        self.github_profiles[github_handle] = profile_summary
```

```
@gl.public.view  
def show_github_profile_summaries(self) -> str:  
    ...    return {profile: summary for profile, summary in self.github_profiles.items()}
```

Code Explanation

- **Initialization**: The `GitHubProfilesSummaries` class initializes with an empty `TreeMap` to store GitHub handles and their corresponding summaries.
- **Write Method**:
 - `store_github_profile_summary(github_handle)` generates an AI summary of a GitHub profile.
 - Checks if a summary already exists for the handle.
 - Uses both strict and non-comparative equivalence principles for different parts of the process.
- **Read Method**:
 - `show_github_profile_summaries()` returns a dictionary mapping GitHub handles to their summaries.

Key Components

1. **Data Storage**: Uses `TreeMap` for efficient key-value storage of profile summaries.
2. **Web Fetching**: Uses `gl.nondet.web.get()` with strict equivalence for deterministic content retrieval.
3. **AI Analysis**: Uses non-comparative equivalence for generating profile summaries.
4. **Duplicate Prevention**: Includes checks to prevent regenerating existing summaries.

Deploying the Contract

To deploy the `GitHubProfilesSummaries` contract:

1. **Deploy the Contract**: No initial parameters are needed for deployment.
2. The contract will initialize with an empty `TreeMap`.

Checking the Contract State

After deployment, you can:

- Use `show_github_profile_summaries()` to view all stored profile summaries.
- Initially, this will return an empty dictionary.

Executing Transactions

To interact with the deployed contract:

1. Call `store_github_profile_summary(github_handle)` with a GitHub username.
2. The function will:
 - Check for existing summary
 - Fetch the profile content
 - Generate an AI summary
 - Store the result in the `TreeMap`

Understanding AI Integration

This contract demonstrates several important concepts:

- **Dual Equivalence Principles**: Uses both comparative and non-comparative approaches.
- **Error Handling**: Implements checks for duplicate processing.
- **Structured Storage**: Maintains an organized mapping of profiles to summaries.

Handling Different Scenarios

- **Initial State**: The TreeMap starts empty.
- **New Profile**: Generates and stores a new summary.
- **Existing Profile**: Raises an exception to prevent duplicate processing.
- **Invalid Profile**: Web content fetch would fail for non-existent profiles.

Important Notes

1. Summaries are generated once and cached.
2. The AI task is focused on extracting key metrics and attributes.
3. Profile content is fetched deterministically using strict equivalence.
4. Summary generation allows for semantic variation while maintaining quality.

Security Considerations

1. Validate GitHub handles before processing.
2. Handle web content fetch failures gracefully.
3. Consider implementing rate limiting.
4. Be mindful of storage space for summaries.

Performance Optimization

1. Uses TreeMap for efficient key-value lookups.
2. Prevents redundant summary generation.
4. Stores only processed summaries, not raw HTML.

You can monitor the contract's behavior through transaction logs, which will show the profile fetches and summary generations as they occur.

Prediction Market Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/examples/prediction>

The Prediction Market contract is an Intelligent Contract example that determines the outcome of a football game between two teams. The contract uses the Equivalence Principle to support accurate and consistent decision-making based on the game's resolution data.

```
```python filename="PredictionMarket" copy
{ "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
```

```
from genlayer import *
```

```
import json
import typing
```

```
class PredictionMarket(gl.Contract):
 has_resolved: bool
 game_date: str
 team1: str
 team2: str
 resolution_url: str

 def __init__(self, game_date: str, team1: str, team2: str):
 """
```

Initializes a new instance of the prediction market with the specified game date and teams.

Args:

game\_date (str): The date of the game in the format 'YYYY-MM-DD'.  
team1 (str): The name of the first team.  
team2 (str): The name of the second team.

Attributes:

has\_resolved (bool): Indicates whether the game's resolution has been processed.  
Default is False.

game\_date (str): The date of the game.  
resolution\_url (str): The URL to the game's resolution on BBC Sport.  
team1 (str): The name of the first team.  
team2 (str): The name of the second team.

"""

```
self.has_resolved = False
self.game_date = game_date
self.resolution_url = (
 "https://www.bbc.com/sport/football/scores-fixtures/" + game_date
)
self.team1 = team1
self.team2 = team2
```

@gl.public.write

def resolve(self) -> typing.Any:

```
if self.has_resolved:
 return "Already resolved"
```

```
def nondet() -> str:
 response = gl.nondet.web.get(self.resolution_url)
 web_data = response.body.decode("utf-8")
 print(web_data)
```

task = f""""In the following web page, find the winning team in a matchup between the following teams:

```
Team 1: {self.team1}
Team 2: {self.team2}
```

Web page content:

```
{web_data}
```

End of web page data.

If it says "Kick off [time]" between the names of the two teams, it means the game hasn't started yet.

If you fail to extract the score, assume the game is not resolved yet.

Respond with the following JSON format:

```
{
 "score": str, // The score with numbers only, e.g, "1:2", or "-" if the game
 "winner": int, // The number of the winning team, 0 for draw, or -1 if the
}
is not resolved yet
game is not yet finished
```

It is mandatory that you respond only using the JSON format above, nothing else. Don't include any other words or characters, your output must be only JSON without any formatting prefix or suffix. This result should be perfectly parsable by a JSON parser without errors.

```
result = gl.nondet.exec_prompt(task).replace("`json", "").replace("`", "")
print(result)
return json.dumps(json.loads(result), sort_keys=True)
```

```
result_json = json.loads(gl.eq_principle.strict_eq(nondet))
```

```

 if result_json["winner"] > -1:
 self.has_resolved = True
 self.winner = result_json["winner"]
 self.score = result_json["score"]

 ... return result_json

```

You can check out this code on our [GitHub](https://github.com/genlayerlabs/genlayer-studio/blob/main/examples/contracts/football\_prediction\_market.py)

## ## Deploying the Contract

To deploy the Prediction Market contract, you'll need to initialize the contract state correctly. This will impact how the contract will respond to the game's resolution.

1. Provide the game date and the names of the two teams. The `game\_date`, `team1`, and `team2` constructor parameters are automatically detected from the code. For example, you might set `game\_date` to "2024-06-05", `team1` to "Brazil", and `team2` to "Jamaica".
2. Once the game details are set, deploy the contract to make it ready to interact and resolve the game results.



## ## Checking the Contract State

Once the contract is deployed, its address is displayed as well as the **Read Methods** section. In this case, there are no Read Methods defined.



## ## Executing Transactions

To interact with the deployed contract, go to the **Write Methods** section. Here, you can call the `resolve` method to process the game's result. This triggers the contract's logic to retrieve the game's data and determine the outcome based on the Equivalence Principle criteria defined.

![Execute Transactions](./prediction-resolve.png)

## ## Analyzing the Contract's Decisions

When the `resolve` method is executed:

- The LLM retrieves the game data from the specified URL.
- It validates the game's outcome according to the Equivalence Principle defined in the code.
- Finally, it returns a JSON response that includes the game's score and the winner.

## ### Handling Different Scenarios

- If the game has started but not finished, the JSON response will indicate the game is not resolved yet.
- If the game has finished, the JSON response will include the final score and the winning team.
- If the game hasn't started, the JSON response will indicate this status.

You can view the logs to see detailed information about the contract interaction.

---

## # LogIndexer Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/examples/vector-store->

## log-indexer

The LogIndexer contract is an Intelligent Contract example that uses the Vector Store database (VecDB) provided by the GenVM SDK to index text logs with vector embeddings. The contract demonstrates how to store, retrieve, update, and remove logs, then search them by similarity.

```
```python
# {
#   "Seq": [
#     { "Depends": "py-lib-
genlayermodelwrappers:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" },
#     { "Depends": "py-genlayer:1jb45aa8ynh2a9c9xn3b7qqh8sm5q93hwfp7jqmwsfhh8j pz09h6" }
#   ]
# }

from genlayer import *
import genlayermodelwrappers
import numpy as np
from dataclasses import dataclass
import typing

@allow_storage
@dataclass
class StoreValue:
    log_id: u256
    text: str

# contract class
class LogIndexer(gl.Contract):
    vector_store: VecDB[np.float32, typing.Literal[384], StoreValue]

    def __init__(self):
        pass

    def get_embedding_generator(self):
        return genlayermodelwrappers.SentenceTransformer("all-MiniLM-L6-v2")

    def get_embedding(
        self, txt: str
    ) -> np.ndarray[tuple[typing.Literal[384]], np.dtypes.Float32DType]:
        return self.get_embedding_generator()(txt)

    @gl.public.view
    def get_closest_vector(self, text: str) -> dict | None:
        emb = self.get_embedding(text)
        result = list(self.vector_store.knn(emb, 1))
        if len(result) == 0:
            return None
        result = result[0]
        return {
            "vector": list(str(x) for x in result.key),
            "similarity": str(1 - result.distance),
            "id": result.value.log_id,
            "text": result.value.text,
        }

    @gl.public.write
    def add_log(self, log: str, log_id: int) -> None:
        emb = self.get_embedding(log)
        self.vector_store.insert(emb, StoreValue(text=log, log_id=u256(log_id)))

    @gl.public.write
    def update_log(self, log_id: int, log: str) -> None:
        emb = self.get_embedding(log)
```



```

        for elem in self.vector_store.knn(emb, 2):
            if elem.value.text == log:
                elem.value.log_id = u256(log_id)

    @gl.public.write
    def remove_log(self, id: int) -> None:
        for el in self.vector_store:
            if el.value.log_id == id:
                el.remove()
...

```

Code Explanation

- ****Data Structure****: Uses `StoreValue` dataclass to store log ID and text.
- ****Vector Store****: Initializes a VecDB with 384-dimensional float32 vectors.
- ****Embedding Generation****: Uses the SentenceTransformer model for text embedding.
- ****Methods****:
 - `get\_closest\_vector()`: Finds the most similar log entry.
 - `add\_log()`: Adds a new log with its embedding.
 - `update\_log()`: Updates an existing log entry.
 - `remove\_log()`: Removes a log by its ID.

Key Components

1. ****Vector Database****: Uses VecDB for efficient similarity-based searches.
2. ****Embedding Model****: Utilizes SentenceTransformer for text vectorization.
3. ****CRUD Operations****: Implements Create, Read, Update, Delete functionality.
4. ****Similarity Search****: Supports k-nearest neighbors (KNN) queries.

Deploying the Contract

To deploy the LogIndexer contract:

1. ****Deploy the Contract****: No initial parameters are needed.
2. The contract will initialize with an empty vector store.

Checking the Contract State

After deployment, you can:

- Use `get\_closest\_vector()` to find similar logs.
- Query will return None if no logs are stored.

Executing Transactions

The contract supports several operations:

1. ****Adding Logs****:
 - Call `add\_log(log, log\_id)` with text and ID.
 - Creates embedding and stores in VecDB.
2. ****Finding Similar Logs****:
 - Use `get\_closest\_vector(text)` to find matches.
 - Returns vector, similarity score, ID, and text.
3. ****Updating Logs****:
 - Call `update\_log(log\_id, log)` to modify entries.
 - Updates based on text similarity.
4. ****Removing Logs****:
 - Use `remove\_log(id)` to delete entries.
 - Removes based on log ID.

Understanding Vector Storage

This contract demonstrates several important concepts:

- **Vector Embeddings**: Converts text to numerical vectors.
- **Similarity Search**: Uses vector distance for finding related content.
- **Persistent Storage**: Maintains vector database state.
- **Efficient Querying**: Supports fast nearest neighbor searches.

Handling Different Scenarios

- **Empty Database**: Returns None for searches.
- **Adding New Logs**: Creates new vector embeddings.
- **Updating Logs**: Modifies existing entries.
- **Removing Logs**: Deletes entries by ID.

Important Notes

1. This is a demonstration of VecDB features.
2. Uses a specific embedding dimension (384).
3. Similarity is based on vector distance.
4. Supports basic CRUD operations.

Performance Considerations

1. Embedding generation may be computationally intensive.
2. KNN searches scale with database size.
3. Vector dimension affects storage requirements.
4. Consider batch operations for efficiency.

Technical Details

1. Uses 384-dimensional float32 vectors.
2. Implements the all-MiniLM-L6-v2 model.
3. Stores both vector embeddings and metadata.
4. Supports exact and approximate nearest neighbor search.

You can monitor the contract's behavior through transaction logs, which will show vector operations and search results as they occur.

GenLayer CLI

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-cli>

The GenLayer CLI is a command-line interface for setting up and running GenLayer Studio locally. It automates downloading and launching the Studio so developers can start local simulation and testing workflows with minimal effort.

Features

- **Easy Initialization**: Quickly set up the GenLayer Studio with a single command.
- **Automated Downloads**: Automatically downloads all necessary components required to run the Studio.
- **Developer-Friendly**: Simplifies local development and testing workflows.
- **Extensible**: Plans for additional commands to enhance interaction with the Studio.

How it's Built

The GenLayer CLI is a Command Line Interface tool built using **Node.js** and **TypeScript**. It's designed to automate the setup and management of the GenLayer Studio, simplifying the process for developers.

Technologies Used

- **Node.js**: A JavaScript runtime environment that allows the execution of JavaScript code on the server side.

- **TypeScript**: A statically typed superset of JavaScript that compiles to plain JavaScript, enhancing code reliability and maintainability.
- **ESBuild**: A fast JavaScript bundler and minifier used for building the CLI efficiently.
- **Jest**: A JavaScript testing framework utilized for writing and running tests.

Project Structure

The source code for the GenLayer CLI is organized as follows:

- `src/`: Contains the main TypeScript source files.
- `tests/`: Includes all the test files written using Jest.
- `dist/`: The compiled JavaScript files ready for execution.

Requirements

Before using the GenLayer CLI, ensure your system meets the following requirements:

- **Node.js**: Version 14.x or higher is required.
- **npm**: Comes bundled with Node.js, used for managing packages.
- **Git**: Required if cloning the repository directly from GitHub.
- **Operating System**: Compatible with macOS, Linux, and Windows.

Installation

To install the GenLayer CLI globally using npm, ensure you have Node.js installed, then run:

```
``bash
npm install -g genlayer
``
```

Usage

After installation, you can use the following command to start the Studio:

```
``bash
genlayer init
``
```

This command will download the necessary components and start the Studio. Once initialized, you can execute further commands (to be implemented) to interact with the Studio.

General Format of Commands

The GenLayer CLI commands follow a consistent syntax pattern:

```
``bash
genlayer <command> [options]
``
```

- `<command>`: The primary action you want the CLI to perform (e.g., `init`, `up`).
- `[options]`: Optional flags that alter the behavior of the command.

Example

To initialize the GenLayer Studio with specific parameters:

```
``bash
genlayer init --numValidators 10 --branch develop
``
```

Testing

The GenLayer CLI uses Jest with ts-jest for testing TypeScript files.

To run tests:

```
``bash
```

```
npm run test
...
```

Further Development

Additional commands are planned to enhance interaction with the GenLayer Studio. Stay tuned for updates.

Repository

You can find the GenLayer CLI repository on GitHub:

[GenLayer CLI Repository](https://github.com/genlayerlabs/genlayer-cli)

Full Reference

The full reference for the GenLayer CLI is available in the [GenLayer CLI Reference] (/references/genlayer-cli).

GenLayer FAQ

The GenLayer FAQ is a collection of useful questions and answers about the project. If you have a question that isn't answered here, please let us know on our [Discord] (https://discord.gg/8Jm4v89VAu) .

How do I set up the GenLayer Development Environment?

To quickly set up the GenLayer Development Environment, run the commands below:

```
$ npm install -g genlayer
$ genlayer init
```

For more detailed setup instructions, please refer to the [Getting Started page] (/getting-started)

Where can I find the GenLayer Studio?

The GenLayer Studio is available at studio.genlayer.com.

How do I deploy an Intelligent Contract?

- Go to studio.genlayer.com or use the [GenLayer CLI] (/getting-started#installation-of-the-genlayer-cli) to start the GenLayer Studio locally.
- Write your contract in Python using the [Intelligent Contract SDK] (/core-concepts/intelligent-contracts).
- Follow the guide [here] (/genlayer-stack/genlayer-studio/execute-transaction) to interact with the Studio and deploy your Intelligent Contract.

What is the GenLayer CLI used for?

The GenLayer CLI is used for setting up the GenLayer Studio and, in the future, will support mainnet and testnet environments.

GenLayer Studio

Source: https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio

GenLayer Studio is an interactive sandbox where developers explore the potential of GenLayer's Intelligent Contracts. It replicates the GenLayer network's execution environment and consensus algorithm, but offers a controlled and local environment to test different ideas and behaviors.

What you can do with the GenLayer Studio:

- **Experiment with Intelligent Contracts:** Intelligent Contracts leverage LLMs, such as GPT-4 or Llama3, to understand natural language and be capable of complex decision making.
- **Access the Web Natively:** GenLayer is the first platform where Intelligent Contracts don't need oracles to access the Internet.
- **Code in Python:** Develop in a familiar, developer-friendly language, where memory and string management are not a big headache.

Explore the Studio

Here is a video introduction to help you get started with the GenLayer Studio:

[Embedded content](https://www.youtube.com/embed/G4N-oErIIBI?si=ZkWRT6Ya2mjJWuP8)

Intelligent Contract SDK

> **Note:**

> Learn how to write your own Intelligent Contracts at [Intelligent Contracts](/build-with-genlayer/intelligent-contracts)

Load Your Contract

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/loading-contract>

Loading an Intelligent Contract in GenLayer Studio means adding your contract file to the Studio, opening it in the editor, and preparing it for deployment and execution. To start using the GenLayer Studio, navigate the Studio interface and select your contract file so you can run and debug it before deploying.

Access your Intelligent Contracts

1. On the left sidebar, click on the **Contracts** icon. This is where you can see the list of your Intelligent Contracts. There are pre-loaded example contracts for you to try out.

![Navigate Contract](/contract-sidebar.png)

2. To add a new contract, click on the **+** button labeled **New Contract** to create a new contract file in the Studio.

![Add a New Contract](/add-contract.png)

3. Click on the **Add From File** button to select and upload your Intelligent Contract file from your local machine.

![Add From File](/upload-file.png)

4. Once your contract is uploaded, you will see it listed under **Your Contracts**.

Run Your Intelligent Contract

1. Click on the contract file to open it in the editor pane. This allows you to review and edit the code if necessary before running it.

![Open the Contract](/open-contract.png)

2. Once you have your contract file open, you have two options to run and debug your contract:

- Click the **Run and Debug** button on the left sidebar.
- Click the **play icon** at the top right of the editor pane.

![Run and Debug](/run-and-debug.png)

Now that you have loaded your Intelligent Contract into the GenLayer Studio, you can proceed to set constructor parameters and deploy it.

Deploy Contracts

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/deploying-contract>

Deploying an Intelligent Contract in GenLayer Studio means setting the constructor parameters that initialize the contract state, then deploying the loaded contract. Once you have loaded your Intelligent Contract into the GenLayer Studio, the next step is to set the constructor parameters and deploy it.

Setting Constructor Parameters

1. After loading your Intelligent Contract, you will see the **Constructor Inputs** section on the left-hand pane. The constructor parameters are [automatically detected from your code if defined properly](#detecting-constructor-parameters).

![Constructor Parameter](/constructor-parameter.png)

2. If you need to manually adjust your constructor parameters, you can write them in JSON format by clicking on the **JSON** button.

![Manual Upload](/manual-upload.png)

Detecting Constructor Parameters

The GenLayer Studio automatically detects the constructor parameters from your code. It analyzes your `__init__` method to identify the parameters and their types. This automatic detection ensures that you have the correct inputs for initializing your contract. It's important to have clear type annotations for each parameter (e.g., `str`, `bool`, `int`, `list`) to enable accurate detection.

Deploying the Contract

After setting the constructor parameters, click on **Deploy** to deploy your contract.

![Deploy Constructor Parameter](/deploy-constructor-parameter.png)

Once completed, you can proceed to execute your transactions and interact with the deployed contract.

Current Intelligent Contract State

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/contract-state>

Current Intelligent Contract State is the deployed contract data shown through read methods in GenLayer Studio, used to verify that initialization succeeded. Once your Intelligent Contract is deployed, you can check its current state to confirm that the contract's data and variables are set as expected.

Viewing the Current Intelligent Contract State

After deploying your contract, you will now see:

- **Contract Address:** This is used for interacting with your contract.
- **Read Methods:** These return information on the current state of the contract.

For example, in the ****Storage**** contract:

- If the constructor parameter ``initial_storage`` was set to ``hello``, the state method ``get_storage`` will return ``hello``.

The result will show the current state based on the getter function called. This allows you to verify that the variables in your contract have been initialized correctly and that they change when transactions are executed.

> ****Note:****

> When the contract state changes, you can always come back and refresh the state by calling these getter functions to see the most current state of your Intelligent Contract.

With the state verified, you can now execute transactions and interact with your deployed contract.

Execute Transactions

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/execute-transaction>

Executing transactions in GenLayer Studio means calling the write methods of a deployed Intelligent Contract to perform specific actions or queries. After deploying your Intelligent Contract and verifying its state, use transaction execution to interact with the contract's methods and functions and review the returned results.

Write Methods

1. In the ****Write Methods**** section, you will see all the write methods available in your Intelligent Contract.
2. Expand the method you want to execute. For example, in the ****Storage**** contract, select the ``update_storage`` method.
3. If the method requires input parameters, a field will appear for you to enter the necessary values. For example, the ``update_storage`` method requires a string called ``new_storage``.

![Deploy Contract](./request-string.png)

4. After entering any required parameters, click the button to execute the method. The Studio will process the transaction and return the result.

Viewing Transaction Results

Once you execute a transaction, the result will be displayed, showing the output of the method called. This allows you to verify that the method has executed correctly and to see the effect of the transaction on your contract's state.

Viewing Current Account Address

You can also view your current account address in the header. This address is used to interact with your contract.

![Select Active Account](./dropdown-account.png)

You can continue interacting with your Intelligent Contract by executing additional transactions or checking the state again to see how it has changed.

Monitoring Node Logs

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/monitoring-node-logs>

Node Logs in GenLayer Studio are real-time logs for debugging and tracking how Intelligent Contracts behave as they interact within the studio. They help developers monitor contract activity, execution feedback, and related node behavior while working in GenLayer Studio.

Key Features of Node Logs

Node Logs capture and display a variety of events, including:

- **Transaction Submission, Status Changes and Finalization:** Track the status and outcomes of submitted transactions.
- **Contract Deployment Statuses:** Monitor the progress and success of contract deployments.
- **Execution Results and Errors:** Review the outputs and any errors encountered during contract execution.
- **Validator Operations and Consensus-Related Messages:** Observe the activities of validators and consensus processes.

Accessing Node Logs

You can view the node logs at the bottom of the GenLayer Studio. This section automatically updates with new log entries as actions are performed within the studio. You can also filter the logs by content as well as by scope and level.

![Node Log](/node-logs.png)

Understanding Node Logs

The logs are structured to provide clear and detailed information:

- **Scope:** Each log is tagged with a label such as RPC, GenVM or Consensus to give you a better understanding of the context of the log.
- **Level:** Log entries are associated with a level and color to facilitate rapid scanning for issues or confirmations.
- **Details:** Expandable log entries offer detailed explanations and data for each event, such as transaction hashes or error messages.

By actively viewing these logs, you can ensure that your Intelligent Contracts are performing as expected and that any issues are promptly addressed. This leads to a smoother development process within the GenLayer Studio.

Accessing and Configuring Validators

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/validators>

GenLayer Studio validators are configurable participants that help achieve consensus and validate transactions through [Optimistic Democracy](/understand-genlayer-protocol/core-concepts/optimistic-democracy). When you initialize the Studio based on your selected LLM provider(s), you are provided with 5 default validators that can be modified to suit your Intelligent Contract's requirements.

1. **Leaders:** Within this consensus model, one validator is selected as the leader for each transaction. The leader's role is to propose how a transaction should be executed based on the transaction data and the Intelligent Contract's logic.
2. **Validators:** After the leader proposes a transaction execution, other validators are responsible for reviewing and validating the proposal. They use the [Equivalence Principle](/understand-genlayer-protocol/core-concepts/optimistic-democracy/equivalence-principle) to determine if the leader's proposal meets the required standards.

Accessing Validators

To access and manage validators, follow these steps:

1. On the left sidebar, click on the **Validators** icon. This will open the validators page.
2. In the validators page, you will see a list with all the validators currently configured in the Studio along with their models and providers.

Configuring Validators

To configure validators, you can add new validators or modify existing ones.

1. Click the **New Validator** button to open the validator creation dialog.

2. **Create New Validator:**

In the validator creation dialog, fill in the required fields:

- **Provider:** Select the provider for the validator.
- **Model:** Choose the model that the validator will use.
- **Stake:** Specify the stake for the validator.
- **Config:** Enter any additional configuration parameters in JSON format.

3. Click **Create** to add the new validator to your configuration.

Modify Validator Details

To modify your validator settings, click on any existing validator and modify the fields based on your needs.

Delete Validator

To delete a validator, click on the delete button beside the validator in the list.

By properly managing and configuring validators, you ensure that your Intelligent Contracts operate smoothly and securely within the GenLayer Studio.

Inference Providers

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/providers>

Inference providers are the model backends that validators use to validate transactions in GenLayer Studio. You can configure those providers and add custom ones to suit your needs. When you create a new validator in the validators page, you will be able to select those providers and the relevant models from the list.

Accessing Providers

You can manage your providers in the **Settings** page:

Configuring Providers

Providers have the following properties:

- **Provider:** The name of the provider. For example, `OpenAI` or `ollama`.
- **Model:** The model attached to this provider. For example, `gpt-4o` or `llama3`.
- **Plugin:** (only for custom providers) When adding a custom provider, you need to specify a plugin that implements the provider. The plugin defines what configuration is necessary for the provider.
- **Provider Config:** Related environment variables, for example API keys or endpoint URLs.
- **Default Validator Config:** This config is used as default when creating a new validator, and will then be applied when making calls to inference providers using said validator.

Adding a new Provider

You can add a new provider by clicking the **New Config** button at the top of the providers list.

Updating an Existing Provider

To modify your validator settings, click on any existing validator and modify the fields based on your needs.

Deleting a Provider

To delete a provider, click on the delete button beside the provider in the list.

Limitations of the GenLayer Studio

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/limitations>

GenLayer Studio limitations are Studio-specific caveats for building and testing Intelligent Contracts outside a full live-network replica. GenLayer Studio supports core contract execution, consensus testing, appeals, web access, and native value transfers, but live-network behavior should be validated where exact gas, chain-layer, EVM, or web-access behavior matters.

Gas Usage

Studio is gasless for EVM-compatible wallet flows. `eth\_gasPrice` returns `0`, `eth\_estimateGas` returns a compatibility estimate, and transaction receipts should not be used to benchmark live-network gas usage or fees.

Chain-Layer and EVM Parity

Studio tracks contract and account state in its local database. Native value transfers are supported for local testing, but Studio does not fully model the live network's chain-layer and ghost-contract behavior.

EVM contract interaction beyond value transfers to EOAs or addresses is not implemented in Studio. `@gl.evm.contract\_interface` calls that depend on EVM contract execution should be validated on a live network.

Web Access

Studio web access depends on its local browser/WebDriver service and the network access available to your Studio environment. Validate production-critical web reads against the

target network before relying on exact rendering, timing, or availability behavior.

Resetting the GenLayer Studio

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/reset-the-studio>

Resetting the GenLayer Studio clears its storage, current status, memory, and contract examples so the workspace returns to a clean state. Use this feature when you want to start fresh, test new contracts, or remove previous configurations.

How to Reset the Studio

To reset the GenLayer Studio, follow these simple steps:

1. ****Access the Settings Page****: begin by navigating to the Settings page within the GenLayer Studio.

2. ****Click "Reset Storage"****: this option allows you to clear all the current data stored in the studio, including the state of deployed contracts, transaction history, and any cached information.

3. ****Confirm the Reset in the Dialog****: this is an important step, as resetting the storage will remove all existing data and cannot be undone.

Once confirmed, the studio will reset its storage, returning to its initial state as if it were newly initialized.

What Does Resetting Do?

Resetting the GenLayer Studio through the Reset Storage option will:

- Clear All Deployed Contracts: Any contracts that you have deployed in the studio will be removed from the frontend memory.
- Reset Transaction History: All previous transactions, including those in various states, will be erased.
- Restore Contract Examples: Any example contracts included in the studio will be restored to their original, unaltered state.

When Should You Reset?

Consider resetting the studio when:

- You want to start with a clean slate for a new development or testing session.
- You encounter issues that might be caused by lingering data or state inconsistencies.
- You have completed a testing phase and need to ensure that previous data does not interfere with new experiments.

Troubleshooting

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/troubleshooting>

GenLayer Studio troubleshooting covers common issues that can prevent the Studio from loading, starting, displaying correctly, or responding as expected. If you encounter any issues with the Studio, use these steps to resolve common problems:

Frontend Not Loading Correctly

The frontend may not load correctly due to caching issues or an outdated version of the contract stored in your browser.

Solution

- ****Refresh Frontend****: Simply reload the page in your browser.
- ****Clear Cache****: Clear your browser's cache by clicking on the left of your address bar -> Cookies and site data -> Manage on device site data -> Delete localhost.

![Node Log](/troubleshoot.png)

Port Conflicts

The Studio may fail to start if some ports required for its operation are already in use by other applications.

Ports used by the Studio:

- **Frontend:** 8080
- **Ollama:** 11434
- **JSON-RPC Server:** 4000
- **GenVM:** 6678
- **Postgres:** 5432
- **Hardhat:** 8545
- **Webrequest:** 5000, 5001

Solution

1. Identify processes using the required ports. Replace `` with the specific port number.

- **Linux / MacOS:**
```bash copy  
lsof -i :<PORT\_NUMBER>  
```
- **Windows:**
```cmd copy  
netstat -aon | find <PORT\_NUMBER>  
```

2. Use the following command to kill the process using the port. Replace `` with the Process ID obtained from the previous step.

- **Linux / MacOS:**
```bash copy  
kill -9 <PID>  
```
- **Windows:**
```cmd copy  
taskkill /PID <PID> /F  
```

Docker not Running

The GenLayer Studio relies on Docker for managing containers and images. If Docker is not running, the Studio cannot function properly.

Solution

- Ensure that Docker (or Docker Desktop) is installed on your system. You can check this by running:

```
```bash copy  
docker info
```
```

- Restart Docker Desktop or the Docker daemon on your system.

Display Issues

The code section or other UI elements may not display correctly due to screen size limitations.

Solution

- Resize your screen to accommodate the UI elements properly. This will help improve the visibility and layout of the Studio.

Studio Not Responding or Throwing Errors

The Studio may become unresponsive or throw errors due to various reasons, such as conflicting processes or corrupted containers.

Solution

You can perform a fresh start by stopping and removing all containers, as well as removing

all images.

****Via Command Line:****

```
```bash
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)
docker rmi $(docker images -q)
```
```

****Via Docker Desktop:****

1. Open Docker Desktop and go to the ****Containers**** section.
3. Select all containers and click ****Delete****.
4. Go to the ****Images**** section.
5. Select all images and click ****Delete****.

Development Tips

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/development-tips>

GenLayer Studio development tips explain how to develop your Intelligent Contract, debug it, and test it in GenLayer Studio.

1. Validators setup

You need to be sure that you have at least one validator. If you don't have any, you can create one in the validators screen by providing:

- Provider
- Model
- Stake (at least 1)
- Config (optional)

2. Add types to your contract method inputs

Adding types to your contract method inputs helps the UI render the correct input fields. For example, if you have a method that takes a string as input, you can define it as follows:

```
```python
def update_storage(self, new_storage:str) -> None:
 ...
 self.storage = new_storage
...
```
```

This will allow the UI to render the form needed to call the contract with the correct input type as follows

![Node Log](/images/run-debug/input-types.png)

3. The constructor

The constructor is a special method called when the contract is deployed. It is used to initialize the contract's storage. You can define it as follows:

```
```python
def __init__(self, initial_storage: str):
 ...
 self.storage = initial_storage
...
```
```

Right now, your contract must include a constructor. Without one, the Studio won't be able to deploy your contract.

4. Deploying a contract and sending transactions

After you have written your contract, you can deploy it by clicking the "Deploy" button. This will deploy the contract and show you the contract's address.

You can then call the contract's methods, shown below in the deployment section on the left panel of the Studio's UI.

![Node Log](/images/run-debug/read-write-methods.png)

5. Debugging your contract

You can debug your contract by adding print statements to your contract methods. The

Studio's Logs section will display these print statements.
Here is an example of printing in a contract method:

```
```python
def update_storage(self, new_storage:str) -> None:
 print("new storage value: ", new_storage)
 self.storage = new_storage
```
```

When a transaction is sent to this method, the print statement will be shown in the Logs section as follows:

![Node Log](/images/run-debug/print-logs.png)

6. What happened when the transaction was executed

After a transaction is executed, you can see the details by clicking on the transaction in the Transactions section (left-bottom corner). This will show you:

- Number: The transaction's number in the database
- Timestamp: The time the transaction was executed
- Type: The transaction's type (deployment or method call)
- Status: The transaction's status (PENDING, PROPOSING, COMMITTING, REVEALING, ACCEPTED, FINALIZED)
- Input: the parameters passed to the method
- Execution: The status of the execution (SUCCESS or ERROR), the leader's configuration, and the validator's consensus votes
- Equivalence principle output: The output of the equivalence principle from the leader

![Node Log](/images/run-debug/transaction-modal.png)

7. Knowing what happens when the LLMs are called

If a contract method calls an LLM, you can see the Equivalence Principle Output from the leader (what's being input in the eq.set call) in the transaction details modal.

![Node Log](/images/run-debug/transaction-modal-eq-output.png)

8. Access to the contract state

If your contract methods change the contract's state, you can query that information by adding a read method to your contract. Read methods on a contract need to be decorated with the `@gl.public.view` decorator. This method should return the state you want to query.

For example:

```
```python
@gl.public.view
def get_balances(self) -> dict[str, int]:
 return self.balances

@gl.public.view
def get_balance_of(self, address: str) -> int:
 return self.balances.get(address, 0)
```
```

These two methods allow you to query the contract's balance state. You can call them by expanding them from the left panel of the Studio's UI and clicking on the "Call Contract" button.

Adding LLM Provider Plugins

Source: <https://docs.genlayer.com/developers/intelligent-contracts/tools/genlayer-studio/advanced-features/custom-plugins>

LLM provider plugins in GenLayer Studio let advanced users integrate custom LLM providers beyond the providers supported out of the box.

> **Note:**

- > This section is intended for advanced users who wish to integrate new LLM provider plugins. This process requires code modifications, and we encourage you to create a Pull Request if you'd like your plugin to be included in the

```

`python
class MyCustomPlugin:
    def __init__(self, plugin_config: dict):
        self.api_key = plugin_config.get('api_key')
        self.base_url = plugin_config.get('base_url', 'https://api.customllm.com')
        # Add any other necessary configuration parameters

    async def call(
        self,
        node_config: dict,
        prompt: str,
        regex: Optional[str],
        return_streaming_channel: Optional[asyncio.Queue],
    ) -> str:
        # Implement the API call to your custom LLM provider
        # This is a placeholder implementation
        async with aiohttp.ClientSession() as session:
            async with session.post(
                f"{self.base_url}/generate",
                json={"prompt": prompt, "config": node_config},
                headers={"Authorization": f"Bearer {self.api_key}"}) as response:
                if response.status == 200:
                    result = await response.json()

```

```

        return result['generated_text']
    else:
        raise Exception(f"API call failed with status {response.status}")

def is_available(self) -> bool:
    # Check if the plugin is properly configured and available
    return bool(self.api_key) and bool(self.base_url)

def is_model_available(self, model: str) -> bool:
    # Check if the specified model is available for this provider
    # This is a placeholder implementation
    available_models = ['custom-gpt-3', 'custom-gpt-4']
    return model in available_models
...

### Registering the Plugin

```

After implementing the `Plugin` protocol, you need to register your new provider in the `llms.py` file under the `get\_llm\_provider` function. This step ensures that the GenLayer Studio recognizes and can utilize your custom plugin.

```

```python
def get_llm_plugin(plugin: str, plugin_config: dict) -> Plugin:
 """
 Function to register new providers
 """
 plugin_map = {
 "ollama": OllamaPlugin,
 "openai": OpenAIPlugin,
 "anthropic": AnthropicPlugin,
 "custom-plugin-key": MyCustomPlugin, # Modify here accordingly
 }

 if plugin not in plugin_map:
 raise ValueError(f"Plugin {plugin} not registered.")

 return plugin_map[plugin](plugin_config)
...

```

## ## Updating the JSON Schema

To maintain consistency and enable proper configuration validation, you must update the [JSON schema](https://json-schema.org/) in the [`providers\_schema.json`](https://github.com/genlayerlabs/genlayer-studio/blob/main/backend/node/create\_nodes/providers\_schema.json) file. This update should include your new provider and its specific configuration options.

The JSON schema plays a crucial role in validating the configuration options for each provider, ensuring that users input correct and compatible settings.

## ## Registering a Provider for the new plugin

New LLM Providers can be registered through the Studio's UI in the **Settings page** or manually through the backend API. Here is an example on how addition of an LLM Provider using your new Plugin could look like

```

```sh
curl --request POST \
  --url http://localhost:4000/api \
  --header 'Content-Type: application/json' \
  --data '{
    "jsonrpc": "2.0",
    "method": "sim_addProvider",
    "params": [
      {

```



```

        "provider": "my-custom-provider",
        "model": "my-custom-model",
        "config": {},
        "plugin": "custom-plugin-key",
        "plugin_config": {
            "api_key": "MY-SECRET-KEY",
            "base_url": "my.custom.url"
        }
    },
    "id": 1
}

```

💡 Build With GenLayer

Source: <https://docs.genlayer.com/developers/intelligent-contracts/ideas>

GenLayer Intelligent Contract ideas are example applications that use web sources, APIs, and LLM calls to understand natural language and act on real-world information. Use these ideas as starting points for building Intelligent Contracts on GenLayer.

Idea	Implementation Details
Prediction Markets	Use GenLayer to directly search the web for relevant information allowing users to place bets on future events and get outcomes automatically determined.
Parametric Insurance	Create insurance that pays out based on real-world events, such as drought conditions or natural disasters, using data fetched from reliable sources.
Bounty Review and Payout	Automatically reward community members for completing tasks like writing articles or developing features by evaluating their work using Intelligent Contracts.
Performance-based Contracting	Set up contracts that pay out based on the completion of pre-agreed jobs, such as developing a website or creating a piece of content, with performance verified through automated checks.
Slashing Monitoring and Insurance	Monitor protocols for any violations and trigger slashing events or provide insurance payouts based on human-in-the-loop evaluations or LLM-based assessments.
Hack Detection and Emergency Pause	Set up contracts that continuously monitor for potential protocol attacks through explorers and news sites, and trigger automated emergency shutdowns if threats are detected.
On-chain Identity Verification	Verify users' identities by checking their social media profiles for specific messages and linking these to their on-chain accounts, enhancing trust in the ecosystem.
Under-collateralized Lending	Enable lending with less collateral by linking real-world identity to on-chain reputation, allowing borrowers to leverage their good standing for better loan terms.
P2P Gambling	Allow two users to place bets on real-world outcomes, with the contract determining the winner based on data from trusted sources.
Decentralized Game Master	Facilitate text-based role-playing games where users submit their actions, and the intelligent contract determines the outcomes, creating a dynamic and interactive gaming experience.
Interoperable Games with NFTs	Develop games that issue and recognize automatically generated NFT items, allowing players to transfer assets seamlessly between different games within the ecosystem.
Unstoppable Organizations	Create autonomous entities like DAOs that can adapt and continue their missions indefinitely as long as they remain funded, including AI DAOs and autonomous trusts.
Retroactive Public Goods Funding	Set up contracts that reward contributions to the

protocol retroactively, incentivizing community members to make valuable improvements and innovations.

| Crowd-sourced Knowledge Database | Implement a system where users are rewarded for finding and summarizing new information on various topics, building a comprehensive knowledge base.

| AI Notary | Create an automated notary service that can confirm the occurrence of online events, providing verifiable records for various purposes.

| AI Arbitration | Develop a dispute resolution system where parties submit their cases to an AI-based arbitrator, which makes decisions based on pre-defined legal frameworks and submitted evidence.

| Private P2P Contracts | Enable two-party contracts that remain private unless a dispute arises, using a commit/reveal scheme to keep terms confidential until necessary.

| Multi-modal Use Cases | Integrate real-world images for performance-based contracting, such as verifying the completion of physical tasks, with cryptographically signed proofs.

| Generative Memes | Create fun and engaging Intelligent Contracts, like ones that track and manipulate token balances in creative ways, leveraging the capabilities of LLMs for humor and experimentation.

| Text-based Intelligent Contracts | Open the possibility for non-developers to create Intelligent Contracts by capturing the entire logic in plain text, making smart contract development more accessible.

| Honeypot Contracts for Security | Set up Intelligent Contracts designed to attract and test adversarial attacks, helping to identify and fix vulnerabilities through real-world testing.

| Fair and Transparent Moderation | Use GenLayer-based arbitration to make fair and transparent moderation decisions in various communities and games, ensuring unbiased and consistent enforcement of rules.

Architecture Overview of Decentralized Applications Using GenLayer

Source: <https://docs.genlayer.com/developers/decentralized-applications/architecture-overview>

A decentralized application (DApp) on GenLayer is an application architecture that connects a frontend to the GenLayer protocol through coordinated components, including AI-powered Intelligent Contracts. These DApps leverage GenLayer's unique blockchain architecture so the components can work together for seamless protocol interaction.

Key Components

Frontend Application

The user interface of the DApp is typically built with web technologies like HTML, CSS, and JavaScript. It communicates with the backend using the GenLayerJS SDK, enabling interaction with the GenLayer protocol and handling user inputs to trigger blockchain transactions.

GenLayerJS SDK

A TypeScript/JavaScript library that abstracts the complexities of blockchain interactions. The SDK provides APIs to read from and write to Intelligent Contracts, manages accounts and queries transactions, and acts as the bridge between the frontend and GenLayer's protocol.

Consensus Layer

The consensus layer implements GenLayer's Optimistic Democracy mechanism to ensure reliable and secure execution of transactions using a validator-driven commit-reveal scheme. It also handles appeals and transaction finality to maintain integrity and fairness.

It is built on ZK-stack rollups to provide scalability and cost-efficiency, secure state updates, and anchoring to Ethereum's security model.

Execution Environment

The execution environment (the GenVM) is the engine that executes Intelligent Contracts. It supports both deterministic and non-deterministic operations, enabling AI integration and web data access.

Ghost Contracts

Ghost contracts are proxy smart contracts on the consensus layer that facilitate interactions between accounts and Intelligent Contracts. They also manage external messages and asset bridging.

Architecture Diagram

```
```mermaid
flowchart TD
 subgraph s1["Frontend Application"]
 n1["GenLayerJS"]
 end
 subgraph s2["Consensus Layer."]
 n2["Consensus Smart Contracts"]
 n3["Ghost Contracts"]
 end
 subgraph s3["GenLayer Node"]
 n4["Execution Environment - GenVM"]
 end
 s1 <--> s3
 s3 <--> s2
```
```

DApp Development Workflow with GenLayer

Source: <https://docs.genlayer.com/developers/decentralized-applications/dapp-development-workflow>

DApp development with GenLayer is the lifecycle for creating, testing, deploying, and connecting decentralized applications built with Intelligent Contracts. This guide explains how developers can move from using **GenLayer Studio** for their first Intelligent Contract to an advanced local development workflow, then finish by building a frontend application with **GenLayerJS**.

1. Starting with GenLayer Studio

GenLayer Studio is an integrated environment designed to streamline the initial stages of Intelligent Contract development, making it accessible for developers of all experience levels.

It provides an interactive environment that serves as a comprehensive sandbox for developing, deploying, and testing Intelligent Contracts in real time. This environment enables developers to experiment with their contracts and see immediate results without the complexities of a production setup.

The platform runs a simulated network with customizable validators that accurately mirror the GenLayer consensus mechanism. This feature allows developers to test their contracts under conditions that closely resemble the actual blockchain environment, ensuring reliable deployment outcomes.

Getting Started:

1. **Set Up the Studio**: Developers initialize the Studio using `genlayer cli` command `genlayer init` which configures the environment and spawns a local validator network. GenLayer Studio is also available as a hosted instace at [studio.genlayer.com] (<https://studio.genlayer.com/>).
2. **Write Your First Contract**: Intelligent Contracts in GenLayer are written in Python, utilizing its extensive libraries and GenVM capabilities like LLM calls and web integration. Refer to [Your First Contract](/developers/intelligent-contracts/your-first-contract) guide for more information.
3. **Deploy and Test**: Deploy your Intelligent Contracts through the Studio interface and test them in the simulated network.

2. Moving to Advanced Workflow

As projects transform to real-world applications, developers should migrate to a local development setup. This approach mirrors frameworks like Hardhat or Foundry and is well-suited for iterative development and comprehensive testing.

Benefits of Local Development:

- **Complete Control**: Developers can configure the environment, validators, and network parameters as needed.
- **Enhanced Debugging**: The local setup allows for advanced debugging of both contracts and transactions.
- **Flexible Testing**: Tests can simulate real-world scenarios, including edge cases and complex interactions.

Workflow Steps:

1. **Set Up Your Local Environment**: Developers can start with the GenLayer boilerplate project, which includes pre-configured templates for local testing.
2. **Write Tests**: Tests are written in Python, focusing on validating contract functionality and ensuring consensus integrity. The boilerplate includes sample tests to accelerate development.
3. **Simulate Transactions**: Run detailed simulations to observe how contracts behave under various network conditions, ensuring robust performance.

3. Building the Frontend with GenLayerJS

The frontend is the user-facing component of a DApp. With **GenLayerJS**, developers can integrate their applications with the GenLayer network, focusing on providing a seamless user experience.

Why Use GenLayerJS?

- **Simplified Interactions**: Abstracts the complexity of blockchain interactions with high-level APIs.
- **Comprehensive Features**: Supports transaction handling, contract interaction, event subscriptions, and account management.
- **TypeScript Integration**: Provides type safety and code reliability for frontend development.

Frontend Development Workflow:

1. **Integrate GenLayerJS**:

Install the SDK and set up a client to connect to the GenLayer network. Configuration options allow connection to various environments, such as testnets or the GenLayer Studio. Refer to [\[GenLayerJS\]\(/developers/decentralized-applications/genlayer-js\)](#) guide for more information.

2. **Read and Write to Contracts**:

Use the SDK's high-level APIs to interact with deployed contracts. For instance, retrieve user balances or update contract state seamlessly.

Refer to [\[Reading Data\]\(/developers/decentralized-applications/reading-data\)](#) and [\[Writing Data\]\(/developers/decentralized-applications/writing-data\)](#) guides for more information.

3. **Monitor Transactions**:

Developers can subscribe to events or query transaction statuses, ensuring users are kept informed of transaction progress and outcomes.

4. **Build the User Interface**:

Combine GenLayerJS with popular frontend frameworks (like React or Angular) to create

intuitive interfaces.

Advanced Tips for Developers

1. ****Leverage GenVM Features****:

The GenVM allows Intelligent Contracts to interact with LLM models and access real-time web data. Developers should design contracts to maximize these capabilities for dynamic and intelligent DApps.

2. ****Optimize Testing****:

Incorporate edge case scenarios in tests to ensure that contracts behave reliably under all conditions.

3. ****Focus on Security****:

Implement robust security measures, including input validation and error handling, to protect contracts from malicious inputs and ensure consensus consistency.

GenLayerJS SDK

Source: <https://docs.genlayer.com/developers/decentralized-applications/genlayer-js>

GenLayerJS SDK is a TypeScript library for building decentralized applications (DApps) on the GenLayer protocol. It provides tools for interacting with the GenLayer network, including client creation, transaction handling, event subscriptions, and more, while using Viem as the underlying blockchain client.

Features

- ****Client Creation****: Easily create and configure a client to connect to GenLayer's network.
- ****Transaction Handling****: Send and manage transactions on the GenLayer network.
- ****Contract Interaction****: Read from and write to smart contracts deployed on GenLayer.
- ****Event Subscriptions****: Subscribe to events and listen for blockchain updates.
- ****TypeScript Support****: Benefit from static typing and improved code reliability.

How it's Built

The GenLayerJS SDK is built using ****TypeScript**** and leverages the ****Viem**** library as the underlying blockchain client. It is designed to provide a high-level, easy-to-use API for interacting with the GenLayer network, abstracting away the complexities of direct blockchain interactions.

Technologies Used

- ****TypeScript****: A statically typed superset of JavaScript that compiles to plain JavaScript, enhancing code reliability and maintainability.
- ****Viem****: A modular and extensible blockchain client for JavaScript and TypeScript.
- ****ESBuild****: A fast JavaScript bundler and minifier used for building the SDK efficiently.
- ****Vitest****: A Vite-native unit test framework used for testing.

Project Structure

The source code for the GenLayerJS SDK is organized as follows:

- ``src/``: Contains the main TypeScript source files.
- ``tests/``: Includes all the test files written using Vitest.
- ``dist/``: The compiled JavaScript files ready for use.

Requirements

Before using the GenLayerJS SDK, ensure your system meets the following requirements:

- **Node.js**: Version 16.x or higher is required.
- **npm**: Comes bundled with Node.js, used for managing packages.
- **Operating System**: Compatible with macOS, Linux, and Windows.

Installation

To install the GenLayerJS SDK, use the following command:

```
```bash
npm install genlayer-js
```
```

Usage

Here's how to initialize the client and connect to the GenLayer Studio:

- **[Reading a transaction](../intelligent-contracts/features/interacting-with-intelligent-contracts)**: Understand how to query transactions on the GenLayer network.
- **[Reading a contract](/developers/decentralized-applications/reading-data)**: Discover how to read data from intelligent contracts on GenLayer.
- **[Writing a contract](../intelligent-contracts/tools/genlayer-studio/contract-state)**: Learn how to write data to intelligent contracts on GenLayer.

General Format of Commands

The GenLayerJS SDK provides functions that follow a consistent syntax pattern:

```
```typescript
client.functionName(parameters);
```
```

- **`client`**: The instance of the GenLayer client.
- **`functionName`**: The primary action you want to perform (e.g., ``getTransaction``, ``readContract``).
- **`parameters`**: An object containing the required parameters for the function.

Further Development

Additional features are planned to enhance interaction with the GenLayer network, including wallet integration and gas estimation. Stay tuned for updates.

Repository

You can find the GenLayerJS SDK repository on GitHub:

[GenLayerJS SDK Repository](https://github.com/genlayerlabs/genlayer-js)

Full Reference

The full reference for the GenLayerJS SDK is available in the [GenLayerJS SDK Reference](/references/genlayer-js).

Querying a Transaction

Source: <https://docs.genlayer.com/developers/decentralized-applications/querying-a-transaction>

Querying a transaction in GenLayer means reading a submitted network transaction by its hash to inspect its details. Use transaction queries to monitor transaction status, debug behavior, and verify transaction details.

Basic Transaction Reading

Here's the simplest way to read a transaction:

```
``typescript
import { simulator } from 'genlayer-js/chains';
import { createClient } from 'genlayer-js';

const client = createClient({
  chain: simulator,
});

const transactionHash = "0x...";

const transaction = await client.getTransaction({
  hash: transactionHash
});

{/*
## Transaction Data Structure
```

When you read a transaction, you get access to its complete data structure:

```
``typescript
interface Transaction {
  hash: `0x${string}` // The unique transaction hash
  from: `0x${string}` // Address of the sender
  to: `0x${string}` | null // Address of the recipient (null for contract deployments)
  nonce: number // Transaction sequence number
  value: bigint // Amount of native tokens transferred
  data: `0x${string}` // Transaction input data
  timestamp: number // Block timestamp when transaction was included
  status: 'success' | 'failure' | 'pending' // Current transaction status
  blockNumber: bigint | null // Block number where transaction was included
  blockHash: `0x${string}` | null // Hash of the block
  // ... additional fields
}
```

Reading Different Transaction Types

Basic Transfer Transaction

```
``typescript
const transferTx = await client.getTransaction({
  hash: "0x123...",
});

console.log({
  from: transferTx.from,
  to: transferTx.to,
  value: transferTx.value,
  status: transferTx.status
});
```

Contract Interaction Transaction

```
``typescript
const contractTx = await client.getTransaction({
  hash: "0x456...",
});

// Decode the transaction input data
const decodedInput = client.decodeTransactionInput({
  data: contractTx.data,
  abi: contractABI, // You need the contract's ABI
```

```

});

console.log({
  contractAddress: contractTx.to,
  functionName: decodedInput.functionName,
  arguments: decodedInput.args
});

### Contract Deployment Transaction
```typescript
const deployTx = await client.getTransaction({
 hash: "0x789...",
});

console.log({
 deployer: deployTx.from,
 contractAddress: deployTx.create, // Address of deployed contract
 deploymentData: deployTx.data
});
``` */}

## Error Handling
```typescript
async function getTransactionWithRetry(
 client: GenLayerClient,
 hash: string,
 maxAttempts = 3
): Promise<Transaction> {
 for (let attempt = 1; attempt <= maxAttempts; attempt++) {
 try {
 const tx = await client.getTransaction({ hash });
 if (!tx) throw new Error('Transaction not found');
 return tx;
 } catch (error) {
 if (attempt === maxAttempts) throw error;
 if (error.message.includes('not found')) {
 // Wait longer between retries for not found errors
 await new Promise(resolve => setTimeout(resolve, 2000 * attempt));
 continue;
 }
 throw error; // Rethrow other errors immediately
 }
 }
 throw new Error('Failed to fetch transaction after max retries');
}
```

## Monitoring Transaction Status
```typescript
async function monitorTransaction(
 client: GenLayerClient,
 hash: string,
 interval = 1000
): Promise<Transaction> {
 return new Promise((resolve, reject) => {
 const checkStatus = async () => {
 try {
 const tx = await client.getTransaction({ hash });

 if (!tx) {
 setTimeout(checkStatus, interval);
 return;
 }
 }
 };
 checkStatus();
 });
}

```



```

 }

 if (tx.status === 'pending') {
 setTimeout(checkStatus, interval);
 return;
 }

 resolve(tx);
 } catch (error) {
 reject(error);
 }
};

checkStatus();
});
}

// Usage
const finalTx = await monitorTransaction(client, "0x...");
\`\`\`

```

---

# Reading from Intelligent Contracts  
Source: <https://docs.genlayer.com/developers/decentralized-applications/reading-data>

Reading from Intelligent Contracts means querying contract state or executing view functions without modifying blockchain state. These read-only operations are free, require no fees, and provide immediate access to contract data.

## ## Understanding View Operations

In GenLayer, functions marked with the `@gl.public.view`` decorator are read-only operations that:

- Don't modify the contract's state
- Can be executed without requiring a transaction
- Return data immediately
- Don't consume gas
- Can be called by any account

## ## Basic Contract Reading

Here's how to read from an Intelligent Contract:

```

\`\`\`typescript
import { simulator } from 'genlayer-js/chains';
import { createClient, createAccount } from 'genlayer-js';

const account = createAccount();
const client = createClient({
 chain: simulator,
 account: account,
});

const result = await client.readContract({
 address: contractAddress,
 functionName: 'get_complete_storage',
 args: [],
});
\`\`\`

```

## ### Parameters Explained

- ``address``: The deployed contract's address on the GenLayer network
- ``functionName``: The name of the view function you want to call

- `args`: An array of arguments that the function accepts (empty if none required)

## ## Common View Operations

Intelligent Contracts typically include several types of view functions:

### ### State Queries

```typescript

// Reading a single value

```
const balance = await client.readContract({
  address: contractAddress,
  functionName: 'get_balance',
  args: [accountAddress],
});
```

// Reading multiple values

```
const userInfo = await client.readContract({
  address: contractAddress,
  functionName: 'get_user_info',
  args: [userId],
});
```

Computed Values

```typescript

// Getting calculated results

```
const totalSupply = await client.readContract({
 address: contractAddress,
 functionName: 'calculate_total_supply',
 args: [],
});
```

### ### Validation Checks

```typescript

// Checking permissions

```
const hasAccess = await client.readContract({
  address: contractAddress,
  functionName: 'check_user_permission',
  args: [userId, 'ADMIN_ROLE'],
});
```

Error Handling

When reading from contracts, you should handle potential errors:

```typescript

```
try {
 const result = await client.readContract({
 address: contractAddress,
 functionName: 'get_data',
 args: [],
 });
 console.log('Data retrieved:', result);
} catch (error) {
 if (error.message.includes('Contract not found')) {
 console.error('Invalid contract address');
 } else if (error.message.includes('Function not found')) {
 console.error('Invalid function name');
 } else {
 console.error('Error reading contract:', error);
 }
}
```

## ## Best Practices

1. **\*\*Cache Results\*\***: For frequently accessed data that doesn't change often, consider caching the results.
2. **\*\*Batch Readings\*\***: When possible, use functions that return multiple values instead of making multiple separate calls.
3. **\*\*Type Safety\*\***: Use TypeScript interfaces to ensure type safety when handling returned data:

---

## # Writing to Intelligent Contracts

Source: <https://docs.genlayer.com/developers/decentralized-applications/writing-data>

Writing to an Intelligent Contract means sending a transaction that modifies the contract's state. Unlike read operations, write operations require fees and must be processed by the network before taking effect.

## ## Understanding Write Operations

In GenLayer, functions that modify state:

- Require a transaction to be sent to the network
- Consume gas (computational resources)
- Need time to be processed and finalized
- Must be signed by an account with sufficient balance to pay for the transaction fees
- Return a transaction hash immediately, but state changes are not instant

## ## Basic Contract Writing

Here's how to write to an Intelligent Contract:

```
``typescript
import { simulator } from 'genlayer-js/chains';
import { createClient, createAccount } from 'genlayer-js';
import type { TransactionStatus } from 'genlayer-js/types';

const account = createAccount();
const client = createClient({
 chain: simulator,
 account: account,
});

// Send the transaction
const transactionHash = await client.writeContract({
 address: contractAddress,
 functionName: 'update_storage',
 args: ['new_data'],
 value: 0, // Optional: amount of native GEN tokens to send
});

// Wait for the transaction to be processed
const receipt = await client.waitForTransactionReceipt({
 hash: transactionHash,
 status: TransactionStatus.FINALIZED, // or 'ACCEPTED'
});
```

## ### Parameters Explained

- ``address``: The deployed contract's address
- ``functionName``: The name of the function to call
- ``args``: Array of arguments for the function
- ``value``: Amount of native tokens to send (in wei)

## ## Transaction Lifecycle

### 1. \*\*Transaction Creation\*\*

```
``typescript
const transactionHash = await client.writeContract({
 address: contractAddress,
 functionName: 'mint_token',
 args: [recipient, amount],
});
```

### 2. \*\*Transaction Status Monitoring\*\*

```
``typescript
// Basic waiting
const receipt = await client.waitForTransactionReceipt({
 hash: transactionHash,
 status: 'FINALIZED',
});

// Advanced monitoring with timeout
const receipt = await client.waitForTransactionReceipt({
 hash: transactionHash,
 status: 'FINALIZED',
 interval: 5_000, // check every 5 seconds
 retries: 10, // maximum number of retries
});
```

## ## Common Write Operations

### ### Calling Payable Methods

Send GEN with a write call using the `value` parameter:

```
``typescript
// Send 5 GEN to a payable method
const hash = await client.writeContract({
 address: contractAddress,
 functionName: 'tip',
 args: [],
 value: BigInt(5) * BigInt(10 ** 18), // 5 GEN in wei
});
```

The `value` is specified in wei (1 GEN =  $10^{18}$  wei). The receiving contract method must be decorated with `@gl.public.write.payable`.

### ### Token Transfers

```
``typescript
// Sending tokens
const hash = await client.writeContract({
 address: tokenContractAddress,
 functionName: 'transfer',
 args: [recipientAddress, amount],
});
```

### ### State Updates

```
``typescript
// Updating user data
const hash = await client.writeContract({
 address: contractAddress,
 functionName: 'update_user_profile',
 args: [userId, newProfile],
});
```

...

## ## Using a Browser Wallet (MetaMask)

When building a browser dApp, transactions are signed by the user's wallet instead of a local private key. You need to:

1. Create a client with the wallet's address and provider
2. Switch the wallet to the correct GenLayer network
3. Send transactions as usual

```
``typescript
import { createClient } from "genlayer-js";
import { studionet } from "genlayer-js/chains";
import { TransactionStatus } from "genlayer-js/types";

// Create client with wallet address (from your wallet connection flow)
const client = createClient({
 chain: studionet,
 account: walletAddress as `0x${string}`,
});

// Switch the wallet to the correct network – must be called before writing
await client.connect("studionet");

// Now write as usual – MetaMask will prompt for signing
const txHash = await client.writeContract({
 address: contractAddress,
 functionName: "create_profile",
 args: ["alice", "Hello world"],
 value: BigInt(0),
});

const receipt = await client.waitForTransactionReceipt({
 hash: txHash,
 status: TransactionStatus.ACCEPTED,
});
```

Available networks: ``"localnet"`, `"studionet"`, `"testnetAsimov"`, `"testnetBradbury"`.`

> **\*\*Important:\*\*** If the wallet is on the wrong chain, the SDK will throw an error telling you which chain the wallet is on vs. which chain the client expects. Always call ``client.connect()`` before sending transactions.

## ## Error Handling

```
``typescript
try {
 const hash = await client.writeContract({
 address: contractAddress,
 functionName: 'update_data',
 args: ['new_data'],
 });

 try {
 const receipt = await client.waitForTransactionReceipt({
 hash,
 status: 'FINALIZED',
 });
 console.log('Transaction successful:', receipt);
 } catch (waitError) {
 console.error('Transaction failed or timed out:', waitError);
 }
} catch (error) {
```

```

 if (error.message.includes('insufficient funds')) {
 console.error('Not enough balance to pay for transaction');
 } else if (error.message.includes('user rejected')) {
 console.error('User rejected the transaction');
 } else {
 console.error('Error sending transaction:', error);
 }
}
}

```

## ## Transaction Status Types

GenLayer transactions can have different status requirements:

```

```typescript
enum TransactionStatus {
    PENDING = "PENDING",
    CANCELED = "CANCELED",
    PROPOSING = "PROPOSING",
    COMMITTING = "COMMITTING",
    REVEALING = "REVEALING",
    ACCEPTED = "ACCEPTED",
    FINALIZED = "FINALIZED",
    UNDETERMINED = "UNDETERMINED",
}

// Wait for just acceptance (faster)
const acceptedReceipt = await client.waitForTransactionReceipt({
    hash: transactionHash,
    status: TransactionStatus.ACCEPTED,
});

// Wait for full finalization
const finalizedReceipt = await client.waitForTransactionReceipt({
    hash: transactionHash,
    status: TransactionStatus.FINALIZED,
});
```

```

## ## Best Practices

1. **\*\*Always Wait for Receipts\*\***: Don't assume a transaction is successful just because you got a hash.
2. **\*\*Handle Timeouts\*\***: Set appropriate timeouts for transaction waiting.
3. **\*\*Implement Retry Logic\*\***: For important transactions, implement retry mechanisms:

```

```typescript
async function sendWithRetry(
    client: GenLayerClient,
    params: WriteContractParameters,
    maxAttempts = 3
): Promise<TransactionReceipt> {
    for (let attempt = 1; attempt <= maxAttempts; attempt++) {
        try {
            const hash = await client.writeContract(params);
            return await client.waitForTransactionReceipt({
                hash,
                status: 'FINALIZED',
                timeout: 30_000 * attempt, // Increase timeout with each attempt
            });
        } catch (error) {
            if (attempt === maxAttempts) throw error;
            console.log(`Attempt ${attempt} failed, retrying...`);
            await new Promise(resolve => setTimeout(resolve, 1000 * attempt));
        }
    }
}
```

```

```

 }
 throw new Error('Max retry attempts reached');
}
...

```

---

## # Testing Contracts in GenLayer Studio

Source: <https://docs.genlayer.com/developers/decentralized-applications/testing>

Testing Intelligent Contracts on GenLayer involves deploying contracts, sending transactions, validating their behavior, and identifying issues. Here is some guidance on how to test using the tools provided in the local development environment and GenLayer Studio.

### ## Testing Workflow in the Local Environment

For a more advanced and controlled testing setup, you can leverage the GenLayer Project Boilerplate and the provided test helpers.

> **Note:**

> Please note that you need the Studio running to run the tests by sending requests to it.

#### ### 1. Create Accounts

Use the `create\_new\_account` helper to generate accounts for testing. These accounts simulate users interacting with the contract.

```

```python
from tools.request import create_new_account

account = create_new_account()
```

```

#### ### 2. Set Up Validators

Validators are essential for processing transactions in GenLayer. Use the `sim\_createRandomValidators` method to initialize them.

```

```python
from tools.request import payload, post_request_localhost

nb_validators = 5
min_stake = 8
max_stake = 12
providers = ["openai"]
models = ["gpt-4o"]
validators_response = post_request_localhost(
    payload("sim_createRandomValidators", nb_validators, min_stake, max_stake, providers,
models)
).json()
```

```

#### ### 3. Deploy the Contract

Deploy your Intelligent Contract using the `deploy\_intelligent\_contract` helper:

```

```python
from tools.request import deploy_intelligent_contract

contract_code = open("contracts/my_contract.py", "r").read()
contract_address, deploy_response = deploy_intelligent_contract(account, contract_code, "{}")
```

```

#### ### 4. Interact with the Contract

Use ``send_transaction`` for writing data to the contract and ``call_contract_method`` for reading data:

```
```python
from tools.request import send_transaction, call_contract_method

# Write data
send_transaction(account, contract_address, "method_name", [arg1, arg2])

# Read data
result = call_contract_method(contract_address, account, "get_state", [])
```
```

### ### 5. Assertions

Validate the responses using provided assertion helpers:

```
```python
from tools.response import assert_dict_struct, has_success_status

assert has_success_status(result)
assert_dict_struct(result, expected_structure)
```
```

### ### 6. Clean Up

After completing the tests, delete the validators or reset the environment:

```
```python
delete_response = post_request_localhost(payload("sim_deleteAllValidators")).json()
```
```

---

### # GenLayer Project Boilerplate

Source: <https://docs.genlayer.com/developers/decentralized-applications/project-boilerplate>

The GenLayer Project Boilerplate is a template for building decentralized applications (DApps) on GenLayer with Intelligent Contracts, frontend integration, testing, and environment configuration. It includes a complete example implementation of a football prediction market that demonstrates best practices and common patterns for GenLayer development.

> **Note:**

> This boilerplate is a work in progress and is not yet ready for production use. You can find the latest version at: [GenLayer Project Boilerplate] (<https://github.com/genlayerlabs/genlayer-project-boilerplate>)

### ## Features

- **Contract Templates**: Ready-to-use intelligent contract templates and examples
- **Testing Framework**: Built-in testing infrastructure for end-to-end testing
- **Frontend Integration**: Vue.js-based frontend setup with GenLayerJS integration
- **Environment Configuration**: Pre-configured environment setup for development
- **Example Implementation**: Full football prediction market implementation

### ## How it's Built

The boilerplate project is structured to provide a development environment for GenLayer applications, combining both backend contract development and frontend user interface.

### ### Technologies Used

- **Python**: For intelligent contract development and testing



- **Vue.js**: Frontend framework for building user interfaces
- **GenLayerJS**: SDK for interacting with GenLayer contracts
- **pytest**: Testing framework for contract validation
- **Vite**: Frontend build tool and development server

### ### Project Structure

```

...
project-root/
├── app/ # Frontend application
│ ├── src/ # Vue.js source files
│ └── .env.example # Frontend environment variables template
├── contracts/ # Intelligent Contracts
│ └── football_prediction_market.py
├── test/ # Test files
└── .env.example # Main environment variables template
...

```

### ## Requirements

Before using the GenLayer Project Boilerplate, ensure your system meets the following requirements:

- **GenLayer Studio**: Running instance required
- **Node.js**: Version 18.x or higher
- **Python**: Version 3.8 or higher

### ### Installation

```

1. Clone the boilerplate repository:
```bash
git clone https://github.com/genlayerlabs/genlayer-project-boilerplate
cd genlayer-project-boilerplate
```

```

2. Set up the environment:

```

```bash
cp .env.example .env
```

```

### ## Usage

#### ### Deploying Contracts

1. Access the GenLayer Simulator UI (default: <http://localhost:8080>)
2. Navigate to "Contracts" section
3. Create a new contract using `/contracts/football_prediction_market.py`
4. Deploy through the "Run and Debug" interface

#### ### Frontend Setup

1. Configure the frontend environment:

```

```bash
cd app
cp .env.example .env
# Add your contract address to VITE_CONTRACT_ADDRESS
```

```

2. Install dependencies and start the development server:

```

```bash
npm install
npm run dev
```

```

### ### Running Tests

Execute the test suite using pytest:

```
```bash
pip install -r requirements.txt
pytest test
```
```

### ## Football Prediction Market Example

The included example contract demonstrates a complete prediction market implementation with the following features:

#### ### Contract Functions

##### ##### Create Predictions:

```
```python
create_prediction(game_date: str, team1: str, team2: str, predicted_winner: str)
```
```

##### ##### Resolve Predictions:

```
```python
resolve_prediction(prediction_id: str)
```
```

##### ##### Query Data:

```
```python
get_player_predictions(player_address: str)
get_player_points(player_address: str)
```
```

### ### Frontend Integration

The Vue.js frontend demonstrates:

- Wallet connection handling
- Contract interaction using GenLayerJS
- User interface for prediction creation and management
- Real-time updates for prediction status

### ## Testing Framework

The boilerplate includes a comprehensive testing suite covering the following scenarios:

#### ### Contract Schema

The schema of the contract is well generated and has the expected methods and variables.

#### ### Test Scenarios

##### ##### Successful Draw Prediction

Tests the scenario where a player correctly predicts a draw between two teams. When the match is resolved as a draw, the player should receive 1 point for their accurate prediction.

##### ##### Successful Winner Prediction

Validates the case where a player correctly predicts the winning team. Upon match resolution confirming their predicted team as the winner, the player should be awarded 1 point.

##### ##### Unsuccessful Prediction

Covers the scenario where a player's prediction doesn't match the actual result. This could be:

- Predicting Team A wins, but Team B wins
- Predicting a draw, but there's a winner
- Predicting a winner, but the match ends in a draw

In all these cases, the player should receive 0 points.

## ## Best Practices

- Always use environment variables for configuration
- Implement comprehensive testing for all contract functions
- Follow the provided folder structure for consistency
- Use TypeScript for frontend development
- Implement proper error handling in both contract and frontend code

---

## # Staking Contract Guide

Source: <https://docs.genlayer.com/developers/staking-guide>

The Staking Contract Guide explains how developers can interact directly with GenLayer staking smart contracts for consensus-level staking operations. It includes Solidity code examples for both validator and delegator operations.

To understand the concepts behind staking, including the epoch system, shares vs stake, validator selection weights, and reward distribution, see [Staking in GenLayer](/understand-genlayer-protocol/core-concepts/optimistic-democracy/staking).

> **Note:**

> For CLI-based staking operations (recommended for most users), see the [GenLayer CLI Staking Commands](/api-references/genlayer-cli#staking-operations-testnet).

## ## Validator Operations

### ### Joining as a Validator

**Requirements:**

- Minimum stake: **42,000 GEN** tokens
- Must run a validator node for consensus participation
- A single address can create multiple validators

**Method 1: Owner as Operator**

```
```solidity
address validatorWallet = genStaking.validatorJoin{value: 42000 ether}();
```
```

**Method 2: Separate Operator Address (Recommended)**

```
```solidity
address operatorAddress = 0x...;
address validatorWallet = genStaking.validatorJoin{value: 42000 ether}(operatorAddress);
```
```

> **Warning:**

> Save the returned **ValidatorWallet address** - you'll need it for all future operations.

**Important Notes:**

- Deposits activate **2 epochs** after joining
- `validatorPrime()` is automatically called by your validator node each epoch
- Monitor priming for proper activation within 2 epochs
- Operator address cannot be reused or be the zero address
- If the operator is already assigned to another validator, the transaction reverts

**Post-Join Steps:**

1. Save the `validatorWallet` address
2. Start your validator node using the operator address
3. Monitor that `validatorPrime(validatorWallet)` executes each epoch
4. Wait 2 epochs and verify activation

### ### Depositing as a Validator

```
```solidity
validatorWallet.validatorDeposit{value: 10000 ether}();
```
```

**\*\*Key Points:\*\***

- Call through the ValidatorWallet contract only
- Deposits follow the **\*\*epoch +2 activation rule\*\***
- Your validator node automatically calls `validatorPrime()` to commit the deposit
- No minimum for additional deposits
- Cannot deposit zero value

**\*\*Activation Timeline:\*\***

```

```
Epoch N:    validatorDeposit() called
Epoch N+1:  validatorPrime() → deposit staged
Epoch N+2:  validatorPrime() → deposit activated
```
```

**### Withdrawing as a Validator**

**\*\*Step 1: Calculate Shares\*\***

```
```solidity
uint256[8] memory validatorState = genStaking.validatorView(validatorWallet);
uint256 currentShares = validatorState[1];
uint256 currentStake = validatorState[0];
uint256 sharesToExit = (currentShares * 30) / 100; // Exit 30%
```
```

**\*\*Step 2: Initiate Exit\*\***

```
```solidity
validatorWallet.validatorExit(sharesToExit);
```
```

**> \*\*Warning:\*\***

**> You must specify **\*\*shares\*\***, not token amounts. Cannot exit more shares than owned.**

**\*\*Important Notes:\*\***

- Exited tokens stop earning rewards immediately
- Tokens are locked for **\*\*7 epochs\*\*** before claiming
- Each exit creates a separate withdrawal

**\*\*Unbonding Timeline:\*\***

```

```
Epoch 10:    validatorExit() called → unbonding starts
Epochs 11-16: Locked period
Epoch 17:    validatorClaim() available
```
```

**### Claiming Withdrawn Funds as a Validator**

```
```solidity
uint256 tokensReceived = genStaking.validatorClaim(validatorWallet);
```
```

**\*\*Important Notes:\*\***

- Must wait **\*\*7 full epochs\*\*** after calling `validatorExit()`
- Claims are permissionless - anyone can trigger them
- Tokens are sent to the validator wallet's owner address
- Multiple exits can be claimed in a single call
- Returns the total GEN received

**\*\*Checking Claim Availability:\*\***

```

```solidity
uint256 exitEpoch = 10;
uint256 claimableAtEpoch = exitEpoch + 7;
uint256 currentEpoch = genStaking.epoch();
if (currentEpoch >= claimableAtEpoch) {
    genStaking.validatorClaim(validatorWallet);
}
```

```

---

## ## Delegator Operations

### ### Joining as a Delegator

#### **\*\*Requirements:\*\***

- Minimum delegation: **\*\*42 GEN\*\*** per validator
- Target validator must exist and be active
- Can delegate to multiple validators

#### **\*\*Basic Delegation:\*\***

```

```solidity
address validatorWallet = 0x...;
genStaking.delegatorJoin{value: 1000 ether}(validatorWallet);
```

```

#### **\*\*Multi-Validator Delegation:\*\***

```

```solidity
genStaking.delegatorJoin{value: 500 ether}(validator1);
genStaking.delegatorJoin{value: 500 ether}(validator2);
genStaking.delegatorJoin{value: 500 ether}(validator3);
```

```

#### **\*\*Important Notes:\*\***

- Follows the **\*\*epoch +2 activation rule\*\***
- The validator's node automatically calls ``validatorPrime()`` to commit your delegation
- Choose validators with good uptime
- Smaller validators often provide higher returns
- Cannot delegate zero value

#### **\*\*Delegation Strategy Tips:\*\***

- Diversify across multiple validators
- Check validator performance and uptime
- Monitor validator quarantine/ban status

### ### Depositing as a Delegator

```

```solidity
genStaking.delegatorDeposit{value: 500 ether}(validatorWallet);
```

```

#### **\*\*Important Notes:\*\***

- Use ``delegatorDeposit()`` to add to existing delegations
- Follows the **\*\*epoch +2 activation rule\*\***
- The validator's node automatically calls ``validatorPrime()``
- No minimum for additional deposits
- Rewards automatically compound

### ### Withdrawing as a Delegator

#### **\*\*Step 1: Calculate Shares\*\***

```
```solidity
uint256 currentShares = genStaking.sharesOf(msg.sender, validatorWallet);
uint256 sharesToExit = (currentShares * 50) / 100; // Exit 50%
```
```

**\*\*Step 2: Initiate Exit\*\***

```
```solidity
genStaking.delegatorExit(validatorWallet, sharesToExit);
```
```

> **\*\*Warning:\*\***  
> You must specify **\*\*shares\*\***, not token amounts.

**\*\*Important Notes:\*\***

- Can exit partial amounts
- Stop earning rewards immediately after exit
- Tokens are locked for **\*\*7 epochs\*\***
- Exit each validator separately if delegating to multiple

**\*\*Unbonding Timeline:\*\***

```
Epoch 5: delegatorExit() called
Epochs 6-11: Locked period
Epoch 12: delegatorClaim() available
```
```

Claiming Withdrawn Funds as a Delegator

```
```solidity
uint256 tokensReceived = genStaking.delegatorClaim(
 delegatorAddress,
 validatorWallet
);
```
```

****Important Notes:****

- Must wait ****7 full epochs**** after calling ``delegatorExit()``
- Claims are permissionless - anyone can trigger them
- Tokens are sent to the delegator's address
- Claim from each validator separately if delegated to multiple

****Multi-Validator Claim Example:****

```
```solidity
genStaking.delegatorClaim(msg.sender, validator1);
genStaking.delegatorClaim(msg.sender, validator2);
genStaking.delegatorClaim(msg.sender, validator3);
```
```

Genesis Epoch 0

Epoch 0 is the ****genesis bootstrapping period**** with special rules designed to facilitate network launch. The normal staking rules are relaxed to allow rapid network bootstrapping.

What is Epoch 0?

Epoch 0 is the ****bootstrapping period**** before the network becomes operational. During epoch 0:

- ****No transactions are processed**** - the network is not yet active
- ****No consensus occurs**** - validators are not yet participating
- Stakes are registered and prepared for activation in epoch 2

****Important****: The network transitions directly from epoch 0 to epoch 2 (epoch 1 is skipped). Validators and delegators who stake in epoch 0 become active in epoch 2.

Special Rules for Epoch 0

| Rule | Normal Epochs (2+) | Epoch 0 |
|-------------------------|-------------------------|-------------------------|
| Validator minimum stake | 42,000 GEN | No minimum to join |
| Delegator minimum stake | 42 GEN | No minimum to join |
| Activation delay | +2 epochs | Active in epoch 2 |
| validatorPrime required | Yes, each epoch | Not required |
| Share calculation | Based on existing ratio | 1:1 (shares = input) |
| Transaction processing | Yes | No (bootstrapping only) |

> ****Warning****

> ****Activation requires meeting minimums****: While you can join with any amount during epoch 0, your stake will only be ****activated in epoch 2**** if it meets the minimum requirements (42,000 GEN for validators, 42 GEN for delegators). Stakes below the minimum remain registered but inactive.

Validators in Epoch 0

****Joining****

```
``solidity
// In epoch 0, any amount is accepted (no 42,000 GEN minimum)
address validatorWallet = genStaking.validatorJoin{value: 1000 ether}(operatorAddress);

// Stake is registered for activation in epoch 2
// No need to call validatorPrime() in epoch 0
``
```

****Key behaviors****

1. ****No minimum stake****: Validators can join with any non-zero amount
2. ****Registered for epoch 2****: Stakes are recorded and will become active when epoch 2 begins
3. ****No priming required****: `validatorPrime()` is not needed during epoch 0
4. ****No consensus participation****: Validators do not process transactions in epoch 0

****Do validators need to take any action in epoch 0 to be active in epoch 2?****

No. Validators who join in epoch 0:

- Have their stake registered during epoch 0
- Become active automatically in epoch 2 (epoch 1 is skipped) ****only if they have at least 42,000 GEN staked****
- Must start calling `validatorPrime()` in epoch 2 for continued participation in epoch 4+

> ****Caution****

> ****Critical****: Validators who joined in epoch 0 with less than 42,000 GEN will ****not be active**** in epoch 2. They must deposit additional funds to meet the minimum requirement before epoch 2 begins.

Delegators in Epoch 0

****Joining****

```
``solidity
// In epoch 0, any amount is accepted (no 42 GEN minimum)
genStaking.delegatorJoin{value: 10 ether}(validatorWallet);

// Delegation is registered for activation in epoch 2
``
```

****Key behaviors:****

1. ****No minimum delegation****: Any non-zero amount accepted
2. ****Registered for epoch 2****: Delegation is recorded and will become active when epoch 2 begins
3. ****No rewards in epoch 0****: Since no transactions are processed, no rewards are earned during epoch 0

****Is a delegation made in epoch 0 active in epoch 2?****

Yes. Delegations made in epoch 0 become active in epoch 2 (epoch 1 is skipped). Unlike normal epochs where you wait +2 epochs, epoch 0 delegations activate as soon as the network becomes operational.

Activation Timeline Comparison

****Normal Epochs (2+):****

...

Epoch N: validatorJoin() or delegatorJoin() called
Epoch N+1: validatorPrime() stages the deposit
Epoch N+2: validatorPrime() activates the deposit → NOW ACTIVE
```

### **\*\*Epoch 0 (Bootstrapping):\*\***

...

Epoch 0: validatorJoin() or delegatorJoin() called → stake registered (not yet active)  
No transactions processed, no consensus  
Epoch 2: Stakes become active (if minimum met), network operational, validatorPrime() required  
```

Share Calculation in Epoch 0

In epoch 0, shares are calculated at a 1:1 ratio with the input amount:

```
```solidity
// Epoch 0: 1,000 GEN deposit = 1,000 shares (1:1 ratio)
genStaking.delegatorJoin{value: 1000 ether}(validatorWallet);
// Result: delegator receives exactly 1,000 shares

// Normal epochs: shares depend on pool ratio
// new_shares = deposit_amount × (total_shares / total_stake)
```
```

This is because there's no existing stake pool to calculate a ratio against. Starting from epoch 2, shares are calculated based on the current stake-to-share ratio.

Transitioning from Epoch 0 to Epoch 2

When the network advances from epoch 0 to epoch 2 (epoch 1 is skipped):

1. ****Epoch 0 stakes that meet minimums become active**** - validators need 42,000 GEN, delegators need 42 GEN
2. ****Normal minimum requirements apply**** for new joins/deposits
3. ****+2 epoch activation delay**** applies to all new deposits
4. ****validatorPrime() becomes mandatory**** for validators to remain in the selection pool
5. ****Existing validators**** must ensure their nodes begin calling `validatorPrime()` in epoch 2

FAQ: Epoch 0 Special Cases

****Q: Can I join as a validator with less than 42,000 GEN in epoch 0?****

A: Yes, any non-zero amount is accepted during epoch 0. However, you will ****not be active**** in epoch 2 unless you have at least 42,000 GEN staked by then.

****Q: If I delegate in epoch 0, when does it become active?****

A: In epoch 2. Unlike normal epochs with a +2 delay, epoch 0 delegations activate when the network becomes operational.

****Q: Do I need to call validatorPrime() in epoch 0?****

A: No. Priming is not required during epoch 0. Your node should start calling it automatically when epoch 2 begins (epoch 1 is skipped).

****Q: Will my epoch 0 stake still be active after epoch 0 ends?****

A: Yes, if you meet the minimum requirements. Stakes from epoch 0 carry forward and remain active in all subsequent epochs.

****Q: What happens to my stake if I joined in epoch 0 but my node doesn't call validatorPrime() in epoch 2?****

A: You'll be excluded from validator selection in epoch 4, but your stake remains. Once priming resumes, you'll be eligible for selection again.

Common Scenarios

Scenario 1: Becoming a Validator

```
```solidity
address validatorWallet = genStaking.validatorJoin{value: 42000 ether}(operatorAddress);
// Start validator node with operator address
// Monitor: Epoch N joined, N+1 staged, N+2 active
```
```

Scenario 2: Delegating to Multiple Validators

```
```solidity
genStaking.delegatorJoin{value: 1000 ether}(validator1);
genStaking.delegatorJoin{value: 2000 ether}(validator2);
genStaking.delegatorJoin{value: 1500 ether}(validator3);

// Later, exit from one validator
uint256 shares = genStaking.sharesOf(msg.sender, validator1);
genStaking.delegatorExit(validator1, shares);

// After 7 epochs:
genStaking.delegatorClaim(msg.sender, validator1);
```
```

Scenario 3: Partial Validator Exit

```
```solidity
uint256[8] memory state = genStaking.validatorView(validatorWallet);
uint256 currentShares = state[1];
uint256 sharesToExit = (currentShares * 25) / 100; // Exit 25%
validatorWallet.validatorExit(sharesToExit);

// After 7 epochs:
genStaking.validatorClaim(validatorWallet);
```
```

Best Practices

For Validators

1. ****Ensure node health****: Your validator node automatically calls ``validatorPrime()``
2. ****Use separate keys****: Cold wallet for owner, hot wallet for operator
3. ****Monitor priming****: Verify deposit activation occurs as expected
4. ****Monitor quarantine status****: Check if your validator is quarantined
5. ****Maintain high uptime****: Downtime leads to idleness strikes
6. ****Set up backup monitoring****: Alert if priming fails

For Delegators

1. ****Diversify****: Spread stake across multiple validators
2. ****Select carefully****: Choose validators with good uptime records
3. ****Verify priming****: Ensure validators are consistently primed
4. ****Consider smaller validators****: They often provide higher returns
5. ****Monitor health****: Regularly check validator status
6. ****Use shares for exits****: Calculate percentages, not absolute amounts

For Both

1. ****Account for unbonding****: Plan for the 7-epoch wait when exiting
2. ****Monitor epochs****: Track current epoch for claim eligibility
3. ****Calculate exits carefully****: Always use share percentages
4. ****Understand compounding****: Rewards automatically increase stake-per-share
5. ****Watch governance****: Parameter changes can affect staking dynamics

Troubleshooting

| Error | Cause | Solution |
|--|--------------------------------|--|
| <code>`ValidatorBelowMinimumStake`</code> | Stake below 42,000 GEN | Deposit at least 42,000 GEN when joining |
| <code>`OperatorAlreadyAssigned`</code> | Operator address in use | Choose a different operator address |
| <code>`ValidatorMayNotDepositZeroValue`</code> | Zero deposit amount | Deposit a non-zero amount |
| <code>`ValidatorExitExceedsShares`</code> | Exiting more shares than owned | Check shares with <code>`validatorView()`</code> |
| Validator not selected | Node not priming | Ensure node runs and calls <code>`validatorPrime()`</code> |
| Stake not activating | Priming not occurring | Monitor node for automatic <code>`validatorPrime()`</code> calls |
| Cannot claim withdrawn funds | Unbonding not complete | Verify 7 epochs have passed with <code>`genStaking.epoch()`</code> |
| Delegation not earning | Validator issues | Check if validator is quarantined/banned; consider switching |

Related Resources

- [Staking Concepts](/understand-genlayer-protocol/core-concepts/optimistic-democracy/staking) - How staking works in GenLayer
- [Unstaking](/understand-genlayer-protocol/core-concepts/optimistic-democracy/unstaking) - Unstaking process details
- [Slashing](/understand-genlayer-protocol/core-concepts/optimistic-democracy/slashing) - Slashing penalties and conditions
- [GenLayer CLI Staking Commands](/api-references/genlayer-cli#staking-operations-testnet) - CLI reference for staking

Setting Up Your GenLayer Validator

Source: <https://docs.genlayer.com/validators/setup-guide>

Running a GenLayer Validator node ensures the security and reliability of the GenLayer

network. Below are the requirements for running a GenLayer validator.

Validators require both a staking deposit and a validator priming transaction to enter the set.

For a deeper understanding of how staking works in GenLayer, see the [Staking documentation] (/understand-genlayer-protocol/core-concepts/optimistic-democracy/staking).

System Requirements

| Resource | Requirement |
|--------------|--------------------------------------|
| **Memory** | 16 GB RAM |
| **CPU** | 8-core (AMD64 only) |
| **Disk** | 128 GB SSD/NVMe |
| **Network** | 100 Mbps |
| **OS** | 64-bit Linux |
| **Software** | Docker, Python 3 (with pip and venv) |

GPU not required unless running LLMs locally. See [detailed requirements] (/validators/system-requirements) for more info.

Choose Your Setup Method

GenLayer offers two ways to set up your validator node. Both methods result in the same configuration—choose the one that fits your workflow.

| Method | Best For |
|-----------------------|--|
| **AI-Assisted Setup** | New validators, quick setup, guided troubleshooting |
| **Manual Setup** | Experienced operators, custom configurations, automation pipelines |

Option 1: AI-Assisted Setup (Recommended)

We've released an interactive AI wizard that guides you through the entire validator setup process step-by-step. Instead of reading through documentation and running commands manually, the wizard handles everything interactively.

What it does

- Walks you through all setup steps interactively
- Checks prerequisites automatically (architecture, RAM, Docker, Node.js, Python)
- Detects common issues before they cause problems (port conflicts, disk space, wrong architecture)
- Generates configurations based on your answers
- Supports multiple deployment options: local machine, remote SSH, GCP, AWS, Azure
- Handles both new validators (staking wizard) and existing validators
- Includes minimal-downtime update procedure for upgrades

How to use it

1. Install [Claude Code](https://claude.ai/code) (Anthropic's CLI tool)

2. Add the GenLayer plugin marketplace and install the plugin:

```
```bash copy
claude /plugin marketplace add genlayerlabs/skills
claude /plugin install genlayernode@genlayerlabs
```
```

3. Run the wizard:

```
```bash copy
claude /genlayernode install
```
```

That's it. The wizard will ask questions and guide you through everything.

Available commands

| Command | Description |
|--|-----------------------|
| <code>`/genlayernode install`</code> | Fresh validator setup |
| <code>`/genlayernode update`</code> | Upgrade existing node |
| <code>`/genlayernode configure grafana`</code> | Set up monitoring |

> **Note:**

> The AI wizard uses the same underlying steps as the manual setup below. You can switch between methods at any point or use the manual guide as a reference.

Browse all available skills at skills.genlayer.com – the marketplace also includes the `genlayer-dev` skill for contract development.

Option 2: Manual Setup

If you prefer to run commands manually, need fine-grained control over each step, or are integrating with existing automation pipelines, follow the detailed instructions in the sections below.

The manual setup covers:

1. **Create the Validator Wallet** – Set up your wallet and stake GEN tokens
2. **Set Up the Validator Node** – Download, configure, and run the node software

Create the Validator Wallet

Before setting up your validator node, you need to create your validator wallet and stake GEN tokens.

Understanding Validator Addresses

> **Note:**

> GenLayer validators use three distinct addresses:

>

> | Address | Description | Where Used |

> |-----|-----|-----|

> | **Owner** | The only address that can withdraw staked funds. Keep this secure (cold wallet). | CLI wizard - signs staking transaction |

> | **Operator** | Hot wallet on your server that signs blocks. Can be same as owner, but separate is recommended. | Node config: `operatorAddress` |

> | **Validator Wallet** | Smart contract created when you join. This is your validator's on-chain identity. | Node config: `validatorWalletAddress` |

>

> The wizard outputs all three addresses at the end. Save them - you'll need the Validator Wallet and Operator addresses for your node configuration.

Prerequisites

- **Node.js** (v18 or higher)
- **GenLayer CLI** - Install the latest version:

```
``bash copy
npm install -g genlayer
```
- **GEN tokens** - You need at least **42,000 GEN** for the minimum self-stake requirement

Using the Validator Wizard

The easiest way to create your validator wallet is using the interactive wizard:

```
```bash copy
genlayer staking wizard
```
```

The wizard guides you through:

1. **Account setup** - Create or select your owner account
2. **Network selection** - Choose testnet-asimov
3. **Balance verification** - Confirms you have at least 42,000 GEN
4. **Operator setup** - Create and export an operator keystore for your validator server
5. **Stake amount** - Enter how much GEN to stake (minimum 42,000)
6. **Validator creation** - Submits the staking transaction
7. **Identity setup** - Set your validator's public profile (moniker, website, etc.)

> **Note:**

> **Save Your Validator Wallet Address!** After the wizard completes, note down your **Validator Wallet** address. You'll need this for your node configuration.

Operator Keystore Export

The wizard creates and exports an operator keystore file for you to transfer to your validator server. If your server is compromised, your staked funds (controlled by owner) remain safe.

Verify Your Validator

After completing the wizard, verify your status:

```
```bash copy
genlayer staking validator-info --validator 0xYourValidatorWallet...
```
```

Managing Your Validator

```
```bash copy
Add more stake
genlayer staking validator-deposit --amount 1000gen

Check active validators
genlayer staking active-validators

Exit (initiates 7-epoch unbonding)
genlayer staking validator-exit --shares 100

Claim after unbonding period
genlayer staking validator-claim

Update identity
genlayer staking set-identity --validator 0x... --moniker "New Name"
```
```

Set Up the Validator Node

Once your validator wallet is created and staked, set up your node software.

Dependencies

LLM Access

Each validator needs access to a Large Language Model (LLM) for executing and evaluating Intelligent Contracts.

It is up to each validator to select the model they want to use.

Possible options:

- Run an open-source model locally on the same machine with a GPU
- Run an open-source model on a different machine
- Connect to a hosted inference provider (OpenAI, Anthropic, Heurist, Atoma network etc.)

See the [GenVM Configuration](/validators/genvm-configuration) page for partner credits, provider setup, and the greybox fallback strategy.

ZKSync Full Node for the GenLayer Chain

Each validator needs access to a [ZKSync Full Node](https://docs.zksync.io/zksync-era/tooling/external-node) connected to the GenLayer chain. Validators use this node to read the chain state and submit transactions.

One full node can be shared between multiple validators. The optimal validator-to-node ratio is currently under evaluation.

Setup

Download the node software

1. Select the version of the node you want to run by checking the available builds

You can use this command to list available versions:

```
```sh copy
curl -s "https://storage.googleapis.com/storage/v1/b/gh-af/o?prefix=genlayer-
node/bin/amd64" | grep -o '"name": *"[^"]*"' | sed -n 's/.*\/(v[^\(]*\)/.*/\1/p' | sort -ru
| grep -v "rc" | head -n 5
```

You should see a list like this

```
```sh
v0.5.12
v0.5.11
v0.5.10
v0.5.9
v0.5.8
```
```

Typically, you will want to run the latest version

2. Download the packaged application

```
```sh copy
export version=v0.5.12 # set your desired version here
wget https://storage.googleapis.com/gh-af/genlayer-node/bin/amd64/${version}/genlayer-
node-linux-amd64-${version}.tar.gz
```

3. Extract the node software

```
```sh copy
mkdir -p ${version}
tar -xzf `genlayer-node-linux-amd64-${version}.tar.gz` -C `./${version}`
```

4. Change the directory

```
```sh copy
cd `./${version}`
```

5. Run Genvm setup

```
```sh copy
```

```
python3 ./third_party/genvm/bin/setup.py
```

It does following:

- Downloads missing runners if any
- Verifies runners hashes
- Sets `elf`/`mach-o` interpreter for all binaries (modules, genvm)
- Rust precompilation for built-in runners

### ### Configuration

Before you can start up the node, you need to configure it.

#### #### `config.yaml`

This is the main configuration file of your node. Without it, your node won't start

The file needs to be located at `configs/node/config.yaml`

You can use the following example configuration. **Note:** For most users, you will only need to modify the `genlayerchainrpcurl`, `genlayerchainwebsocketurl`, and the `consensus` section for your target network (see [Network-Specific Consensus Configuration](#network-specific-consensus-configuration) below).

```
```yaml copy
# rollup configuration
rollup:
  genlayerchainrpcurl: "TODO: Set your GenLayer Chain ZKSync HTTP RPC URL here" # GenLayer
Chain RPC URL
  genlayerchainwebsocketurl: "TODO: Set your GenLayer Chain ZKSync WebSocket RPC URL here" #
GenLayer Chain WebSocket URL
  provider: "TODO: Set your GenLayer Chain ZKSync provider" # Rollup operator name; surfaces
as the "provider" label on RPC-driven metrics. Set explicitly per deployment.
# consensus contracts configuration
consensus:
  # Testnet - Phase 5
  consensusaddress: "0xe66B434bc83805f380509642429eC8e43AE9874a" # AddressManager Smart
Contract Address
  genesis: 17326 # (Optional) Genesis block number for this consensus deployment. If not
provided, it will be auto-detected by searching for the first log from the ConsensusMain
contract.
  # chain_submit_buffer: "30s" # (Optional) Slack reserved at the end of each consensus step
to submit the resulting on-chain tx (commit vote, reveal vote, proposed receipt) before the
slot closes. Tune to underlying settlement chain latency. Default 30s.
# data directory
datadir: "./data/node"
# logging configuration
logging:
  level: "INFO"
  # json: `true` for json output to console, false for human readable log formatting
  json: false
  # Configuration for https://github.com/natefinch/lumberjack
  file:
    # enabled: set to `true` to save logs to a folder
    enabled: true
    level: "DEBUG"
    # folder: path to the folder where to store logs. Relative paths go under `datadir`.
    folder: logs
    # maxsize: maximum size in megabytes of the log file before it gets rotated.
    maxsize: 10
    # maxage: maximum number of days to retain old log files based on the timestamp encoded
in their filename.
    maxage: 7
    # maxbackups: maximum number of old log files to retain. Set to 0 for no limit
    maxbackups: 100
```

```

# localtime: determines if the time used for formatting the timestamps in backup files
is the computer's local time. Set to `false` to use UTC time.
localtime: false
# compress: determines if the rotated log files should be compressed using gzip
compress: true
# node configuration
node:
# Uncomment if the ID of the node is different from the validator wallet address.
# It is used to identify the node in the network.
# id: "node"
# Mode can be "validator" or "full".
# Default value is "validator".
mode: "validator"
# Address of the ValidatorWallet contract (required for validator mode)
validatorWalletAddress: ""
# Address of the operator that owns the ValidatorWallet
operatorAddress: ""
admin:
port: 9155
rpc:
# host: 0.0.0.0 # bind address for the HTTP listener (default: all interfaces)
port: 9151 # RPC server port
# allowed_origins: unified browser-origin allowlist applied to BOTH
# the HTTP transport (CORS) and the WS upgrade (gorilla CheckOrigin).
# Exact-match URLs only – no wildcard syntax beyond "*". Omit or
# leave empty to keep the permissive default ["*"]. Operators can
# override per-transport for WS via rpc.websocket.origins (rare).
# allowed_origins:
#   - "https://wallet.example.com"
#   - "https://app.example.com"
# websocket: inbound JSON-RPC WS endpoint for eth_subscribe /
# zks_subscribe plus all non-subscribe passthrough. Omit this whole
# block (the default) to keep WS disabled.
#
# When enabled, the default is single-listener mode: WS upgrades are
# served on the existing RPC listener (port above) – one port, one
# firewall entry, one reverse-proxy upstream. To run a dedicated WS
# listener instead (Geth-conventional dual-port pattern), set `port`
# to a value other than 0 and other than rpc.port (e.g. 9152) and
# set `host` for that listener.
# websocket:
#   enabled: true
#   # port: 0           # 0 or unset → single-listener on rpc.port (default)
#   # host: 0.0.0.0     # only used in dual-listener mode
#   path: "/ws"         # ws upgrade path – disambiguates HTTP RPC (POST /)
#                       # from WS upgrade (GET /ws) at the URL level.
#                       # Set to "/" for legacy behaviour (root path).
#   origins:           # WS-only override of rpc.allowed_origins (rare;
#                       # leave unset to inherit the unified allowlist)
#     - "*"             # permissive default – tighten via rpc.allowed_origins
#   max_conns: 1024
#   max_conns_per_ip: 16
#   max_subs_per_conn: 32
#   outbox_size: 256    # per-connection bounded queue; full → connection dropped
#   write_timeout: 10s
#   ping_interval: 30s
#   max_message_bytes: 10485760 # 10 MB cap on a single inbound frame (parity with
rpc_max_request_body_bytes)
# ratelimit_whitelist: IPs or CIDRs that bypass rate limiting entirely
# ratelimit_whitelist:
#   - "127.0.0.1"
#   - "::1"           # IPv6 loopback (localhost on macOS)
#   - "10.0.0.0/8"
endpoints:
# Group-level configuration (enables/disables all methods in a group).

```



```

# When a group is enabled, ALL methods in that group are available.
# Rate limits (req/s per IP) apply by default even without config:
#   genlayer: 20, genlayer_debug: 2, ethereum: 50, zksync: 50
#   gen_call: 2, gen_getContractSchema: 2 (method-level overrides)
groups:
  genlayer: true # gen_* methods (default ratelimit: 20 req/s per IP)
  genlayer_debug: true # gen_dbg_* methods (default ratelimit: 2 req/s per IP)
  ethereum: true # eth_* proxy methods (default ratelimit: 50 req/s per IP)
  # includes: eth_blockNumber, eth_getBlockByNumber, eth_getBlockByHash,
  #   eth_sendRawTransaction, eth_getTransactionReceipt, etc.
  zksync: true # zks_* proxy methods (default ratelimit: 50 req/s per IP)
  # includes: zks_getTransaction, zks_getBlockDetails, etc.
# Object format with custom rate limit (overrides defaults):
# groups:
#   genlayer:
#     enabled: true
#     ratelimit: 100          # custom: 100 req/s per IP instead of default 20
#
# Method-level configuration (optional, overrides group settings).
# Methods inherit their group's enabled state by default.
# Use this section to:
#   - Disable specific methods in an enabled group
#   - Enable specific methods in a disabled group
#   - Set per-method rate limits
#
# Example: enable all eth_* except eth_sendRawTransaction:
# groups:
#   ethereum: true
# methods:
#   eth_sendRawTransaction: false
#
# Example: disable all eth_* except eth_sendRawTransaction:
# groups:
#   ethereum: false
# methods:
#   eth_sendRawTransaction: true
# methods:
#   gen_call: true           # default ratelimit: 2 req/s per IP (GenVM-bound)
#   gen_getContractSchema: true # default ratelimit: 2 req/s per IP (GenVM-bound)
#   eth_blockNumber: true
#   eth_sendRawTransaction: true
#   zks_getTransaction: true
# Object format with custom rate limit:
# methods:
#   gen_call:
#     enabled: true
#     ratelimit: 5          # custom: 5 req/s per IP instead of default 2
#   gen_getContractSchema:
#     enabled: true
#     ratelimit: 0          # explicitly disable rate limiting for this method
ops:
  port: 9153 # Metrics port
  endpoints:
    metrics: true # Enable metrics endpoint
    health: true # Enable health endpoint
    balance: true # Enable balance endpoint
# genvm configuration
genvm:
  root_dir: ./third_party/genvm
  start_manager: true # if true node will start genvm manager itself
  manager_url: http://127.0.0.1:3999
  permits: 8 # Leave empty for autodiscovery, put a number for updating genvm permits on
startup
  sync_permits: 64 # Optional: permits used while !isSynced (bulk replay). Leave empty to
reuse the steady-state `permits` value.

```

```

    sync_wait_timeout: "20s" # Max time to wait for GenVM to sync to a requested block before
    returning an error
# Advanced configuration
merkleforest:
    maxdepth: 16
    dbpath: "./data/node/merkle/forest/data.db"
    indexdbpath: "./data/node/merkle/index.db"
merkletree:
    maxdepth: 16
    dbpath: "./data/node/merkle/tree/"
# metrics configuration
metrics:
    interval: "15s" # Default interval for all collectors (can be overridden per collector)
    collectors:
        node:
            enabled: true
            # interval: "30s" # Optional: Override default interval for this collector
        genvm:
            enabled: true
            # interval: "20s" # Optional: Override default interval for this collector
            # enable_simulated_traffic: false # Optional: Enable simulated network traffic
metrics for GenVM processes (default: false)
    # simulated_rx_bytes_per_process: 1024 # Optional: Simulated received bytes per
process per interval (default: 1024)
    # simulated_tx_bytes_per_process: 512 # Optional: Simulated transmitted bytes per
process per interval (default: 512)
    webdriver:
        enabled: true
        # interval: "60s" # Optional: Override default interval for this collector
...

```

Network-Specific Consensus Configuration

Set the `consensus` section in your `config.yaml` according to the network you want to join:

Testnet Asimov

```

``yaml copy
# Asimov Phase 5 network consensus configuration
consensus:
    consensusaddress: "0xe66B434bc83805f380509642429eC8e43AE9874a"
    genesis: 17326
...

```

Testnet Bradbury

```

``yaml copy
# Bradbury Phase 1 network consensus configuration
consensus:
    consensusaddress: "0x8aCE036C8C3C5D603dB546b031302FCf149648E8"
    genesis: 501711
...

```

Overriding Configuration with Environment Variables

Any configuration value in `config.yaml` can be overridden using environment variables with the prefix `GENLAYERNODE\_`.

****Pattern:****

- Replace dots (`.`) with underscores (`\_`)
- Convert to uppercase

- Add the `GENLAYERNODE\_` prefix

****Examples:****

Config Key	Environment Variable
`rollup.genlayerchainrpcurl`	`GENLAYERNODE_ROLLUP_GENLAYERCHAINRPCURL`
`rollup.genlayerchainwebsocketurl`	`GENLAYERNODE_ROLLUP_GENLAYERCHAINWEBSOCKETURL`
`consensus.contractmainaddress`	`GENLAYERNODE_CONSENSUS_CONTRACTMAINADDRESS`
`consensus.contractdataaddress`	`GENLAYERNODE_CONSENSUS_CONTRACTDATAADDRESS`
`consensus.genesis`	`GENLAYERNODE_CONSENSUS_GENESIS`
`node.mode`	`GENLAYERNODE_NODE_MODE`
`node.validatorwalletaddress`	`GENLAYERNODE_NODE_VALIDATORWALLETADDRESS`
`node.operatoraddress`	`GENLAYERNODE_NODE_OPERATORADDRESS`
`node.rpc.port`	`GENLAYERNODE_NODE_RPC_PORT`
`logging.level`	`GENLAYERNODE_LOGGING_LEVEL`

****Usage example:****

```
```sh copy
Override the RPC port
export GENLAYERNODE_CONSENSUS_CONTRACTMAINADDRESS="0x..."

Set validator wallet address
export GENLAYERNODE_NODE_VALIDATORWALLETADDRESS="0x..."

Set logging level
export GENLAYERNODE_LOGGING_LEVEL="DEBUG"
```
```

Environment variables take precedence over values in `config.yaml`, making them ideal for Docker deployments or sensitive values you don't want in configuration files.

GenVM Configuration

See the [GenVM Configuration](/validators/genvm-configuration) page for detailed LLM provider setup, configuration files, and advanced features like greyboxing.

Import the Operator Key

The operator key is used by your node to sign blocks and perform validator duties.

Option 1: Import from CLI Wizard (Recommended)

If you used `genlayer staking wizard`, it exported an operator keystore file. Transfer this file to your validator server and import it:

```
```sh copy
./bin/genlayernode account import \
 --password "your node password" \
 --passphrase "password you set when exporting from wizard" \
 --path "/path/to/operator-keystore.json" \
 -c $(pwd)/configs/node/config.yaml \
 --setup
```
```

You should see:

```
```sh
Account imported:
 Address: 0xA0b12Fd2f3F7e86fEC458D114A5E7a6f571160a8
 Account setup as a validator
```
```

Option 2: Generate New Operator Key

You can also generate a new operator key directly on the server:

```
```sh copy
./bin/genlayernode account new -c $(pwd)/configs/node/config.yaml --setup --password "your
secret password"
```
```

Then use this address when running the wizard on your local machine.

Restoring Your Operator Key

To restore from a backup (e.g., after migrating to a new server):

```
```sh copy
./bin/genlayernode account import \
 --password "your node password" \
 --passphrase "your backup encryption passphrase" \
 --path "/path/to/your/secure/backup.key" \
 -c $(pwd)/configs/node/config.yaml \
 --setup
```
```

> **Warning:**

> Always verify that your imported key works by checking the operator address matches what's configured for your validator.

Backing Up Your Operator Key

After setting up your operator key, back it up securely:

```
```sh copy
./bin/genlayernode account export \
 --password "your node password" \
 --address "your operator address" \
 --passphrase "your backup encryption passphrase" \
 --path "/path/to/your/secure/backup.key" \
 -c $(pwd)/configs/node/config.yaml
```
```

> **Warning:**

> **Important: Back up your operator key!** Losing access means you'll need to set up a new operator and update your validator configuration. Store the backup securely.

To print the private key from your backup file, use the `--print` flag. **Keep this private key secure and never share it.**

Running the node

Once you have configured everything, you are ready to start the node.

> **Warning:**

> **Important:** Before starting the node, ensure you have:

- > 1. **Operator key** imported into the node (see "Import the Operator Key" above)
- > 2. **Validator wallet address** - obtained after joining as validator via `genlayer staking wizard` or `validator-join`

>

> Without both configured, your node will run as a full node instead of a validator.

Running the Node using the binary

1. Set the LLM Provider API Key

Set the appropriate environment variable for your chosen LLM provider. See the [GenVM Configuration](/validators/genvm-configuration) page for details on LLM providers and

obtaining API credits.

```
```sh copy
For Heurist
export HEURISTKEY='your_heurist_api_key'

For Comput3
export COMPUT3KEY='your_comput3_api_key'

For io.net
export IOINTELLIGENCE_API_KEY='your_ionet_api_key'

For Chutes
export CHUTES_API_KEY='your_chutes_api_key'

For Morpheus
export MORPHEUS_API_KEY='your_morpheus_api_key'

For other providers, use the appropriate environment variable name
```
```

2. Run the WebDriver container

```
```sh copy
docker compose up -d # Starts the WebDriver needed by the GenVM web module
```
```

3. (Optional) Run two services (modules) in background (this is a crucial step for running _Intelligent_ contracts). This can be done automatically or manually.

- To start them automatically in node configuration set `genvm.manage\_modules` to `true`
- To start them manually run

```
```bash copy
./third_party/genvm/bin/genvm-modules web & ./third_party/genvm/bin/genvm-modules llm
&
```
```

> ****Note:****

> Note: If you are using the default configuration, `genvm.manage\_modules` is set to `true` by default, meaning the node will manage these modules automatically.

4. Checking Your Configuration

To ensure your node is correctly configured, you can run the following command:

```
```sh copy
./bin/genlayernode doctor
```
```

The `doctor` command now includes comprehensive GenVM diagnostics integration to validate:

- Consensus contract configuration and accessibility
- GenVM module connectivity and health status
- LLM provider configuration and API connectivity
- Network configuration and ZKSync node accessibility

5. Run the node

```
```sh copy
./bin/genlayernode run -c $(pwd)/configs/node/config.yaml --password "your secret password" # The same password you used when creating the account
```
```

> ****Note:****

> If you are running the node via SSH, the process might terminate if your

> connection drops. To prevent this, consider using a terminal multiplexer like
> `screen` or `tmux` to keep the node running in the background even if your SSH
> session ends. You can find a guide on using `screen`
> [here](https://www.networkworld.com/article/967925/how-the-linux-screen-tool-can-save-your-tasks-and-your-sanity-if-ssh-is-interrupted.html).

Running the Node using docker-compose

You can also run the GenLayer node using Docker and Docker Compose.

1. Create a `docker-compose.yaml` file with the following content:

```
```yaml copy
services:
 webdriver-container:
 container_name: genlayer-node-webdriver
 image: yeagerai/genlayer-genvm-webdriver:0.0.11
 shm_size: 2gb
 security_opt:
 - no-new-privileges:true
 environment:
 PORT: 4444
 ports:
 - "${WEBDRIVER_PORT:-4444}:4444"
 restart: unless-stopped
 genlayer-node:
 image: yeagerai/genlayer-node:${NODE_VERSION:-latest}
 entrypoint: ["sh", "-c", "/app/bin/genlayernode run --password ${NODE_PASSWORD:-12345678}"]
 container_name: genlayer-node
 restart: unless-stopped
 env_file:
 - path: ./env
 required: false
 ports:
 - "${NODE_RPC_PORT:-9151}:9151"
 - "${NODE_OPS_PORT:-9153}:9153"
 volumes:
 - ${NODE_CONFIG_PATH:-./configs/node/config.yaml}:/app/configs/node/config.yaml:ro
 - ${NODE_DATA_PATH:-./data}:/app/data
 - ./genvm-module-web-docker.yaml:/app/third_party/genvm/config/genvm-module-web.yaml
 - /var/run/docker.sock:/var/run/docker.sock:ro # required for webdriver metrics
 collection
 security_opt:
 - no-new-privileges:true
 logging:
 driver: "json-file"
 options:
 max-size: "100m"
 max-file: "3"
 compress: "true"
 depends_on:
 - webdriver-container
 profiles:
 - node
...
```
```

2. Create `genvm-module-web-docker.yaml` for Docker networking:

The extracted tarball includes `third\_party/genvm/config/genvm-module-web.yaml` configured for `localhost`, which works when running the binary directly. For docker-compose, you need a version that points to the webdriver service name.

Create `genvm-module-web-docker.yaml` in the same directory as your `docker-compose.yaml`:

- Copy the contents of `third\_party/genvm/config/genvm-module-web.yaml` into `genvm-module-web-docker.yaml`.
- Modify the `webdriver\_host` configuration to use the service name `webdriver-container` instead of `localhost`:

```
```yaml copy
webdriver_host: http://webdriver-container:4444
```
```

This allows the node container to reach the webdriver container using Docker's internal networking.

The key difference:

- **Binary execution**: uses `localhost:4444` (webdriver runs on same machine)
- **Docker-compose**: uses `webdriver-container:4444` (container service name)

3. Set the required environment variables in a `.env` file:

```
> Note:
> This `.env` file serves two purposes: it configures Docker Compose variables (like
`NODE_VERSION`, `NODE_PASSWORD`, ports) and can also include `GENLAYERNODE_*` variables to
override node configuration values as described in [Overriding Configuration with
Environment Variables](#overriding-configuration-with-environment-variables).
```

```
```env copy
GenLayer Node Release Configuration

=====
WebDriver Configuration (required for GenVM)
=====
WEBDRIVER_PORT=4444

=====
Node Service Configuration (optional - use node profile)
=====
Docker image version
NODE_VERSION=v0.4.0

Keystore password (used to unlock the pre-imported wallet)
NODE_PASSWORD=12345678

Path to node configuration file
NODE_CONFIG_PATH=./configs/node/config.yaml

Path to node data directory (for persistence)
NODE_DATA_PATH=./data

Port mappings
NODE_RPC_PORT=9151
NODE_OPS_PORT=9153

LLM API Key (required for GenVM LLM module)
HEURISTKEY=
COMPUT3KEY=
IOINTELLIGENCE_API_KEY=
ANTHROPICKEY=
XAIKEY=
GEMINIKEY=
ATOMAKEY=
CHUTES_API_KEY=
MORPHEUS_API_KEY=
```
```

or simply use the provided `docker-compose.yaml`, `.env.example` and the `genvm-module-web-docker.yaml` from the extracted tarball.

4. Checking Your Configuration

To ensure your node is correctly configured, you can run the following command:

```
```sh copy
source .env && docker run --rm --env-file ./env \
 -v ${NODE_CONFIG_PATH:-./configs/node/config.yaml}:/app/configs/node/config.yaml \
 yeagerai/genlayer-node:${NODE_VERSION:-v0.4.0} \
 ./bin/genlayernode doctor
```
```

5. Start the services using Docker Compose:

```
```sh copy
source .env && docker compose --profile node up -d
```
```

Telemetry

For a quick setup using Docker Compose:

1. ****Configure your `.env` file**** with the required variables for monitoring:

```
```env copy
Central monitoring server endpoints for GenLayer Foundation
CENTRAL_MONITORING_URL=https://prometheus-prod-66-prod-us-east-
3.grafana.net/api/prom/push
CENTRAL_LOKI_URL=https://logs-prod-042.grafana.net/loki/api/v1/push

Authentication for central monitoring
Metrics (Prometheus) credentials
CENTRAL_MONITORING_USERNAME=your-metrics-username
CENTRAL_MONITORING_PASSWORD=your-metrics-password
Logs (Loki) credentials
CENTRAL_LOKI_USERNAME=your-logs-username
CENTRAL_LOKI_PASSWORD=your-logs-password

Node identification
NODE_ID=validator-001
VALIDATOR_NAME=MyValidator
```
```

2. ****Run docker compose****:

```
```bash copy
docker compose --profile monitoring up -d
```
```

3. ****Verify your node is pushing metrics**** by checking the [GenLayer Foundation public dashboard](https://genlayerfoundation.grafana.net/public-dashboards/66a372d856ea44e78cf9ac21a344f792)

For detailed monitoring setup including Prometheus metrics, Grafana dashboards, and centralized logging with Alloy, see the [Monitoring Guide](/validators/monitoring).

Monitoring & Telemetry
Source: <https://docs.genlayer.com/validators/monitoring>

Monitoring and telemetry for GenLayer validators means collecting validator performance, health, and resource usage metrics through endpoints ready for Prometheus and other monitoring tools. GenLayer validators expose comprehensive metrics so you can monitor your

validator's performance, health, and resource usage.

Accessing Metrics

The metrics endpoint is exposed on the operations port (default: 9153) configured in your `config.yaml`:

```
```yaml
node:
 ops:
 port: 9153 # Metrics port
 endpoints:
 metrics: true # Enable metrics endpoint
...
```
```

Once your node is running, you can access the metrics at:

```
```
http://localhost:9153/metrics
```
```

Available Metrics

The validator exposes various metric collectors that can be individually configured:

- ****Node Metrics****: Core validator performance metrics including block processing, transaction handling, and consensus participation
- ****GenVM Metrics****: Virtual machine performance metrics, including execution times and resource usage
- ****WebDriver Metrics****: Metrics related to web access and external data fetching

Configuring Metrics Collection

You can customize metrics collection in your `config.yaml`:

```
```yaml
metrics:
 interval: "15s" # Default collection interval
 collectors:
 node:
 enabled: true
 interval: "30s" # Override interval for specific collector
 genvm:
 enabled: true
 interval: "20s"
 webdriver:
 enabled: true
 interval: "60s"
...
```
```

Example Metrics Query

To check if metrics are working correctly:

```
```bash copy
Get all available metrics
curl http://localhost:9153/metrics

Check specific metric (example)
curl -s http://localhost:9153/metrics | grep genlayer_node_
```
```

> ****Note:****

> The metrics endpoint also provides `/health` and `/balance` endpoints on the same port for additional monitoring capabilities.

Monitoring Best Practices

1. ****Set up alerts**** for critical metrics like node synchronization status and missed blocks
2. ****Monitor resource usage**** to ensure your validator has sufficient CPU, memory, and disk space
3. ****Track GenVM performance**** to optimize LLM provider selection and configuration
4. ****Use visualization tools**** like Grafana to create dashboards for easy monitoring

> ****Note:****

> For production validators, we recommend setting up a complete monitoring stack with Prometheus and Grafana. This enables real-time visibility into your validator's performance and helps identify issues before they impact your validator's operation.

Logs and Metrics Forwarding

You can forward your logs and metrics to external systems for centralized monitoring and alerting by using the service `alloy` provided in the `docker-compose.yaml` from the extracted tarball.

Centralized Push to GenLayer Foundation Grafana Cloud (using Alloy)

To contribute your node's metrics and logs to the centralized GenLayer Foundation Grafana Cloud dashboard (improving aggregate network visibility, alerts, and community monitoring), use the built-in Alloy service.

****Why contribute?****

- Helps the Foundation and community track overall testnet health (validator participation, latency, resource usage).
- May positively influence testnet points/rewards (visible healthy nodes are prioritized).
- Setup takes 10-15 minutes once credentials are provided.

****Prerequisites****

- Metrics enabled in `config.yaml` (`endpoints.metrics: true` – default in recent versions).
- Ops port 9153 exposed in docker-compose (`ports: - "9153:9153"`).
- Credentials from the Foundation team (ask in #testnet-asimov):
 - `CENTRAL\_MONITORING\_URL` – Prometheus remote write URL
 - `CENTRAL\_LOKI\_URL` – Loki push URL
 - `CENTRAL\_MONITORING\_USERNAME` / `CENTRAL\_MONITORING\_PASSWORD` – Metrics (Prometheus) credentials
 - `CENTRAL\_LOKI\_USERNAME` / `CENTRAL\_LOKI\_PASSWORD` – Logs (Loki) credentials

****Steps****

1. ****Create or update .env**** (next to your docker-compose.yaml):

```
```.env
```

```
Central monitoring server endpoints for GenLayer Foundation
```

```
CENTRAL_MONITORING_URL=https://prometheus-prod-66-prod-us-east-3.grafana.net/api/prom/push
```

```
CENTRAL_LOKI_URL=https://logs-prod-042.grafana.net/loki/api/v1/push
```

```
Authentication for central monitoring
```

```
Metrics (Prometheus) credentials
```

```
CENTRAL_MONITORING_USERNAME=your-metrics-username
```

```
CENTRAL_MONITORING_PASSWORD=your-metrics-password
```

```
Logs (Loki) credentials
```

```
CENTRAL_LOKI_USERNAME=your-logs-username
```

```
CENTRAL_LOKI_PASSWORD=your-logs-password
```

```
Node identification
```

```
NODE_ID=validator-001
```

```
VALIDATOR_NAME=MyValidator
```

```
NETWORK_NAME=asimov-phase5
```

```
Usually defaults are fine
```

```
NODE_METRICS_ENDPOINT=host.docker.internal:9153
```

```
LOG_FILE_PATTERN=/var/log/genlayer/node*.log
METRICS_SCRAPE_INTERVAL=15s
````
```

2. ****Add or verify the Alloy service in docker-compose.yml**** (copy if missing):

```
``yaml
# Grafana Alloy for both logs and metrics forwarding
# Supports both single node and multi-node configurations
#
# Single Node Mode:
#   Set NODE_ID, VALIDATOR_NAME, NODE_METRICS_ENDPOINT in .env
#   docker compose --profile monitoring up -d
#
# Multi-Node Mode:
#   Set SCRAPE_TARGETS_JSON in .env
#   docker compose --profile monitoring up -d
alloy:
  image: grafana/alloy:v1.16.1
  container_name: genlayer-node-alloy
  command:
    - run
    - /etc/alloy/config.river
    - --server.http.listen-addr=0.0.0.0:12345
    - --storage.path=/var/lib/alloy/data
  volumes:
    - ./alloy-config.river:/etc/alloy/config.river:ro
    - ./alloy-healthcheck.sh:/etc/alloy/healthcheck.sh:ro
    - ${NODE_LOGS_PATH:-./data/node/logs}:/var/log/genlayer:ro
    - alloy_data:/var/lib/alloy
  healthcheck:
    test: ["CMD", "sh", "/etc/alloy/healthcheck.sh"]
    interval: 120s
    timeout: 15s
    retries: 1
    start_period: 120s
  environment:
    # Central monitoring endpoints
    - CENTRAL_LOKI_URL=${CENTRAL_LOKI_URL:-https://logs-prod-042.grafana.net/loki/api/v1/push}
    - CENTRAL_MONITORING_URL=${CENTRAL_MONITORING_URL:-https://prometheus-prod-66-prod-us-east-3.grafana.net/api/prom/push}

    # Metrics (Prometheus) authentication
    - CENTRAL_MONITORING_USERNAME=${CENTRAL_MONITORING_USERNAME:-telemetric}
    - CENTRAL_MONITORING_PASSWORD=${CENTRAL_MONITORING_PASSWORD:-12345678}

    # Logs (Loki) authentication
    - CENTRAL_LOKI_USERNAME=${CENTRAL_LOKI_USERNAME:-telemetric}
    - CENTRAL_LOKI_PASSWORD=${CENTRAL_LOKI_PASSWORD:-12345678}

    # Single node configuration
    - NODE_ID=${NODE_ID:-validator-001}
    - VALIDATOR_NAME=${VALIDATOR_NAME:-MyValidator}
    - NODE_METRICS_ENDPOINT=${NODE_METRICS_ENDPOINT:-host.docker.internal:9153}

    # Multi-node configuration
    # When set, overrides single node config above
    - SCRAPE_TARGETS_JSON=${SCRAPE_TARGETS_JSON:-}

    # Scraping configuration
    - METRICS_SCRAPE_INTERVAL=${METRICS_SCRAPE_INTERVAL:-60s}
    - METRICS_SCRAPE_TIMEOUT=${METRICS_SCRAPE_TIMEOUT:-10s}
    - ALLOY_SELF_MONITORING_INTERVAL=${ALLOY_SELF_MONITORING_INTERVAL:-60s}
```

```

# Log collection configuration
- LOG_FILE_PATTERN=${LOG_FILE_PATTERN:-/var/log/genlayer/node*.log}

# Log batching configuration
- LOKI_BATCH_SIZE=${LOKI_BATCH_SIZE:-1MiB}
- LOKI_BATCH_WAIT=${LOKI_BATCH_WAIT:-60s}
ports:
- "12345:12345" # Alloy UI for debugging
restart: unless-stopped
profiles:
- monitoring
extra_hosts:
- "host.docker.internal:host-gateway"

volumes:
  alloy_data:
  `

```

3. ****Create or update ./alloy-config.river**** (use the provided version – it handles logs and metrics forwarding):

```

````river
// Grafana Alloy Configuration for GenLayer Node Telemetry
// Handles both log collection and metrics forwarding

// =====
// Log Collection and Forwarding
// =====

// Discovery component to find log files using local.file_match
// Supports different log file patterns:
// - Single node: "/var/log/genlayer/node.log"
// - Multi-node: "/var/log/genlayer/*/logs/node.log" (each node in subdirectory)
// - Custom pattern via LOG_FILE_PATTERN env var
local.file_match "genlayer_logs" {
 path_targets = [{
 __path__ = coalesce(sys.env("LOG_FILE_PATTERN"), "/var/log/genlayer/node*.log"),
 }]
}

// Relabel to add metadata labels to log entries
discovery.relabel "add_labels" {
 targets = local.file_match.genlayer_logs.targets

 // Add instance label from environment variable
 rule {
 target_label = "instance"
 replacement = sys.env("NODE_ID")
 }

 // Add validator_name label from environment variable
 rule {
 target_label = "validator_name"
 replacement = sys.env("VALIDATOR_NAME")
 }

 // Add component label
 rule {
 target_label = "component"
 replacement = "alloy"
 }

 // Add job label
 rule {
 target_label = "job"
 }
}

```

```

 replacement = "genlayer-node"
 }
}

// Source component to read log files
loki.source.file "genlayer" {
 targets = discovery.relabel.add_labels.output
 forward_to = [loki.write.central.receiver]

 // Tail from end to avoid ingesting entire log history on startup
 tail_from_end = true
}

// Write logs to central Loki instance
loki.write "central" {
 endpoint {
 url = sys.env("CENTRAL_LOKI_URL")

 // HTTP Basic Authentication
 basic_auth {
 username = sys.env("CENTRAL_LOKI_USERNAME")
 password = sys.env("CENTRAL_LOKI_PASSWORD")
 }

 // Enable retry with default exponential backoff
 // Note: Alloy's loki.write doesn't have a retry block; retries are handled
 automatically
 // with exponential backoff by default when the endpoint is unreachable

 // Configurable batch settings for efficient log sending
 batch_size = coalesce(sys.env("LOKI_BATCH_SIZE"), "1MiB") // Maximum batch size before
 sending
 batch_wait = coalesce(sys.env("LOKI_BATCH_WAIT"), "60s") // Maximum wait time before
 sending partial batch
 }
}

// =====
// Prometheus Metrics Collection and Forwarding
// =====

// Scrape metrics from GenLayer node(s)
// Supports both single node and multi-node configurations
//
// Single Node Mode:
// Set NODE_METRICS_ENDPOINT, NODE_ID, VALIDATOR_NAME
//
// Multi-Node Mode:
// Set SCRAPE_TARGETS_JSON with JSON array of target objects
// Example:
// [{"__address__": "host.docker.internal:9250", "instance": "0x...", "validator_name": "node-1"}]
//
// Note: The "network" label is emitted by the node itself (auto-detected from consensus
// address),
// so it does not need to be configured here.
prometheus.scrape "genlayer_node" {
 // Dynamic targets based on environment variable
 // If SCRAPE_TARGETS_JSON is set, use it (multi-node mode)
 // Otherwise, build single target from individual env vars (single node mode)
 targets = encoding.from_json(coalesce(sys.env("SCRAPE_TARGETS_JSON"), string.format(
 [{"__address__": \"%s\", \"instance\": \"%s\", \"validator_name\": \"%s\"}],
 coalesce(sys.env("NODE_METRICS_ENDPOINT"), "host.docker.internal:9153"),
 coalesce(sys.env("NODE_ID"), "local"), coalesce(sys.env("VALIDATOR_NAME"), "default")))))

 forward_to = [prometheus.relabel.metrics.receiver]

```

```

// Configurable scrape intervals
scrape_interval = coalesce(sys.env("METRICS_SCRAPE_INTERVAL"), "60s")
scrape_timeout = coalesce(sys.env("METRICS_SCRAPE_TIMEOUT"), "10s")
}

// Relabel metrics to filter before forwarding
prometheus.relabel "metrics" {
 forward_to = [prometheus.remote_write.central.receiver]

 // Option 1: Forward all metrics (default)
 // Currently forwarding all metrics from the node.

 // Option 2: Only keep genlayer_node_* metrics to reduce bandwidth (recommended)
 // To enable filtering and reduce bandwidth, uncomment the following rule:
 /*
 rule {
 source_labels = ["__name__"]
 regex = "genlayer_node_.*"
 action = "keep"
 }
 */
}

// Remote write configuration for sending metrics to central Prometheus
prometheus.remote_write "central" {
 endpoint {
 url = sys.env("CENTRAL_MONITORING_URL")

 // HTTP Basic Authentication
 basic_auth {
 username = sys.env("CENTRAL_MONITORING_USERNAME")
 password = sys.env("CENTRAL_MONITORING_PASSWORD")
 }

 // Queue configuration for reliability
 queue_config {
 capacity = 10000
 max_shards = 5
 max_samples_per_send = 500
 batch_send_deadline = coalesce(sys.env("METRICS_BATCH_SEND_DEADLINE"), "60s")
 }
 }
}

// =====
// Alloy Self-Monitoring
// =====

// Alloy internal exporter for health monitoring
prometheus.exporter.self "alloy" {}

// Expose Alloy's own metrics on the HTTP server
prometheus.scrape "alloy" {
 targets = prometheus.exporter.self.alloy.targets
 forward_to = [] // Not forwarding Alloy metrics to reduce noise

 // Configurable scrape interval for Alloy's internal health monitoring
 scrape_interval = coalesce(sys.env("ALLOY_SELF_MONITORING_INTERVAL"), "60s")
}
...

```

4. **\*\*Start Alloy\*\*:**

```
``bash copy
```

```
docker compose --profile monitoring up -d
```

## 5. **Verify it works**:

- Open Alloy UI: <http://localhost:12345/targets> – the "genlayer\_node" scrape target should show status **UP**.
- Check logs for successful sends:

```
```bash copy
docker logs genlayer-node-alloy | grep "sent batch"
```

```
```bash copy
docker logs genlayer-node-alloy | grep "remote_write"
```

Look for messages indicating successful batch sending (no error codes like 401, 403, 500).

- In Foundation Grafana Cloud: search for metrics with labels  
`instance="\${NODE\_ID}"` or `validator\_name="\${VALIDATOR\_NAME}"`  
(example: `genlayer\_node\_uptime\_seconds{instance="0xYourID"}`).

## ## Troubleshooting

### ### Using the Alloy UI

The Grafana Alloy service includes a built-in web UI for troubleshooting and monitoring the telemetry pipeline. Access it at:

```
...
http://localhost:12345
```

The Alloy UI provides:

- **Targets view** (`/targets`): Shows the status of all scrape targets. Check if your node's metrics endpoint shows as **UP**.
- **Graph view** (`/graph`): Explore and query collected metrics locally before they're forwarded.
- **Component health**: View the status of all Alloy pipeline components (scrapers, relabelers, writers).

> **Note**:

> If metrics are not appearing in the central Grafana Cloud, first verify they show correctly in the local Alloy UI. This helps isolate whether the issue is with metrics collection or forwarding.

### ### Common Issues

- **No local metrics**:

```
```bash copy
curl http://localhost:9153/metrics
```

– it should return Prometheus-formatted data.

- **Authentication errors (401/403)**: Double-check `CENTRAL\_MONITORING\_USERNAME`, `CENTRAL\_MONITORING\_PASSWORD`, `CENTRAL\_LOKI\_USERNAME`, and `CENTRAL\_LOKI\_PASSWORD` in `.env`.

- **No data pushed**: Ensure URLs in `.env` have no trailing slash.

- **Target showing DOWN in Alloy UI**: Verify your node is running and the ops port (9153) is accessible. Check that `NODE\_METRICS\_ENDPOINT` in `.env` is correct.

Getting Help

Share Alloy logs when asking for assistance:

```
```bash copy
docker logs genlayer-node-alloy
```
```

System Requirements

Source: <https://docs.genlayer.com/validators/system-requirements>

System Requirements for a GenLayer validator node are the **initial** recommended hardware and software guidelines for running one. These guidelines may change as the network grows and evolves.

RAM

- **Recommended:** 16 GB
- **Why:**
 - GenLayer's `_genvm_` can spawn multiple Sub-VMs for contract calls and non-deterministic blocks.
 - Each Sub-VM can consume up to ~4 GB of RAM for storage.

CPU

- **Recommended:** Modern multi-core processor with at least 8 cores/16 threads

Architecture

- **Recommended:** ``AMD64``
- ``ARM64`` is not supported at this time

Storage

- **Recommended Disk Space:** 128 GB+
- **Preferred Type:** SSD or NVMe (M.2)

Network

- 100 Mbps connection
- **Recommended:** 1 Gbps+

GPU (Optional)

- GPU is **Not Required**
- If you want to run LLMs locally, you will need to select appropriate hardware (typically a CUDA-compatible GPU with sufficient VRAM for the model you intend to use)

> **Note:**

> These requirements are a starting point. As GenLayer evolves and usage patterns change (e.g., more complex AI-driven Intelligent Contracts), the recommended hardware may change.

Software

- Operating System - 64-bit Linux (Ubuntu, Debian, CentOS, etc.)
- ``docker`` - for running the WebDriver container
- ``python3``, ``python3-pip`` and ``python3-venv`` for GenVM setup

GenVM Configuration

Source: <https://docs.genlayer.com/validators/genvm-configuration>

GenVM Configuration defines the LLM providers and web module settings a validator's GenVM needs to process AI-powered contract logic and fetch live internet data. This page covers how to configure both.

****Why this matters:**** The LLM providers you choose directly affect your validator's operating costs and performance. Faster, cheaper models reduce your costs per transaction. Higher-quality models improve your consensus accuracy, which affects your rewards and avoids slashing. By tuning your provider setup – choosing the right models, configuring fallback chains, running local inference, or applying input filters – you can optimize the economics of running your validator.

LLM Provider Credits

```
> **Note:**
> GenLayer has partnered with multiple LLM providers to offer free credits for validators:
>
> **[Heurist](https://www.heurist.ai/)** – free [API credits](https://dev-api-form.heurist.ai) with referral code _"genlayer"_.
>
> **[Comput3](https://genlayer.comput3.ai/)** – free [API credits](https://genlayer.comput3.ai/) for llama3, hermes3, and qwen3 models.
>
> **[io.net](/partners/ionet)** – create an account at [id.io.net](https://id.io.net/login) and [request free credits](https://form.typeform.com/to/pDmCCViV).
>
> **[Chutes](/partners/chutes)** – create an account at [chutes.ai](https://chutes.ai) and generate an API key from settings.
>
> **[Morpheus](/partners/morpheus)** – obtain a [Morpheus API key](https://mor.org/) for DeepSeek, Llama, Qwen, and more via OpenAI-compatible API.
```

Hot-Reload

You can modify configuration files on the fly and reload without stopping your node. This is important when experimenting with providers or tuning your setup.

```
``bash
PORT=3999 # GenVM manager port

# Restart LLM module (reloads Lua script + YAML config)
curl -X POST "http://127.0.0.1:${PORT}/module/stop" \
  -H 'Content-Type: application/json' \
  -d '{"module_type": "Llm"}'

curl -X POST "http://127.0.0.1:${PORT}/module/start" \
  -H 'Content-Type: application/json' \
  -d '{"module_type": "Web", "config": null}'
...`
```

Repeat for each GenVM instance if running multiple. For a full restart: ``sudo systemctl restart genlayer-node``.

Configuration Files

All configuration lives at ``third_party/genvm/config/``. Three files:

| File | What it controls | Should you modify? |
|--------------------------------------|---|---|
| <code>`genvm-module-llm.yaml`</code> | LLM providers, models, prompt templates, Lua script selection | **Yes** – this is your main configuration file |
| <code>`genvm-module-web.yaml`</code> | Web access, webdriver, URL restrictions | Rarely – only for advanced URL policy changes |
| <code>`genvm-manager.yaml`</code> | Concurrency permits, VM pool | Rarely – only if tuning for your hardware |

Configuring Your LLM Backends

The ``backends`` section in ``genvm-module-llm.yaml`` defines which LLMs your validator can use. The Lua script (configured via ``lua_script_path``) then selects from these backends at runtime.

Backend entry format

```
```yaml
backends:
 my-provider:
 # Your name for this backend
 enabled: true # Set false to disable without removing
 host: https://api.example.com
 provider: openai-compatible # Provider type (see table below)
 key: ${ENV[MY_API_KEY]} # API key via environment variable
 models:
 model-name:
 enabled: true
 supports_json: true # Can return structured JSON output
 supports_image: false # Can process image inputs
 use_max_completion_tokens: false
 meta: {} # Custom metadata (used by greyboxing strategies)
...
```
```

Model capabilities

Intelligent Contract LLM requests have two axes:

- **Input:** text only, or text + up to 2 images
- **Output:** free-form text, or structured JSON

This gives four combinations: text→text, text→JSON, images+text→text, images+text→JSON. The capability flags control which models are eligible:

- ``supports_json: true`` – model can produce structured JSON output
- ``supports_image: true`` – model can accept image inputs

****Your validator must support all combinations across your model set.**** Not every model needs to support everything – that's where routing comes in. You can use a cheap text model for simple prompts and a vision-capable model for image requests. But if a contract sends an image prompt and none of your enabled models support images, the request will fail.

The ``supports_json`` and ``supports_image`` flags tell the Lua script which models are eligible for each request type. Set them accurately for each model.

Supported provider types

| Type | Description | Example services |
|----------------------------------|---|--|
| <code>`openai-compatible`</code> | OpenAI-compatible chat/completions API | OpenAI, OpenRouter, Heurist, Comput3, Chutes, Morpheus, X.ai |
| <code>`ollama`</code> | Ollama's native <code>`/api/generate`</code> endpoint | Local Ollama instance |
| <code>`anthropic`</code> | Anthropic's Messages API | Claude models |
| <code>`google`</code> | Google's Generative Language API | Gemini models |

Example: Adding a local Ollama instance

Add this to the ``backends`` section. No API key needed – Ollama runs locally. Follow [Ollama's install guide](https://ollama.com) to set up the server and pull a model.

```
```yaml
backends:
 my-ollama:
 host: http://localhost:11434
```
```

```

provider: ollama
models:
  llama3.3:
    supports_json: true
    supports_image: false
...

```

Example: Adding OpenRouter

[OpenRouter](https://openrouter.ai) provides access to many models through a single API key:

```

```yaml
backends:
 openrouter:
 host: https://openrouter.ai/api
 provider: openai-compatible
 key: ${ENV[OPENROUTERKEY]}
 models:
 deepseek/deepseek-v3.2:
 supports_json: true
 anthropic/claude-haiku-4.5:
 supports_json: true
 supports_image: true
 openai/gpt-5.1-mini:
 supports_json: true
 supports_image: true
...

```

### ### API key environment variables

Set these in your ``.env`` file before starting the node. You only need to set the keys for providers you're actually using. Keys use ``${ENV[VARIABLE_NAME]}`` syntax in the YAML.

| Variable                 | Provider      |
|--------------------------|---------------|
| --- ---                  |               |
| `OPENROUTERKEY`          | OpenRouter    |
| `OPENAIKEY`              | OpenAI        |
| `ANTHROPICKEY`           | Anthropic     |
| `GEMINIKEY`              | Google Gemini |
| `HEURISTKEY`             | Heurist       |
| `XAIKEY`                 | X.ai (Grok)   |
| `COMPUT3KEY`             | Comput3       |
| `IOINTELLIGENCE_API_KEY` | io.net        |
| `CHUTES_API_KEY`         | Chutes        |
| `MORPHEUS_API_KEY`       | Morpheus      |

> **\*\*Note:\*\***

> Enabled backends without a configured API key will log warnings. Either set the key or disable the backend with ``enabled: false``.

### ## Greyboxing: Optimizing Your Validator

Greyboxing is how you configure and optimize your validator's LLM execution. From the network's point of view, each validator is a **\*\*grey box\*\*** – an attacker can see that LLMs are being used, but cannot know which model will process a given request, how it's configured, or what filters are applied. This per-validator opacity is a security property: it makes prompt injection and manipulation attacks much harder because there's no single configuration to target.

As a validator operator, greyboxing is also your primary lever for **\*\*economics\*\*** – by choosing the right models, configuring fallback chains, and applying filters, you control your operating costs and consensus quality.

### ### Choosing models

Start with a capable model and optimize down over time. Your validator needs to produce correct consensus results – using a model that's too weak will hurt your accuracy, leading to missed rewards or slashing. Once you're confident in your setup, you can experiment with cheaper alternatives.

**\*\*Recommended starting points:\*\***

- **\*\*Frontier tier:\*\*** Claude Sonnet 4.6, GPT-5, Gemini 3 Flash – high accuracy, higher cost
- **\*\*Strong open-source:\*\*** DeepSeek V3.2, Qwen3-235B, Llama 4 Maverick – good accuracy, lower cost
- **\*\*Budget tier:\*\*** Nemotron, smaller Llama variants – lowest cost, test carefully before relying on these

For image-capable requests, you need at least one vision model (e.g., GPT-5, Gemini Flash, Claude Sonnet).

A practical approach: use a frontier model as your primary, with cheaper models as fallback. The greyboxing ordered fallback strategy (below) makes this easy.

### ### What you can control

Greyboxing encompasses several layers of configuration, all implemented in the Lua script that ``lua_script_path`` points to:

**\*\*Model selection\*\*** – Choose which LLM handles each request. You can route text prompts to a cheap fast model and image prompts to a vision-capable one. This can be random, an ordered fallback chain, or a custom router.

**\*\*Input filtering\*\*** – Apply filters to prompts before they reach the LLM. Available text filters: ``NFKC`` (Unicode normalization), ``NormalizeWS`` (whitespace), ``RmZeroWidth`` (remove zero-width characters). Available image filters: ``Denoise``, ``JPEG`` (compression), ``GuassainNoise``, ``Unsharpen``. Filtering normalizes inputs across validators and can improve consensus.

**\*\*LLM parameters\*\*** – Temperature, max tokens, system prompts. Configured per-model in YAML or overridden in your Lua script.

**\*\*Cost management\*\*** – In future versions, your greyboxing script will be able to track execution costs and decide when a transaction is too expensive to process. As a validator, if the cost of LLM calls exceeds what makes economic sense, you can timeout the transaction rather than executing at a loss.

### ### Built-in strategies

GenVM ships with two Lua scripts. Switch between them by changing ``lua_script_path`` in ``genvm-module-llm.yaml``.

#### #### Default: Random selection

**\*\*Script:\*\*** ``genvm-llm-default.lua``

Iterates through all enabled backends that match the required capabilities (JSON support, image support) and tries them in arbitrary order. Falls back to the next provider on overload errors (429, 503, 408, 504, 529).

This is the simplest strategy – works out of the box with no tuning.

#### #### Ordered fallback chain

**\*\*Script:\*\*** ``genvm-llm-greybox.lua``

Uses a fixed priority chain configured via ``meta.greybox`` fields in the YAML. Each model declares which chain it belongs to (text, image, or both) and its priority. Lower number = tried first.

```yaml``

```

models:
  deepseek/deepseek-v3.2:
    supports_json: true
    meta:
      greybox: { text: 1 }          # text chain, priority 1 (tried first)
  openai/gpt-5.1-mini:
    supports_json: true
    supports_image: true
    meta:
      greybox: { image: 1 }        # image chain, priority 1
  anthropic/claude-haiku-4.5:
    supports_json: true
    supports_image: true
    meta:
      greybox: { text: 3, image: 3 } # both chains, lower priority
...

**To enable:**

```bash
sudo systemctl stop genlayer-node

Set your OpenRouter key (or whichever provider you're using)
echo "OPENROUTERKEY=sk-or-v1-your-key-here" >> /opt/genlayer-node/.env

Apply the release config with greybox metadata
VERSION=$(readlink /opt/genlayer-node/bin | sed 's|/bin||; s|.|/|'|)
cp /opt/genlayer-node/${VERSION}/third_party/genvm/config/genvm-modules-llm-release.yaml \
 /opt/genlayer-node/${VERSION}/third_party/genvm/config/genvm-module-llm.yaml

Switch to greybox script
sed -i 's/genvm-llm-default.lua/genvm-llm-greybox.lua/' \
 /opt/genlayer-node/${VERSION}/third_party/genvm/config/genvm-module-llm.yaml

sudo systemctl start genlayer-node
```

**Verify:**

```bash
sudo journalctl -u genlayer-node --no-hostname | grep "greybox"
Expected: greybox: success provider: openrouter model: deepseek/deepseek-v3.2
```

**To revert:** change `genvm-llm-greybox.lua` back to `genvm-llm-default.lua` and restart.

#### Writing a custom strategy

For full control, write your own Lua script. Create a `.lua` file in the config directory and point `lua_script_path` to it.

Your script must export two global functions:

```lua
local lib = require("lib-genvm")
local llm = require("lib-llm")

function ExecPrompt(ctx, args, remaining_gen)
 -- args.prompt: the text prompt
 -- args.response_format: "text" or "json"
 -- args.images: table of image bytes (may be nil)
 -- remaining_gen: gas budget (for future cost tracking)

 -- Transform the raw args into internal format
 local mapped = llm.exec_prompt_transform(args)

```

```

-- Apply input filters
args.prompt = lib.rs.filter_text(args.prompt, {'NFKC', 'NormalizeWS'})

-- Select providers that match requirements
local providers = llm.select_providers_for(mapped.prompt, mapped.format)

-- Execute on a specific provider
local result = llm.rs.exec_prompt_in_provider(ctx, {
 provider = "my-provider",
 model = "model-name",
 prompt = mapped.prompt,
 format = mapped.format,
})

result.consumed_gen = 0 -- Cost tracking (future)
return result
end

function ExecPromptTemplate(ctx, args, remaining_gen)
 -- Called for Equivalence Principle template prompts
 -- args.template: "EqComparative", "EqNonComparativeValidator", "EqNonComparativeLeader"
 -- args.vars: template variable substitutions

 local mapped = llm.exec_prompt_template_transform(args)

 -- ... your provider selection logic ...
end
` ``

```

## ## Prompt Templates

The ``prompt_templates`` section in ``genvm-module-llm.yaml`` configures how your validator acts as a **`**judge**`** in [Equivalence Principle](/developers/intelligent-contracts/features/non-determinism) scenarios. Your validator isn't just running raw LLM calls – in certain consensus situations, it needs to evaluate whether two outputs are equivalent or whether a result meets specified criteria.

Three templates, using ``#{variable}`` syntax for substitution:

| Template                                    | When it's used                        | What it returns               |
|---------------------------------------------|---------------------------------------|-------------------------------|
| <code>`eq_comparative`</code>               | Comparing leader vs validator outputs | Boolean: are they equivalent? |
| <code>`eq_non_comparative_leader`</code>    | Leader producing a result             | Text output                   |
| <code>`eq_non_comparative_validator`</code> | Validator checking leader's result    | Boolean + reasoning           |

You can tune these prompts to improve your validator's judgment quality. Better prompts → more accurate consensus → higher rewards.

## ## Lua Scripting API

### ### Core functions (``lib.rs``)

Available in all Lua scripts:

| Function                                                                       | Description                                                                         |
|--------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <code>`lib.rs.request(ctx, {method, url, headers, body, json, sign})`</code>   | HTTP request. <code>`json: true`</code> returns parsed response.                    |
| <code>`lib.rs.json_parse(str)`</code>                                          | Parse JSON string to Lua table                                                      |
| <code>`lib.rs.json_stringify(value)`</code>                                    | Serialize Lua table to JSON                                                         |
| <code>`lib.rs.base64_encode(data)`</code> / <code>`base64_decode(data)`</code> | Base64 encode/decode                                                                |
| <code>`lib.rs.filter_text(text, filters)`</code>                               | Text filters: <code>`"NFC"`, `"NFKC"`, `"NormalizeWS"`, `"RmZeroWidth"`, ...</code> |
| <code>`lib.rs.filter_image(bytes, filters)`</code>                             | Image filters: <code>`{Denoise=0.5}`, `{JPEG=0.9}`, ...</code>                      |

```
`{GuassainNoise=0.1}`, `{Unsharpen={0.5, 0.5}}` |
| `lib.rs.split_url(url)` | Parse URL → `{schema, host, port}` |
| `lib.rs.url_encode(text)` | URL-encode text |
| `lib.rs.sleep_seconds(n)` | Async sleep |
| `lib.rs.random_bytes(n)` / `random_float()` | Random data generation |
| `lib.rs.user_error({causes, fatal, ctx})` | Raise a user error |
| `lib.rs.as_user_error(err)` | Try to convert error to user error (nil if not) |
| `lib.log({level, message, ...})` | Structured logging (trace, debug, info, warn, error) |
```

### ### LLM functions (`llm.rs`)

| Function                                                  | Description                                                 |
|-----------------------------------------------------------|-------------------------------------------------------------|
| <code>llm.rs.exec_prompt_in_provider(ctx, request)</code> | Execute prompt on a specific provider/model                 |
| <code>llm.providers</code>                                | Table of all configured backends and models                 |
| <code>llm.templates</code>                                | Equivalence Principle prompt templates                      |
| <code>llm.select_providers_for(prompt, format)</code>     | Filter backends by capability (JSON, images)                |
| <code>llm.exec_prompt_transform(args)</code>              | Transform raw prompt args to internal format                |
| <code>llm.exec_prompt_template_transform(args)</code>     | Transform template args with variable substitution          |
| <code>llm.overloaded_statuses</code>                      | HTTP codes indicating overload: `{408, 429, 503, 504, 529}` |

### ### Web functions (`web.rs`)

| Function                        | Description                                             |
|---------------------------------|---------------------------------------------------------|
| <code>web.config</code>         | Web module configuration                                |
| <code>web.allowed_tld</code>    | Set of allowed top-level domains                        |
| <code>web.check_url(url)</code> | Validate URL against schema, port, and TLD restrictions |

### ## Advanced: Web Module

The web module (`genvm-module-web.yaml`) enables contracts to fetch web pages, make HTTP requests, and capture screenshots. You generally don't need to modify this.

The Lua script (`genvm-web-default.lua`) implements two functions:

- `Render(ctx, payload)` – render a page as text, HTML, or screenshot
- `Request(ctx, payload)` – make HTTP requests (GET, POST, etc.)

**\*\*What you might customize:\*\***

- `extra_tld: ["local"]` – allow additional TLDs beyond the standard set
- `always_allow_hosts: ["my-api.internal"]` – bypass URL checks for specific hosts

### ## Advanced: GenVM Manager

The manager (`genvm-manager.yaml`) controls concurrency. The main setting is `permits` – max concurrent GenVM executions.

> **\*\*Note:\*\***

> On macOS (native), auto-detection is not supported. Set `permits` manually (e.g., `32`).

### ## Troubleshooting

**\*\*Warnings for unconfigured providers:\*\*** Enabled backends without API keys log warnings. Set the key in `.env` or disable with `enabled: false`.

**\*\*"module\_failed\_to\_start":\*\*** Check that the Lua script exists at the path in `lua_script_path`, API keys are set, and YAML syntax is valid.

**\*\*"NO\_PROVIDER\_FOR\_PROMPT":\*\*** No enabled backend matches the request's capabilities. Ensure at least one has `supports_json: true` (and `supports_image: true` for image prompts).

**\*\*All models exhausted:\*\*** All providers returned overload errors. Check API key validity and

provider status pages. Add more backends as fallback.

---

## # Validator Upgrade Guide

Source: <https://docs.genlayer.com/validators/upgrade>

A GenLayer validator upgrade is either **non-breaking** or **breaking**, and the upgrade type determines whether the database carries over and how much downtime to expect. Before upgrading, check the [changelog](/validators/changelog) for the target version to confirm the correct procedure.

| Upgrade Type | How to Identify                                                              | Database        | Downtime              |
|--------------|------------------------------------------------------------------------------|-----------------|-----------------------|
| Non-breaking | Patch/minor release, no consensus contract changes                           | Carries over    | Minimal               |
| Breaking     | Changelog mentions "upgrade consensus contracts" or new consensus deployment | Must be deleted | Full restart required |

> **Note:**

> When in doubt, treat it as a breaking change – the only cost is a longer sync time.

---

### ## Non-Breaking Upgrade

No re-staking is required. You can minimize downtime by preparing the new version while your current node is still running.

1. **Download and extract into a fresh folder** – See [download instructions](/validators/setup-guide#download-the-node-software). Always use a clean directory.
2. **Configure the new version** – Set up `config.yaml` and `.env` as described in the [configuration section](/validators/setup-guide#configuration). You can reference your previous settings, but **do not blindly copy config files** – new versions may include updated defaults or new fields.
3. **Copy the data folder** – Copy `data/` from your old installation to the new one (contains database and keystore). Skip this if your `datadir` points outside the project directory.
4. **Validate** – Run `./bin/genlayernode doctor` and ensure all checks pass.
5. **Switch over** – Stop the old node, then start the new one.

---

### ## Breaking Change Upgrade

Breaking changes involve a new consensus deployment. The old database is incompatible, so it cannot be reused. **Re-staking is required** – you must stake again using the [validator wizard](/validators/setup-guide#using-the-validator-wizard).

> **Warning:**

> **Always back up your keystore before a breaking upgrade.** You can copy the `data/node/keystore/` directory or use `genlayernode account export`. See [backing up your operator key](/validators/setup-guide#backing-up-your-operator-key).

1. **Read the changelog** – Check the [changelog](/validators/changelog) for version-specific instructions and what changed.
2. **Back up your keystore** – Copy `data/node/keystore/` to a safe location.
3. **Download and extract into a fresh folder** – See [download instructions](/validators/setup-guide#download-the-node-software).
4. **Configure from scratch** – Create `config.yaml` and `.env` using the new example files as a base. **Do not copy your old config** – the schema may have changed. Refer to the [configuration section](/validators/setup-guide#configuration) for the current format.
5. **Import your operator key** – Copy your backed-up keystore into `data/node/keystore/`, or re-import using the CLI. See [restoring your operator key](/validators/setup-guide#restoring-your-operator-key).



6. **\*\*Do NOT copy the old database\*\*** – Delete `data/node/genlayer.db` if you accidentally copied the full `data/` folder. The node will create a fresh database and sync from genesis.
7. **\*\*Validate\*\*** – Run `./bin/genlayernode doctor` and ensure all checks pass.
8. **\*\*Stop the old node, start the new one.\*\***

---

## ## Migration Notes

### ### v0.4.x to v0.5.0 (Breaking)

- The two consensus contract addresses (`contractmainaddress` and `contractdataaddress`) have been replaced by a single `consensusaddress` field. Do not copy your old consensus config – use the new example `config.yaml`.
- The GenLayer Chain ZKSync RPC URLs (`genlayerchainrpcurl` and `genlayerchainwebsocketurl`) have changed. Update both the HTTP RPC and WebSocket URLs in your `config.yaml` to the new endpoints.
- Database must be wiped (the node creates a fresh one on startup).
- Re-staking is required – breaking changes involve a new consensus deployment, so you must stake again using the [validator wizard](/validators/setup-guide#using-the-validator-wizard).

---

## ## Quick Troubleshooting

- **\*\*Node won't start\*\*** – Run `./bin/genlayernode doctor`. If it reports unknown config fields, reconfigure from the new example.
- **\*\*Not participating in consensus\*\*** – Verify the consensus contract address, `validatorWalletAddress`, and `operatorAddress` are correct.
- **\*\*Key errors\*\*** – Your operator key wasn't imported. See [restoring your operator key](/validators/setup-guide#restoring-your-operator-key).
- **\*\*Slow sync after breaking upgrade\*\*** – Expected. Ensure `genesis` is set in `config.yaml` to avoid scanning from block 0.

---

## # Changelog

Source: <https://docs.genlayer.com/validators/changelog>

### ## v0.5.12

#### ### Misc

- Update dependency ethers to ~5.8.0
- Update all non-major dependencies

#### ### Bug fixes

- Self-recover from sync stall wedge
- Reclassify zkSync not-found errors

#### ### Security

- Update module golang.org/x/net to v0.55.0
- Update module golang.org/x/crypto to v0.52.0

### ## v0.5.11

#### ### New features

- Add node health state machine
- Scope admin tokens per role
- Add circuit breaker for RPC outages
- Add bearer auth for admin endpoints

- Unify CORS and WS origins allowlist
- Inbound WebSocket JSON-RPC endpoint

#### ### Misc

- Tag runtime metrics by network
- Cap PreloadAccepted concurrency
- Improve historical sync by sync-mode cleanup
- Reframe /snapshot examples for SSH access; drop cloud refs
- Default inbound WS path to /ws
- Default WS example to single-listener
- Multiplex WS subscriptions
- Trace context cancellations on all watchers
- Remove unused WaitMined method

#### ### Bug fixes

- Guard nil GasFeeCap on bump retry
- Ensure GenVM precompile cache before manager start
- Keep sync ticker running while Synced
- Create export file when --path is missing
- Autodetect GenVM permit capacity
- RPC debug error codes and id default
- Trust XFF + cap WS frame size
- Never halt sync on bloom mismatch
- Refresh version on skip-sync
- Route receipt polls to poll probes
- Handle -32602 decode-rejection and cap GasFeeCap
- Cap RPC body/batch and admin loopback bind

#### ### Security

- Update module go.opentelemetry.io/otel to v1.41.0

#### ## v0.5.10

##### ### New features

- Resilient WS RPC for fallback path

##### ### Bug fixes

- Close activator-binding race
- Exit finalize loop on post-Submit race
- Persist gas high-water across cycles
- Lift gas ceiling on retry-count
- Bound cancel gasFeeCap and lift bump ceiling on blocker lookup
- Fall back to WS on block-not-found
- Repair self-cancellation recovery
- Make chain-submit buffer configurable and stop collapsing on short step\_timeout
- Add fault label to RPC error metric
- Unwedge replay and filter sync

##### ### Misc

- Label RPC metrics with provider
- Add chain nonce and desync gauges metrics
- Log timestamps after acquireTransaction

#### ## v0.5.9

##### ### Bug fixes

- Count real provider failures on SLA
- Harden nonce management

- Check per-block bloom filters for multi-block ranges
- Pool HTTP transport for chain RPC
- Self-cancel stuck nonce on mempool replacement exhaustion
- Prevent sync panic and self-dependency cycle during tx replay
- Resume sync after WS reconnect

#### ### Misc

- Use RFC3339Nano for zerolog timestamps
- Extract encoding functions into dedicated encoding package
- Move consensus entity types to entities package
- Add gen\_dbg\_traceTransaction API doc

#### ## v0.5.8

#### ### New features

- Add `gen\_syncing` endpoint for sync status visibility
- Add bounded sync wait for RPC readiness
- Add per-IP per-method RPC rate limiting

#### ### Bug fixes

- Defer GenVM precompile to container startup to avoid AVX-512 incompatibility
- Replace stuck transactions with gas bump
- Skip duty recovery when behind and handle chained missing tx data dependencies
- Add hiccup retry to sync pipeline
- Recover prime after reorg
- Fix RPC max block range exceeded in event router
- Return -32001 for missing contract errors
- Pass transaction value to GenVM
- Standardize RPC error codes per EIP-1474
- Retry tx lookup for RPC indexing lag

#### ### Misc

- Consolidate genvm configs
- Add `gen\_getTransactionStatus` docs
- Add `gen\_syncing` API docs and clarify RPC config group vs method precedence

#### ## v0.5.7

#### ### New features

- Add LLM greyboxing with configurable chain order via YAML meta

#### ### Bug fixes

- Use PendingAt for idle activator rotation
- Remove unnecessary workaround from view calls
- Restart sync pipeline after RPC failures
- Suppress shutdown errors in watchers
- Reduce FindAcceptanceBlock range to fit RPC limit

#### ### Misc

- Include Alloy healthcheck script in release tarball
- Increase Alloy push intervals to 60s

#### ## v0.5.6

#### ### Bug fixes

- Add priority admission control

## ## v0.5.5

### ### New features

- Add network name to health endpoint
- Add network label to logger and all metrics
- Add metrics to rollup RPC transport and WebSocket

### ### Bug fixes

- Recover from cache miss on late lifecycle events
- Wait for queue head before finalize with recipient-aware queue management
- Suppress context canceled errors on shutdown
- Raise gas bump ceiling and retries
- Stuck nonce and finalize resilience
- Patch GenVM dylib references on macOS during download

### ### Misc

- Downgrade finalized-not-in-cache log to Debug

## ## v0.5.4

### ### New features

- Add `--network` flag to doctor command for multi-network validation

## ## v0.5.3

### ### New features

- Add network label to telemetry metrics and logs

## ## v0.5.2

### ### New features

- Add recovery for missing TxData

### ### Bug fixes

- Invalidate timeout cache on SetTimeouts
- Dedup propose on duplicate events
- Stop retrying on ValidatorSelectionFailed
- Prevent sync error race on ctx cancel
- Recover sync on missing TxData
- Reject stale block in idle check
- Increment leader revealed metric counter
- Sync commit idle timeout with contract timestamps

### ### Misc

- Use CtxLogger in HTTP handler constructors

## ## v0.5.1

### ### New features

- Add transaction manager for nonce resilience
- Add leader\_results input and nondetDisagreementCallNo to gen\_call
- Handle hanging transactions via processIdleness
- Add Morpheus as OpenAI-compatible LLM provider

### ### Misc

- Dedup duplicate NewTransaction events per block
- Add CanFinalize pre-check before finalization
- Make sync idempotent
- Per-recipient head-by-head idleness finalization

#### ### Bug fixes

- Detect ZkSync nonce inconsistency error
- Capture RPC error message when data is not decodable
- Correct consensus address and genesis block
- Enrich context errors with last failure
- Fetch on-chain data in Accepted/Undetermined handlers

#### ## v0.5.0

##### ### New features

- Upgrade consensus contracts from v0.4 to v0.5
- Add gen\_getContractCode endpoint
- Return full VMResult data from gen\_call

##### ### Bug fixes

- Check pending queue head before activation
- Enable and fix gen\_call write simulation
- Exclude stack field from console output
- Move GenVM execution outside retry loop to avoid double GenVM execution on submitting failure
- Use ConsoleWriter for caller field filtering
- Remove nested retry from epoch finalization
- Add logging for silent tx submission failures
- Retry ProcessIdleness on failure in lastVoteWatcher
- Handle shutdown errors gracefully
- Remove --create-venv=false flag from post-install scripts
- Install Python dependencies globally to avoid venv build issues
- Update validator priming logic to use current epoch
- Epoch advance scheduler not running for epoch 0
- Prevent panic on closing done channel during event processing
- Handle new consensus ResultType
- New behavior for finalizing epochs

##### ### Misc

- Inject txID into context for automatic log propagation
- Enable validator idle reveal idleness
- Add profiling with pyroscope
- Fix to handle zksync local-node compatibility

#### ## v0.4.5

##### ### New features

- Decouple epoch advance from finalization, allowing them to run as independent processes with separate retry mechanisms
- Implement sync waiting mechanism to ensure GenVM has latest state before contract execution
- Add `genlayer\_node\_info` Prometheus metric exposing node and protocol version

##### ### Bug fixes

- Add canPrime check to verify epoch state before validator priming, preventing priming when epochs are out of sync
- Return user-friendly error instead of system error when sync times out
- Fix genvm executor process handling

### ### Misc

- Improve GenVM logging with context-aware logger
- Update health checker to use ZkSync connectivity checks
- Increase default genvm permits to 8 in config.yaml.example

### ## v0.4.4

#### ### New features

- Improve logging with context-aware tracing and cleaner console output
- Refactor ZkSync client to split RPC and WSS connections for better reliability
- Move storage writing to genvm for improved performance

#### ### Bug fixes

- Skip priming check for full nodes
- Add Docker socket mount for webdriver metrics collection in docker-compose configurations
- Prevent incorrect sync state by updating blocksBehind calculation
- Fix memory leak from threads package
- Fix memory leak in consensus processing
- Fix bug in latest transaction with effect calculation
- Fix wallet mutex using read lock instead of write lock when unlocking accounts

### ### Misc

- Fix signer for full nodes
- Improve logs for http servers
- Improve signer initialization for faster and more reliable transaction signing
- Add logs for previous activator on idleness checks
- Migrate zksync-go to go-ethereum

### ## v0.4.3

#### ### Bug fixes

- Fix node storage handling for non-accepted transactions like LeaderTimeout
- Fix validator priming logic
- Fix default in release/.env.example

### ## v0.4.2

#### ### Bug fixes

- Telemetry config and scraping
- Fix genvm readiness attempts

### ## v0.4.1

#### ### New features

- Add show and list subcommands for account management
- Split authentication credentials for Prometheus and Loki
- Add IsPrimed health check for unprimed validators
- Add primed state and warning for unprimed validators

### ### Misc

- Refactor txcalldata decoding to a new encoding package
- Remove unused `pkg/cryptoutils/address.go`
- Update deployment configuration options for network and cleanup
- Refactor gas estimator

#### ### Bug fixes

- Implement indefinite retry for validator priming until primed
- Implement health check and timeout handling for GenVM manager
- Fix undetermined transaction rotations when ProcessIdleness
- Improve message handling to prevent dropped events
- Add GenVM contracts cache and enhance sync progress logging
- Fix sync filtering process
- Improve messages around genlayerRPC chain connectivity
- Stop retrying activation after transaction is activated

## v0.4.0

### ### New features

- Integrate the latest GenLayer Consensus version v0.4 with Staking
  - Introduce validator wallet
  - Handle validator priming
  - Finalize epochs
  - Advance epochs when epochs finalize
- Handle automatic gas increase
- Implement ProcessIdleness and enhance event subscription
- Handle validator commit idleness for leader
- Implement leader idleness detection and rotation
- Add detailed ban information with epoch and permanent flag
- Handle unfinished transactions with batch finalization
- Add nonceSafe for granular transaction locking
- Add sanity check for operator address in validator wallet configuration
- Add missing fields from consensus transaction data to API response
- Add validator telemetry push monitoring
- Add actionable fix suggestions to WebDriver error messages in doctor command
- Add zkSync connectivity checks
- Add Staking Contract address comparison to doctor checks
- Add libertai as a new LLM provider
- Add transaction count metrics for consensus roles
- Implement consensus sync status metrics
- Add TimeoutsSet event router for dynamic timeout configuration
- Add enhanced transaction failure debugging with trace support
- Sync data with finalized and appeal started events
- Add timeout delay mechanism for RPC resilience
- Add status and blockNumber parameters to gen\_call
- Enhance health endpoint with version info and validator with ban and permanent ban detection
- Update genvm version to v0.2.7
- Store and expose genvm execution information
- Add sign admin endpoint for genvm

### ### Bug fixes

- GenVM architecture on arm64
- Fix sync issue when there are no consensus events
- Use latest block instead of timestamp for transaction queries
- Update WebDriver check for Puppeteer implementation
- Resolve multi-transaction nonce issue
- Remove duplicated check logic and fix status check on specific block
- Leader rotation timeout and receipt cleanup
- Change transaction finalization to allow all nodes to finalize
- GenVM non determinism
- Add retry logic to prevent validator banning on transient errors
- GenVM sanity checks
- Cancel context and wait for group when genvm tester fails
- Decoding calldata into complex types
- Don't try activating already activated transactions
- Still process sync failed TX for DB consistency
- Add correct encoding for getState
- Only sync transactions with ResultCodeReturn and add sanity check
- Use block number for transaction data retrieval

- Validator votes idleness
- Leader idleness
- Accidental deadlock in genvm
- Lazy validators commit order

#### ### Misc

- Simplify environment variable configuration using .env file
- Add environment variable override support for config values
- Update webdriver image to v0.0.9
- Update RPC and WebSocket endpoints in config.yaml.example
- Correct status for validatorTimeout after appeal time window
- Validate presence of abigen in dependency check
- Rename environment validation functions for clarity
- Use cockroachdb errors package for error handling
- Remove wallet.yaml file handling
- Rename archive to full

#### ## v0.3.11

##### ### New features

- Add validator telemetry push monitoring

##### ### Bug fixes

- Validator banning on parallel transactions executions

##### ### Misc

- Document health RPC endpoint

#### ## v0.3.10

##### ### New features

- Add actionable fix suggestions to WebDriver error messages in doctor command

##### ### Bug fixes

- Add retry logic to prevent validator banning on transient errors

#### ## v0.3.9

##### ### New features

- Add doctor command zkSync connectivity checks

##### ### Misc

- Simplify contract initialization with auto-discovery

##### ### Bug fixes

- Remove response body size limit for WebDriver

#### ## v0.3.8

##### ### New features

- Add Staking Contract address comparison to doctor checks
- Add libertai as a new LLM provider

##### ### Misc



- Enhance metrics logging by adding blocks behind calculation
- Print address for account not found
- Fix typo in telemetry configuration comment

#### ### Bug fixes

- Enable CORS support for HTTP methods in the API
- Move sync success log after state update
- Don't panic with metrics + print panics

#### ## v0.3.7

#### ### New features

- Add transaction count metrics for consensus roles
- Implement consensus sync status metrics
- Add node uptime collector
- Add disk storage usage metrics for monitoring
- Add comprehensive metrics collection for ops visibility

#### ## v0.3.6

#### ### Misc

- Add enhanced transaction failure debugging with trace support

#### ## v0.3.5

#### ### New features

- Add Transport with exponential backoff retry logic
- Add timeout delay mechanism for RPC resilience
- Add status and blockNumber parameters to gen\_call
- Enhance health endpoint with version info and validator ban detection
- Add balance endpoint for validator nodes
- Add gas estimation failover to all consensus transaction functions
- Implement gen\_getContractState endpoint
- Add ionet as inference provider
- Add gen\_getTransactionStatus endpoint
- Implement gen\_getTransactionReceipt
- Add log upload system for validator diagnostics

#### ### Misc

- Add comprehensive API documentation system and operations endpoints
- Move retry logic to infrastructure and update DI
- Improve timeout detection with type-based checking
- Enhance WebDriver health check validation

#### ### Bug fixes

- Fix validator idleness commit flow
- Handle methods and groups configuration for RPC methods

#### ## v0.3.4

#### ### New features

- Added genesis block configuration support for faster node startup
- Enhanced `doctor` command with GenVM diagnostics integration
- Updated consensus contract addresses for network upgrade

#### ### Misc

- Improved node synchronization performance with genesis block hints

- Enhanced validator configuration validation and error reporting

## v0.3.3

### New features

- Added [Comput3](/validators/setup-guide#genvm-configuration) as a supported LLM provider for validators

## v0.3.2

### New features

- Add command `doctor` to check Consensus Contract configuration

### Misc

- Implement retry logic with double-checked locking for `GetTimeouts`
- Update ABI for consensus contracts.
- Decode error returns from the network to improve error handling.

## v0.3.1

### Misc

- Add logs details

## v0.3.0

### New features

- Added appeals functionality
- Added `gen_getContractSchema`
- Added admin checkpoint endpoint
- Added admin "stop at block" endpoint
- Handle leader timeout
- Handle validator timeout
- Added `version` command

### Bug fixes

- Mitigate TxAccepted events being dropped
- Waiting on event timeouts is now absolute
- Patch up genvm timeouts
- Add retry mechanism for genvm contracts interaction
- Listen to leader timeout event to finalize transaction

## v0.2.0-testnet007

### New features

- Activator idle replacement
- Leader idle replacement
- Validator idle replacement
- Check for genvm on startup
- Log file support for node
- Ability to export private key in account command

### Bug fixes

- Build info logs now display correctly
- Fixed config relative path
- Node now fails fast when accepted transaction processing fails

### Misc

- Updated module [github.com/btcsuite/btcd](https://github.com/btcsuite/btcd) to v0.24.2 (security)
- Improved sync speed on node startup

## v0.2.0-testnet005

### New features

- Discover Consensus main genesis from deployment (contracts)

### Bug fixes

- All accepted sync transactions are now processed atomically per block
- Fixed deadlock on cancelled context

## v0.2.0-testnet004

### Bug fixes

- Updated genvm to v0.0.18

## v0.2.0-testnet003

### New features

- Updated genvm to v0.0.17

## v0.2.0-testnet002

### New features

- Use different address to run e2e tests
- Updated consensus contract addresses in config

## v0.2.0-testnet001

### New features

- Updated genvm to v0.0.16

### Bug fixes

- Log level is no longer ignored
- Revived trie dumping and enhanced logging
- Always kill genvm modules

### Misc

- Updated module [golang.org/x/net](https://golang.org/x/net) to v0.38.0 (security)

---

# API References

Source: <https://docs.genlayer.com/api-references>

- [GenLayer CLI](/api-references/genlayer-cli)
- [GenLayerJS](/api-references/genlayer-js)
- [GenLayerPY](/api-references/genlayer-py)
- [GenLayer Test](/api-references/genlayer-test)
- [GenLayer Node API](/api-references/genlayer-node)
- [GenLayer SDK](<https://sdk.genlayer.com/>)

---

# GenLayer CLI

Source: <https://docs.genlayer.com/api-references/genlayer-cli>

## ## Description

The GenLayer CLI is designed to streamline the setup and local execution of the GenLayer simulator. This tool automates the process of downloading and launching the GenLayer simulator, making it easy to start simulating and testing locally with minimal setup.

## ## Installation

Before installing the GenLayer CLI, ensure you have Node.js installed on your system. You can then install the CLI globally using npm:

```
```bash
npm install -g genlayer
```
```

## ### Linux Dependencies

On some Linux distributions with minimal setups (like Debian netinst or Docker images), you may need to manually install libsecret:

```
```bash
# Ubuntu/Debian
sudo apt-get install libsecret-1-0
```

```
# CentOS/RHEL/Fedora
sudo yum install libsecret
# or for newer versions
sudo dnf install libsecret
```

```
# Arch Linux
sudo pacman -S libsecret
```
```

The GenLayer CLI uses the `keytar` library for secure key storage, which relies on `libsecret` on Linux systems.

## ## Usage

Each command includes syntax, usage information, and examples to help you effectively use the CLI for interacting with the GenLayer environment.

## ### Command line syntax

General syntax for using the GenLayer CLI:

```
```bash
genlayer command [command options] [arguments...]
```
```

## ### Commands and usage

### #### Initialize

Prepares and verifies your environment to run the GenLayer Studio.

```
```bash
USAGE:
  genlayer init [options]
```

OPTIONS:

| | |
|---------------------------------|-------------------------------------|
| --numValidators <numValidators> | Number of validators (default: "5") |
| --headless | Headless mode (default: false) |
| --reset-db | Reset Database (default: false) |

```
--localnet-version <localnetVersion>  Select a specific localnet version
--ollama                                Enable Ollama container (default: false)
```

EXAMPLES:

```
genlayer init
genlayer init --numValidators 10 --headless --reset-db --localnet-version v0.10.2
...genlayer init --ollama
```

Version Compatibility

The GenLayer CLI always uses the latest compatible version of the environment, ensuring that you benefit from the most recent features, bug fixes, and optimizations without requiring manual updates. If a specific version is needed, you can specify it using the `--localnet-version` option when initializing the environment.

```
```bash
genlayer init --localnet-version v0.10.2
```

#### #### Start GenLayer environment

Launches the GenLayer environment and the Studio, initializing a fresh set of database and accounts.

```
```bash
USAGE:
  genlayer up [options]
```

OPTIONS:

| | |
|--|--|
| <code>--reset-validators</code> | Remove all current validators and create new random ones |
| (default: false) | |
| <code>--numValidators <numValidators></code> | Number of validators (default: "5") |
| <code>--headless</code> | Headless mode (default: false) |
| <code>--reset-db</code> | Reset Database (default: false) |
| <code>--ollama</code> | Enable Ollama container (default: false) |

EXAMPLES:

```
genlayer up
genlayer up --reset-validators --numValidators 8 --headless --reset-db
...genlayer up --ollama
```

Stop GenLayer environment

Stops all running GenLayer Localnet services.

```
```bash
USAGE:
 genlayer stop
```

#### #### Create a New GenLayer Project

Initialize a new GenLayer project using a local template.

```
```bash
USAGE:
  genlayer new <projectName> [options]
```

OPTIONS:

| | |
|---------------------------------------|--|
| <code>--path <directory></code> | Specify the directory for the new project (default: ".") |
| <code>--overwrite</code> | Overwrite existing directory if it exists (default: false) |

EXAMPLES:

```
genlayer new myProject
genlayer new myProject --path ./customDir
...genlayer new myProject --overwrite
```

Manage CLI Configuration

Configure the GenLayer CLI settings.

```
```bash
USAGE:
 genlayer config <command> [options]

COMMANDS:
 set <key=value> Set a configuration value
 get [key] Get the current configuration
 reset <key> Reset a configuration value to its default
```

```
EXAMPLES:
 genlayer config get
 genlayer config get defaultOllamaModel
 genlayer config set defaultOllamaModel=deepseek-r1
 ...genlayer config reset keyPairPath
```

#### #### Network Management

Manage network configuration for contract operations.

```
```bash
USAGE:
  genlayer network set [network]  Set the network to use
  genlayer network info            Show current network configuration and contract
addresses
  genlayer network list            List available networks
```

```
EXAMPLES:
  genlayer network set
  genlayer network set testnet
  genlayer network set mainnet
  genlayer network info
  ...genlayer network list
```

Deploy and Call Intelligent Contracts

Deploy and interact with intelligent contracts.

```
```bash
USAGE:
 genlayer deploy [options]
 genlayer call <contractAddress> <method> [options]
 genlayer write <contractAddress> <method> [options]
 genlayer schema <contractAddress> [options]
```

```
OPTIONS (deploy):
 --contract <contractPath> (Optional) Path to the intelligent contract to deploy
 --rpc <rpcUrl> RPC URL for the network
 --args <args...> Contract arguments (see Argument Types below)
```

```
OPTIONS (call):
 --rpc <rpcUrl> RPC URL for the network
 --args <args...> Method arguments (see Argument Types below)
```

```
OPTIONS (write):
```

```
--rpc <rpcUrl> RPC URL for the network
--args <args...> Method arguments (see Argument Types below)
```

```
OPTIONS (schema):
 --rpc <rpcUrl> RPC URL for the network
```

#### EXAMPLES:

```
genlayer deploy
genlayer deploy --contract ./my_contract.gpy
genlayer deploy --contract ./my_contract.gpy --args "arg1" "arg2" 123
genlayer call 0x123456789abcdef greet --args "Hello World!"
genlayer write 0x123456789abcdef updateValue --args 42
genlayer write 0x123456789abcdef sendReward --args
0x6857Ed54CbafaA74Fc0357145eC0ee1536ca45A0
genlayer write 0x123456789abcdef setScores --args '[1, 2, 3]'
genlayer write 0x123456789abcdef setConfig --args '{"timeout": 30, "retries": 5}'
...genlayer schema 0x123456789abcdef
```

#### ##### Argument Types

The `--args` option automatically detects and converts values to the correct type:

| Type        | Syntax                                   | Example                                         |
|-------------|------------------------------------------|-------------------------------------------------|
| Boolean     | `true`, `false`                          | `--args true false`                             |
| Null        | `null`                                   | `--args null`                                   |
| Integer     | numeric value                            | `--args 42 -1`                                  |
| Hex integer | `0x` prefix                              | `--args 0x1a`                                   |
| String      | any other value                          | `--args hello "multi word"`                     |
| Address     | 40 hex chars with `0x` or `addr#` prefix | `--args 0x6857...a0` or `--args addr#6857...a0` |
| Bytes       | `b#` prefix + hex                        | `--args b#deadbeef`                             |
| Array       | JSON array in quotes                     | `--args '[1, 2, "three"]'`                      |
| Dict        | JSON object in quotes                    | `--args '{"key": "value"}'`                     |

Large numbers that exceed JavaScript's safe integer range are automatically handled as BigInt to preserve precision.

#### ##### Deploy Behavior

- If `--contract` is specified, the command will **deploy** the given contract.
- If `--contract` is omitted, the CLI will **search** for scripts inside the `deploy` folder, sort them, and execute them sequentially.

#### ##### Call vs Write

- `call` - Calls a contract method without sending a transaction or changing the state (read-only)
- `write` - Sends a transaction to a contract method that modifies the state

#### ##### Schema

- `schema` - Retrieves the contract schema

#### #### Transaction Operations

```
```bash
```

USAGE:

```
genlayer receipt <txId>          Get transaction receipt
genlayer appeal <txId>          Appeal a transaction
genlayer appeal-bond <txId>      Show minimum appeal bond required
```

OPTIONS (receipt):

```
--status <status>      Status to wait for (default: FINALIZED)
--retries <retries>      Number of retries (default: 100)
--interval <interval>    Interval between retries in ms (default: 5000)
--stdout                 Print only stdout from the receipt
```

--stderr Print only stderr from the receipt

OPTIONS (appeal):

--bond <amount> Appeal bond amount (e.g. 500gen, 0.5gen). Auto-calculated if omitted
--rpc <rpcUrl> RPC URL override

EXAMPLES:

```
# Check the minimum bond required to appeal
genlayer appeal-bond 0x1234...

# Appeal with auto-calculated bond
genlayer appeal 0x1234...

# Appeal with explicit bond
genlayer appeal 0x1234... --bond 500gen
...
```

Transaction Trace

Inspect execution traces for debugging:

```
```bash
genlayer transactions trace <txId> [--round N] [--rpc URL]
```
```

Account Management

View and manage your account.

```
```bash
```

USAGE:

|                                     |                                                       |
|-------------------------------------|-------------------------------------------------------|
| genlayer account                    | Show account info (address, balance, network, status) |
| genlayer account create [options]   | Create a new account                                  |
| genlayer account send <to> <amount> | Send GEN to an address                                |
| genlayer account unlock             | Unlock account (cache key in OS keychain)             |
| genlayer account lock               | Lock account (remove key from OS keychain)            |

OPTIONS (create):

--output <path>              Path to save the keystore (default: "./keypair.json")  
--overwrite                    Overwrite existing file (default: false)

EXAMPLES:

```
genlayer account
genlayer account create
genlayer account create --output ./my_key.json --overwrite
genlayer account send 0x123...abc 10gen
genlayer account send 0x123...abc 0.5gen
genlayer account unlock
genlayer account lock
...
```

#### Update Resources

Manage and update models or configurations.

```
```bash
```

USAGE:

genlayer update ollama [options]

OPTIONS:

--model [model-name] Specify the model to update or pull
--remove Remove the specified model instead of updating

EXAMPLES:


```

genlayer update ollama
genlayer update ollama --model deepseek-r1
genlayer update ollama --model deepseek-r1 --remove
...

```

Localnet Validator Management

Manage localnet validator operations.

```
```bash
```

USAGE:

```
genlayer localnet validators <command> [options]
```

COMMANDS:

|                                       |                                                            |
|---------------------------------------|------------------------------------------------------------|
| get [--address <validatorAddress>]    | Retrieve details of a specific validator or all validators |
| delete [--address <validatorAddress>] | Delete a specific validator or all validators              |
| count                                 | Count all validators                                       |
| update <validatorAddress> [options]   | Update a validator details                                 |
| create-random [options]               | Create random validators                                   |
| create [options]                      | Create a new validator                                     |

OPTIONS (update):

|                       |                                   |
|-----------------------|-----------------------------------|
| --stake <stake>       | New stake for the validator       |
| --provider <provider> | New provider for the validator    |
| --model <model>       | New model for the validator       |
| --config <config>     | New JSON config for the validator |

OPTIONS (create-random):

|                            |                                                              |
|----------------------------|--------------------------------------------------------------|
| --count <count>            | Number of validators to create (default: "1")                |
| --providers <providers...> | Space-separated list of provider names (e.g., openai ollama) |
| --models <models...>       | Space-separated list of model names (e.g., gpt-4 gpt-4o)     |

OPTIONS (create):

|                       |                                               |
|-----------------------|-----------------------------------------------|
| --stake <stake>       | Stake amount for the validator (default: "1") |
| --config <config>     | Optional JSON configuration for the validator |
| --provider <provider> | Specify the provider for the validator        |
| --model <model>       | Specify the model for the validator           |

EXAMPLES:

```

genlayer localnet validators get
genlayer localnet validators get --address 0x123456789abcdef

genlayer localnet validators count
genlayer localnet validators delete --address 0x123456789abcdef
genlayer localnet validators update 0x123456789abcdef --stake 100 --provider openai --
model gpt-4

genlayer localnet validators create
genlayer localnet validators create --stake 50 --provider openai --model gpt-4
genlayer localnet validators create-random --count 3 --providers openai --models gpt-4
gpt-4o
...

```

#### #### Staking Operations

Manage staking for validators and delegators on testnet-bradbury (or testnet-asimov).  
Staking is not available on localnet/studio.

```
```bash
```

USAGE:

```
genlayer staking <command> [options]
```

COMMANDS:

| | |
|-------------------------------|--|
| validator-join [options] | Join as a validator by staking tokens |
| validator-deposit [options] | Make an additional deposit as a validator |
| validator-exit [options] | Exit as a validator by withdrawing shares |
| validator-claim [options] | Claim validator withdrawals after unbonding period |
| validator-prime [validator] | Prime a validator for the next epoch |
| prime-all [options] | Prime all validators that need priming |
| delegator-join [options] | Join as a delegator by staking with a validator |
| delegator-exit [options] | Exit as a delegator by withdrawing shares |
| delegator-claim [options] | Claim delegator withdrawals after unbonding period |
| validator-info [validator] | Get information about a validator (--debug for raw data) |
| validator-history [validator] | Show slash and reward history for a validator |
| delegation-info [validator] | Get delegation info for a delegator with a validator |
| epoch-info [options] | Get current/previous epoch info (--epoch <n> for specific) |
| validators [options] | Show validator set with stake, primed status, and weight |
| active-validators [options] | List all active validators |
| quarantined-validators | List all quarantined validators |
| banned-validators | List all banned validators |

COMMON OPTIONS (all commands):

| | |
|-----------------------------|---|
| --network <network> | Network to use (localnet, testnet-bradbury, testnet-asimov) |
| --rpc <rpcUrl> | RPC URL override |
| --staking-address <address> | Staking contract address override |

OPTIONS (validator-join):

| | |
|----------------------|---|
| --amount <amount> | Amount to stake (in wei or with 'gen' suffix) |
| --operator <address> | Operator address (defaults to signer) |

OPTIONS (delegator-join):

| | |
|-----------------------|---|
| --validator <address> | Validator address to delegate to |
| --amount <amount> | Amount to stake (in wei or with 'gen' suffix) |

OPTIONS (exit commands):

| | |
|-----------------------|--|
| --shares <shares> | Number of shares to withdraw |
| --validator <address> | Validator address (for delegator commands) |

EXAMPLES:

```
# Get epoch info (uses --network to specify testnet-bradbury)
genlayer staking epoch-info --network testnet-bradbury
```

```
# Or set network globally first
genlayer network set testnet-bradbury
```

```
# Join as validator with 42000 GEN
genlayer staking validator-join --amount 42000gen
```

```
# Join as delegator with 42 GEN
genlayer staking delegator-join --validator 0x... --amount 42gen
```

```
# Check validator info
genlayer staking validator-info --validator 0x...
```

```
# Output:
```

```
# {
#   validator: '0xa8f1BF1e5e709593b4468d7ac5DC315Ea3CAe130',
#   vStake: '0.01 GEN',
#   vShares: '10000000000000000',
#   dStake: '0 GEN',
#   dShares: '0',
#   vDeposit: '0 GEN',
#   vWithdrawal: '0 GEN',
#   epoch: '0',
#   live: true,
#   banned: 'Not banned'
# }
```

Get current epoch info (shows current + previous epoch)

genlayer staking epoch-info

Output:

✓ Epoch info

#

Current Epoch: 5 (started 9h 30m ago)

Next Epoch: in 14h 30m

Validators: 33

...

#

Previous Epoch: 4 (finalized)

Inflation: 1732904.66 GEN

Claimed: 0 GEN

Unclaimed: 1732904.66 GEN

...

Query specific epoch data

genlayer staking epoch-info --epoch 4

List active validators

genlayer staking active-validators

Output:

{

count: 6,

validators: [

'0xa8f1BF1e5e709593b4468d7ac5DC315Ea3CAe130',

'0xe9246A020cbb4fC6C46e60677981879c9219e8B9',

...

]

}

Show validator set table with stake, status, weight

genlayer staking validators

genlayer staking validators --all # Include banned validators

Show validator slash/reward history (testnet only, default: last 10 epochs)

genlayer staking validator-history 0x...

genlayer staking validator-history 0x... --epochs 5 # Last 5 epochs

genlayer staking validator-history 0x... --from-epoch 3 # From epoch 3

genlayer staking validator-history 0x... --all # Complete history (slow)

genlayer staking validator-history 0x... --limit 20

Output:

#

| # | Time | Epoch | Type | Details | GL TxId / Block |
|---|-------------|-------|--------|----------|-----------------|
| # | | | | | |
| # | 12-11 14:20 | 5 | REWARD | Val: ... | block 4725136 |
| # | 12-10 18:39 | 4 | SLASH | 1.00% | 0x52db90a9... |
| # | | | | | |

Exit and claim (requires validator wallet address)

genlayer staking validator-exit --validator 0x... --shares 100

genlayer staking validator-claim --validator 0x...

Prime a validator for next epoch

genlayer staking validator-prime 0x...

Prime all validators that need priming (anyone can call)

...genlayer staking prime-all

Running the CLI from the repository

First, install the dependencies and start the build process

```
```bash
npm install
npm run dev
```
```

This will continuously rebuild the CLI from the source

Then in another window execute the CLI commands like so:

```
```bash
node dist/index.js init
```
```

Guides

- [Validator Guide](docs/validator-guide.md) - How to become a validator on GenLayer testnet
- [Delegator Guide](docs/delegator-guide.md) - How to delegate GEN to a validator

Documentation

For detailed information on how to use GenLayer CLI, please refer to our [documentation](https://docs.genlayer.com/).

Contributing

We welcome contributions to GenLayerJS SDK! Whether it's new features, improved infrastructure, or better documentation, your input is valuable. Please read our [CONTRIBUTING](https://github.com/yeagerai/genlayer-js/blob/main/CONTRIBUTING.md) guide for guidelines on how to submit your contributions.

License

This project is licensed under the ... License - see the LICENSE file for details.

GenLayerJS

Source: <https://docs.genlayer.com/api-references/genlayer-js>

About

GenLayerJS SDK is a TypeScript library designed for developers building decentralized applications (Dapps) on the GenLayer protocol. This SDK provides a comprehensive set of tools to interact with the GenLayer network, including client creation, transaction handling, event subscriptions, and more, all while leveraging the power of Viem as the underlying blockchain client.

Prerequisites

Before installing GenLayerJS SDK, ensure you have the following prerequisites installed:

- Node.js ($\geq 16.x$)
- npm ($\geq 7.x$)

Installation and Usage

To install the GenLayerJS SDK, use the following command:

```
```bash
$ npm install genlayer-js
```
```

Here's how to initialize the client and connect to the GenLayer Simulator:

```

### Reading a Transaction
```typescript
import { localnet } from 'genlayer-js/chains';
import { createClient } from "genlayer-js";

const client = createClient({
 chain: localnet,
});

const transactionHash = "0x...";

const transaction = await client.getTransaction({ hash: transactionHash })
```

### Waiting for Transaction Receipt
```typescript
import { localnet } from 'genlayer-js/chains';
import { createClient } from "genlayer-js";
import { TransactionStatus } from "genlayer-js/types";

const client = createClient({
 chain: localnet,
});

// Get simplified receipt (default - removes binary data, keeps execution results)
const receipt = await client.waitForTransactionReceipt({
 hash: "0x...",
 status: TransactionStatus.FINALIZED,
 fullTransaction: false // Default - simplified for readability
});

// Get complete receipt with all fields
const fullReceipt = await client.waitForTransactionReceipt({
 hash: "0x...",
 status: TransactionStatus.FINALIZED,
 fullTransaction: true // Complete receipt with all internal data
});
```

### Reading a contract
```typescript
import { localnet } from 'genlayer-js/chains';
import { createClient } from "genlayer-js";

const client = createClient({
 chain: localnet,
});

const result = await client.readContract({
 // account: account, Account is optional when reading from contracts
 address: contractAddress,
 functionName: 'get_complete_storage',
 args: []
 stateStatus: "accepted",
});
```

### Writing a transaction
```typescript
import { localnet } from 'genlayer-js/chains';
import { createClient, createAccount } from "genlayer-js";

const client = createClient({
 network: localnet,

```

```
});

const account = createAccount();
const transactionHash = await client.writeContract({
 account: account, // using this account for this transaction
 address: contractAddress,
 functionName: 'account',
 args: ['new_storage'],
 value: 0, // value is optional, if you want to send some native token to the contract
});

const receipt = await client.waitForTransactionReceipt({
 hash: txHash,
 status: TransactionStatus.FINALIZED, // or ACCEPTED
 fullTransaction: false // False by default - returns simplified receipt for better
 readability
})
...

```

### ### Checking execution results

A transaction can be finalized by consensus but still have a failed execution. Always check `txExecutionResult` before reading contract state:

```
``typescript
import { ExecutionResult, TransactionStatus } from "genlayer-js/types";

const receipt = await client.waitForTransactionReceipt({
 hash: txHash,
 status: TransactionStatus.FINALIZED,
});

if (receipt.txExecutionResultName === ExecutionResult.FINISHED_WITH_RETURN) {
 // Execution succeeded – safe to read state
 const result = await client.readContract({
 address: contractAddress,
 functionName: "get_storage",
 args: [],
 });
} else if (receipt.txExecutionResultName === ExecutionResult.FINISHED_WITH_ERROR) {
 // Execution failed – contract state was not modified
 console.error("Contract execution failed");
} else {
 // NOT_VOTED – execution hasn't completed
 console.warn("Execution result not yet available");
}
...

```

### ### Fetching emitted messages and triggered transactions

Transactions can emit messages to other contracts. These messages create new child transactions when processed:

```
``typescript
const tx = await client.getTransaction({ hash: txHash });

// Messages emitted by the contract during execution
console.log(tx.messages);
// [{messageType, recipient, value, data, onAcceptance, saltNonce}, ...]

// Child transaction IDs created from those messages (separate call)
const childTxIds = await client.getTriggeredTransactionIds({ hash: txHash });
console.log(childTxIds);
// ["0xabc...", "0xdef..."]

```

```
...
```

### ### Debugging transaction execution

Use `debugTraceTransaction` to inspect the full execution trace of a transaction, including return data, errors, and GenVM logs:

```
``typescript
const trace = await client.debugTraceTransaction({
 hash: txHash,
 round: 0, // optional, defaults to 0
});

console.log(trace.result_code); // 0=success, 1=user error, 2=VM error
console.log(trace.return_data); // hex-encoded contract return data
console.log(trace.stderr); // standard error output
console.log(trace.genvm_log); // detailed GenVM execution logs
``
```

### ### Using with a wallet provider (MetaMask)

When building a browser dApp, create two clients: one for reads (no wallet needed) and one for writes (signed by the wallet). This follows the standard viem pattern and keeps concerns separated.

```
``typescript
import { createClient } from "genlayer-js";
import { testnetBradbury } from "genlayer-js/chains";
import { TransactionStatus } from "genlayer-js/types";

// Read client – talks directly to GenLayer RPC, no wallet needed
const readClient = createClient({
 chain: testnetBradbury,
});

// Write client – signs transactions through the wallet
const writeClient = createClient({
 chain: testnetBradbury,
 account: address as `0x${string}`, // from wallet connection
 provider: window.ethereum, // or from a wallet SDK
});

// Use readClient for all reads
const result = await readClient.readContract({
 address: contractAddress,
 functionName: "get_storage",
 args: [],
});

const tx = await readClient.getTransaction({ hash: txHash });

// Use writeClient for transactions (MetaMask popup)
const txHash = await writeClient.writeContract({
 address: contractAddress,
 functionName: "update_storage",
 args: ["new_value"],
 value: BigInt(0),
});

// Either client can wait for receipts
const receipt = await readClient.waitForTransactionReceipt({
 hash: txHash,
 status: TransactionStatus.ACCEPTED,
});
``
```

### ### Switching the wallet to the correct network

When using MetaMask or another browser wallet, the wallet may be connected to a different chain than what your client is configured for. Use `client.connect()` to switch the wallet to the correct GenLayer network before sending transactions:

```
```typescript
import { createClient } from "genlayer-js";
import { studionet } from "genlayer-js/chains";

const client = createClient({
  chain: studionet,
  account: address as `0x${string}`,
});

// Switch MetaMask to the correct chain (adds the network if not present)
await client.connect("studionet");

// Now transactions will go to the right network
const txHash = await client.writeContract({
  address: contractAddress,
  functionName: "create_profile",
  args: ["alice", "Hello world"],
  value: BigInt(0),
});
```
```

Available networks: `"localnet"`, `"studionet"`, `"testnetAsimov"`, `"testnetBradbury"`.

> **Note:** If the wallet is on the wrong chain when you call `writeContract`, the SDK will throw a clear error telling you which chain the wallet is on vs. which chain the client expects. Call `client.connect()` to resolve this.

### ### Staking Operations

The SDK provides staking functionality for validators and delegators on testnet-bradbury (and testnet-asimov).

```
```typescript
import { testnetBradbury } from 'genlayer-js/chains';
import { createClient, createAccount } from "genlayer-js";

const account = createAccount();
const client = createClient({
  chain: testnetBradbury,
  account,
});

// Get epoch info (includes timing estimates and inflation data)
const epochInfo = await client.getEpochInfo();
// {
//   currentEpoch: 2n,
//   epochMinDuration: 86400n,           // 1 day in seconds
//   currentEpochStart: Date,
//   currentEpochEnd: Date | null,
//   nextEpochEstimate: Date | null,
//   validatorMinStake: "0.01 GEN",
//   delegatorMinStake: "42 GEN",
//   activeValidatorsCount: 6n,
//   inflation: "1000 GEN",              // Total inflation for current epoch
//   inflationRaw: 10000000000000000000n,
//   totalWeight: 5000000000000000000000n, // Total stake weight
//   totalClaimed: "500 GEN",           // Total claimed rewards
// }
```



```
// Get active validators
const validators = await client.getActiveValidators();

// Check if address is a validator
const isValidator = await client.isValidator("0x...");

// Get validator info
const validatorInfo = await client.getValidatorInfo("0x...");

// Join as validator (requires account with funds)
const result = await client.validatorJoin({ amount: "42000gen" });

// Join as delegator
const delegateResult = await client.delegatorJoin({
  validator: "0x...",
  amount: "42gen",
});
```

Key Features

- * **Client Creation**: Easily create and configure a client to connect to GenLayer's network.
- * **Transaction Handling**: Send and manage transactions on the GenLayer network.
- * **Staking**: Full staking support for validators and delegators on testnet-bradbury and testnet-asimov.
- * **Wallet Integration**: Seamless integration with MetaMask for managing user accounts.
- * **Gas Estimation**: Estimate gas fees for executing transactions on GenLayer.

* under development

Documentation

For detailed information on how to use GenLayerJS SDK, please refer to our [documentation] (<https://docs.genlayer.com/>).

Contributing

We welcome contributions to GenLayerJS SDK! Whether it's new features, improved infrastructure, or better documentation, your input is valuable. Please read our [CONTRIBUTING] (<https://github.com/genlayerlabs/genlayer-js/blob/main/CONTRIBUTING.md>) guide for guidelines on how to submit your contributions.

License

This project is licensed under the MIT License - see the LICENSE file for details.

Contract Methods

Source: <https://docs.genlayer.com/api-references/genlayer-js/contracts>

Methods for deploying, reading, writing, and simulating GenLayer intelligent contracts.

getContractCode

Retrieves the source code of a deployed contract.

| Parameter | Type | Required | Description |
|-----------|-----------|----------|-------------|
| address | `Address` | yes | |

****Returns:**** `string`

getContractSchema

Gets the schema (methods and constructor) of a deployed contract.

| Parameter | Type | Required | Description |
|-----------|-----------|----------|-------------|
| address | `Address` | yes | |

****Returns:**** `ContractSchema`

getContractSchemaForCode

Generates a schema for contract code without deploying it.

| Parameter | Type | Required | Description |
|--------------|--------------------------|----------|-------------|
| contractCode | `string` \ `Uint8Array` | yes | |

****Returns:**** `ContractSchema`

readContract

Executes a read-only contract call without modifying state.

| Parameter | Type | Required | Description |
|------------------------|--|----------|-------------|
| account | `Account` | no | |
| address | `Address` | yes | |
| functionName | `string` | yes | |
| args | `CallldataEncodable[]` | no | |
| kwargs | `Map<string, CallldataEncodable>` \ `{[key: string]: CallldataEncodable}` | no | |
| rawReturn | `RawReturn` | no | |
| jsonSafeReturn | `boolean` | no | |
| leaderOnly | `boolean` | no | |
| transactionHashVariant | `TransactionHashVariant` | no | |

****Returns:**** `RawReturn extends true ? 0x\${string} : CallldataEncodable`

simulateWriteContract

Simulates a state-modifying contract call without executing on-chain.

| Parameter | Type | Required | Description |
|------------------------|--|----------|-------------|
| account | `Account` | no | |
| address | `Address` | yes | |
| functionName | `string` | yes | |
| args | `CallldataEncodable[]` | no | |
| kwargs | `Map<string, CallldataEncodable>` \ `{[key: string]: CallldataEncodable}` | no | |
| rawReturn | `RawReturn` | no | |
| leaderOnly | `boolean` | no | |
| transactionHashVariant | `TransactionHashVariant` | no | |

****Returns:**** `RawReturn extends true ? 0x\${string} : CallldataEncodable`

writeContract

Executes a state-modifying function on a contract through consensus. Returns the transaction hash.

| Parameter | Type | Required | Description |
|-----------------------|---|----------|-------------|
| account | `Account` | no | |
| address | `Address` | yes | |
| functionName | `string` | yes | |
| args | `CalldataEncodable[]` | no | |
| kwargs | `Map<string, CalldataEncodable>` \ `{[key: string]: CalldataEncodable}` | no | |
| value | `bigint` | yes | |
| leaderOnly | `boolean` | no | |
| consensusMaxRotations | `number` | no | |

****Returns:**** `0x\${string}`

deployContract

Deploys a new intelligent contract to GenLayer. Returns the transaction hash.

| Parameter | Type | Required | Description |
|-----------------------|---|----------|-------------|
| account | `Account` | no | |
| code | `string` \ `Uint8Array` | yes | |
| args | `CalldataEncodable[]` | no | |
| kwargs | `Map<string, CalldataEncodable>` \ `{[key: string]: CalldataEncodable}` | no | |
| leaderOnly | `boolean` | no | |
| consensusMaxRotations | `number` | no | |

getMinAppealBond

Calculates the minimum bond required to appeal a transaction.

| Parameter | Type | Required | Description |
|-----------|----------------|----------|-------------|
| txId | `0x\${string}` | yes | |

****Returns:**** `bigint`

getRoundNumber

Returns the current consensus round number for a transaction.

| Parameter | Type | Required | Description |
|-----------|----------------|----------|-------------|
| txId | `0x\${string}` | yes | |

****Returns:**** `bigint`

getRoundData

Returns detailed data for a specific consensus round.

| Parameter | Type | Required | Description |
|-----------|------|----------|-------------|
|-----------|------|----------|-------------|

| |
|-------------------------------|
| ----- ----- ----- ----- |
| txId ``0x\${string}`` yes |
| round `bigint` yes |

getLastRoundData

Returns the current round number and its data for a transaction.

| |
|---|
| Parameter Type Required Description |
| ----- ----- ----- ----- |
| txId ``0x\${string}`` yes |

canAppeal

Checks if a transaction can be appealed.

| |
|---|
| Parameter Type Required Description |
| ----- ----- ----- ----- |
| txId ``0x\${string}`` yes |

****Returns:**** `boolean`

appealTransaction

Appeals a consensus transaction to trigger a new round of validation.

| |
|---|
| Parameter Type Required Description |
| ----- ----- ----- ----- |
| account `Account` no |
| txId ``0x\${string}`` yes |
| value `bigint` no |

finalizeTransaction

Finalizes a single GenLayer transaction that is ready to be finalized. Returns the EVM transaction hash.

| |
|---|
| Parameter Type Required Description |
| ----- ----- ----- ----- |
| account `Account` no |
| txId ``0x\${string}`` yes |

****Returns:**** `0x\${string}`

finalizeIdlenessTx

Batch-finalizes idle GenLayer transactions (those stuck without progressing). Returns the EVM transaction hash.

| |
|---|
| Parameter Type Required Description |
| ----- ----- ----- ----- |
| account `Account` no |
| txIds `readonly 0x\${string}`[]` yes |

****Returns:**** `0x\${string}`

Transaction Methods

Source: <https://docs.genlayer.com/api-references/genlayer-js/transactions>

Methods for fetching transactions, waiting for receipts, estimating gas, and debugging execution traces.

waitForTransactionReceipt

Polls until a transaction reaches the specified status. Returns the transaction receipt.

| Parameter | Type | Required | Description |
|-----------------|---------------------|----------|-------------|
| hash | `TransactionHash` | yes | |
| status | `TransactionStatus` | yes | |
| interval | `number` | no | |
| retries | `number` | no | |
| fullTransaction | `boolean` | no | |

****Returns:**** `GenLayerTransaction`

getTransaction

Fetches transaction data including status, execution result, and consensus details.

| Parameter | Type | Required | Description |
|-----------|-------------------|----------|-------------|
| hash | `TransactionHash` | yes | |

****Returns:**** `GenLayerTransaction`

getTriggeredTransactionIds

Returns transaction IDs of child transactions created from emitted messages.

| Parameter | Type | Required | Description |
|-----------|-------------------|----------|-------------|
| hash | `TransactionHash` | yes | |

****Returns:**** `TransactionHash[]`

debugTraceTransaction

Fetches the full execution trace including return data, stdout, stderr, and GenVM logs.

| Parameter | Type | Required | Description |
|-----------|-------------------|----------|-------------|
| hash | `TransactionHash` | yes | |
| round | `number` | no | |

****Returns:**** `DebugTraceResult`

cancelTransaction

Cancels a pending transaction. Studio networks only.

| Parameter | Type | Required | Description |
|-----------|-------------------|----------|-------------|
| hash | `TransactionHash` | yes | |

****Returns:**** `{transaction\_hash: string; status: string}`

getTransactionQueuePosition

Returns the queue slot position of a transaction in the pending queue.

| Parameter | Type | Required | Description |
|-----------|-------------------|----------|-------------|
| hash | `TransactionHash` | yes | |

****Returns:**** `number`

estimateTransactionGas

Estimates gas required for a transaction.

| Parameter | Type | Required | Description |
|-----------|----------------|----------|-------------|
| from | `Address` | no | |
| to | `Address` | yes | |
| data | `0x\${string}` | no | |
| value | `bigint` | no | |

****Returns:**** `bigint`

Staking Methods

Source: <https://docs.genlayer.com/api-references/genlayer-js/staking>

Methods for validator and delegator staking operations, epoch queries, and network status.

validatorJoin

Joins as a validator with the specified stake amount.

| Parameter | Type | Required | Description |
|-----------|----------------------|----------|-------------|
| amount | `bigint` \ `string` | yes | |
| operator | `Address` | no | |

****Returns:**** `ValidatorJoinResult`

validatorDeposit

Adds additional self-stake to an active validator position. The underlying Staking contract requires `msg.sender == ValidatorWallet`, so the call is routed through the wallet's own `validatorDeposit` forwarder (which re-enters Staking with the correct sender).

| Parameter | Type | Required | Description |
|-----------|---------------------|----------|-------------|
| amount | `bigint` \ `string` | yes | |
| validator | `Address` | yes | |

****Returns:**** `StakingTransactionResult`

validatorExit

Exits a validator position by burning the specified shares. Same msg.sender constraint as validatorDeposit – routed via the wallet.

| Parameter | Type | Required | Description |
|-----------|---------------------|----------|-------------|
| shares | `bigint` \ `string` | yes | |
| validator | `Address` | yes | |

****Returns:**** `StakingTransactionResult`

validatorClaim

Claims pending validator withdrawals.

| Parameter | Type | Required | Description |
|-----------|-----------|----------|-------------|
| validator | `Address` | no | |

****Returns:**** `StakingTransactionResult & {claimedAmount: bigint}`

validatorPrime

Primes a validator for participation in the next epoch.

| Parameter | Type | Required | Description |
|-----------|-----------|----------|-------------|
| validator | `Address` | yes | |

****Returns:**** `StakingTransactionResult`

setOperator

Sets the operator address for a validator wallet.

| Parameter | Type | Required | Description |
|-----------|-----------|----------|-------------|
| validator | `Address` | yes | |
| operator | `Address` | yes | |

****Returns:**** `StakingTransactionResult`

setIdentity

Sets validator identity information (name, website, social links).

| Parameter | Type | Required | Description |
|-----------|------|----------|-------------|
|-----------|------|----------|-------------|

| Parameter | Type | Required | Description |
|-------------|-----------|----------|-------------|
| validator | `Address` | yes | |
| moniker | `string` | yes | |
| logoUri | `string` | no | |
| website | `string` | no | |
| description | `string` | no | |
| email | `string` | no | |
| twitter | `string` | no | |
| telegram | `string` | no | |
| github | `string` | no | |
| extraCid | `string` | no | |

****Returns:**** `StakingTransactionResult`

delegatorJoin

Delegates stake to a validator.

| Parameter | Type | Required | Description |
|-----------|----------------------|----------|-------------|
| validator | `Address` | yes | |
| amount | `bigint` \ `string` | yes | |

****Returns:**** `DelegatorJoinResult`

delegatorExit

Exits a delegation by burning the specified shares.

| Parameter | Type | Required | Description |
|-----------|----------------------|----------|-------------|
| validator | `Address` | yes | |
| shares | `bigint` \ `string` | yes | |

****Returns:**** `StakingTransactionResult`

delegatorClaim

Claims pending delegator withdrawals.

| Parameter | Type | Required | Description |
|-----------|-----------|----------|-------------|
| validator | `Address` | yes | |
| delegator | `Address` | no | |

****Returns:**** `StakingTransactionResult`

isValidator

Checks if an address is an active validator.

| Parameter | Type | Required | Description |
|-----------|-----------|----------|-------------|
| address | `Address` | yes | |

****Returns:**** `boolean`

getValidatorInfo

Returns comprehensive information about a validator including stake, identity, and status.

| Parameter | Type | Required | Description |
|-----------|-----------|----------|-------------|
| validator | `Address` | yes | |

****Returns:**** `ValidatorInfo`

getStakeInfo

Returns delegation stake information for a delegator-validator pair.

| Parameter | Type | Required | Description |
|-----------|-----------|----------|-------------|
| delegator | `Address` | yes | |
| validator | `Address` | yes | |

****Returns:**** `StakeInfo`

getEpochInfo

Returns current epoch information including timing, stake requirements, and inflation data.

No parameters.

****Returns:**** `EpochInfo`

getEpochData

Returns detailed data for a specific epoch.

| Parameter | Type | Required | Description |
|-------------|----------|----------|-------------|
| epochNumber | `bigint` | yes | |

****Returns:**** `EpochData`

getActiveValidators

Returns addresses of all currently active validators.

No parameters.

****Returns:**** `Address[]`

getActiveValidatorsCount

Returns the count of active validators.

No parameters.

****Returns:**** `bigint`

getQuarantinedValidators

Returns addresses of validators currently in quarantine.

No parameters.

****Returns:**** `Address[]`

getBannedValidators

Returns banned validators with ban duration and permanent ban status.

| Parameter | Type | Required | Description |
|------------|--------|----------|-------------|
| startIndex | bigint | no | |
| size | bigint | no | |

****Returns:**** `BannedValidatorInfo[]`

getQuarantinedValidatorsDetailed

Returns detailed quarantine information with pagination.

| Parameter | Type | Required | Description |
|------------|--------|----------|-------------|
| startIndex | bigint | no | |
| size | bigint | no | |

****Returns:**** `BannedValidatorInfo[]`

GenLayerPY

Source: <https://docs.genlayer.com/api-references/genlayer-py>

About

GenLayerPY SDK is a python library designed for developers building decentralized applications (Dapps) on the GenLayer protocol. This SDK provides a comprehensive set of tools to interact with the GenLayer network, including client creation, transaction handling, event subscriptions, and more, all while leveraging the power of web3.py as the underlying blockchain client.

Prerequisites

Before installing GenLayerPY SDK, ensure you have the following prerequisites installed:

- Python (>=3.12)

Installation and Usage

To install the GenLayerPY SDK, use the following command:

```
```bash
$ pip install genlayer-py
```
```

Here's how to initialize the client and connect to the GenLayer Simulator:

```
### Reading a Transaction
```python
from genlayer_py import create_client
from genlayer_py.chains import localnet

client = create_client(
 chain=localnet,
)

transaction_hash = "0x..."

transaction = client.get_transaction(hash=transaction_hash)
...

Waiting for Transaction Receipt
```python
from genlayer_py import create_client
from genlayer_py.chains import localnet
from genlayer_py.types import TransactionStatus

client = create_client(chain=localnet)

# Get simplified receipt (default - removes binary data, keeps execution results)
receipt = client.wait_for_transaction_receipt(
    transaction_hash="0x...",
    status=TransactionStatus.FINALIZED,
    full_transaction=False # Default - simplified for readability
)

# Get complete receipt with all fields
full_receipt = client.wait_for_transaction_receipt(
    transaction_hash="0x...",
    status=TransactionStatus.FINALIZED,
    full_transaction=True # Complete receipt with all internal data
)
...

### Reading a contract
```python
from genlayer_py import create_client
from genlayer_py.chains import localnet

client = create_client(
 chain=localnet,
)

result = client.read_contract(
 address=contract_address,
 function_name='get_complete_storage',
 args=[],
 state_status='accepted'
)
...

Writing a transaction
```python
from genlayer_py.chains import localnet
from genlayer_py import create_client, create_account

client = create_client(
    chain=localnet,
```

```

)

account = create_account()

transaction_hash = client.write_contract(
    account=account,
    transaction=transaction,
    address=contract_address,
    function_name='account',
    args=['new_storage'],
    value=0, // value is optional, if you want to send some native token to the contract
)
receipt = client.wait_for_transaction_receipt(
    hash=transaction_hash,
    status=TransactionStatus.FINALIZED, // or ACCEPTED
    full_transaction=False // False by default - returns simplified receipt for better
readability
)
...

```

Checking execution results

A transaction can be finalized by consensus but still have a failed execution. Always check `tx\_execution\_result` before reading contract state:

```

```python
from genlayer_py import create_client, create_account
from genlayer_py.chains import testnet_bradbury
from genlayer_py.types import TransactionStatus, ExecutionResult

client = create_client(chain=testnet_bradbury, account=create_account())

receipt = client.wait_for_transaction_receipt(
 transaction_hash=tx_hash,
 status=TransactionStatus.FINALIZED,
)

if receipt.get("tx_execution_result_name") == ExecutionResult.FINISHED_WITH_RETURN.value:
 # Execution succeeded – safe to read state
 result = client.read_contract(
 address=contract_address,
 function_name="get_storage",
 args=[],
)
elif receipt.get("tx_execution_result_name") == ExecutionResult.FINISHED_WITH_ERROR.value:
 # Execution failed – contract state was not modified
 raise RuntimeError("Contract execution failed")
else:
 # NOT_VOTED – execution hasn't completed
 print("Execution result not yet available")
...

```

### ### Fetching emitted messages and triggered transactions

Transactions can emit messages to other contracts. These messages create new child transactions when processed:

```

```python
tx = client.get_transaction(transaction_hash=tx_hash)

# Messages emitted by the contract during execution
print(tx["messages"])
# [{"messageType": 1, "recipient": "0x...", "value": 0, "data": "0x...", "onAcceptance":
True, "saltNonce": 0}, ...]

```

```
# Child transaction IDs created from those messages (separate call)
child_tx_ids = client.get_triggered_transaction_ids(transaction_hash=tx_hash)
print(child_tx_ids)
# ["0xabc...", "0xdef..."]
````
```

### ### Debugging transaction execution

Use `debug_trace_transaction` to inspect the full execution trace of a transaction, including return data, errors, and GenVM logs:

```
``python
trace = client.debug_trace_transaction(
 transaction_hash=tx_hash,
 round=0, # optional, defaults to 0
)

print(trace["result_code"]) # 0=success, 1=user error, 2=VM error
print(trace["return_data"]) # hex-encoded contract return data
print(trace["stderr"]) # standard error output
print(trace["genvm_log"]) # detailed GenVM execution logs
````
```

Key Features

- * **Client Creation**: Easily create and configure a client to connect to GenLayer's network.
- * **Transaction Handling**: Send and manage transactions on the GenLayer network.
- * **Gas Estimation**: Estimate gas fees for executing transactions on GenLayer.

`_*` under development

Documentation

For detailed information on how to use GenLayerPY SDK, please refer to our [documentation] (<https://docs.genlayer.com/api-references/genlayer-py>).

Contributing

We welcome contributions to GenLayerPY SDK! Whether it's new features, improved infrastructure, or better documentation, your input is valuable. Please read our [CONTRIBUTING] (<https://github.com/genlayerlabs/genlayer-py/blob/main/CONTRIBUTING.md>) guide for guidelines on how to submit your contributions.

License

This project is licensed under the MIT License - see the LICENSE file for details.

GenLayerPY SDK API Reference
Source: <https://docs.genlayer.com/api-references/genlayer-py/api>

Auto-generated from source docstrings.

Client Methods

Client for interacting with the GenLayer network.

Provides methods for deploying and calling intelligent contracts, managing transactions, and staking operations.

fund_account

Funds an account with test tokens. Localnet only.

```
```python
client.fund_account(address: Union, amount: int)
```
```

| Parameter | Type | Required | Default |
|-----------|---------|----------|---------|
| address | `Union` | yes | |
| amount | `int` | yes | |

****Returns:**** `HexBytes`

get_current_nonce

Returns the current nonce (transaction count) for an account.

```
```python
client.get_current_nonce(address: Union = None, block_identifier: Union = None)
```
```

| Parameter | Type | Required | Default |
|------------------|---------|----------|---------|
| address | `Union` | no | None |
| block_identifier | `Union` | no | None |

****Returns:**** `Nonce`

initialize_consensus_smart_contract

Initializes the consensus contract configuration from the network.

```
```python
client.initialize_consensus_smart_contract(force_reset: bool = False)
```
```

| Parameter | Type | Required | Default |
|-------------|--------|----------|---------|
| force_reset | `bool` | no | False |

****Returns:**** `None`

read_contract

Executes a read-only contract call without modifying state.

```
```python
client.read_contract(address: Union, function_name: str, args: Optional = None, kwargs:
Optional = None, account: Optional = None, raw_return: bool = False,
transaction_hash_variant: TransactionHashVariant = <TransactionHashVariant.LATEST_NONFINAL:
'latest-nonfinal'>, sim_config: Optional = None)
```
```

| Parameter | Type | Required | Default |
|---------------|------------|----------|---------|
| address | `Union` | yes | |
| function_name | `str` | yes | |
| args | `Optional` | no | None |
| kwargs | `Optional` | no | None |

| | | | |
|--------------------------|--------------------------|----|-------|
| account | 'Optional' | no | None |
| raw_return | 'bool' | no | False |
| transaction_hash_variant | 'TransactionHashVariant' | no | |

<TransactionHashVariant.LATEST_NONFINAL: 'latest-nonfinal'> |

| | | | |
|------------|------------|----|------|
| sim_config | 'Optional' | no | None |
|------------|------------|----|------|

write_contract

Executes a state-modifying function on a contract through consensus. Returns the transaction hash.

```
```python
```

```
client.write_contract(address: Union, function_name: str, account: Optional = None,
consensus_max_rotations: Optional = None, value: int = 0, leader_only: bool = False, args:
Optional = None, kwargs: Optional = None, sim_config: Optional = None)
```

```
```
```

| Parameter | Type | Required | Default |
|-------------------------|------------|----------|---------|
| ----- ----- ----- ----- | | | |
| address | 'Union' | yes | |
| function_name | 'str' | yes | |
| account | 'Optional' | no | None |
| consensus_max_rotations | 'Optional' | no | None |
| value | 'int' | no | 0 |
| leader_only | 'bool' | no | False |
| args | 'Optional' | no | None |
| kwargs | 'Optional' | no | None |
| sim_config | 'Optional' | no | None |

simulate_write_contract

Simulates a state-modifying contract call without executing on-chain. Localnet only.

```
```python
```

```
client.simulate_write_contract(address: Union, function_name: str, account: Optional = None,
args: Optional = None, kwargs: Optional = None, sim_config: Optional = None,
transaction_hash_variant: TransactionHashVariant = <TransactionHashVariant.LATEST_NONFINAL:
'latest-nonfinal'>)
```

```
```
```

| Parameter | Type | Required | Default |
|--------------------------|--------------------------|----------|---------|
| ----- ----- ----- ----- | | | |
| address | 'Union' | yes | |
| function_name | 'str' | yes | |
| account | 'Optional' | no | None |
| args | 'Optional' | no | None |
| kwargs | 'Optional' | no | None |
| sim_config | 'Optional' | no | None |
| transaction_hash_variant | 'TransactionHashVariant' | no | |

<TransactionHashVariant.LATEST_NONFINAL: 'latest-nonfinal'> |

deploy_contract

Deploys a new intelligent contract to GenLayer. Returns the transaction hash.

```
```python
```

```
client.deploy_contract(code: Union, account: Optional = None, args: Optional = None, kwargs:
Optional = None, consensus_max_rotations: Optional = None, leader_only: bool = False,
sim_config: Optional = None)
```

...

| Parameter               | Type       | Required | Default |
|-------------------------|------------|----------|---------|
| code                    | `Union`    | yes      |         |
| account                 | `Optional` | no       | None    |
| args                    | `Optional` | no       | None    |
| kwargs                  | `Optional` | no       | None    |
| consensus_max_rotations | `Optional` | no       | None    |
| leader_only             | `bool`     | no       | False   |
| sim_config              | `Optional` | no       | None    |

---

### ### get\_contract\_schema

Gets the schema (methods and constructor) of a deployed contract. Localnet only.

```
```python
client.get_contract_schema(address: Union)
```
```

| Parameter | Type    | Required | Default |
|-----------|---------|----------|---------|
| address   | `Union` | yes      |         |

**\*\*Returns:\*\*** `ContractSchema`

---

### ### get\_contract\_schema\_for\_code

Generates a schema for contract code without deploying it. Localnet only.

```
```python
client.get_contract_schema_for_code(contract_code: AnyStr)
```
```

| Parameter     | Type     | Required | Default |
|---------------|----------|----------|---------|
| contract_code | `AnyStr` | yes      |         |

**\*\*Returns:\*\*** `ContractSchema`

---

### ### appeal\_transaction

Appeals a consensus transaction to trigger a new round of validation.  
Returns the original transaction\_id (appeals operate on the same tx).

```
```python
client.appeal_transaction(transaction_id: HexStr, account: Optional = None, value: int = 0)
```
```

| Parameter      | Type       | Required | Default |
|----------------|------------|----------|---------|
| transaction_id | `HexStr`   | yes      |         |
| account        | `Optional` | no       | None    |
| value          | `int`      | no       | 0       |

---

### ### wait\_for\_transaction\_receipt



Polls until a transaction reaches the specified status. Returns the transaction receipt.

```
```python
client.wait_for_transaction_receipt(transaction_hash: Union, status: TransactionStatus =
<TransactionStatus.ACCEPTED: 'ACCEPTED'>, interval: int = 3000, retries: int = 10,
full_transaction: bool = False)
```
```

| Parameter        | Type                | Required | Default                                  |
|------------------|---------------------|----------|------------------------------------------|
| transaction_hash | `Union`             | yes      |                                          |
| status           | `TransactionStatus` | no       | <TransactionStatus.ACCEPTED: 'ACCEPTED'> |
| interval         | `int`               | no       | 3000                                     |
| retries          | `int`               | no       | 10                                       |
| full_transaction | `bool`              | no       | False                                    |

**\*\*Returns:\*\*** `GenLayerTransaction`

---

### ### get\_transaction

Fetches transaction data including status, execution result, and consensus details.

```
```python
client.get_transaction(transaction_hash: Union)
```
```

| Parameter        | Type    | Required | Default |
|------------------|---------|----------|---------|
| transaction_hash | `Union` | yes      |         |

**\*\*Returns:\*\*** `GenLayerTransaction`

---

### ### get\_triggered\_transaction\_ids

Returns transaction IDs of child transactions created from emitted messages.

```
```python
client.get_triggered_transaction_ids(transaction_hash: Union)
```
```

| Parameter        | Type    | Required | Default |
|------------------|---------|----------|---------|
| transaction_hash | `Union` | yes      |         |

**\*\*Returns:\*\*** `list`

---

### ### debug\_trace\_transaction

Fetches the full execution trace including return data, stdout, stderr, and GenVM logs.

```
```python
client.debug_trace_transaction(transaction_hash: Union, round: int = 0)
```
```

| Parameter        | Type    | Required | Default |
|------------------|---------|----------|---------|
| transaction_hash | `Union` | yes      |         |
| round            | `int`   | no       | 0       |

**\*\*Returns:\*\*** `dict`

---

## ## Types and Enums

### ### TransactionStatus

Status of a GenLayer transaction in the consensus lifecycle.

```python

```
TransactionStatus.UNINITIALIZED = "UNINITIALIZED"
TransactionStatus.PENDING = "PENDING"
TransactionStatus.PROPOSING = "PROPOSING"
TransactionStatus.COMMITTING = "COMMITTING"
TransactionStatus.REVEALING = "REVEALING"
TransactionStatus.ACCEPTED = "ACCEPTED"
TransactionStatus.UNDETERMINED = "UNDETERMINED"
TransactionStatus.FINALIZED = "FINALIZED"
TransactionStatus.CANCELED = "CANCELED"
TransactionStatus.APPEAL_REVEALING = "APPEAL_REVEALING"
TransactionStatus.APPEAL_COMMITTING = "APPEAL_COMMITTING"
TransactionStatus.READY_TO_FINALIZE = "READY_TO_FINALIZE"
TransactionStatus.VALIDATORS_TIMEOUT = "VALIDATORS_TIMEOUT"
TransactionStatus.LEADER_TIMEOUT = "LEADER_TIMEOUT"
```

```

---

### ### TransactionResult

Consensus voting result across validators.

```python

```
TransactionResult.IDLE = "IDLE"
TransactionResult.AGREE = "AGREE"
TransactionResult.DISAGREE = "DISAGREE"
TransactionResult.TIMEOUT = "TIMEOUT"
TransactionResult.DETERMINISTIC_VIOLATION = "DETERMINISTIC_VIOLATION"
TransactionResult.NO_MAJORITY = "NO_MAJORITY"
TransactionResult.MAJORITY_AGREE = "MAJORITY_AGREE"
TransactionResult.MAJORITY_DISAGREE = "MAJORITY_DISAGREE"
```

```

---

### ### ExecutionResult

Result of contract execution by the GenVM.

```python

```
ExecutionResult.NOT_VOTED = "NOT_VOTED"
ExecutionResult.FINISHED_WITH_RETURN = "FINISHED_WITH_RETURN"
ExecutionResult.FINISHED_WITH_ERROR = "FINISHED_WITH_ERROR"
```

```

---

### ### VoteType

```
str(object='') -> str
str(bytes_or_buffer[, encoding[, errors]]) -> str
```

Create a new string object from the given object. If encoding or errors is specified, then the object must expose a data buffer

that will be decoded using the given encoding and error handler.  
Otherwise, returns the result of object.\_\_str\_\_() (if defined)  
or repr(object).  
encoding defaults to 'utf-8'.  
errors defaults to 'strict'.

```
```python
VoteType.NOT_VOTED = "NOT_VOTED"
VoteType.AGREE = "AGREE"
VoteType.DISAGREE = "DISAGREE"
VoteType.TIMEOUT = "TIMEOUT"
VoteType.DETERMINISTIC_VIOLATION = "DETERMINISTIC_VIOLATION"
```
```

---

---

# GenLayer Testing Suite  
Source: <https://docs.genlayer.com/api-references/genlayer-test>

A pytest-based testing framework for [GenLayer](<https://docs.genlayer.com/>) intelligent contracts. Built on top of [genlayer-py](<https://docs.genlayer.com/api-references/genlayer-py>).

```
```bash
pip install genlayer-test
```
```

## ## Two Ways to Test

The testing suite provides two execution modes. Pick the one that fits your workflow:

| Direct Mode                                                                                                                            | Studio Mode |
|----------------------------------------------------------------------------------------------------------------------------------------|-------------|
| <b>How it works</b>   Runs contract Python code directly in-memory   Deploys to GenLayer Studio, interacts via RPC                     |             |
| <b>Speed</b>   ~milliseconds per test   ~minutes per test                                                                              |             |
| <b>Prerequisites</b>   Python >= 3.12   Python >= 3.12 + GenLayer Studio (Docker)                                                      |             |
| <b>Best for</b>   Unit tests, rapid development, CI/CD   Integration tests, consensus validation, testnet                              |             |
| <b>Mocking</b>   Foundry-style cheatcodes ( <code>mock_web</code> , <code>mock_llm</code> )   Mock validators with transaction context |             |

**Start with Direct Mode.** It's faster, simpler, and doesn't require Docker. Use Studio Mode when you need full network behavior, multi-validator consensus, or testnet deployment.

---

## ## Direct Mode

Run contracts directly in Python – no simulator, no Docker, no network. Tests execute in milliseconds.

### ### Quick Start

```
```python
def test_storage(direct_vm, direct_deploy):
    # Deploy contract in-memory
    storage = direct_deploy("contracts/Storage.py", "initial")

    # Read state directly
    assert storage.get_storage() == "initial"

    # Write state directly
```
```

```

 storage.update_storage("updated")
... assert storage.get_storage() == "updated"

```

Run with pytest:

```

```bash
pytest tests/ -v
```

```

### Fixtures

| Fixture                                        | Description                |
|------------------------------------------------|----------------------------|
| `direct_vm`                                    | VM context with cheatcodes |
| `direct_deploy`                                | Deploy contracts directly  |
| `direct_alice`, `direct_bob`, `direct_charlie` | Test addresses             |
| `direct_owner`                                 | Default sender address     |
| `direct_accounts`                              | List of 10 test addresses  |

### Cheatcodes

```

```python
# Change sender
direct_vm.sender = alice

# Prank (temporary sender change)
with direct_vm.prank(bob):
    contract.method() # Called as bob

# Snapshots (captures full state: storage, mocks, sender, validators)
snap_id = direct_vm.snapshot()
contract.modify_state()
direct_vm.revert(snap_id) # Full state restored

# Expect revert
with direct_vm.expect_revert("Insufficient balance"):
    contract.transfer(bob, 1000000)

# Mock web/LLM (regex pattern matching)
direct_vm.mock_web(r"api\.example\.com", {"status": 200, "body": "{}"})
direct_vm.mock_llm(r"analyze.*", "positive sentiment")

# Test validator consensus logic
contract.update_price() # Runs leader_fn, captures validator
direct_vm.clear_mocks() # Swap mocks for validator
direct_vm.mock_llm(r".*", "different result")
assert direct_vm.run_validator() is False # Validator disagrees

# Strict mocks (detect unused mocks)
direct_vm.strict_mocks = True

# Pickling validation (catch production serialization issues)
direct_vm.check_pickling = True
```

```

**\*\*[Full Direct Mode Documentation](docs/direct-runner.md)\*\*** – fixtures, cheatcodes, validator testing, limitations, and complete examples.

---

## Studio Mode

Deploy contracts to a running GenLayer Studio instance and interact via RPC. This gives you full network behavior including multi-validator consensus.

### ### Prerequisites

- Python >= 3.12
- GenLayer Studio running (Docker)

### ### Quick Start

```
```python
from gltest import get_contract_factory, get_default_account
from gltest.assertions import tx_execution_succeeded

factory = get_contract_factory("MyContract")
contract = factory.deploy()

# Read method – returns value directly
result = contract.get_value().call()

# Write method – returns transaction receipt
tx_receipt = contract.set_value(args=["new_value"]).transact()
assert tx_execution_succeeded(tx_receipt)
```
```

Run with the `gltest` CLI:

```
```bash
gltest                                # Run all tests
gltest tests/test_mycontract.py      # Specific file
gltest --network studionet           # Specific network
gltest --leader-only                 # Skip consensus (faster)
gltest -v                             # Verbose output
```
```

### ### Configuration

Create a `gltest.config.yaml` in your project root:

```
```yaml
networks:
  default: localnet

  localnet:
    url: "http://127.0.0.1:4000/api"
    leader_only: false

  studionet:
    # Pre-configured – accounts auto-generated

  testnet_asimov:
    accounts:
      - "${ACCOUNT_PRIVATE_KEY_1}"
      - "${ACCOUNT_PRIVATE_KEY_2}"
    from: "${ACCOUNT_PRIVATE_KEY_1}"

paths:
  contracts: "contracts"
  artifacts: "artifacts"

environment: .env
```
```

Key options:

- **Networks**: `localnet` and `studionet` work out of the box. `testnet\_asimov` requires account keys.
- **Paths**: Where your contracts and artifacts live.

- **\*\*Environment\*\***: ``.env`` file for private keys.

Override via CLI:

```
```bash
gltest --network testnet_asimov
gltest --contracts-dir custom/contracts/path
gltest --rpc-url http://custom:4000/api
gltest --chain-type localnet
```
```

### ### Contract Deployment

```
```python
from gltest import get_contract_factory, get_default_account
from gltest.assertions import tx_execution_succeeded
```

```
factory = get_contract_factory("Storage")
```

```
# deploy() returns the contract instance (recommended)
```

```
contract = factory.deploy(
    args=["initial_value"],
    account=get_default_account(),
    consensus_max_rotations=3,
)
```

```
# deploy_contract_tx() returns only the receipt
```

```
receipt = factory.deploy_contract_tx(args=["initial_value"])
assert tx_execution_succeeded(receipt)
```
```

### ### Read and Write Methods

```
```python
```

```
# Read – call() returns the value
```

```
result = contract.get_storage().call()
```

```
# Write – transact() returns a receipt
```

```
tx_receipt = contract.update_storage(args=["new_value"]).transact(
    value=0,
    consensus_max_rotations=3,
    wait_interval=1000,
    wait_retries=10,
)
```

```
assert tx_execution_succeeded(tx_receipt)
```
```

### ### Assertions

```
```python
```

```
from gltest.assertions import tx_execution_succeeded, tx_execution_failed
```

```
assert tx_execution_succeeded(tx_receipt)
```

```
assert tx_execution_failed(tx_receipt)
```

```
# Regex matching on stdout/stderr (localnet/studionet only)
```

```
assert tx_execution_succeeded(tx_receipt, match_std_out=r".*code \d+")
assert tx_execution_failed(tx_receipt, match_std_err=r"Method.*failed")
```
```

### ### Fixtures

| Fixture                  | Scope   | Description                            |
|--------------------------|---------|----------------------------------------|
| <code>`gl_client`</code> | session | GenLayer client for network operations |

|                   | session |                                  |
|-------------------|---------|----------------------------------|
| `default_account` | session | Default account for transactions |
| `accounts`        | session | List of test accounts            |

```

```python
def test_workflow(gl_client, default_account, accounts):
    factory = get_contract_factory("MyContract")
    contract = factory.deploy(account=default_account)

    tx_receipt = contract.some_method(args=["value"], account=accounts[1])
    assert tx_execution_succeeded(tx_receipt)
...

```

Mock LLM Responses

Simulate LLM responses for deterministic tests:

```

```python
from gltest import get_contract_factory, get_validator_factory
from gltest.types import MockedLLMResponse

mock_response: MockedLLMResponse = {
 "nondet_exec_prompt": {
 "analyze this": "positive sentiment"
 },
 "eq_principle_prompt_comparative": {
 "values match": True
 }
}

validator_factory = get_validator_factory()
validators = validator_factory.batch_create_mock_validators(
 count=5,
 mock_llm_response=mock_response
)

transaction_context = {
 "validators": [v.to_dict() for v in validators],
 "genvm_datetime": "2024-01-01T00:00:00Z"
}

factory = get_contract_factory("LLMContract")
contract = factory.deploy(transaction_context=transaction_context)
result = contract.analyze_text(args=["analyze this"]).transact(
 transaction_context=transaction_context
)
...

```

Mock keys map to GenLayer methods:

| Mock Key                              | GenLayer Method                          |
|---------------------------------------|------------------------------------------|
| `nondet_exec_prompt`                  | `gl.nondet.exec_prompt`                  |
| `eq_principle_prompt_comparative`     | `gl.eq_principle.prompt_comparative`     |
| `eq_principle_prompt_non_comparative` | `gl.eq_principle.prompt_non_comparative` |

The system performs **substring matching** on the internal user message – your mock key must appear within the message.

### ### Mock Web Responses

Simulate HTTP responses for contracts that call `gl.nondet.web.get()`, etc.:

```

```python
from gltest.types import MockedWebResponse
import json

```

```

mock_web_response: MockedWebResponse = {
    "nondet_web_request": {
        "https://api.example.com/price": {
            "method": "GET",
            "status": 200,
            "body": json.dumps({"price": 100.50})
        }
    }
}

validators = validator_factory.batch_create_mock_validators(
    count=5,
    mock_web_response=mock_web_response
)

```

You can combine both `mock\_llm\_response` and `mock\_web\_response` in a single `batch\_create\_mock\_validators` call. URL matching is exact (including query parameters).

Custom Validators

```

```python
from gltest import get_validator_factory

factory = get_validator_factory()

Real validators with specific LLM providers
validators = factory.batch_create_validators(
 count=5,
 stake=10,
 provider="openai",
 model="gpt-4o",
 config={"temperature": 0.7},
 plugin="openai-compatible",
 plugin_config={"api_key_env_var": "OPENAI_API_KEY"}
)

Use in transaction context
transaction_context = {
 "validators": [v.to_dict() for v in validators],
 "genvm_datetime": "2024-03-15T14:30:00Z"
}

```

### ### Statistical Analysis

For LLM-based contracts, `.analyze()` runs multiple simulations to measure consistency:

```

```python
analysis = contract.process_with_llm(args=["input"]).analyze(
    provider="openai",
    model="gpt-4o",
    runs=100,
)

print(f"Success rate: {analysis.success_rate:.2f}%")
print(f"Reliability: {analysis.reliability_score:.2f}%")
print(f"Unique states: {analysis.unique_states}")

```

****[Full Studio Mode Documentation](docs/studio-runner.md)**** – configuration reference, all CLI flags, mock LLM/web details, custom validators, statistical analysis, and complete examples.

Example Contract

```
```python
from genlayer import *

class Storage(gl.Contract):
 storage: str

 def __init__(self, initial_storage: str):
 self.storage = initial_storage

 @gl.public.view
 def get_storage(self) -> str:
 return self.storage

 @gl.public.write
 def update_storage(self, new_storage: str) -> None:
 self.storage = new_storage
...

```

## ### Project Structure

```
...
my-project/
├── contracts/
│ └── Storage.py
├── tests/
│ ├── test_direct.py # Direct mode tests (fast)
│ └── test_integration.py # Studio mode tests
└── gltest.config.yaml # Studio mode config
...

```

For more examples, see the [contracts directory](tests/examples/contracts).

## ## Troubleshooting

**\*\*Contract not found\*\***: Ensure contracts are in `contracts/` or specify `--contracts-dir`. Contracts must inherit from `gl.Contract`.

**\*\*Transaction timeouts\*\*** (Studio mode): Increase `wait\_interval` and `wait\_retries` in `.transact()`.

**\*\*Consensus failures\*\*** (Studio mode): Increase `consensus\_max\_rotations` or use `--leader-only` for faster iteration.

**\*\*Environment issues\*\***: Verify Python  $\geq 3.12$ . For Studio mode, check Docker is running (`docker ps`).

## ## Contributing

See our [Contributing Guide](CONTRIBUTING.md).

## ## License

MIT – see [LICENSE](LICENSE).

## ## Support

- [Documentation](https://docs.genlayer.com/api-references/genlayer-test)
- [Discord](https://discord.gg/qjCU4AwnKE)
- [GitHub Issues](https://github.com/genlayerlabs/genlayer-testing-suite/issues)
- [Twitter](https://x.com/GenLayer)

```

Integration Testing
Source: https://docs.genlayer.com/api-references/genlayer-test/integration

Test contracts against a running GenLayer network (localnet, studionet, or testnet).

Setup Functions

`get_contract_factory`

Get a ContractFactory instance for a contract.

Args:
 contract_name: Name of the contract to load from artifacts
 contract_file_path: Path to the contract file to load directly

Note: Exactly one of contract_name or contract_file_path must be provided.

```python
get_contract_factory(contract_name: Optional = None, contract_file_path: Union = None)
```

Parameters:

| Parameter | Type | Required | Default |
|----------------------|------------|----------|---------|
| `contract_name` | `Optional` | No | `None` |
| `contract_file_path` | `Union` | No | `None` |

Returns: `ContractFactory`

`get_default_account`

Returns the default account for the current network.

Returns: `LocalAccount`

`get_accounts`

Returns all configured accounts for the current network.

Returns: `List`

`create_accounts`

Creates n new accounts with random private keys.

```python
create_accounts(n_accounts: int)
```

Parameters:

| Parameter | Type | Required | Default |
|--------------|-------|----------|---------|
| `n_accounts` | `int` | Yes | |

`get_gl_client`

```

Get the GenLayer client instance.

```

`get_validator_factory`

Returns: `ValidatorFactory`
```

```

ContractFactory
```

A factory for deploying contracts.

```
`factory.deploy`
```

Deploy the contract and return a Contract instance (convenience method).

This is a convenience method that handles receipt validation and contract instantiation automatically.

```
```python
factory.deploy(args: Optional = None, account: Optional = None, consensus_max_rotations:
Optional = None, wait_interval: Optional = None, wait_retries: Optional = None,
wait_transaction_status: TransactionStatus = <TransactionStatus.ACCEPTED: 'ACCEPTED'>,
wait_triggered_transactions: bool = False, wait_triggered_transactions_status:
TransactionStatus = <TransactionStatus.ACCEPTED: 'ACCEPTED'>, transaction_context: Optional
= None)
```
```

**Parameters:**

| Parameter                            | Type                | Required | Default                                    |
|--------------------------------------|---------------------|----------|--------------------------------------------|
| `args`                               | `Optional`          | No       | `None`                                     |
| `account`                            | `Optional`          | No       | `None`                                     |
| `consensus_max_rotations`            | `Optional`          | No       | `None`                                     |
| `wait_interval`                      | `Optional`          | No       | `None`                                     |
| `wait_retries`                       | `Optional`          | No       | `None`                                     |
| `wait_transaction_status`            | `TransactionStatus` | No       | `<TransactionStatus.ACCEPTED: 'ACCEPTED'>` |
| `wait_triggered_transactions`        | `bool`              | No       | `False`                                    |
| `wait_triggered_transactions_status` | `TransactionStatus` | No       | `<TransactionStatus.ACCEPTED: 'ACCEPTED'>` |
| `transaction_context`                | `Optional`          | No       | `None`                                     |

**Returns:** `Contract`

```

```

```
`factory.deploy_contract_tx`
```

Deploy the contract and return the transaction receipt.

```
```python
factory.deploy_contract_tx(args: Optional = None, account: Optional = None,
consensus_max_rotations: Optional = None, wait_interval: Optional = None, wait_retries:
Optional = None, wait_transaction_status: TransactionStatus = <TransactionStatus.ACCEPTED:
'ACCEPTED'>, wait_triggered_transactions: bool = False, wait_triggered_transactions_status:
TransactionStatus = <TransactionStatus.ACCEPTED: 'ACCEPTED'>, transaction_context: Optional
= None)
```
```

**Parameters:**

| Parameter | Type | Required | Default |
|-----------|------|----------|---------|
|           |      |          |         |

```

| `args` | `Optional` | No | `None` |
| `account` | `Optional` | No | `None` |
| `consensus_max_rotations` | `Optional` | No | `None` |
| `wait_interval` | `Optional` | No | `None` |
| `wait_retries` | `Optional` | No | `None` |
| `wait_transaction_status` | `TransactionStatus` | No | `<TransactionStatus.ACCEPTED: 'ACCEPTED'>` |
| `wait_triggered_transactions` | `bool` | No | `False` |
| `wait_triggered_transactions_status` | `TransactionStatus` | No | `<TransactionStatus.ACCEPTED: 'ACCEPTED'>` |
| `transaction_context` | `Optional` | No | `None` |

```

**\*\*Returns:\*\*** `GenLayerTransaction`

---

### `factory.build\_contract`

Build contract from address

```

```python
factory.build_contract(contract_address: Union, account: Optional = None)
```

```

**\*\*Parameters:\*\***

| Parameter          | Type       | Required | Default |
|--------------------|------------|----------|---------|
| `contract_address` | `Union`    | Yes      |         |
| `account`          | `Optional` | No       | `None`  |

**\*\*Returns:\*\*** `Contract`

---

## ContractFunction

ContractFunction(method\_name: str, read\_only: bool, call\_method: Optional[Callable] = None, analyze\_method: Optional[Callable] = None, transact\_method: Optional[Callable] = None)

### `contract.method\_name.call`

Executes a read-only contract method call.

```

```python
contract.method_name.call(transaction_hash_variant: TransactionHashVariant =
<TransactionHashVariant.LATEST_NONFINAL: 'latest-nonfinal'>, transaction_context: Optional =
None)
```

```

**\*\*Parameters:\*\***

| Parameter                  | Type                     | Required | Default |
|----------------------------|--------------------------|----------|---------|
| `transaction_hash_variant` | `TransactionHashVariant` | No       |         |
| `transaction_context`      | `Optional`               | No       | `None`  |

---

### `contract.method\_name.transact`

Executes a state-changing contract method through consensus. Returns the transaction receipt.

```

```python

```

```
contract.method_name.transact(value: int = 0, consensus_max_rotations: Optional = None,
wait_transaction_status: TransactionStatus = <TransactionStatus.ACCEPTED: 'ACCEPTED'>,
wait_interval: Optional = None, wait_retries: Optional = None, wait_triggered_transactions:
bool = False, wait_triggered_transactions_status: TransactionStatus =
<TransactionStatus.ACCEPTED: 'ACCEPTED'>, transaction_context: Optional = None)
```
```

**\*\*Parameters:\*\***

| Parameter                            | Type                | Required | Default                                    |
|--------------------------------------|---------------------|----------|--------------------------------------------|
| `value`                              | `int`               | No       | `0`                                        |
| `consensus_max_rotations`            | `Optional`          | No       | `None`                                     |
| `wait_transaction_status`            | `TransactionStatus` | No       | `<TransactionStatus.ACCEPTED: 'ACCEPTED'>` |
| `wait_interval`                      | `Optional`          | No       | `None`                                     |
| `wait_retries`                       | `Optional`          | No       | `None`                                     |
| `wait_triggered_transactions`        | `bool`              | No       | `False`                                    |
| `wait_triggered_transactions_status` | `TransactionStatus` | No       | `<TransactionStatus.ACCEPTED: 'ACCEPTED'>` |
| `transaction_context`                | `Optional`          | No       | `None`                                     |

---

### `contract.method\_name.analyze`

Runs statistical analysis of method behavior across multiple executions.

```
```python
contract.method_name.analyze(provider: str, model: str, config: Optional = None, plugin:
Optional = None, plugin_config: Optional = None, runs: int = 100, genvm_datetime: Optional =
None)
```
```

**\*\*Parameters:\*\***

| Parameter        | Type       | Required | Default |
|------------------|------------|----------|---------|
| `provider`       | `str`      | Yes      |         |
| `model`          | `str`      | Yes      |         |
| `config`         | `Optional` | No       | `None`  |
| `plugin`         | `Optional` | No       | `None`  |
| `plugin_config`  | `Optional` | No       | `None`  |
| `runs`           | `int`      | No       | `100`   |
| `genvm_datetime` | `Optional` | No       | `None`  |

---

## ValidatorFactory

### `validator\_factory.create\_validator`

```
```python
validator_factory.create_validator(stake: int, provider: str, model: str, config: Dict,
plugin: str, plugin_config: Dict)
```
```

**\*\*Parameters:\*\***

| Parameter  | Type   | Required | Default |
|------------|--------|----------|---------|
| `stake`    | `int`  | Yes      |         |
| `provider` | `str`  | Yes      |         |
| `model`    | `str`  | Yes      |         |
| `config`   | `Dict` | Yes      |         |
| `plugin`   | `str`  | Yes      |         |

```
| `plugin_config` | `Dict` | Yes | |
```

```
Returns: `Validator`
```

```

```

```
`validator_factory.batch_create_validators`
```

```
```python
```

```
validator_factory.batch_create_validators(count: int, stake: int, provider: str, model: str,  
config: Dict, plugin: str, plugin_config: Dict)
```

```
```
```

```
Parameters:
```

| Parameter       | Type   | Required | Default |
|-----------------|--------|----------|---------|
| `count`         | `int`  | Yes      |         |
| `stake`         | `int`  | Yes      |         |
| `provider`      | `str`  | Yes      |         |
| `model`         | `str`  | Yes      |         |
| `config`        | `Dict` | Yes      |         |
| `plugin`        | `str`  | Yes      |         |
| `plugin_config` | `Dict` | Yes      |         |

```
Returns: `List`
```

```

```

```
`validator_factory.create_mock_validator`
```

```
```python
```

```
validator_factory.create_mock_validator(mock_llm_response: Optional = None,  
mock_web_response: Optional = None)
```

```
```
```

```
Parameters:
```

| Parameter           | Type       | Required | Default |
|---------------------|------------|----------|---------|
| `mock_llm_response` | `Optional` | No       | `None`  |
| `mock_web_response` | `Optional` | No       | `None`  |

```
Returns: `Validator`
```

```

```

```
`validator_factory.batch_create_mock_validators`
```

```
```python
```

```
validator_factory.batch_create_mock_validators(count: int, mock_llm_response: Optional =  
None, mock_web_response: Optional = None)
```

```
```
```

```
Parameters:
```

| Parameter           | Type       | Required | Default |
|---------------------|------------|----------|---------|
| `count`             | `int`      | Yes      |         |
| `mock_llm_response` | `Optional` | No       | `None`  |
| `mock_web_response` | `Optional` | No       | `None`  |

```
Returns: `List`
```

```

```

## Validator

Validator(stake: int, provider: str, model: str, config: Dict[str, Any], plugin: str, plugin\_config: Dict[str, Any], mock\_enabled: bool, mock\_llm\_response: Optional[gltest.types.MockedLLMResponse], mock\_web\_response: Optional[gltest.types.MockedWebResponse])

### `validator.to\_dict`

**\*\*Returns:\*\*** `Dict`

---

### `validator.clone`

**\*\*Returns:\*\*** `Validator`

---

### `validator.batch\_clone`

```
```python
validator.batch_clone(count: int)
```
```

**\*\*Parameters:\*\***

| Parameter | Type  | Required | Default |
|-----------|-------|----------|---------|
| `count`   | `int` | Yes      |         |

**\*\*Returns:\*\*** `List`

---

---

# Direct Mode Testing

Source: <https://docs.genlayer.com/api-references/genlayer-test/direct>

Run contracts directly in Python without a network. Provides Foundry-style cheatcodes for fast test execution.

## Setup Functions

### `deploy\_contract`

Deploy a contract and return an instance.

```
```python
deploy_contract(contract_path: Path, vm: 'VMContext', args: Any, sdk_version: Optional[str] = None, kwargs: Any)
```
```

**\*\*Parameters:\*\***

| Parameter       | Type            | Required | Default |
|-----------------|-----------------|----------|---------|
| `contract_path` | `Path`          | Yes      |         |
| `vm`            | `VMContext`     | Yes      |         |
| `args`          | `Any`           | Yes      |         |
| `sdk_version`   | `Optional[str]` | No       | `None`  |
| `kwargs`        | `Any`           | Yes      |         |

**\*\*Returns:\*\*** `Any`

```

`load_contract_class`

Load a contract class from file.

Sets up SDK paths, WASI mock, and message context.

```python
load_contract_class(contract_path: Path, vm: 'VMContext', sdk_version: Optional[str] = None)
```

Parameters:

| Parameter | Type | Required | Default |
|-----|-----|-----|-----|
| `contract_path` | `Path` | Yes | |
| `vm` | `VMContext` | Yes | |
| `sdk_version` | `Optional[str]` | No | `None` |

Returns: `Type[Any]`

`create_address`

Create a deterministic address from seed string.

```python
create_address(seed: str)
```

Parameters:

| Parameter | Type | Required | Default |
|-----|-----|-----|-----|
| `seed` | `str` | Yes | |

Returns: `Any`

`create_test_addresses`

Create a list of test addresses.

```python
create_test_addresses(count: int = 10)
```

Parameters:

| Parameter | Type | Required | Default |
|-----|-----|-----|-----|
| `count` | `int` | No | `10` |

Returns: `list`

VMContext

Test VM context providing Foundry-style cheatcodes.

Usage:
 vm = VMContext()
 vm.sender = Address("0x" + "a" * 40)
 vm.mock_web("api.example.com", {"status": 200, "body": "{}"})

```



```

 with vm.activate():
 contract = deploy_contract("Token.py", vm, owner)
 contract.transfer(bob, 100)

`vm.activate`

Activate this VM context for contract execution.
Uses proper cleanup via ExitStack for resource management.

Patches datetime.datetime so that datetime.now() returns the
warped time set via vm.warp(). This is dynamic: calling warp()
mid-test updates _datetime and subsequent now() calls reflect it.

`vm.warp`

Set block timestamp (ISO format).

```python
vm.warp(timestamp: str)
```

Parameters:

| Parameter | Type | Required | Default |
|-------------|-------|----------|---------|
| `timestamp` | `str` | Yes | |

Returns: `None`

`vm.deal`

Set balance for an address.

```python
vm.deal(address: Any, amount: int)
```

Parameters:

| Parameter | Type | Required | Default |
|-----------|-------|----------|---------|
| `address` | `Any` | Yes | |
| `amount` | `int` | Yes | |

Returns: `None`

`vm.snapshot`

Take a snapshot of current state. Returns snapshot ID.

Returns: `int`

`vm.revert`

Revert to a previous snapshot.

```

```
```python
vm.revert(snapshot_id: int)
```
```

**\*\*Parameters:\*\***

| Parameter     | Type  | Required | Default |
|---------------|-------|----------|---------|
| `snapshot_id` | `int` | Yes      |         |

**\*\*Returns:\*\*** `None`

---

### `vm.mock\_web`

Mock web requests matching URL pattern.

```
```python
vm.mock_web(url_pattern: str, response: MockedWebResponseData)
```
```

**\*\*Parameters:\*\***

| Parameter     | Type                    | Required | Default |
|---------------|-------------------------|----------|---------|
| `url_pattern` | `str`                   | Yes      |         |
| `response`    | `MockedWebResponseData` | Yes      |         |

**\*\*Returns:\*\*** `None`

---

### `vm.mock\_llm`

Mock LLM prompts matching pattern.

```
```python
vm.mock_llm(prompt_pattern: str, response: str)
```
```

**\*\*Parameters:\*\***

| Parameter        | Type  | Required | Default |
|------------------|-------|----------|---------|
| `prompt_pattern` | `str` | Yes      |         |
| `response`       | `str` | Yes      |         |

**\*\*Returns:\*\*** `None`

---

### `vm.clear\_mocks`

Clear all registered mocks.

**\*\*Returns:\*\*** `None`

---

### `vm.prank`

Context manager to temporarily change sender.

```
```python
```

```
vm.prank(address: Any)
```
```

```
Parameters:
```

| Parameter | Type  | Required | Default |
|-----------|-------|----------|---------|
| `address` | `Any` | Yes      |         |

```

```

```
`vm.startPrank`
```

Start pranking as address (persists until stopPrank).

```
```python
vm.startPrank(address: Any)
```
```

```
Parameters:
```

| Parameter | Type  | Required | Default |
|-----------|-------|----------|---------|
| `address` | `Any` | Yes      |         |

```
Returns: `None`
```

```

```

```
`vm.stopPrank`
```

Stop the current prank.

```
Returns: `None`
```

```

```

```
`vm.expect_revert`
```

Context manager expecting the next call to revert.

Catches ContractRollback (gl.rollback) and any Exception raised by contract code (ValueError, RuntimeError, etc.). If *message* is given, the exception text must contain it.

```
```python
vm.expect_revert(message: Optional[str] = None)
```
```

```
Parameters:
```

| Parameter | Type            | Required | Default |
|-----------|-----------------|----------|---------|
| `message` | `Optional[str]` | No       | `None`  |

```

```

```
`vm.run_validator`
```

Run a captured validator function from a prior run\_nondet call.

Each ``gl.vm.run\_nondet`` call in a contract appends an entry to an internal list. Use *index* to select which one (default -1, the most recent).

Mocks still apply: the validator typically re-runs `leader_fn` internally, which hits the current web/LLM mocks. Swap mocks between the contract call and `run_validator()` to simulate the validator seeing different external data.

Args:

- `leader_result`: Override the leader's return value.
- `leader_error`: Simulate a leader exception (`gl.vm.UserError`).
- `index`: Which captured validator to run (-1 = last).

Returns:

The bool returned by the validator function.

```
```python
```

```
vm.run_validator(leader_result: Any = <object object at 0x7f3bdfd04ad0>, leader_error: Optional[Exception] = None, index: int = -1)
```

Parameters:

| Parameter | Type | Required | Default |
|----------------------------|----------------------------------|----------|--|
| <code>leader_result</code> | <code>Any</code> | No | <code><object object at 0x7f3bdfd04ad0></code> |
| <code>leader_error</code> | <code>Optional[Exception]</code> | No | <code>None</code> |
| <code>index</code> | <code>int</code> | No | <code>-1</code> |

Returns: `bool`

`vm.clear_validators`

Clear the captured validator list.

Returns: `None`

`glsim` – Local GenLayer Network

Source: <https://docs.genlayer.com/api-references/genlayer-test/glsim>

A lightweight single-process GenLayer network that runs contracts via direct mode. Supports real LLM and web calls without Docker or WASM.

Installation

```
```bash
```

```
pip install genlayer-test[sim]
```

## Usage

```
```bash
```

```
# Start with defaults (port 4000, 5 validators)
glsim
```

```
# Custom configuration
```

```
glsim --port 8000 --validators 3 --llm-provider openai:gpt-4o
```

```
# Deterministic mode (reproducible addresses)
```

```
glsim --seed my-test-seed
```

CLI Options

| Option | Default | Description |
|------------------------------|-----------|--|
| <code>--port</code> | 4000 | RPC server port |
| <code>--host</code> | 127.0.0.1 | Bind address |
| <code>--validators</code> | 5 | Number of validators |
| <code>--max-rotations</code> | 3 | Max leader rotations |
| <code>--chain-id</code> | 61127 | Network chain ID |
| <code>--llm-provider</code> | None | LLM provider:model (e.g., <code>openai:gpt-4o</code>) |
| <code>--no-browser</code> | false | Use httpx instead of Playwright for web |
| <code>--seed</code> | None | Deterministic seed for addresses |
| <code>-v, --verbose</code> | false | Verbose logging |

JSON-RPC Methods

glsim exposes a JSON-RPC 2.0 endpoint at `POST /api` compatible with both GenLayer Studio and standard Ethereum methods:

GenLayer Methods

- `gen_call` - Call a contract method
- `gen_get_contract_schema` - Get contract schema
- `gen_get_contract_schema_for_code` - Get schema from code
- `gen_get_transaction_status` - Get transaction status

Simulator Methods

- `sim_deploy` - Deploy a contract
- `sim_call` - Call a contract method
- `sim_read` - Read contract state
- `sim_fund_account` - Fund an account
- `sim_get_balance` - Get account balance
- `sim_create_snapshot` - Create state snapshot
- `sim_restore_snapshot` - Restore snapshot
- `sim_install_mock` - Install web/LLM mocks
- `sim_get_mock` - Get current mocks
- `sim_increase_time` - Advance time (Anvil-style)
- `sim_set_time` - Set time to specific datetime

Ethereum Compatible

- `eth_chainId`, `eth_blockNumber`, `eth_getBalance`
- `eth_getTransactionReceipt`, `eth_getTransactionByHash`
- `eth_estimateGas`, `eth_gasPrice`, `eth_sendRawTransaction`

GenLayer Node API

Source: <https://docs.genlayer.com/api-references/genlayer-node>

The GenLayer Node provides a [JSON-RPC API](<https://www.jsonrpc.org/specification>) for interacting with it. This API allows you to execute contract calls, retrieve transaction information, and perform various blockchain operations.

GenLayer Methods

gen_call

Executes a new message call immediately without creating a transaction on the blockchain. This method supports read operations, write operations, and contract deployments.

****Method:**** `gen_call`

****Parameters:****

- `request` (object, required): The contract call request parameters
 - `from` (string, required): The address making the call

- ``to`` (string, required): The contract address to call
- ``data`` (string, required): The encoded function call data
- ``type`` (string, required): The type of call (``"read"``, ``"write"``, or ``"deploy"``)
- ``blockNumber`` (string, optional): The block number to query at (hex-encoded). If not provided, uses latest block
- ``status`` (string, optional): The transaction status filter (``"finalized"`` or ``"accepted"``). If not provided, uses accepted state
- ``value`` (string, optional): The value to send with the call (hex-encoded)
- ``leader_results`` (array of strings or null, optional): When provided, GenVM runs in validator mode (executing ``validator_fn`` instead of ``leader_fn`` in ``run_nondet`` calls). Each element is a hex-encoded equivalence output from a previous leader execution. When omitted or null, GenVM runs in leader mode

****Returns:**** A structured JSON object containing execution results and debug information

****Response Fields:****

- ``data`` (string): Hex-encoded return value from contract execution
- ``eqOutputs`` (array of strings): Equivalence outputs from GenVM execution, each as a hex-encoded string. These are deterministic outputs used for consensus validation. Populated in leader mode; empty in validator mode
- ``status`` (object): Execution status
 - ``code`` (number): Status code (0=success, 1=user error, 2=vm error, 3=internal error)
 - ``message`` (string): Human-readable status message
- ``stdout`` (string): Standard output captured during execution
- ``stderr`` (string): Standard error captured during execution
- ``logs`` (array): GenVM log entries, each as a JSON object with fields like ``level``, ``message``, ``target``, ``file``, ``ts``
- ``events`` (array): Contract events emitted during execution
 - ``topics`` (array of strings): Event topics as hex-encoded strings
 - ``data`` (string): Event data as hex-encoded string
- ``messages`` (array): Messages emitted by the contract
 - ``messageType`` (number): Type identifier for the message
 - ``recipient`` (string): Recipient address
 - ``value`` (string): Value in hex format
 - ``data`` (string): Message data as hex-encoded string
 - ``onAcceptance`` (boolean): Whether message is sent on acceptance
 - ``saltNonce`` (string): Salt nonce in hex format
- ``nondetDisagreementCallNo`` (number or null): When running in validator mode (with ``leader_results``), indicates which ``run_nondet`` call the validator disagreed on. Null when there is no disagreement or when running in leader mode

****Example - Deploy Call:****

```

```json
{
 "jsonrpc": "2.0",
 "method": "gen_call",
 "params": [
 {
 "type": "deploy",
 "data":
"0xf9023db90223232076302e312e300a23207b2022446570656e6473223a202270792d67656e6c617965723a313
5716669766a7679383038303072683939387063786d64326d38766131777132717a71687a3835306e38676763723
46939713022207d0a0a66726f6d2067656e6c6179657220696d706f7274202a0a0a2320636f6e7472616374206
36c6173730a636c6173732053746f7261676528676c2e436f6e7472616374293a0a2020202073746f726167653a2
07374720a0a202020202320636f6e7374727563746f720a20202020646566205f5f696e69745f5f2873656c662c2
0696e697469616c5f73746f726167653a20737472293a0a202020202020202073656c662e73746f72616765203d2
0696e697469616c5f73746f726167650a0a20202020232072656164206d6574686f6473206d75737420626520616
e6e6f7461746564207769746820766965770a2020202040676c2e7075626c69632e766965770a202020206465662
06765745f73746f726167652873656c6629202d3e207374723a0a20202020202020202072657475726e2073656c662
e73746f726167650a0a2020202023207772697465206d6574686f640a2020202040676c2e7075626c69632e77726
974650a20202020646566207570646174655f73746f726167652873656c662c206e65775f73746f726167653a207
3747229202d3e204e6f6e653a0a202020202020202073656c662e73746f72616765203d206e65775f73746f72616
7650a950e04617267730d6c696e697469616c2076616c756500",
]
]
}
```

```

```

        "from": "0xf6A60752C24Cd3BFcAf8d387a5EB62d3746F44EE",
        "to": "0x0000000000000000000000000000000000000000",
        "status": "finalized"
    }
},
    "id": 1
}

```

****Example Response (Deploy Success):****

```

```json
{
 "jsonrpc": "2.0",
 "result": {
 "data": "00",
 "eqOutputs": [],
 "status": {
 "code": 0,
 "message": "success"
 },
 "stdout": "",
 "stderr": "",
 "logs": [
 {
 "file": "common/src/lib.rs:111",
 "level": "info",
 "message": "logging initialized",
 "target": "genvm_common",
 "ts": "1770221913210",
 "version": "v0.2.11-aarch64-macos-release"
 },
 {
 "file": "src/exe/run.rs:150",
 "genvm_id": "1",
 "level": "info",
 "message": "genvm id",
 "target": "genvm::exe::run",
 "ts": "1770221913212"
 },
 {
 "file": "src/lib.rs:299",
 "level": "info",
 "message": "metrics",
 "metrics": {
 "gvm": {
 "host": { "time": 654 },
 "llm_module": { "calls": 0, "time": 0 },
 "supervisor": { "compilation_time": 0, "compiled_modules": 0, "precompile_hits":
2 },
 "web_module": { "calls": 0, "time": 0 }
 },
 "llm": null,
 "web": null
 },
 "target": "genvm",
 "ts": "1770221913590"
 }
],
 "events": [],
 "messages": [],
 "nondetDisagreementCallNo": null
 },
 "id": 1
}

```





response includes the messages array:

```
```json
{
  "jsonrpc": "2.0",
  "result": {
    "data": "00",
    "eqOutputs": [],
    "status": {
      "code": 0,
      "message": "success"
    },
    "stdout": "",
    "stderr": "",
    "logs": [
      {
        "file": "src/lib.rs:299",
        "level": "info",
        "message": "metrics",
        "target": "genvm",
        "ts": "1770225770000"
      }
    ],
    "events": [],
    "messages": [
      {
        "messageType": 1,
        "recipient": "0xc346f80c86adf1d71accfc4ef6efe7482b25be40",
        "value": "0x0",
        "data": "0xf86eb86a16046172677331d4023078393164463338654243623930384335643635...",
        "onAcceptance": false,
        "saltNonce": "0x0"
      },
      {
        "messageType": 1,
        "recipient": "0xad26513a4315b0ea8a5c800322b35092a83dfd3d",
        "value": "0x0",
        "data": "0xf86eb86a16046172677331d4023078333738653339374533316346383538333630...",
        "onAcceptance": false,
        "saltNonce": "0x0"
      }
    ],
    "nondetDisagreementCallNo": null
  },
  "id": 1
}
```
```

**\*\*Notes:\*\***

- The ``blockNumber`` parameter can be either hex-encoded with a ``0x`` prefix (e.g., ``"0x151ec5"``) or decimal (e.g., ``"1384133"``)
- Valid ``status`` values are ``"accepted"`` (default) and ``"finalized"``
- When both parameters are omitted, the call executes against the latest accepted state
- For ``read`` and ``write`` type calls, if the contract doesn't exist at the specified block/status combination, an error is returned with code ``-32001`` (Resource not found) and message ``"contract not found at address 0x..."``
- For ``deploy`` type calls, the ``status`` and ``blockNumber`` parameters are not used for contract state validation since no contract exists yet

### gen\_getContractSchema

Retrieve the schema/interface of a GenLayer contract to understand its available methods and properties.

```
```json
{
  "jsonrpc": "2.0",
  "result": {
    "ctor": {
      "kwparams": {},
      "params": [["initial_storage", "string"]]
    },
    "methods": {
      "get_storage": {
        "kwparams": {},
        "params": [],
        "readonly": true,

```

```

        "ret": "string"
    },
    "update_storage": {
        "kwparams": {},
        "params": [["new_storage", "string"]],
        "payable": false,
        "readonly": false,
        "ret": "null"
    }
}
},
"id": 1
}
},
}

```

gen_getContractState

Retrieves the current state of a GenLayer intelligent contract at a specific block height and transaction status.

****Method:**** `gen\_getContractState`

****Parameters:****

- `request` (object, required): The contract state request parameters
 - `address` (string, required): The contract address to query
 - `blockNumber` (string, optional): The block number to query at (hex-encoded). If not provided, uses latest block
 - `status` (string, optional): The transaction status filter (`"finalized"` or `"accepted"`). If not provided, uses accepted state

****Returns:**** Hex-encoded contract state data

****Example Request - Latest Accepted State:****

```

```json
{
 "jsonrpc": "2.0",
 "method": "gen_getContractState",
 "params": [
 {
 "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f"
 }
],
 "id": 1
}
```

```

****Example Request - Specific Block and Status:****

```

```json
{
 "jsonrpc": "2.0",
 "method": "gen_getContractState",
 "params": [
 {
 "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f",
 "blockNumber": "0x151ec5",
 "status": "accepted"
 }
],
 "id": 1
}
```

```

****Example Response:****

```
```json
{
 "jsonrpc": "2.0",
 "result": "0xd9960e0773746f726167656c6e757064617465642074657374207374",
 "id": 1
},..
```

## **\*\*Notes:\*\***

- The ``blockNumber`` parameter should be hex-encoded with a ``0x`` prefix
- Valid ``status`` values are ``"accepted"`` (default) and ``"finalized"``
- The returned state is hex-encoded binary data with a ``0x`` prefix
- Empty contract state returns ``"0x"``
- If the contract doesn't exist at the specified block/status, an error will be returned

## **\*\*cURL Example:\*\***

```
```bash
# Get latest accepted state
curl -X POST http://localhost:9151 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "gen_getContractState",
    "params": [{
      "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f"
    }],
    "id": 1
  }'

# Get state at specific block with accepted status
curl -X POST http://localhost:9151 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "gen_getContractState",
    "params": [{
      "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f",
      "blockNumber": "0x151ec5",
      "status": "accepted"
    }],
    "id": 1
  },..
```

gen_getContractCode

Retrieves the code of a deployed GenLayer intelligent contract at a specific block height and transaction status.

****Method:**** ``gen_getContractCode``

****Parameters:****

- ``request`` (object, required): The contract code request parameters
 - ``address`` (string, required): The contract address to query
 - ``blockNumber`` (string, optional): The block number to query at (hex-encoded). If not provided, uses latest block
 - ``status`` (string, optional): The transaction status filter (``"finalized"`` or ``"accepted"``). If not provided, uses accepted state

****Returns:**** Base64-encoded contract code

****Example Request - Latest Accepted State:****

```
```json
{
 "jsonrpc": "2.0",
 "method": "gen_getContractCode",
 "params": [
 {
 "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f"
 }
],
 "id": 1
},
...
```

**\*\*Example Request - Specific Block and Status:\*\***

```
```json
{
  "jsonrpc": "2.0",
  "method": "gen_getContractCode",
  "params": [
    {
      "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f",
      "blockNumber": "0x151ec5",
      "status": "finalized"
    }
  ],
  "id": 1
},
...
```

****Example Response:****

```
```json
{
 "jsonrpc": "2.0",
 "result":
 "IyB7ICJEZXB1bmRzIjogInB5LWdlbmXheWVyOnRlc3QiIH0KcmZyb20gZ2VubGF5ZXIgaW1wb3J0ICoKCgpjbGFzcyB
 TdG9yYWdlKGdsLkNvbnRyYWN0KToKICAgIHNOb3JhZ2U6IHN0cgoKICAgIGRlZiBfX2luaXRfXyhzZWxmLCBpbm10aWF
 sX3N0b3JhZ2U6IHN0cik6CiAgICAgICAgc2VsZi5zdG9yYWdlID0gaW5pdGlhbF9zdG9yYWdlCgogICAgQGdsLnB1Ymx
 pYy52aWV3CiAgICBkZWYgZ2V0X3N0b3JhZ2Uoc2VsZikgLT4gc3RyOgogICAgICAgIHJldHVybiBzZWxmLnN0b3JhZ2U
 KCiAgICBAZ2wucHVibGljLndyaXRlCiAgICBkZWYgdXBkYXRlX3N0b3JhZ2Uoc2VsZiwgbmV3X3N0b3JhZ2U6IHN0cik
 gLT4gTm9uZToKICAgICAgICBzZWxmLnN0b3JhZ2UgPSBuZXdfc3RvcnFnZQo=",
 "id": 1
},
...
```

**\*\*Notes:\*\***

- The `blockNumber` parameter should be hex-encoded with a `0x` prefix
- Valid `status` values are `"accepted"` (default) and `"finalized"`
- The returned code is base64-encoded
- If the contract doesn't exist at the specified block/status, an error will be returned
- If the address does not have code (not a contract), an error will be returned

**\*\*cURL Example:\*\***

```
```bash
# Get latest accepted code
curl -X POST http://localhost:9151 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",

```

```

    "method": "gen_getContractCode",
    "params": [{
        "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f"
    }],
    "id": 1
}'

```

Get code at specific block with finalized status

```

curl -X POST http://localhost:9151 \
-H "Content-Type: application/json" \
-d '{
    "jsonrpc": "2.0",
    "method": "gen_getContractCode",
    "params": [{
        "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f",
        "blockNumber": "0x151ec5",
        "status": "finalized"
    }],
    "id": 1
}'
..

```

gen_getTransactionReceipt

Retrieves the transaction receipt for a given transaction ID. This endpoint is a proxy to the underlying consensus layer's `getTransactionAllData` function, providing detailed information about a transaction that has been processed.

****Method:**** `gen\_getTransactionReceipt`

****Parameters:****

- `request` (object, required):
 - `txId` (`string`, required): The 32-byte transaction hash as a hexadecimal string with a `0x` prefix.

****Returns:****

A `Transaction` object which contains the details of the transaction. The structure is based on `ITransactionsTransaction` from the consensus contracts. The object has the following fields:

- `id` (`string`): The transaction ID as a hex string.
- `txOrigin` (`address`): The original address that initiated the transaction.
- `sender` (`address`): The address of the sender.
- `recipient` (`address`): The destination address of the transaction.
- `activator` (`address`): The address of the validator that activated the transaction.
- `status` (`uint8`): The current status of the transaction.
- `previousStatus` (`uint8`): The previous status of the transaction.
- `txSlot` (`uint256`): The transaction slot number.
- `initialRotations` (`uint256`): The initial number of rotations.
- `numOfInitialValidators` (`uint256`): The number of initial validators.
- `randomSeed` (`string`): The random seed used for the transaction as a hex string.
- `txExecutionHash` (`string`): The execution hash of the transaction as a hex string.
- `txCallData` (`string`): The transaction call data as a hex string.
- `eqBlocksOutputs` (`string`): The equivalent blocks outputs as a hex string.
- `timestamps` (object): Object containing various timestamps:
 - `Created` (`uint256`): When the transaction was created.
 - `Pending` (`uint256`): When the transaction became pending.
 - `Activated` (`uint256`): When the transaction was activated.
 - `Proposed` (`uint256`): When the transaction was proposed.
 - `Committed` (`uint256`): When the transaction was committed.
 - `LastVote` (`uint256`): When the last vote was cast.
 - `AppealSubmitted` (`uint256`): When an appeal was submitted.
 - `LeaderRevealed` (`uint256`): When the leader was revealed.

- `result` (`uint256`): The result of the transaction execution.
- `readStateBlockRanges` (`array`): Array of block range objects, each containing:
 - `ActivationBlock` (`uint256`): The activation block number.
 - `ProcessingBlock` (`uint256`): The processing block number.
 - `ProposalBlock` (`uint256`): The proposal block number.
- `roundData` (`array`): Array of round data objects containing:
 - `round` (`uint256`): The round number.
 - `leaderIndex` (`uint256`): Index of the round leader.
 - `votesCommitted` (`uint256`): Number of votes committed.
 - `votesRevealed` (`uint256`): Number of votes revealed.
 - `appealBond` (`uint256`): The appeal bond amount.
 - `rotationsLeft` (`uint256`): Number of rotations remaining.
 - `result` (`uint256`): The round result.
 - `roundValidators` (`array`): Array of validator addresses for this round.
 - `validatorVotes` (`string`): Base64-encoded validator votes.
 - `validatorVotesHash` (`array`): Array of vote hashes as hex strings.
 - `validatorResultHash` (`array`): Array of result hashes as hex strings.

****Example Request:****

```

```json
{
 "jsonrpc": "2.0",
 "method": "gen_getTransactionReceipt",
 "params": [
 {
 "txId": "0x635060dd514082096d18c8eb64682cc6a944f9ce1ae6982febf7a71e9f656f49"
 }
],
 "id": 1
}
```

```

****Example Response:****

```

```json
{
 "jsonrpc": "2.0",
 "id": 1,
 "result": {
 "id": "0x635060dd514082096d18c8eb64682cc6a944f9ce1ae6982febf7a71e9f656f49",
 "txOrigin": "0x5b70759760a85d92bf972b67ee8558e8e8da87d4",
 "sender": "0x5b70759760a85d92bf972b67ee8558e8e8da87d4",
 "recipient": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f",
 "activator": "0xf27b07d36ff5b9dcd37fd3a09811d6456dd4483a",
 "status": 14,
 "previousStatus": 0,
 "txSlot": 1,
 "initialRotations": 1,
 "numOfInitialValidators": 5,
 "randomSeed": "0x3d0bdf63bce1cf84b155f1557bb15e303b903b1b559325b6492794a5e498954",
 "txExecutionHash": "0x76744dc491b1725eb2ba08cf516855c14aac310e36711368c04442fd39d61853",
 "txCallData":
 "f5b31604617267730da4017570646174656420746573742073746f72616765066d6574686f64747570646174655
 f73746f7261676500",
 "eqBlocksOutputs": "c0",
 "timestamps": {
 "Created": 1750774777,
 "Pending": 1750774777,
 "Activated": 1750774781,
 "Proposed": 1750774786,
 "Committed": 1750774791,
 "LastVote": 0,
 "AppealSubmitted": 0,
 "LeaderRevealed": 0
 }
 }
}
```

```


| | Field | Type | Description |
|-------------------|--------------|--------|---|
| ----- ----- ----- | `status` | string | Human-readable status name (e.g., "FINALIZED", "ACCEPTED", "PENDING") |
| | `statusCode` | uint8 | Numeric status code |

Status Codes:

| Code | Status |
|------|--------------------|
| 0 | UNINITIALIZED |
| 1 | PENDING |
| 2 | PROPOSING |
| 3 | COMMITTING |
| 4 | REVEALING |
| 5 | ACCEPTED |
| 6 | UNDETERMINED |
| 7 | FINALIZED |
| 8 | CANCELED |
| 9 | APPEAL_REVEALING |
| 10 | APPEAL_COMMITTING |
| 11 | READY_TO_FINALIZE |
| 12 | VALIDATORS_TIMEOUT |
| 13 | LEADER_TIMEOUT |

Example Request:

```

```json
{
 "jsonrpc": "2.0",
 "method": "gen_getTransactionStatus",
 "params": [
 {
 "txId": "0x563f046c187d711127c51213ca62e2e4fee52009a98f0989a73a0a0382d21890"
 }
],
 "id": 1
}
```

```

Example Response:

```

```json
{
 "jsonrpc": "2.0",
 "result": {
 "status": "FINALIZED",
 "statusCode": 7
 },
 "id": 1
}
```

```

Notes:

- This endpoint queries the consensus contract on-chain via `getTransactionData`
- Use this for polling – it's cheaper than `gen\_getTransactionReceipt` which returns the full receipt
- The `timestamp` parameter is passed to the on-chain call and affects how the contract reports the status

gen_syncing

Returns the GenVM sync status, indicating how far the node's local GenVM state is behind the chain tip.

****Method:**** `gen\_syncing`

****Parameters:**** None

****Returns:**** Object with sync status information

****Response Fields:****

| Field | Type | Description |
|----------------|--------|---|
| `syncedBlock` | string | Hex-encoded block number that GenVM has synced to |
| `latestBlock` | string | Hex-encoded latest block number on the chain |
| `blocksBehind` | number | Number of blocks GenVM is behind the chain tip (0 = fully synced) |

****Example Request:****

```
```json
{
 "jsonrpc": "2.0",
 "method": "gen_syncing",
 "params": [],
 "id": 1
}
```

**\*\*Example Response (Synced):\*\***

```
```json
{
  "jsonrpc": "2.0",
  "result": {
    "syncedBlock": "0x151ec5",
    "latestBlock": "0x151ec5",
    "blocksBehind": 0
  },
  "id": 1
}
```

****Example Response (Behind):****

```
```json
{
 "jsonrpc": "2.0",
 "result": {
 "syncedBlock": "0x151e00",
 "latestBlock": "0x151ec5",
 "blocksBehind": 197
 },
 "id": 1
}
```

**\*\*Notes:\*\***

- This endpoint always returns an object (never `false` like `eth\_syncing`)
- Use `blocksBehind == 0` to determine if the node is fully synced
- When the node is catching up, `gen\_call` requests without an explicit `blockNumber` will serve data at the `syncedBlock` rather than waiting for full sync

## ## Debug Methods

These methods are available for debugging and testing purposes during development.

### ### gen\_dbg\_ping

Simple connectivity test method that returns a "pong" response.

**\*\*Method:\*\*** `gen\_dbg\_ping`

**\*\*Parameters:\*\*** None

**\*\*Returns:\*\*** "pong" string

**\*\*Example:\*\***

```
```json
{
  "jsonrpc": "2.0",
  "method": "gen_dbg_ping",
  "params": [],
  "id": 1
},,,
```

****Response:****

```
```json
{
 "jsonrpc": "2.0",
 "result": "pong",
 "id": 1
},,,
```

### ### gen\_dbg\_trie

Get trie information for debugging node internal DB state.

**\*\*Method:\*\*** `gen\_dbg\_trie`

**\*\*Parameters:\*\***

- `request` (object, required): The trie debug request
- `txID` (string, required): Transaction ID (hex-encoded)
- `round` (integer, required): Round number

**\*\*Returns:\*\*** Trie debug information string

**\*\*Example:\*\***

```
```json
{
  "jsonrpc": "2.0",
  "method": "gen_dbg_trie",
  "params": [
    {
      "txID": "0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b6742d35Cc6634C0532925a3b8",
      "round": 0
    }
  ],
  "id": 1
},,,
```

****Response:****

```
```json
{
```



```

 "file": "src/lib.rs:82",
 "level": "error",
 "message": "vm error",
 "error": {"causes": ["VMError(invalid_contract absent_runner_comment)"]}
 }
],
"storage_proof": "0x99ff0d9125e1fc9531a11262e15aeb2c60509a078c4cc4c64cefdfb06ff68647",
"run_time": "0s",
"eq_outputs": []
},
"id": 1
}
...

```

#### **\*\*Notes:\*\***

- If the trace isn't stored locally (e.g., the node restarted or data was pruned), the endpoint will replay the transaction automatically.
- This endpoint is in the `genlayer\_debug` group and must be enabled in the node configuration.
- Use `result\_code` to determine execution outcome: 0 = contract returned successfully, 1 = contract raised a user error, 2 = GenVM encountered an error.

#### **## Ethereum Compatibility**

The GenLayer Node also supports Ethereum-compatible methods that are proxied to the underlying infrastructure. These methods follow the standard [Ethereum JSON-RPC specification](https://ethereum.org/en/developers/docs/apis/json-rpc/) and are prefixed with `eth\_`.

#### **\*\*Examples of supported Ethereum methods:\*\***

- `eth\_blockNumber`
- `eth\_getBalance`
- `eth\_sendTransaction`
- `eth\_call`
- And other standard Ethereum JSON-RPC methods

#### **## zkSync Compatibility**

[zkSync-compatible](https://docs.zksync.io/zksync-protocol/api/zks-rpc) methods are also supported and proxied to the underlying infrastructure. These methods are prefixed with `zksync\_`.

#### **## Usage Examples**

##### **### cURL**

```

```bash
# Test connectivity
curl -X POST http://localhost:9151 \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "gen_dbg_ping",
  "params": [],
  "id": 1
}'

```

```

# Execute a contract call
curl -X POST http://localhost:9151 \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "method": "gen_call",

```

```

    "params": [{
      "from": "0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b6",
      "to": "0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b6",
      "data": "0x70a08231000000000000000000000000742d35cc6634c0532925a3b8d4c9db96c4b4d8b6",
      "type": "read",
      "transaction_hash_variant": "latest-nonfinal"
    }],
    "id": 1
  }'

# Get contract schema
curl -X POST http://localhost:9151 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "gen_getContractSchema",
    "params": [{
      "code":
"IyB7ICJEZXB1bmRzIjogInB5LWdlbmVwYyOnRlc3QiIH0KcmZyb20gZ2VubGF5ZXIgaW1wb3J0ICoKCgojIGNvbnRyYWN0IGNsYXNzCmNsYXNzIFN0b3JhZ2UoZ2wuQ29udHJhY3QpOgogICAgc3RvcnFnZTogc3RyCgogICAgIyBjb25zdHJ1Y3RvcgogICAgZGVmIF9faw5pdF9fKHNLbGYsIGluaXRpYWxf3RvcnFnZTogc3RyKToKICAgICAgICBzZWxlnN0b3JhZ2UgPSBpbm10aWFsX3N0b3JhZ2UKCiAgICAgIyBjb25zdHJ1Y3RvcgogICAgZGVmIGdlbF9zdG9yYWdlKHNLbGYpIC0+IHN0cjoKICAgICAgICBzZXR1cm4gc2VsZi5zdG9yYWdlCgogICAgIyB3cm10ZSBtZXRob2QKICAgIEBnbC5wdWJsawMud3JpdGUKICAgIGRlZiB1cGRhdGVfc3RvcnFnZShzZWxlcCBuZXdfc3RvcnFnZTogc3RyKSAtpiB0b25l0gogICAgICAgIHNlbGYuc3RvcnFnZSA9IG5ld19zdG9yYWdlCg=="
    }],
    "id": 1
  }'

# Get debug trie information
curl -X POST http://localhost:9151 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "gen_dbg_trie",
    "params": [{
      "txID": "0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b6742d35Cc6634C0532925a3b8",
      "round": 0
    }],
    "id": 1
  }'

# Get transaction receipt
curl -X POST http://localhost:9151 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "gen_getTransactionReceipt",
    "params": [{
      "txId": "0x635060dd514082096d18c8eb64682cc6a944f9ce1ae6982febf7a71e9f656f49"
    }],
    "id": 1
  }'
..

```

Ops Methods

These methods provide operational endpoints for monitoring the GenLayer node.

balance

Returns the configured operator's account balance in multiple formats including hexadecimal, wei, and GEN units.

****Method:**** `GET /balance`

****Parameters:**** None

****Returns:**** JSON object containing the operator's balance in three formats

****Example Request:****

```
```bash
curl -X GET http://localhost:9153/balance
```
```

****Example Response:****

```
```json
{
 "balance_hex": "0x0de0b6b3a7640000",
 "balance_wei": "1000000000000000000",
 "balance_gen": "1.000"
}
```
```

****Response Fields:****

- `balance\_hex` (string): The balance encoded as hexadecimal with "0x" prefix
- `balance\_wei` (string): The balance in wei as a decimal string
- `balance\_gen` (string): The balance in GEN units formatted with 3 decimal places

****Notes:****

- This endpoint is only available when the node is running in validator mode
- The balance is always retrieved from the latest block
- The operator address is configured at node startup
- If the operator address is not configured, the zero address (0x0000...0000) is used
- Balance conversions: 1 GEN = 1,000,000,000,000,000,000 wei (10^{18} wei)
- Returns HTTP 503 if the node is not in validator mode
- Returns HTTP 500 if there's an error retrieving the balance

****Example Response (Zero Balance):****

```
```json
{
 "balance_hex": "0x",
 "balance_wei": "0",
 "balance_gen": "0.000"
}
```
```

****Example Response (Large Balance):****

```
```json
{
 "balance_hex": "0x3635c9adc5dea00000",
 "balance_wei": "1000000000000000000",
 "balance_gen": "1000.000"
}
```
```

****Error Response (Not in Validator Mode):****

```
```json
{
 "message": "Balance endpoint not available in rollup mode"
}
```
```

****cURL Example with Headers:****

```
```bash
curl -X GET http://localhost:9153/balance \
 -H "Accept: application/json" \
 -v
```
```

health

Returns the health status of the GenLayer node along with version information and the status of individual health checks.

****Method:**** `GET /health`

****Parameters:**** None

****Returns:**** JSON object containing the overall health status, version information, and individual check results

****Example Request:****

```
```bash
curl -X GET http://localhost:9153/health
```
```

****Example Response:****

```
```json
{
 "status": "up",
 "node_version": "v1.2.3",
 "protocol_version": "consensus-v2.1",
 "checks": {
 "test-check": {
 "status": "up",
 "timestamp": "2024-01-15T10:30:45Z"
 }
 }
}
```
```

****Response Fields:****

- `status` (string): Overall health status - "up" if all checks pass, "down" if any check fails
- `node\_version` (string): The version of the GenLayer node software
- `protocol\_version` (string): The consensus protocol version
- `checks` (object, optional): Map of individual health check results
 - `[check-name]` (object): Results for a specific health check
 - `status` (string): Check status - "up" or "down"
 - `timestamp` (string): ISO 8601 timestamp of when the check was performed
 - `error` (string, optional): Error message if check failed

****HTTP Status Codes:****

- `200 OK`: Node is healthy (all checks passing)
- `503 Service Unavailable`: Node is unhealthy (one or more checks failing)

****Notes:****

- The endpoint has a 5-second timeout for all health checks
- The `checks` field is only included when health checks are configured
- Version fields will show "unset" if version information is not available

- Individual health checks are executed concurrently with results aggregated in the response

****Example Response (Unhealthy):****

```
```json
{
 "status": "down",
 "node_version": "v1.2.3",
 "protocol_version": "consensus-v2.1",
 "checks": {
 "database": {
 "status": "down",
 "error": "connection timeout",
 "timestamp": "2024-01-15T10:30:45Z"
 },
 "consensus": {
 "status": "up",
 "timestamp": "2024-01-15T10:30:45Z"
 }
 }
}
```
```

****cURL Example with Headers:****

```
```bash
curl -X GET http://localhost:9153/health \
 -H "Accept: application/json" \
 -v
```
```

metrics

Exposes Prometheus-compatible metrics for monitoring node performance and health.

****Method:**** `GET /metrics`

****Parameters:**** None

****Returns:**** Prometheus-format text response with all collected metrics

****Example Request:****

```
```bash
curl -X GET http://localhost:9153/metrics
```
```

****Example Response:****

```
...
# HELP genlayer_node_cpu_usage_percent Current CPU usage percentage (0-100)
# TYPE genlayer_node_cpu_usage_percent gauge
genlayer_node_cpu_usage_percent{component="node"} 15.5
genlayer_node_cpu_usage_percent{component="genvm-llm"} 45.2
genlayer_node_cpu_usage_percent{component="genvm-web"} 12.8
genlayer_node_cpu_usage_percent{component="webdriver"} 5.3

# HELP genlayer_node_memory_rss_bytes Resident Set Size (RSS) memory usage in bytes
# TYPE genlayer_node_memory_rss_bytes gauge
genlayer_node_memory_rss_bytes{component="node"} 524288000
genlayer_node_memory_rss_bytes{component="genvm-llm"} 2147483648

# HELP genlayer_node_network_rx_bytes_total Total bytes received across all network
interfaces
# TYPE genlayer_node_network_rx_bytes_total counter
```

```
genlayer_node_network_rx_bytes_total{component="node"} 1073741824
```

Available Metrics

The node collects metrics for three main components:

- **Node** - The main node process
- **GenVM** - The GenVM modules (LLM and Web)
- **WebDriver** - The WebDriver container

CPU Metrics

genlayer_node_cpu_usage_percent

Current CPU usage percentage (0-100).

Type: Gauge

Labels:

- `component`: One of `node`, `genvm-llm`, `genvm-web`, `webdriver`

Example:

```
genlayer_node_cpu_usage_percent{component="node"} 15.5
genlayer_node_cpu_usage_percent{component="genvm-llm"} 45.2
genlayer_node_cpu_usage_percent{component="genvm-web"} 12.8
genlayer_node_cpu_usage_percent{component="webdriver"} 5.3
```

Memory Metrics

genlayer_node_memory_rss_bytes

Resident Set Size (RSS) memory usage in bytes.

Type: Gauge

Labels:

- `component`: One of `node`, `genvm-llm`, `genvm-web`, `webdriver`

Example:

```
genlayer_node_memory_rss_bytes{component="node"} 524288000
genlayer_node_memory_rss_bytes{component="genvm-llm"} 2147483648
```

genlayer_node_memory_vms_bytes

Virtual Memory Size (VMS) in bytes.

Type: Gauge

Labels:

- `component`: One of `node`, `genvm-llm`, `genvm-web`, `webdriver`

Note: Only available for node and GenVM components.

genlayer_node_memory_usage_bytes

Current memory usage in bytes (WebDriver only).

Type: Gauge

Labels:

- `component`: `webdriver`

****genlayer_node_memory_limit_bytes****

Memory limit in bytes (WebDriver only).

****Type:** Gauge**

****Labels:****

- `component`: `webdriver`

****genlayer_node_memory_percent****

Memory usage percentage.

****Type:** Gauge**

****Labels:****

- `component`: One of `genvm-llm`, `genvm-web`, `node`, `webdriver`

- `node`: Node address (hex format)

****genlayer_node_memory_peak_bytes****

Peak memory usage in bytes (GenVM components only).

****Type:** Gauge**

****Labels:****

- `component`: One of `genvm-llm`, `genvm-web`

- `node`: Node address (hex format)

****Network Metrics****

****genlayer_node_network_rx_bytes_total****

Total bytes received across all network interfaces.

****Type:** Counter**

****Labels:****

- `component`: One of `node`, `genvm-llm`, `genvm-web`, `webdriver`

- `node`: Node address (hex format)

****Example:****

```
genlayer_node_network_rx_bytes_total{component="node",node="0x84b6cbd007511352d3fea26834c5c39a440903c4"} 96879942
```

```
genlayer_node_network_rx_bytes_total{component="webdriver",node="0x84b6cbd007511352d3fea26834c5c39a440903c4"} 84
```

****genlayer_node_network_tx_bytes_total****

Total bytes transmitted across all network interfaces.

****Type:** Counter**

****Labels:****

- `component`: One of `node`, `genvm-llm`, `genvm-web`, `webdriver`

- `node`: Node address (hex format)

****Disk Metrics****

****genlayer_node_disk_free_bytes****

Available free disk space in bytes.

****Type:** Gauge**

****Labels:****

- `component`: `node`
- `directory`: One of `genlayer.db`, `keystore`, `logs`
- `node`: Node address (hex format)

****genlayer_node_disk_total_bytes****

Total disk space in bytes.

****Type:** Gauge**

****Labels:****

- `component`: `node`
- `directory`: One of `genlayer.db`, `keystore`, `logs`
- `node`: Node address (hex format)

****genlayer_node_disk_usage_bytes****

Used disk space in bytes.

****Type:** Gauge**

****Labels:****

- `component`: `node`
- `directory`: One of `genlayer.db`, `keystore`, `logs`
- `node`: Node address (hex format)

****genlayer_node_disk_usage_percent****

Used disk space as percentage.

****Type:** Gauge**

****Labels:****

- `component`: `node`
- `directory`: One of `genlayer.db`, `keystore`, `logs`
- `node`: Node address (hex format)

****Node Status Metrics****

****genlayer_node_blocks_behind****

Number of blocks behind the latest block.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_latest_block****

Latest block number in the network.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_processing_block****

Currently processing block number.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_synced_block****

Last synced block number.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_synced****

Node synchronization status (1 = synced, 0 = not synced).

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_uptime_seconds****

Node uptime in seconds.

****Type:** Gauge**

****Labels:****

- `component`: `node`

- `node`: Node address (hex format)

****Transaction Metrics****

****genlayer_node_transactions_accepted_synced_total****

Total number of accepted and synced transactions.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_transactions_activated_total****

Total number of activated transactions.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_transactions_leader_proposed_total****

Total number of transactions proposed as leader.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_transactions_leader_revealed_total****

Total number of transactions revealed as leader.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_transactions_validator_commit_total****

Total number of validator commit votes.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_transactions_validator_reveal_total****

Total number of validator reveal votes.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****Go Runtime Metrics****

The endpoint also exposes standard Go runtime metrics including:

- `go\_gc\_duration\_seconds` - Garbage collection duration summary
- `go\_goroutines` - Number of active goroutines
- `go\_memstats\_\*` - Go memory statistics (heap, stack, GC)
- `go\_threads` - Number of OS threads

****Process Metrics****

Standard process metrics are also available:

- `process\_cpu\_seconds\_total` - Total CPU time
- `process\_resident\_memory\_bytes` - Resident memory size
- `process\_virtual\_memory\_bytes` - Virtual memory size
- `process\_open\_fds` - Number of open file descriptors
- `process\_network\_receive\_bytes\_total` - Network bytes received
- `process\_network\_transmit\_bytes\_total` - Network bytes transmitted

****Prometheus Handler Metrics****

Metrics about the metrics endpoint itself:

- `promhttp\_metric\_handler\_requests\_total` - Total scrape requests by status code
- `promhttp\_metric\_handler\_requests\_in\_flight` - Current scrapes being served

Configuration

Metrics collection can be configured in the node configuration file:

```yaml

metrics:

  interval: "15s"           # Default interval for all collectors

  collectors:

    node:

      enabled: true       # Enable/disable node metrics

      interval: "30s"    # Optional: Override default interval

    genvm:

      enabled: true       # Enable/disable GenVM metrics

      # Uses default interval (15s)

```
webdriver:
 enabled: true # Enable/disable WebDriver metrics
 interval: "60s" # Optional: Override default interval
...
```

#### **\*\*Default Values\*\***

- `metrics.interval`: 15s (applies to all collectors by default)
- `enabled`: true (for each collector)
- `interval`: Uses metrics.interval if not specified per collector

#### **\*\*Configuration Priority\*\***

1. Collector-specific `interval` (if specified)
2. Global `metrics.interval` (if specified)
3. System default (15s)

#### **#### Troubleshooting**

##### **\*\*No Metrics for GenVM\*\***

GenVM processes are ephemeral and only run when executing smart contracts. If you don't see GenVM metrics, it's likely that no contracts are currently being executed.

##### **\*\*WebDriver Metrics Missing\*\***

Ensure the WebDriver container is running:

```
```bash
docker ps | grep genlayer-node-webdriver
```
```

If not running, start it:

```
```bash
task docker:webdriver:docker-compose:up:detach
```
```

##### **\*\*Metrics Not Updating\*\***

Check if metrics collection is enabled in your configuration and that the node has been restarted after configuration changes.

#### **#### Performance Considerations**

- Collection intervals can be increased to reduce overhead further
- Each collector runs independently and won't block others if one fails

#### **### snapshot**

Produces a consistent snapshot of the node's state at the current block. Two modes selected by the request body: stream the snapshot back as a gzipped tar response (operator path), or write it to a destination directory on the node's filesystem (zero-downtime upgrade-controller path).

**\*\*Method:\*\*** `POST /snapshot`

**\*\*Authentication:\*\*** required. Send the operator token in either header:

...

Authorization: Bearer <token>

X-Admin-Auth: <token>

...

The token is the value of `GENLAYERNODE\_NODE\_OPERATOR\_TOKEN` on the node process. When that variable is unset, the endpoint returns `401` to every caller regardless of header value (default-deny). The other ops routes (`/metrics`, `/health`, `/balance`) are unaffected and remain public.

**\*\*Network exposure:\*\*** the ops server is intended for in-host or in-cluster access (Prometheus scraping `/metrics`, k8s probes hitting `/health`). The endpoint may not be reachable from outside the host depending on the deployment's firewall. The recommended operator workflow is therefore one of:

- SSH onto the node and call the endpoint via `127.0.0.1:9153`, or
- Open an SSH tunnel from the operator workstation to the node (`ssh -L 9153:127.0.0.1:9153 <node>`) and call `localhost:9153` through it.

**\*\*Parameters:\*\***

- `destDir` (string, optional): Absolute path on the node's filesystem. When set, switches to write-to-path mode; when omitted or empty, the snapshot is streamed back as `application/gzip`.

**\*\*Returns:\*\***

- Stream mode (`destDir` omitted): gzipped tar archive in the response body, `Content-Type: application/gzip`.
- Write-to-path mode (`destDir` set): empty body, `204 No Content`.

**\*\*Example Request – Stream Mode (operator):\*\***

On the node (after `ssh <node>`):

```
```bash
curl -X POST http://127.0.0.1:9153/snapshot \
  -H "X-Admin-Auth: $GENLAYERNODE_NODE_OPERATOR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{}' \
  -o snapshot.tar.gz
```
```

Or via SSH tunnel from a workstation (forwards `9153` to the node's loopback):

```
```bash
ssh -L 9153:127.0.0.1:9153 -N <node> &
curl -X POST http://localhost:9153/snapshot \
  -H "X-Admin-Auth: $GENLAYERNODE_NODE_OPERATOR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{}' \
  -o snapshot.tar.gz
```
```

The response body is a stream – `curl` can write it to a file (as above) or pipe it through any tool that reads stdin, without staging on local disk.

**\*\*Example Response – Stream Mode:\*\***

```
...
HTTP/1.1 200 OK
Content-Type: application/gzip
Content-Disposition: attachment; filename="snapshot.tar.gz"

<gzipped tar archive of the snapshot's files>
```
```

****Example Request – Write-to-Path Mode (upgrade controller):****

```
```bash
curl -X POST http://127.0.0.1:9153/snapshot \
 -H "X-Admin-Auth: $GENLAYERNODE_NODE_OPERATOR_TOKEN" \
 -H "Content-Type: application/json" \
 -d '{"destDir": "/var/snapshots/asimov-100"}'
```
```


****Example Response – Write-to-Path Mode:****

```
```\nHTTP/1.1 204 No Content\n```
```

## **\*\*HTTP Status Codes:\*\***

- `200 OK`: Stream mode succeeded; body carries the gzipped tar archive.
- `204 No Content`: Write-to-path mode succeeded; snapshot is on disk at `destDir`.
- `400 Bad Request`: `destDir` is not absolute, or already exists.
- `401 Unauthorized`: Missing, invalid, or empty operator token (also returned when `GENLAYERNODE\_NODE\_OPERATOR\_TOKEN` is unset on the node).
- `500 Internal Server Error`: Snapshot creation failed (e.g., disk full, DB checkpoint error).

## **\*\*Notes:\*\***

- A snapshot may be several GB. In stream mode the connection stays open for the duration of the snapshot – configure reverse proxies / load balancers with adequate read timeouts and disable response buffering.
- In stream mode, a dropped connection leaves no on-disk artifact on the node; retry the request from scratch.
- In write-to-path mode, `destDir` must be an absolute path that does not yet exist; the node creates it. Path-traversal hardening beyond these checks is a separate (Sev: Medium) follow-up – treat the operator token as a credential that can write under the node user's filesystem permissions.
- **\*\*Docker deploys\*\***: in write-to-path mode, `destDir` is resolved inside the node container's filesystem, not on the host. To make the resulting directory reachable from the host (or from another container), use a volume mount that maps `destDir` identically on both sides, or stick with stream mode. Stream mode is unaffected – the response body is sent over HTTP regardless of where the node runs.
- The snapshot includes the full chain database (SSTables, MANIFEST, WAL) at the moment of capture. Treat the artifact as sensitive – anyone holding it can reconstruct the node's state.
- Token comparison is constant-time and length-padded so neither the value nor the length of the secret can be inferred from response timing.
- Rotation: update `GENLAYERNODE\_NODE\_OPERATOR\_TOKEN` and restart the node. Tokens are read once at startup and not hot-reloaded.

**\*\*Forward-compatibility note:\*\*** when the zero-downtime upgrade controller (Phase 3) lands, `/snapshot` will additionally accept a second role (`upgrade`) so the controller can drive the snapshot + stop + start sequence with its own credential. The operator-token path documented here remains the same – no migration is required for operator tooling.

## **\*\*cURL Example with Bearer header:\*\***

```
```\nbash\ncurl -X POST http://127.0.0.1:9153/snapshot \\\n  -H "Authorization: Bearer $GENLAYERNODE_NODE_OPERATOR_TOKEN" \\\n  -H "Content-Type: application/json" \\\n  -d '{}' \\\n  -o snapshot.tar.gz\n```
```

gen_call
Source: https://docs.genlayer.com/api-references/genlayer-node/gen/gen_call

gen_call

Executes a new message call immediately without creating a transaction on the blockchain. This method supports read operations, write operations, and contract deployments.

```
{
  "jsonrpc": "2.0",
  "method": "gen_call",
  "params": [
    {
      "type": "deploy",
      "data":
"0xf9023db90223232076302e312e300a23207b2022446570656e6473223a202270792d67656e6c617965723a313
5716669766a7679383038303072683939387063786d64326d38766131777132717a71687a3835306e38676763723
46939713022207d0a0a66726f6d2067656e6c6179657220696d706f7274202a0a0a2320636f6e7472616374206
36c6173730a636c6173732053746f7261676528676c2e436f6e7472616374293a0a2020202073746f726167653a2
07374720a0a2020202320636f6e7374727563746f720a20202020646566205f5f696e69745f5f2873656c662c2
0696e697469616c5f73746f726167653a20737472293a0a20202020202073656c662e73746f72616765203d2
0696e697469616c5f73746f726167650a0a20202020232072656164206d6574686f6473206d75737420626520616
```

```

e6e6f7461746564207769746820766965770a2020202040676c2e7075626c69632e766965770a202020206465662
06765745f73746f726167652873656c6629202d3e207374723a0a202020202020202072657475726e2073656c662
e73746f726167650a0a2020202023207772697465206d6574686f640a2020202040676c2e7075626c69632e77726
974650a20202020646566207570646174655f73746f726167652873656c662c206e65775f73746f726167653a207
3747229202d3e204e6f6e653a0a202020202020202073656c662e73746f72616765203d206e65775f73746f72616
7650a950e04617267730d6c696e697469616c2076616c756500",
    "from": "0xf6A60752C24Cd3BFcAf8d387a5EB62d3746F44EE",
    "to": "0x0000000000000000000000000000000000000000000000000000000000000000",
    "status": "finalized"
  }
],
  "id": 1
},
...

```

****Example Response (Deploy Success):****

```

```json
{
 "jsonrpc": "2.0",
 "result": {
 "data": "00",
 "eqOutputs": [],
 "status": {
 "code": 0,
 "message": "success"
 },
 "stdout": "",
 "stderr": "",
 "logs": [
 {
 "file": "common/src/lib.rs:111",
 "level": "info",
 "message": "logging initialized",
 "target": "genvm_common",
 "ts": "1770221913210",
 "version": "v0.2.11-aarch64-macos-release"
 },
 {
 "file": "src/exe/run.rs:150",
 "genvm_id": "1",
 "level": "info",
 "message": "genvm id",
 "target": "genvm::exe::run",
 "ts": "1770221913212"
 },
 {
 "file": "src/lib.rs:299",
 "level": "info",
 "message": "metrics",
 "metrics": {
 "gvm": {
 "host": { "time": 654 },
 "llm_module": { "calls": 0, "time": 0 },
 "supervisor": { "compilation_time": 0, "compiled_modules": 0, "precompile_hits":
2 },
 "web_module": { "calls": 0, "time": 0 }
 },
 "llm": null,
 "web": null
 },
 "target": "genvm",
 "ts": "1770221913590"
 }
]
 },
 "id": 1,
}

```

```

 "events": [],
 "messages": [],
 "nondetDisagreementCallNo": null
 },
 "id": 1
},
{

```

**\*\*Example - Read Call (Latest Accepted State):\*\***

```

```json
{
  "jsonrpc": "2.0",
  "method": "gen_call",
  "params": [
    {
      "type": "read",
      "data": "0xd8960e066d6574686f646c6765745f686176655f636f696e00",
      "from": "0xf6A60752C24Cd3BFcAf8d387a5EB62d3746F44EE",
      "to": "0xCe5712E63fd5475288aB1B7c0a368B9417357b81",
      "gas": "0x5208",
      "value": "0x0"
    }
  ],
  "id": 1
},
{

```

****Example - Read Call with Specific Block and Status:****

```

```json
{
 "jsonrpc": "2.0",
 "method": "gen_call",
 "params": [
 {
 "type": "read",
 "data": "0xd8960e066d6574686f646c6765745f686176655f636f696e00",
 "from": "0xf6A60752C24Cd3BFcAf8d387a5EB62d3746F44EE",
 "to": "0xCe5712E63fd5475288aB1B7c0a368B9417357b81",
 "blockNumber": "0x151ec5",
 "status": "accepted"
 }
],
 "id": 1
},
{

```

**\*\*Example - Write Call:\*\***

```

```json
{
  "jsonrpc": "2.0",
  "method": "gen_call",
  "params": [
    {
      "from": "0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b6",
      "to": "0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b6",
      "data": "0xa9059cbb000000000000000000000000742d35cc6634c0532925a3b8d4c9db96c4b4d8b6",
      "type": "write",
      "gas": "0x5208",
      "value": "0x0"
    }
  ],
  "id": 1

```

```
}...
```

****Example Response with Messages (Router Forward Call):****

When a contract emits internal messages (e.g., a router contract forwarding calls), the response includes the messages array:

```
```json
{
 "jsonrpc": "2.0",
 "result": {
 "data": "00",
 "eqOutputs": [],
 "status": {
 "code": 0,
 "message": "success"
 },
 "stdout": "",
 "stderr": "",
 "logs": [
 {
 "file": "src/lib.rs:299",
 "level": "info",
 "message": "metrics",
 "target": "genvm",
 "ts": "1770225770000"
 }
],
 "events": [],
 "messages": [
 {
 "messageType": 1,
 "recipient": "0xc346f80c86adf1d71accfc4ef6efe7482b25be40",
 "value": "0x0",
 "data": "0xf86eb86a16046172677331d4023078393164463338654243623930384335643635...",
 "onAcceptance": false,
 "saltNonce": "0x0"
 },
 {
 "messageType": 1,
 "recipient": "0xad26513a4315b0ea8a5c800322b35092a83dfd3d",
 "value": "0x0",
 "data": "0xf86eb86a16046172677331d4023078333738653339374533316346383538333630...",
 "onAcceptance": false,
 "saltNonce": "0x0"
 }
],
 "nondetDisagreementCallNo": null
 },
 "id": 1
}...
```

## **\*\*Notes:\*\***

- The ``blockNumber`` parameter can be either hex-encoded with a ``0x`` prefix (e.g., ``"0x151ec5"``) or decimal (e.g., ``"1384133"``)
- Valid ``status`` values are ``"accepted"`` (default) and ``"finalized"``
- When both parameters are omitted, the call executes against the latest accepted state
- For ``read`` and ``write`` type calls, if the contract doesn't exist at the specified block/status combination, an error is returned with code ``-32001`` (Resource not found) and message ``"contract not found at address 0x..."``
- For ``deploy`` type calls, the ``status`` and ``blockNumber`` parameters are not used for contract state validation since no contract exists yet

```
gen_getContractSchema
Source: https://docs.genlayer.com/api-references/genlayer-node/gen/gen_getContractSchema
```

### gen\_getContractSchema

Retrieve the schema/interface of a GenLayer contract to understand its available methods and properties.

```
Method: `gen_getContractSchema`
```

**\*\*Parameters:\*\***

```
- `request` (object, required): The contract schema request
- `code` (string, required): Base64-encoded contract code
```

```
Returns: Contract schema object with methods and properties
```

```
{
 "ctor": { // constructor info
 "kwparams": {}, // dict
 "params": [["value", "type"]] // list of tuples
 },
 "methods": { // dict from method name to method info
 "method_name": {
 "kwparams": {}, // dict
 "params": [["value", "type"]], // list of tuples
 "payable": false, // bool
 "readonly": false, // bool
 "ret": "null" // string
 },
 },
}
```

**\*\*Example:\*\***

```
{
 "jsonrpc": "2.0",
 "method": "gen_getContractSchema",
 "params": [
 {
 "code":
 "IyB7ICJEZXB1bmRzIjogInB5LWdlbmXheWVyOnRlc3QiIH0KCmZyb20gZ2VubGF5ZXIgaW1wb3J0ICoKCGojIGNvbvNyYWN0IGNsYXNzCmNsYXNzIFN0b3JhZ2UoZ2wuQ29udHJhY3QpOgogICAgc3RvcnFnZTogc3RyCgogICAgIyBjb25zdHJ1Y3RvcgogICAgZGVmIF9faW5pdF9fKHNBGYSIGluaXRpYWxfzc3RvcnFnZTogc3RyKTOKICAgICAgICBzZWxmLnN0b3JhZ2UgPSBpbml0aWFsX3N0b3JhZ2UKCiAgICAjaHJlYWQgbwV0aG9kcyBtdXN0IGJlIGFubm90YXRlZCB3aXRoIHZpZXcKICAgIEBnbC5wdWJsawMudmlldwogICAgZGVmIGdldF9zdG9yYWdlKHNBGYPIC0+IHN0cjokICAgICAgICByZXRxcm4gc2VsZi5zdG9yYWdlCgogICAgIyB3cm10ZSBtZXRob2QKICAgIEBnbC5wdWJsawMud3JpdGUkICAgIGRlZiB1cGRhdGVfc3RvcnFnZShzZWxmcCBuZXdfc3RvcnFnZTogc3RyKSAtPiBOb25lOgogICAgICAgIHNBGYYuc3RvcnFnZSA9IG5ld19zdG9yYWdlCg=="
 }
],
 "id": 1
}
```

**\*\*Response:\*\***

```
```json
{
```

```

"jsonrpc": "2.0",
"result": {
  "ctor": {
    "kwparams": {},
    "params": [["initial_storage", "string"]]
  },
  "methods": {
    "get_storage": {
      "kwparams": {},
      "params": [],
      "readonly": true,
      "ret": "string"
    },
    "update_storage": {
      "kwparams": {},
      "params": [["new_storage", "string"]],
      "payable": false,
      "readonly": false,
      "ret": "null"
    }
  }
},
"id": 1
}...

```

gen_getContractState

Retrieves the current state of a GenLayer intelligent contract at a specific block height and transaction status.

****Method:**** `gen\_getContractState`

****Parameters:****

- `request` (object, required): The contract state request parameters
 - `address` (string, required): The contract address to query
 - `blockNumber` (string, optional): The block number to query at (hex-encoded). If not provided, uses latest block
 - `status` (string, optional): The transaction status filter (`"finalized"` or `"accepted"`). If not provided, uses accepted state

****Returns:**** Hex-encoded contract state data

****Example Request - Latest Accepted State:****

```

```json
{
 "jsonrpc": "2.0",
 "method": "gen_getContractState",
 "params": [
 {
 "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f"
 }
],
 "id": 1
}...

```

**\*\*Example Request - Specific Block and Status:\*\***

```

```json
{

```

```

"jsonrpc": "2.0",
"method": "gen_getContractState",
"params": [
  {
    "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f",
    "blockNumber": "0x151ec5",
    "status": "accepted"
  }
],
"id": 1
}

```

****Example Response:****

```

```json
{
 "jsonrpc": "2.0",
 "result": "0xd9960e0773746f726167656c6e757064617465642074657374207374",
 "id": 1
}

```

#### **\*\*Notes:\*\***

- The `blockNumber` parameter should be hex-encoded with a `0x` prefix
- Valid `status` values are `"accepted"` (default) and `"finalized"`
- The returned state is hex-encoded binary data with a `0x` prefix
- Empty contract state returns `0x`
- If the contract doesn't exist at the specified block/status, an error will be returned

#### **\*\*cURL Example:\*\***

```

```bash
# Get latest accepted state
Source: https://docs.genlayer.com/api-references/genlayer-node/gen/gen_getContractState
curl -X POST http://localhost:9151 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "gen_getContractState",
    "params": [{
      "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f"
    }],
    "id": 1
  }'

```

```

# Get state at specific block with accepted status
curl -X POST http://localhost:9151 \
  -H "Content-Type: application/json" \
  -d '{
    "jsonrpc": "2.0",
    "method": "gen_getContractState",
    "params": [{
      "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f",
      "blockNumber": "0x151ec5",
      "status": "accepted"
    }],
    "id": 1
  }'

```

gen_getContractCode


```
{
  "jsonrpc": "2.0",
  "result": "IyB7ICJEZXB1bmRzIjogInB5LWdlbmxeWVyonRlc3QiIH0KcmZyb20gZ2VubGF5ZXIGaw1wb3J0ICoKCgpjbGZcyBTdG9yYWdlKGdsLkNvbnRyYWN0KToKICAgIAHN0b3JhZ2U6IHN0cgoKICAgIGRlZiBfX2luaXRfXyhzZWxmLCBpbml0aWFsX3N0b3JhZ2U6IHN0cik6CiAgICAgICAgc2VsZi5zdG9yYWdlID0gaW5pdGlhbF9zdG9yYWdlCgogICAgQGdsLnB1YmxuPyY52awB3CiAgICBkZWYgZ2V0X3N0b3JhZ2Uoc2VsZikgLt4gc3RyOgogICAgICAgIHJldHVybiBzZWxmLnN0b3JhZ2UwPCAgICBAZ2wucHViGlJlndyaXRlCiAgICBkZWYgdXBkYXRlX3N0b3JhZ2Uoc2VsZiwgbmV3X3N0b3JhZ2U6IHN0cikgLt4gTm9uZToKICAgICAgICBzZWxmLnN0b3JhZ2UgPSBuZXdfc3RvcnFnZQo=",
  "id": 1
}
```

****Notes:****

- The `blockNumber` parameter should be hex-encoded with a `0x` prefix
- Valid `status` values are `"accepted"` (default) and `"finalized"`
- The returned code is base64-encoded
- If the contract doesn't exist at the specified block/status, an error will be returned
- If the address does not have code (not a contract), an error will be returned

****cURL Example:****

```
```bash
```

```
Get latest accepted code
```

```
Source: https://docs.genlayer.com/api-references/genlayer-node/gen/gen_getContractCode
```

```
curl -X POST http://localhost:9151 \
 -H "Content-Type: application/json" \
 -d '{
 "jsonrpc": "2.0",
 "method": "gen_getContractCode",
 "params": [{
 "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f"
 }],
 "id": 1
 }'
```

```
Get code at specific block with finalized status
```

```
curl -X POST http://localhost:9151 \
 -H "Content-Type: application/json" \
 -d '{
 "jsonrpc": "2.0",
 "method": "gen_getContractCode",
 "params": [{
 "address": "0x73ca5a2b51edf506ceaf110a41780ec51294d89f",
 "blockNumber": "0x151ec5",
 "status": "finalized"
 }],
 "id": 1
 }'
```

```
```
```

```
---
```

```
# gen_getTransactionReceipt
```

```
Source: https://docs.genlayer.com/api-references/genlayer-node/gen/gen\_getTransactionReceipt
```

gen_getTransactionReceipt

Retrieves the transaction receipt for a given transaction ID. This endpoint is a proxy to the underlying consensus layer's `getTransactionAllData` function, providing detailed information about a transaction that has been processed.

****Method:**** `gen\_getTransactionReceipt`

****Parameters:****

- `request` (object, required):
 - `txId` (`string`, required): The 32-byte transaction hash as a hexadecimal string with a `0x` prefix.

****Returns:****

A `Transaction` object which contains the details of the transaction. The structure is based on `ITransactionsTransaction` from the consensus contracts. The object has the following fields:

- `id` (`string`): The transaction ID as a hex string.

- `txOrigin` (`address`): The original address that initiated the transaction.
- `sender` (`address`): The address of the sender.
- `recipient` (`address`): The destination address of the transaction.
- `activator` (`address`): The address of the validator that activated the transaction.
- `status` (`uint8`): The current status of the transaction.
- `previousStatus` (`uint8`): The previous status of the transaction.
- `txSlot` (`uint256`): The transaction slot number.
- `initialRotations` (`uint256`): The initial number of rotations.
- `numOfInitialValidators` (`uint256`): The number of initial validators.
- `randomSeed` (`string`): The random seed used for the transaction as a hex string.
- `txExecutionHash` (`string`): The execution hash of the transaction as a hex string.
- `txCallData` (`string`): The transaction call data as a hex string.
- `eqBlocksOutputs` (`string`): The equivalent blocks outputs as a hex string.
- `timestamps` (`object`): Object containing various timestamps:
 - `Created` (`uint256`): When the transaction was created.
 - `Pending` (`uint256`): When the transaction became pending.
 - `Activated` (`uint256`): When the transaction was activated.
 - `Proposed` (`uint256`): When the transaction was proposed.
 - `Committed` (`uint256`): When the transaction was committed.
 - `LastVote` (`uint256`): When the last vote was cast.
 - `AppealSubmitted` (`uint256`): When an appeal was submitted.
 - `LeaderRevealed` (`uint256`): When the leader was revealed.
- `result` (`uint256`): The result of the transaction execution.
- `readStateBlockRanges` (`array`): Array of block range objects, each containing:
 - `ActivationBlock` (`uint256`): The activation block number.
 - `ProcessingBlock` (`uint256`): The processing block number.
 - `ProposalBlock` (`uint256`): The proposal block number.
- `roundData` (`array`): Array of round data objects containing:
 - `round` (`uint256`): The round number.
 - `leaderIndex` (`uint256`): Index of the round leader.
 - `votesCommitted` (`uint256`): Number of votes committed.
 - `votesRevealed` (`uint256`): Number of votes revealed.
 - `appealBond` (`uint256`): The appeal bond amount.
 - `rotationsLeft` (`uint256`): Number of rotations remaining.
 - `result` (`uint256`): The round result.
 - `roundValidators` (`array`): Array of validator addresses for this round.
 - `validatorVotes` (`string`): Base64-encoded validator votes.
 - `validatorVotesHash` (`array`): Array of vote hashes as hex strings.
 - `validatorResultHash` (`array`): Array of result hashes as hex strings.

****Example Request:****

```

```json
{
 "jsonrpc": "2.0",
 "method": "gen_getTransactionReceipt",
 "params": [
 {
 "txId": "0x635060dd514082096d18c8eb64682cc6a944f9ce1ae6982febf7a71e9f656f49"
 }
],
 "id": 1
}
```

```

****Example Response:****

```

```json
{
 "jsonrpc": "2.0",
 "id": 1,
 "result": {
 "id": "0x635060dd514082096d18c8eb64682cc6a944f9ce1ae6982febf7a71e9f656f49",
 "txOrigin": "0x5b70759760a85d92bf972b67ee8558e8e8da87d4",
 "sender": "0x5b70759760a85d92bf972b67ee8558e8e8da87d4",

```

[illegible]

```

}
...

gen_getTransactionStatus
Source: https://docs.genlayer.com/api-references/genlayer-node/gen/gen_getTransactionStatus

gen_getTransactionStatus

Returns the current consensus status of a transaction. This is a lightweight endpoint that returns only the status without full receipt data – use it for polling transaction progress.

Method: `gen_getTransactionStatus`

Parameters:

- `request` (object, required):
 - `txId` (string, required): The transaction hash (hex-encoded with `0x` prefix)
 - `timestamp` (integer, optional): Unix timestamp for the query. If not provided, uses current time

Returns: Object with status string and numeric code

Response Fields:

| Field | Type | Description |
|-----|-----|-----|
| `status` | string | Human-readable status name (e.g., "FINALIZED", "ACCEPTED", "PENDING")
| `statusCode` | uint8 | Numeric status code |

Status Codes:

| Code | Status |
|-----|-----|
| 0 | UNINITIALIZED |
| 1 | PENDING |
| 2 | PROPOSING |
| 3 | COMMITTING |
| 4 | REVEALING |
| 5 | ACCEPTED |
| 6 | UNDETERMINED |
| 7 | FINALIZED |
| 8 | CANCELED |
| 9 | APPEAL_REVEALING |
| 10 | APPEAL_COMMITTING |
| 11 | READY_TO_FINALIZE |
| 12 | VALIDATORS_TIMEOUT |
| 13 | LEADER_TIMEOUT |

Example Request:

```json
{
  "jsonrpc": "2.0",
  "method": "gen_getTransactionStatus",
  "params": [
    {
      "txId": "0x563f046c187d711127c51213ca62e2e4fee52009a98f0989a73a0a0382d21890"
    }
  ],
  "id": 1
}
...

```

****Example Response:****

```
```json
{
 "jsonrpc": "2.0",
 "result": {
 "status": "FINALIZED",
 "statusCode": 7
 },
 "id": 1
}
```
```

****Notes:****

- This endpoint queries the consensus contract on-chain via `getTransactionData`
- Use this for polling – it's cheaper than `gen\_getTransactionReceipt` which returns the full receipt
- The `timestamp` parameter is passed to the on-chain call and affects how the contract reports the status

gen_syncing
Source: https://docs.genlayer.com/api-references/genlayer-node/gen/gen_syncing

gen_syncing

Returns the GenVM sync status, indicating how far the node's local GenVM state is behind the chain tip.

****Method:**** `gen\_syncing`

****Parameters:**** None

****Returns:**** Object with sync status information

****Response Fields:****

| Field | Type | Description |
|----------------|--------|---|
| `syncedBlock` | string | Hex-encoded block number that GenVM has synced to |
| `latestBlock` | string | Hex-encoded latest block number on the chain |
| `blocksBehind` | number | Number of blocks GenVM is behind the chain tip (0 = fully synced) |

****Example Request:****

```
```json
{
 "jsonrpc": "2.0",
 "method": "gen_syncing",
 "params": [],
 "id": 1
}
```
```

****Example Response (Synced):****

```
```json
{
 "jsonrpc": "2.0",
 "result": {
 "syncedBlock": "0x151ec5",

```

```

 "latestBlock": "0x151ec5",
 "blocksBehind": 0
 },
 "id": 1
}

```

**\*\*Example Response (Behind):\*\***

```

```json
{
  "jsonrpc": "2.0",
  "result": {
    "syncedBlock": "0x151e00",
    "latestBlock": "0x151ec5",
    "blocksBehind": 197
  },
  "id": 1
}

```

****Notes:****

- This endpoint always returns an object (never `false` like `eth\_syncing`)
- Use `blocksBehind == 0` to determine if the node is fully synced
- When the node is catching up, `gen\_call` requests without an explicit `blockNumber` will serve data at the `syncedBlock` rather than waiting for full sync

gen_dbg_ping
Source: https://docs.genlayer.com/api-references/genlayer-node/debug/gen_dbg_ping

gen_dbg_ping

Simple connectivity test method that returns a "pong" response.

****Method:**** `gen\_dbg\_ping`

****Parameters:**** None

****Returns:**** "pong" string

****Example:****

```

```json
{
 "jsonrpc": "2.0",
 "method": "gen_dbg_ping",
 "params": [],
 "id": 1
}

```

**\*\*Response:\*\***

```

```json
{
  "jsonrpc": "2.0",
  "result": "pong",
  "id": 1
}

```

- `txID` (string, required): Transaction ID (hex-encoded with `0x` prefix)
- `round` (integer, optional): Consensus round number (default: 0)

****Returns:**** Transaction execution trace object

****Response Fields:****

| Field | Type | Description |
|------------------|--------|---|
| `transaction_id` | string | Transaction hash |
| `result_code` | number | 0=success, 1=user error, 2=VM error |
| `return_data` | string | Hex-encoded contract return data |
| `stdout` | string | Standard output captured during execution |
| `stderr` | string | Standard error captured during execution |
| `genvm_log` | array | Detailed GenVM execution log entries |
| `storage_proof` | string | Hex-encoded storage proof hash |
| `run_time` | string | Execution duration |
| `eq_outputs` | array | Hex-encoded equivalence principle outputs |

****Example:****

```
```json
{
 "jsonrpc": "2.0",
 "method": "gen_dbg_traceTransaction",
 "params": [{"txID": "0x563f046c187d711127c51213ca62e2e4fee52009a98f0989a73a0a0382d21890",
"round": 0}],
 "id": 1
}
```
```

****Response:****

```
```json
{
 "jsonrpc": "2.0",
 "result": {
 "transaction_id": "0x563f046c187d711127c51213ca62e2e4fee52009a98f0989a73a0a0382d21890",
 "result_code": 2,
 "return_data": "0x2e0464617461b402696e76616c69645f636f6e74726163374...",
 "stdout": "",
 "stderr": "",
 "genvm_log": [
 {
 "file": "src/lib.rs:82",
 "level": "error",
 "message": "vm error",
 "error": {"causes": ["VMError(invalid_contract absent_runner_comment)"]}
 }
],
 "storage_proof": "0x99ff0d9125e1fc9531a11262e15aeb2c60509a078c4cc4c64cefdfeb06ff68647",
 "run_time": "0s",
 "eq_outputs": []
 },
 "id": 1
}
```
```

****Notes:****

- If the trace isn't stored locally (e.g., the node restarted or data was pruned), the endpoint will replay the transaction automatically.
- This endpoint is in the `genlayer\_debug` group and must be enabled in the node configuration.
- Use `result\_code` to determine execution outcome: 0 = contract returned successfully, 1 =

contract raised a user error, 2 = GenVM encountered an error.

balance

Source: <https://docs.genlayer.com/api-references/genlayer-node/ops/balance>

balance

Returns the configured operator's account balance in multiple formats including hexadecimal, wei, and GEN units.

****Method:**** `GET /balance`

****Parameters:**** None

****Returns:**** JSON object containing the operator's balance in three formats

****Example Request:****

```
```bash
curl -X GET http://localhost:9153/balance
```
```

****Example Response:****

```
```json
{
 "balance_hex": "0x0de0b6b3a7640000",
 "balance_wei": "10000000000000000000",
 "balance_gen": "1.000"
}
```
```

****Response Fields:****

- `balance\_hex` (string): The balance encoded as hexadecimal with "0x" prefix
- `balance\_wei` (string): The balance in wei as a decimal string
- `balance\_gen` (string): The balance in GEN units formatted with 3 decimal places

****Notes:****

- This endpoint is only available when the node is running in validator mode
- The balance is always retrieved from the latest block
- The operator address is configured at node startup
- If the operator address is not configured, the zero address (0x0000...0000) is used
- Balance conversions: 1 GEN = 1,000,000,000,000,000,000 wei (10^{18} wei)
- Returns HTTP 503 if the node is not in validator mode
- Returns HTTP 500 if there's an error retrieving the balance

****Example Response (Zero Balance):****

```
```json
{
 "balance_hex": "0x",
 "balance_wei": "0",
 "balance_gen": "0.000"
}
```
```

****Example Response (Large Balance):****

```
```json
{
 "balance_hex": "0x3635c9adc5dea00000",

```

```
 "balance_wei": "1000000000000000000000",
 "balance_gen": "1000.000"
},

```

**\*\*Error Response (Not in Validator Mode):\*\***

```
```json
{
  "message": "Balance endpoint not available in rollup mode"
},

```

****cURL Example with Headers:****

```
```bash
curl -X GET http://localhost:9153/balance \
 -H "Accept: application/json" \
 -v
```

```

health
Source: <https://docs.genlayer.com/api-references/genlayer-node/ops/health>

health

Returns the health status of the GenLayer node along with version information and the status of individual health checks.

****Method:**** `GET /health`

****Parameters:**** None

****Returns:**** JSON object containing the overall health status, version information, and individual check results

****Example Request:****

```
```bash
curl -X GET http://localhost:9153/health
```

```

****Example Response:****

```
```json
{
 "status": "up",
 "node_version": "v1.2.3",
 "protocol_version": "consensus-v2.1",
 "checks": {
 "test-check": {
 "status": "up",
 "timestamp": "2024-01-15T10:30:45Z"
 }
 }
},

```

**\*\*Response Fields:\*\***

- `status` (string): Overall health status - "up" if all checks pass, "down" if any check fails
- `node\_version` (string): The version of the GenLayer node software

- `protocol\_version` (string): The consensus protocol version
- `checks` (object, optional): Map of individual health check results
  - `[check-name]` (object): Results for a specific health check
    - `status` (string): Check status - "up" or "down"
    - `timestamp` (string): ISO 8601 timestamp of when the check was performed
    - `error` (string, optional): Error message if check failed

**\*\*HTTP Status Codes:\*\***

- `200 OK`: Node is healthy (all checks passing)
- `503 Service Unavailable`: Node is unhealthy (one or more checks failing)

**\*\*Notes:\*\***

- The endpoint has a 5-second timeout for all health checks
- The `checks` field is only included when health checks are configured
- Version fields will show "unset" if version information is not available
- Individual health checks are executed concurrently with results aggregated in the response

**\*\*Example Response (Unhealthy):\*\***

```
```json
{
  "status": "down",
  "node_version": "v1.2.3",
  "protocol_version": "consensus-v2.1",
  "checks": {
    "database": {
      "status": "down",
      "error": "connection timeout",
      "timestamp": "2024-01-15T10:30:45Z"
    },
    "consensus": {
      "status": "up",
      "timestamp": "2024-01-15T10:30:45Z"
    }
  }
}
```
```

**\*\*cURL Example with Headers:\*\***

```
```bash
curl -X GET http://localhost:9153/health \
  -H "Accept: application/json" \
  -v
```
```

---

**### metrics**

Exposes Prometheus-compatible metrics for monitoring node performance and health.

**\*\*Method:\*\*** `GET /metrics`

**\*\*Parameters:\*\*** None

**\*\*Returns:\*\*** Prometheus-format text response with all collected metrics

**\*\*Example Request:\*\***

```
```bash
curl -X GET http://localhost:9153/metrics
```
```

## **\*\*Example Response:\*\***

```
...
HELP genlayer_node_cpu_usage_percent Current CPU usage percentage (0-100)
Source: https://docs.genlayer.com/api-references/genlayer-node/ops/metrics
TYPE genlayer_node_cpu_usage_percent gauge
genlayer_node_cpu_usage_percent{component="node"} 15.5
genlayer_node_cpu_usage_percent{component="genvm-llm"} 45.2
genlayer_node_cpu_usage_percent{component="genvm-web"} 12.8
genlayer_node_cpu_usage_percent{component="webdriver"} 5.3

HELP genlayer_node_memory_rss_bytes Resident Set Size (RSS) memory usage in bytes
TYPE genlayer_node_memory_rss_bytes gauge
genlayer_node_memory_rss_bytes{component="node"} 524288000
genlayer_node_memory_rss_bytes{component="genvm-llm"} 2147483648

HELP genlayer_node_network_rx_bytes_total Total bytes received across all network
interfaces
TYPE genlayer_node_network_rx_bytes_total counter
genlayer_node_network_rx_bytes_total{component="node"} 1073741824
...
```

## **#### Available Metrics**

The node collects metrics for three main components:

- **\*\*Node\*\*** - The main node process
- **\*\*GenVM\*\*** - The GenVM modules (LLM and Web)
- **\*\*WebDriver\*\*** - The WebDriver container

### **\*\*CPU Metrics\*\***

#### **\*\*genlayer\_node\_cpu\_usage\_percent\*\***

Current CPU usage percentage (0-100).

**\*\*Type:\*\*** Gauge

**\*\*Labels:\*\***

- `component`: One of `node`, `genvm-llm`, `genvm-web`, `webdriver`

**\*\*Example:\*\***

```
...
genlayer_node_cpu_usage_percent{component="node"} 15.5
genlayer_node_cpu_usage_percent{component="genvm-llm"} 45.2
genlayer_node_cpu_usage_percent{component="genvm-web"} 12.8
genlayer_node_cpu_usage_percent{component="webdriver"} 5.3
...
```

### **\*\*Memory Metrics\*\***

#### **\*\*genlayer\_node\_memory\_rss\_bytes\*\***

Resident Set Size (RSS) memory usage in bytes.

**\*\*Type:\*\*** Gauge

**\*\*Labels:\*\***

- `component`: One of `node`, `genvm-llm`, `genvm-web`, `webdriver`

**\*\*Example:\*\***

```
...
genlayer_node_memory_rss_bytes{component="node"} 524288000
genlayer_node_memory_rss_bytes{component="genvm-llm"} 2147483648
...
```

**\*\*genlayer\_node\_memory\_vms\_bytes\*\***

Virtual Memory Size (VMS) in bytes.

**\*\*Type:\*\* Gauge**

**\*\*Labels:\*\***

- `component`: One of `node`, `genvm-llm`, `genvm-web`, `webdriver`

**\*\*Note:\*\*** Only available for node and GenVM components.

**\*\*genlayer\_node\_memory\_usage\_bytes\*\***

Current memory usage in bytes (WebDriver only).

**\*\*Type:\*\* Gauge**

**\*\*Labels:\*\***

- `component`: `webdriver`

**\*\*genlayer\_node\_memory\_limit\_bytes\*\***

Memory limit in bytes (WebDriver only).

**\*\*Type:\*\* Gauge**

**\*\*Labels:\*\***

- `component`: `webdriver`

**\*\*genlayer\_node\_memory\_percent\*\***

Memory usage percentage.

**\*\*Type:\*\* Gauge**

**\*\*Labels:\*\***

- `component`: One of `genvm-llm`, `genvm-web`, `node`, `webdriver`

- `node`: Node address (hex format)

**\*\*genlayer\_node\_memory\_peak\_bytes\*\***

Peak memory usage in bytes (GenVM components only).

**\*\*Type:\*\* Gauge**

**\*\*Labels:\*\***

- `component`: One of `genvm-llm`, `genvm-web`

- `node`: Node address (hex format)

**\*\*Network Metrics\*\***

**\*\*genlayer\_node\_network\_rx\_bytes\_total\*\***

Total bytes received across all network interfaces.

**\*\*Type:\*\* Counter**

**\*\*Labels:\*\***

- `component`: One of `node`, `genvm-llm`, `genvm-web`, `webdriver`

- `node`: Node address (hex format)

**\*\*Example:\*\***

```

genlayer_node_network_rx_bytes_total{component="node",node="0x84b6cbd007511352d3fea26834c5c3

```
9a440903c4"} 96879942
genlayer_node_network_rx_bytes_total{component="webdriver",node="0x84b6cbd007511352d3fea2683
4c5c39a440903c4"} 84
``,``
```

****genlayer_node_network_tx_bytes_total****

Total bytes transmitted across all network interfaces.

****Type:** Counter**

****Labels:****

- `component`: One of `node`, `genvm-llm`, `genvm-web`, `webdriver`
- `node`: Node address (hex format)

****Disk Metrics****

****genlayer_node_disk_free_bytes****

Available free disk space in bytes.

****Type:** Gauge**

****Labels:****

- `component`: `node`
- `directory`: One of `genlayer.db`, `keystore`, `logs`
- `node`: Node address (hex format)

****genlayer_node_disk_total_bytes****

Total disk space in bytes.

****Type:** Gauge**

****Labels:****

- `component`: `node`
- `directory`: One of `genlayer.db`, `keystore`, `logs`
- `node`: Node address (hex format)

****genlayer_node_disk_usage_bytes****

Used disk space in bytes.

****Type:** Gauge**

****Labels:****

- `component`: `node`
- `directory`: One of `genlayer.db`, `keystore`, `logs`
- `node`: Node address (hex format)

****genlayer_node_disk_usage_percent****

Used disk space as percentage.

****Type:** Gauge**

****Labels:****

- `component`: `node`
- `directory`: One of `genlayer.db`, `keystore`, `logs`
- `node`: Node address (hex format)

****Node Status Metrics****

****genlayer_node_blocks_behind****

Number of blocks behind the latest block.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_latest_block****

Latest block number in the network.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_processing_block****

Currently processing block number.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_synced_block****

Last synced block number.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_synced****

Node synchronization status (1 = synced, 0 = not synced).

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_uptime_seconds****

Node uptime in seconds.

****Type:** Gauge**

****Labels:****

- `component`: `node`

- `node`: Node address (hex format)

****Transaction Metrics****

****genlayer_node_transactions_accepted_synced_total****

Total number of accepted and synced transactions.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_transactions_activated_total****

Total number of activated transactions.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_transactions_leader_proposed_total****

Total number of transactions proposed as leader.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_transactions_leader_revealed_total****

Total number of transactions revealed as leader.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_transactions_validator_commit_total****

Total number of validator commit votes.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****genlayer_node_transactions_validator_reveal_total****

Total number of validator reveal votes.

****Type:** Gauge**

****Labels:****

- `node`: Node address (hex format)

****Go Runtime Metrics****

The endpoint also exposes standard Go runtime metrics including:

- `go\_gc\_duration\_seconds` - Garbage collection duration summary
- `go\_goroutines` - Number of active goroutines
- `go\_memstats\_\*` - Go memory statistics (heap, stack, GC)
- `go\_threads` - Number of OS threads

****Process Metrics****

Standard process metrics are also available:

- `process\_cpu\_seconds\_total` - Total CPU time
- `process\_resident\_memory\_bytes` - Resident memory size
- `process\_virtual\_memory\_bytes` - Virtual memory size
- `process\_open\_fds` - Number of open file descriptors
- `process\_network\_receive\_bytes\_total` - Network bytes received
- `process\_network\_transmit\_bytes\_total` - Network bytes transmitted

****Prometheus Handler Metrics****

Metrics about the metrics endpoint itself:

- `promhttp\_metric\_handler\_requests\_total` - Total scrape requests by status code
- `promhttp\_metric\_handler\_requests\_in\_flight` - Current scrapes being served

Configuration

Metrics collection can be configured in the node configuration file:

```
```yaml
metrics:
 interval: "15s" # Default interval for all collectors
 collectors:
 node:
 enabled: true # Enable/disable node metrics
 interval: "30s" # Optional: Override default interval
 genvm:
 enabled: true # Enable/disable GenVM metrics
 # Uses default interval (15s)
 webdriver:
 enabled: true # Enable/disable WebDriver metrics
 interval: "60s" # Optional: Override default interval
...
```
```

Default Values

- `metrics.interval`: 15s (applies to all collectors by default)
- `enabled`: true (for each collector)
- `interval`: Uses metrics.interval if not specified per collector

Configuration Priority

1. Collector-specific `interval` (if specified)
2. Global `metrics.interval` (if specified)
3. System default (15s)

Troubleshooting

No Metrics for GenVM

GenVM processes are ephemeral and only run when executing smart contracts. If you don't see GenVM metrics, it's likely that no contracts are currently being executed.

WebDriver Metrics Missing

Ensure the WebDriver container is running:

```
```bash
docker ps | grep genlayer-node-webdriver
```
```

If not running, start it:

```
```bash
task docker:webdriver:docker-compose:up:detach
```
```

Metrics Not Updating

Check if metrics collection is enabled in your configuration and that the node has been restarted after configuration changes.

Performance Considerations

- Collection intervals can be increased to reduce overhead further
- Each collector runs independently and won't block others if one fails

snapshot
Source: <https://docs.genlayer.com/api-references/genlayer-node/ops/snapshot>

snapshot

Produces a consistent snapshot of the node's state at the current block. Two modes selected by the request body: stream the snapshot back as a gzipped tar response (operator path), or write it to a destination directory on the node's filesystem (zero-downtime upgrade-controller path).

****Method:**** `POST /snapshot`

****Authentication:**** required. Send the operator token in either header:

...

Authorization: Bearer <token>

X-Admin-Auth: <token>

...

The token is the value of `GENLAYERNODE\_NODE\_OPERATOR\_TOKEN` on the node process. When that variable is unset, the endpoint returns `401` to every caller regardless of header value (default-deny). The other ops routes (`/metrics`, `/health`, `/balance`) are unaffected and remain public.

****Network exposure:**** the ops server is intended for in-host or in-cluster access (Prometheus scraping `/metrics`, k8s probes hitting `/health`). The endpoint may not be reachable from outside the host depending on the deployment's firewall. The recommended operator workflow is therefore one of:

- SSH onto the node and call the endpoint via `127.0.0.1:9153`, or
- Open an SSH tunnel from the operator workstation to the node (`ssh -L 9153:127.0.0.1:9153 <node>`) and call `localhost:9153` through it.

****Parameters:****

- `destDir` (string, optional): Absolute path on the node's filesystem. When set, switches to write-to-path mode; when omitted or empty, the snapshot is streamed back as `application/gzip`.

****Returns:****

- Stream mode (`destDir` omitted): gzipped tar archive in the response body, `Content-Type: application/gzip`.
- Write-to-path mode (`destDir` set): empty body, `204 No Content`.

****Example Request – Stream Mode (operator):****

On the node (after `ssh <node>`):

```bash

```
curl -X POST http://127.0.0.1:9153/snapshot \
 -H "X-Admin-Auth: $GENLAYERNODE_NODE_OPERATOR_TOKEN" \
 -H "Content-Type: application/json" \
 -d '{}' \
 -o snapshot.tar.gz
```

```

Or via SSH tunnel from a workstation (forwards `9153` to the node's loopback):

```bash

```
ssh -L 9153:127.0.0.1:9153 -N <node> &
curl -X POST http://localhost:9153/snapshot \
 -H "X-Admin-Auth: $GENLAYERNODE_NODE_OPERATOR_TOKEN" \
 -H "Content-Type: application/json" \
 -d '{}' \
```

```
..-o snapshot.tar.gz
...
```

The response body is a stream – `curl` can write it to a file (as above) or pipe it through any tool that reads stdin, without staging on local disk.

**\*\*Example Response – Stream Mode:\*\***

```
...
HTTP/1.1 200 OK
Content-Type: application/gzip
Content-Disposition: attachment; filename="snapshot.tar.gz"

<gzipped tar archive of the snapshot's files>
...
```

**\*\*Example Request – Write-to-Path Mode (upgrade controller):\*\***

```
```bash
curl -X POST http://127.0.0.1:9153/snapshot \
  -H "X-Admin-Auth: $GENLAYERNODE_NODE_OPERATOR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"destDir": "/var/snapshots/asimov-100"}'
```
```

**\*\*Example Response – Write-to-Path Mode:\*\***

```
...
HTTP/1.1 204 No Content
...
```

**\*\*HTTP Status Codes:\*\***

- `200 OK`: Stream mode succeeded; body carries the gzipped tar archive.
- `204 No Content`: Write-to-path mode succeeded; snapshot is on disk at `destDir`.
- `400 Bad Request`: `destDir` is not absolute, or already exists.
- `401 Unauthorized`: Missing, invalid, or empty operator token (also returned when `GENLAYERNODE\_NODE\_OPERATOR\_TOKEN` is unset on the node).
- `500 Internal Server Error`: Snapshot creation failed (e.g., disk full, DB checkpoint error).

**\*\*Notes:\*\***

- A snapshot may be several GB. In stream mode the connection stays open for the duration of the snapshot – configure reverse proxies / load balancers with adequate read timeouts and disable response buffering.
- In stream mode, a dropped connection leaves no on-disk artifact on the node; retry the request from scratch.
- In write-to-path mode, `destDir` must be an absolute path that does not yet exist; the node creates it. Path-traversal hardening beyond these checks is a separate (Sev: Medium) follow-up – treat the operator token as a credential that can write under the node user's filesystem permissions.
- **\*\*Docker deploys\*\***: in write-to-path mode, `destDir` is resolved inside the node container's filesystem, not on the host. To make the resulting directory reachable from the host (or from another container), use a volume mount that maps `destDir` identically on both sides, or stick with stream mode. Stream mode is unaffected – the response body is sent over HTTP regardless of where the node runs.
- The snapshot includes the full chain database (SSTables, MANIFEST, WAL) at the moment of capture. Treat the artifact as sensitive – anyone holding it can reconstruct the node's state.
- Token comparison is constant-time and length-padded so neither the value nor the length of the secret can be inferred from response timing.
- Rotation: update `GENLAYERNODE\_NODE\_OPERATOR\_TOKEN` and restart the node. Tokens are read once at startup and not hot-reloaded.

**\*\*Forward-compatibility note:\*\*** when the zero-downtime upgrade controller (Phase 3) lands, `/snapshot` will additionally accept a second role (`upgrade`) so the controller can drive the snapshot + stop + start sequence with its own credential. The operator-token path documented here remains the same – no migration is required for operator tooling.

**\*\*cURL Example with Bearer header:\*\***

```
```bash
curl -X POST http://127.0.0.1:9153/snapshot \
  -H "Authorization: Bearer $GENLAYERNODE_NODE_OPERATOR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{}' \
  -o snapshot.tar.gz
```
```

---

# genvm-linter  
Source: <https://docs.genlayer.com/api-references/genlayer-linter>

Fast validation and schema extraction for GenLayer intelligent contracts.

## ## Installation

```
```bash
pip install genvm-linter
```
```

## ## Usage

```
```bash
# Run both lint and validate (default)
genvm-lint check contract.py

# Fast AST safety checks only (~50ms)
genvm-lint lint contract.py

# Full SDK semantic validation (~200ms cached)
genvm-lint validate contract.py

# Extract ABI schema
genvm-lint schema contract.py
genvm-lint schema contract.py --output abi.json

# Pyright type checking with SDK auto-configured
genvm-lint typecheck contract.py
genvm-lint typecheck contract.py --strict      # Strict mode
genvm-lint typecheck contract.py --all        # Show all errors (disable SDK suppressions)

# IDE setup – download SDK and output extraPaths for Pylance
genvm-lint setup                               # Latest version
genvm-lint setup --contract contract.py        # Auto-detect version from header
genvm-lint setup --version v0.2.12            # Specific version
genvm-lint setup --json                        # JSON output for IDE integration

# Pre-download GenVM artifacts
genvm-lint download                            # Latest
genvm-lint download --version v0.2.12         # Specific version
genvm-lint download --list                     # Show cached

# JSON output (all commands)
genvm-lint check contract.py --json
```
```

## ## How It Works

### ### Layer 1: AST Lint Checks (Fast)

- Forbidden imports (`random`, `os`, `time`, etc.)
- Non-deterministic patterns (`float()`, `time.time()`)
- Structure validation (dependency header)
- Semantic consensus rule (GL-S03)

### ### Semantic Rules (GL-S)

These rules run as part of Layer 1 and catch common mistakes in GenLayer consensus patterns.

#### #### GL-S03 – `eq\_principle\_strict\_eq` nondeterminism mismatch (ERROR)

Flags `eq\_principle\_strict\_eq` (all API generations: `gl.eq\_principle.strict\_eq`, `eq\_principle\_strict\_eq`, `eq\_principle.strict\_eq`) wrapping a lambda or function that **\*\*returns raw nondeterministic output directly\*\***. LLM outputs and raw web content vary across validators, so strict-equality consensus will always fail.

**\*\*Detection is conservative\*\*** – only flagged when the wrapped function/lambda returns the nondeterministic value without any transformation. Processed output (boolean comparisons, `json.loads`, sorted results, etc.) is not flagged.

Nondeterministic calls detected in two forms:

- **\*\*Namespace-qualified\*\*** (v0.1.0 and v0.1.3+ APIs): `gl.exec\_prompt`, `gl.get\_webpage`, `gl.nondet.exec\_prompt`, `gl.nondet.web.render`, and their `genlayer.\*` equivalents
- **\*\*Bare names\*\*** (direct SDK import convention): `exec\_prompt`, `get\_webpage` – matched when called without a namespace prefix, representing  
`from genlayer.std.nondet\_fns import exec\_prompt` style imports

Flagged patterns:

```
```python
# Bad – namespace-qualified, LLM output is non-deterministic (v0.1.0 API)
gl.eq_principle.strict_eq(lambda: gl.exec_prompt("What is 2+2?"))

# Bad – namespace-qualified, raw web content varies (v0.1.0 API)
gl.eq_principle.strict_eq(lambda: gl.get_webpage("https://example.com"))

# Bad – namespace-qualified, v0.1.3+ API
gl.eq_principle.strict_eq(lambda: gl.nondet.exec_prompt("What is 2+2?"))
gl.eq_principle.strict_eq(lambda: gl.nondet.web.render("https://example.com"))

# Bad – bare name (direct import: from genlayer.std.nondet_fns import exec_prompt)
gl.eq_principle.strict_eq(lambda: exec_prompt("What is 2+2?"))

# Bad – named function returning raw nondet
def fetch():
    return gl.exec_prompt("What is 2+2?")

gl.eq_principle.strict_eq(fetch)

# Bad – simple passthrough variable
def fetch():
    result = gl.exec_prompt("What is 2+2?")
    return result

gl.eq_principle.strict_eq(fetch)
```
```

Not flagged (processed output or third-party calls):

```
```python
# OK – bool is deterministic
```

```
gl.eq_principle.strict_eq(lambda: "Paris" in gl.get_webpage("https://example.com"))
```

OK – comparison produces deterministic bool

```
def classify():
    data = gl.exec_prompt("Answer YES or NO")
    return data == "YES"
```

```
gl.eq_principle.strict_eq(classify)
```

OK – third-party: my_service.exec_prompt is not a GL SDK call

```
gl.eq_principle.strict_eq(lambda: my_service.exec_prompt("What is 2+2?"))
```

****Alias handling:**** GL-S03 matches by name convention, not by import tracking. Bare names ``exec_prompt`` and ``get_webpage`` are flagged when called without a namespace prefix (representing direct SDK imports such as ``from genlayer.std.nondet_fns import exec_prompt``). Calls through a non-SDK object – e.g. ``my_service.exec_prompt()`` – are ****not**** flagged because ``my_service.exec_prompt`` is not in the matched name list.

Fix – switch to a comparative principle:

```
```python
gl.eq_principle.prompt_comparative(
 lambda: gl.exec_prompt("What is 2+2?", response_format=int),
 principle="Return YES if both answers are numerically equal",
)
```
```

Layer 2: SDK Validation (Accurate)

- Downloads GenVM release artifacts (cached at `~/cache/genvm-linter/``)
- Loads exact SDK version specified in contract header
- Validates types, decorators, storage fields
- Extracts ABI schema

Exit Codes

- ``0`` - All checks passed
- ``1`` - Lint or validation errors
- ``2`` - Contract file not found
- ``3`` - SDK download failed

IDE Integration

VS Code Extension

This linter is used by the [GenLayer VS Code Extension]

(<https://github.com/genlayerlabs/vscode-extension>) for real-time contract validation.

Manual Pylance Setup

Use ``genvm-lint setup`` to configure Pylance with the correct SDK paths. This gives you hover docs, go-to-definition, and type checking without the extension.

```
```bash
genvm-lint setup --contract contract.py
```
```

Add the output paths to your VS Code ``settings.json``:

```
```json
{
 "python.analysis.extraPaths": ["<output paths>"],
 "python.analysis.reportMissingModuleSource": "none"
}
```
```

Type Checking

`genvm-lint typecheck` runs Pyright with the SDK auto-configured. By default it suppresses SDK-internal noise (dynamic attributes, NewType compat). Use `--all` to see everything, `--strict` for strict mode.

Development

```
```bash
git clone https://github.com/genlayerlabs/genvm-linter.git
cd genvm-linter
pip install -e ".[dev]"
pytest
```
```

License

MIT

GenLayer Labs

Source: <https://docs.genlayer.com/partners/genlayerlabs>

[GenLayer Labs](<https://www.genlayerlabs.com/>) is a pioneering AI research lab and one of the core developers of GenLayer. GenLayer Labs focuses on using AI to enhance the performance, security, and scalability of blockchain networks, with work dedicated to revolutionizing human-AI interaction. By embedding AI into the blockchain's core, GenLayer Labs enables decentralized applications to make smart decisions autonomously, opening new possibilities for automation and efficiency in various industries.

We encourage other developers and organizations to collaborate with GenLayer Labs on this groundbreaking project. Together, we can push the boundaries of what AI and blockchain technology can achieve.

Heurist

Source: <https://docs.genlayer.com/partners/heurist>

[Heurist](<https://www.heurist.ai/>) is a Layer 2 network for AI model hosting and inference, built on the ZK Stack. Heurist offers serverless access to open-source AI models through a decentralized network, making AI deployment easy and efficient for developers. This approach promotes transparency and reduces bias, which is especially useful for projects like GenLayer that require detailed AI analysis.

In collaboration with GenLayer, Heurist provides APIs for open-source large language models (LLMs) to enable developers building Intelligent Contracts to use AI at low or no cost. Developers building on GenLayer can obtain free [Heurist API credits](<https://dev-api-form.heurist.ai>) by using the referral code: `_"genlayer"_`.

This partnership underscores our commitment to providing robust tools and scalable solutions for decentralized applications.

io.net

Source: <https://docs.genlayer.com/partners/ionet>

io.intelligence is an AI infrastructure and API platform from [io.net](<https://io.net>) that gives the IO community and AI developers API access to advanced AI models and AI agents. The platform lets users access and integrate pre-trained open-source models and custom AI agents into applications through API calls, powered by [io.net](<https://io.net>)'s decentralized distributed compute network.

Through its partnership with GenLayer, io.net provides access to over 25 top open-source models, including the latest versions of Deepseek, Qwen, Llama, and Mistral, through one unified API endpoint. Developers on GenLayer can receive free daily compute tokens to access io.intelligence for Enterprise by completing a short [form](https://form.typeform.com/to/pDmCCViV).

API Reference: <https://docs.io.net/reference/get-started-with-io-intelligence-api>

Comput3

Source: <https://docs.genlayer.com/partners/comput3>

[Comput3](https://genlayer.comput3.ai/) is a decentralized compute network that provides serverless access to open-source AI models through distributed infrastructure. The platform gives developers access to powerful AI capabilities without requiring local hardware.

In partnership with GenLayer, Comput3 provides access to inferencing APIs with support for llama3, hermes3, and qwen3 models. GenLayer Validators can use the Comput3.ai inferencing API to power their Intelligent Contracts with high-quality AI responses, and can obtain free [Comput3 API credits](https://genlayer.comput3.ai/) to get started with their validator setup.

This partnership enhances the GenLayer ecosystem by providing reliable and scalable AI inference capabilities for decentralized applications.

Using Chutes with GenLayer Validators

Source: <https://docs.genlayer.com/partners/chutes>

Chutes is a serverless inference platform for open-source AI models that GenLayer validators can access through the Chutes API. Chutes runs on decentralized GPU infrastructure powered by Bittensor, so validators can call AI models through a simple HTTP API without running their own GPU infrastructure.

This guide explains how GenLayer validators can use Chutes to access AI models, authenticate requests, and run model inference from a validator environment. GenLayer Validators can request a discount on Chutes models by [filling out this form](https://docs.google.com/forms/d/e/1FAIpQLSd6I_Q0aiCalDQLX9FvzQ3hfFaKR4j1jaDEoa-jfpXhLclmog/viewform).

Why Chutes

Running large AI models locally requires significant GPU infrastructure.

Chutes allows validators to run AI workloads without managing GPUs, by providing:

- Serverless model inference
- Decentralized GPU compute
- A simple REST API
- Access to many open-source AI models

Validators can integrate Chutes into their systems to run AI workloads while keeping infrastructure lightweight.

Resources

- [Chutes Website](https://chutes.ai)
- [Chutes Documentation](https://chutes.ai/docs)

- [Chutes API Reference](https://api.chutes.ai/docs)
- [Available Models (API)](https://llm.chutes.ai/v1/models)
- [Model Browser (Web UI)](https://chutes.ai/app?type=llm)

1. Create a Chutes Account

First create an account: https://chutes.ai

Chutes supports multiple login methods including:

- Google login
- Fingerprint-based account identity

When creating your account, store your fingerprint securely. It is an important identifier tied to your account.

2. Generate an API Key

After creating an account, generate an API key.

API keys allow applications to authenticate requests to the Chutes API.

Steps:

1. Log into your Chutes account
2. Open account settings
3. Navigate to API Keys
4. Generate a new key
5. Store the key securely

Your API key will be used in authenticated API requests.

3. Authenticate Requests

The Chutes API uses Bearer token authentication.

Include your API key as a Bearer token in the `Authorization` header. You can verify your key is working by listing the available models:

```
```bash
curl https://llm.chutes.ai/v1/models \
 -H "Authorization: Bearer $CHUTES_API_KEY"
```
```

The API follows standard REST conventions and returns JSON responses.

4. Run Your First Model from a Validator

Validators can call the Chutes inference API directly from their validator environment.

Chutes provides an OpenAI-compatible inference endpoint for running language models.

Endpoint: `https://llm.chutes.ai/v1/chat/completions`

This naming convention is the same format commonly used on Hugging Face, where models are referenced as: `organization/model`

For example: `deepseek-ai/DeepSeek-R1`

The list of available models changes frequently. You can retrieve the latest models here:

- API endpoint: <https://llm.chutes.ai/v1/models>
- Browse them visually: <https://chutes.ai/app?type=llm>

Example request from a validator environment:

```
```bash
curl https://llm.chutes.ai/v1/chat/completions \
 -H "Authorization: Bearer $CHUTES_API_KEY" \
 -H "Content-Type: application/json" \
 -d '{
 "model": "model_team/model_name",
 "messages": [
 {
 "role": "user",
 "content": "Explain decentralized AI infrastructure in one sentence."
 }
]
 }'
```
```

Example response:

```
```json
{
 "choices": [
 {
 "message": {
 "role": "assistant",
 "content": "Decentralized AI infrastructure allows developers and validators to access distributed compute resources for running AI models without owning the underlying hardware."
 }
 }
]
}
```
```

Validators can then use the returned output inside their application logic.

****Example use cases include:****

- Evaluating prompts or responses
- Generating summaries or structured outputs
- Integrating AI into validator tooling

Because the Chutes API is stateless and accessed via HTTPS, it can be integrated into validator environments without additional infrastructure.

5. Discover Available Models

The list of models available on Chutes changes frequently.

You can retrieve the current list using: <https://llm.chutes.ai/v1/models>

****Example:****

```
```bash
curl https://llm.chutes.ai/v1/models
```
```

This endpoint returns the latest models available for inference.

Models can also be browsed in the Chutes web interface: [<https://chutes.ai/app?type=llm>]
(<https://chutes.ai/app?type=llm>)

Best Practices

For validator deployments:

- Create a dedicated API key
- Store the key in environment variables
- Avoid committing API keys to source control

****Example:****

```
```bash
export CHUTES_API_KEY=your_api_key_here
```
```

Next Steps

- [Explore the full API reference](<https://api.chutes.ai/docs>)
- [View the latest available models](<https://llm.chutes.ai/v1/models>)
- [Browse models in the Chutes app](<https://chutes.ai/app?type=llm>)
- [Learn more about Chutes](<https://chutes.ai/docs>)

LibertAI

Source: <https://docs.genlayer.com/partners/libertai>

[LibertAI](<https://libertai.io>) is a decentralized AI platform that provides confidential AI services through [Aleph Cloud](<https://aleph.cloud>) GPUs and TEEs, designed to be more secure and resilient than traditional centralized alternatives while protecting user privacy. LibertAI also offers tools for AI Agents to become self-sustainable & more!

In partnership with GenLayer, [LibertAI](<https://libertai.io>) provides access to top open-source models with complete data confidentiality through one unified API endpoint. Developers on GenLayer can receive free API credits visible in the [LibertAI console](<https://console.libertai.io>). Complete a short [form](<https://tally.so/r/n9AvdK>) to get started.

API Reference: <https://docs.libertai.io/apis>

Morpheus

Source: <https://docs.genlayer.com/partners/morpheus>

[Morpheus](<https://mor.org/>) is a decentralized AI inference network where open-source model providers compete on a peer-to-peer marketplace. The network routes requests across independent compute providers instead of relying on a single centralized API, offering models like DeepSeek, Llama, Qwen, and others, with the available model list updating dynamically as providers join and leave the network.

In partnership with GenLayer, Morpheus provides decentralized LLM inference for Intelligent Contract validators through an OpenAI-compatible API. GenLayer Studio fetches available models from the Morpheus marketplace in real time, so validators always see the latest options. Developers can obtain a [Morpheus API key](<https://mor.org/>) to get started.

This partnership brings decentralized, censorship-resistant AI inference to the GenLayer ecosystem, ensuring that Intelligent Contracts are not dependent on any single provider.

FAQ

Source: <https://docs.genlayer.com/FAQ>

What is GenLayer?

GenLayer is the **adjudication layer for the agentic economy**. It uses decentralized AI-validator consensus to resolve contracts that require judgment, not just code. It's built as an L2 on zkSync Elastic Chain, anchoring to Ethereum's security. Contracts on GenLayer – called **Intelligent Contracts** – can interpret language, process images, fetch live web data, and reach consensus on contested or subjective outcomes. See [What is GenLayer] (/understand-genlayer-protocol/what-is-genlayer).

How is GenLayer different from other blockchains?

Traditional smart contracts can only execute deterministic logic and rely on external oracles for off-chain data. GenLayer integrates AI at the protocol level – contracts natively access LLMs, the web, and image processing. No oracles, no intermediaries. Where Bitcoin reached consensus on the *order* of transactions and Ethereum on the *execution* of code, GenLayer reaches consensus on the **meaning** of transactions. See [What is GenLayer] (/understand-genlayer-protocol/what-is-genlayer#how-it-compares).

Is GenLayer trying to replace courts?

No. GenLayer is built to provide a **first-instance, internet-native adjudication layer** for disputes that arrive too quickly, too often, and too globally for traditional courts to handle in time – particularly the disputes the agentic economy is starting to produce at machine speed. It is designed to complement, not replace, existing legal frameworks.

Where does GenLayer fit in the agentic-commerce stack?

The agentic-commerce stack – Coinbase's [x402](<https://www.x402.org/>) for payments, Ethereum's [ERC-8004](<https://eips.ethereum.org/EIPS/eip-8004>) for trustless agent identity, the Linux Foundation's [A2A](<https://a2a-protocol.org>) for agent interoperability, plus Stripe/OpenAI's ACP, Visa's Trusted Agent Protocol, Google's AP2, and Mastercard's Agent Pay – engineers the happy path. None of these standards ships dispute resolution. GenLayer is the layer they reach for when a commitment becomes contested.

What language are Intelligent Contracts written in?

Python. Intelligent Contracts are Python classes extending `gl.Contract`. No Solidity required. See [Your First Contract] (/developers/intelligent-contracts/first-contract).

What is Optimistic Democracy?

GenLayer's consensus mechanism. A leader validator executes the transaction and proposes a result. Other validators independently verify it and vote. If a majority agrees, the transaction is accepted. Anyone can appeal within a finality window, triggering re-evaluation by a larger validator set. See [How GenLayer Works] (/understand-genlayer-protocol/optimistic-democracy-how-genlayer-works).

How do validators handle non-deterministic AI results?

Since LLMs and web data produce varying outputs, GenLayer provides multiple strategies: strict equality (for deterministic operations), LLM-based comparison (validators compare outputs against criteria), and custom validation functions (developers define their own leader/validator logic). See [Non-determinism] (/developers/intelligent-contracts/features/non-determinism).

Can contracts process images?

Yes. Pass images to LLMs via `gl.nondet.exec_prompt(images=[...])`. Contracts can also capture web page screenshots with `gl.nondet.web.render(url, mode='screenshot')` and analyze

them. See [Image Processing](/developers/intelligent-contracts/features/image-processing).

Can contracts send and receive GEN (native token)?

Yes. Use `@gl.public.write.payable` to receive value, `gl.message.value` to access the amount sent, and `emit_transfer()` to send value to other contracts or EOAs. See [Value Transfers](/developers/intelligent-contracts/features/value-transfers).

Do I need to be an AI expert?

No. The GenLayer SDK handles LLM calls and consensus. You write Python and call `gl.nondet.exec_prompt()` with a prompt. Validators handle model execution. GenLayer Studio provides a browser-based IDE for writing, deploying, and testing contracts.

Can I run normal deterministic transactions?

Yes. GenLayer fully supports deterministic logic – standard state updates, value transfers, and contract interactions all work without any AI components. You can mix deterministic and non-deterministic operations in the same contract.

What are ghost contracts?

Every Intelligent Contract has a corresponding ghost contract on the GenLayer Chain (EVM layer) at the same address. Ghost contracts hold the contract's GEN balance, relay transactions to consensus, and execute external messages. See [Ghost Contracts](/developers/intelligent-contracts/features/messages#ghost-contracts).

What can I build?

Anything requiring judgment that deterministic code alone can't resolve: performance and milestone adjudication (bounties, grants, retro funding), adjudication inside the agentic-commerce stack (agent SLAs, escrow on ambiguous completion, reputation disputes), prediction markets, insurance claims, content verification, compliance automation, AI-governed DAOs, and more. See [Use Cases](/understand-genlayer-protocol/typical-use-cases) and [projects building on GenLayer](https://portal.genlayer.foundation/#/).

How do I get started?

1. Try [GenLayer Studio](https://studio.genlayer.com) in the browser – no setup needed
2. Or install the [GenLayer CLI](/api-references/genlayer-cli) for local development
3. Follow [Your First Contract](/developers/intelligent-contracts/first-contract)
4. Use [genlayer-js](/api-references/genlayer-js) or [genlayer-py](/api-references/genlayer-py) for app integration

How is the network secured?

Through delegated proof of stake and Optimistic Democracy:

- Token holders delegate stake to validators, incentivizing honest behavior
- Multiple validators independently verify each transaction
- Appeals provide a failsafe – dishonest validators get slashed
- The system anchors to Ethereum via zkSync for finality

How can I contribute?

- Join the community on [Discord](https://discord.gg/8Jm4v89VAu) or [Telegram](https://t.me/genlayer)
- Contribute to the [GitHub repositories](https://github.com/genlayerlabs)
- [We're hiring](https://x.com/GenLayer/jobs)